

The Backend Engineer's Library

System Design and Architecture

Volume 7

Contents

Principles of Scalable System Design

Back-of-the-Envelope Estimation and Capacity Planning

Caching Strategies at Scale

Load Balancing and Traffic Management

Data Modeling for Scale

Monolith, Microservices, and Between

Event-Driven Architecture

API Gateways, BFF, and Edge

Rate Limiting, Quotas, and Fairness

Multi-Region and Geo-Distributed Systems

Designing for Failure: Bulkheads, Circuit Breakers, and Load Shedding

Design Case Studies: Feed, Chat, Notifications, and a Global KV Store

Chapter 1 — Principles of Scalable System Design

What this chapter covers. Every system design interview starts with "design Twitter" and every production system starts with "why is this slow at 10× load?" This chapter builds the vocabulary and mental models that make the rest of the volume coherent. Scalability is not a feature you bolt on; it is a set of deliberate trade-offs — between consistency and availability, latency and throughput, simplicity and flexibility — applied consistently across every layer. We define what scalability means along its four axes (load, data volume, geographic distribution, organizational scale), introduce six principles that recur in every large system from Google's Borg to a three-service startup that survived product-market fit, show how to quantify scale with Little's Law and queueing theory, and walk through a reference architecture that maps each principle to concrete components. The chapter closes with the antipatterns that turn scalable designs back into distributed monoliths.

Learning goals — after this chapter you should be able to:

- Distinguish scalability (handling growth) from performance (speed at a fixed load) and elasticity (speed of scaling), and name the four dimensions along which a backend system must scale.
- State and apply six principles — horizontal scaling with stateless services, partitioning, replication, decoupling, designing for failure, and operability — to a greenfield design discussion.
- Use Little's Law and basic queueing intuition to relate concurrency, throughput, and latency, and explain why utilization above ~70% is a warning sign even when latency still looks acceptable.
- Sketch a canonical three-tier-plus-async reference architecture and justify the placement of load balancers, caches, queues, and databases within it.
- Identify common scalability antipatterns (stateful app servers, synchronous fan-out, single-writer bottlenecks, unbounded queues) in an architecture review.

- Explain where this volume's concerns end and neighboring volumes begin: Vol 3 for wire-level load balancing, Vol 6 for consistency theory, Vol 11 for operational resilience, and Vol 8 for API contracts.

What "scalable" actually means

Engineers use "scalable" the way food packaging uses "natural" — enthusiastically and imprecisely. A useful definition must be falsifiable. A system is *scalable* with respect to a growth dimension if the marginal cost of handling additional load along that dimension grows at most linearly, and ideally sublinearly, while preserving its correctness and latency SLOs.

Four dimensions matter for backend systems:

Dimension	What grows	Canonical question	Bottleneck that appears first
Load (request volume)	Requests/sec, concurrent connections	"Can we handle 10× traffic after a launch?"	CPU, thread pools, connection limits
Data (volume + velocity)	Rows, bytes, events/sec	"What happens when the primary table hits 2 TB?"	Storage, index size, replication lag
Geography (distribution)	Regions, edge PoPs, user distance	"Can a user in Jakarta get p95 < 200 ms?"	Speed of light, cross-region consistency
Organization (team scale)	Services, repositories, deploy frequency	"Can 40 teams deploy independently?"	Coupling, coordination cost, blast radius

A system can be scalable along one axis and brittle along another. A single beefy Postgres instance with a large buffer pool may handle 10× read load via replicas (load-scalable) but collapse when the dataset no longer fits on one machine's storage (not data-scalable) or when 15 teams

contend on one schema migration (not organizationally scalable). Naming the axis forces honesty about what was designed for and what was not.

Three terms are routinely conflated with scalability and must be separated:

- **Performance** is a point measurement: latency or throughput at a *fixed* load. A system can be fast and unscalable (a hand-tuned single-node cache) or scalable and not yet fast (a correctly partitioned system running on under-provisioned hardware).
- **Elasticity** is the *speed and automation* of scaling. A system that requires a weekend maintenance window and a DBA to add a shard is scalable (it can grow) but not elastic.
- **Availability** is the fraction of time the system serves correctly. Scalability and availability are allies — horizontal scaling removes single points of failure — but they are not the same. Adding nodes can improve availability while hurting scalability if coordination costs rise superlinearly.

Boundary note. This chapter frames scalability decisions at the *system* level — which components to replicate, partition, or decouple. The mechanism of distributing traffic among replicas at the *wire* level (L4/L7 algorithms, consistent hashing on the data path, health checking) is covered in Vol 3, Chapter 9 — Load Balancing: L4, L7, and Algorithms and revisited for system-level traffic policy in Chapter 4 of this volume. Consistency models and impossibility results that constrain how far replication can go are in Vol 6 — Distributed Systems. Operational aspects of scaling (autoscaling, capacity planning in production) are in Vol 11 and Vol 12.

Scalability is an economic claim

Every scalability argument is implicitly an economic one. "We scale horizontally" means "we can add capacity in small, commodity increments whose cost is predictable." Vertical scaling — a bigger machine — is not wrong; it is often the cheapest path to 5-10× growth. It becomes wrong when the cost curve goes vertical: the next-larger instance type costs 4× for 1.6× performance, or the largest available machine is simply not large enough. The inflection point is where the economics flip, and the architecture must already be ready for horizontal scaling before that

point arrives. Retrofitting statelessness onto a stateful system under load is the most expensive migration most teams will ever attempt.

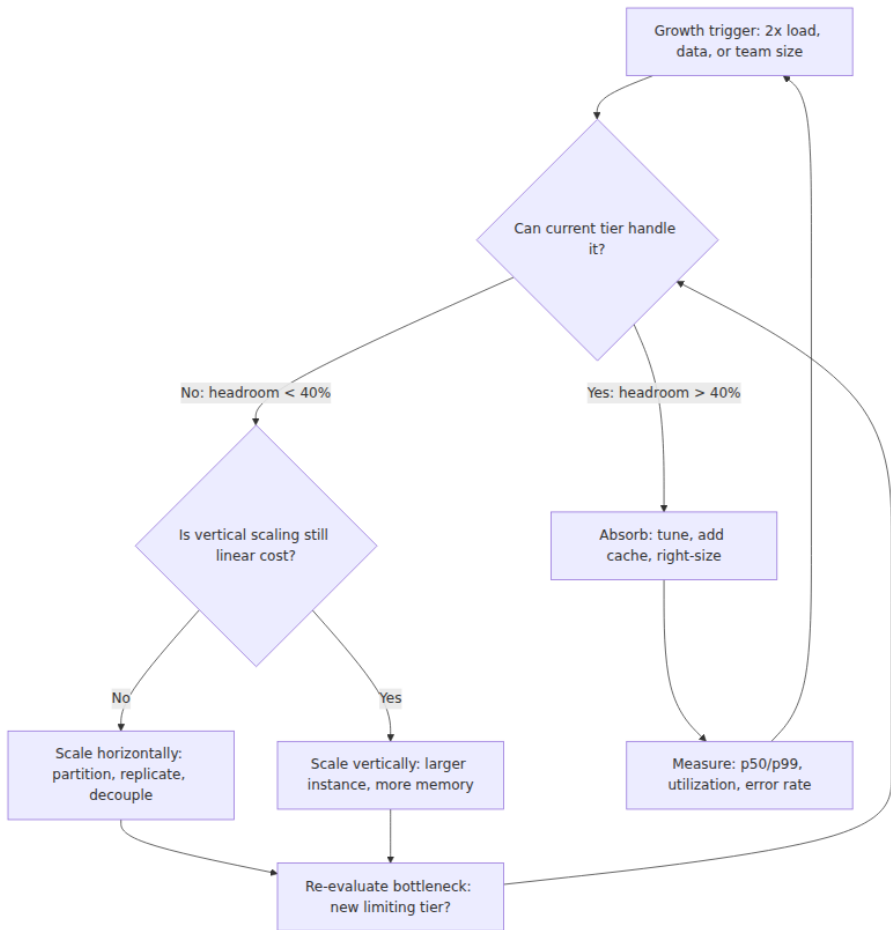


Figure 1-1: The scaling decision loop. Most systems cycle through "absorb → vertical → horizontal" as they grow; the key is detecting the cost inflection before it becomes an emergency.

Principle 1 — Scale horizontally with stateless services

The single most consequential architectural decision is whether application servers hold state that matters across requests. A *stateless* service stores no session, no local cache that must be coherent, and no in-memory queue that would be lost on restart. Any instance can handle any request; any instance can die without data loss.

Statelessness is what makes horizontal scaling trivial. If N instances handle R requests per second, $N+1$ instances handle roughly $R \times (N+1)/N$, minus load-balancer overhead. No data migration, no rebalancing, no "drain this node before terminating."

Achieving statelessness requires pushing state to the right place:

- **Session state** → external store (Redis, database) or stateless tokens (JWTs validated with a shared key; see Vol 9, Chapter 5).
- **Local caches** → external caches (Redis, Memcached) or accept per-instance caches with explicit staleness bounds.
- **In-flight work** → durable queues (Kafka, SQS) so a crash does not lose tasks.

A minimal stateless service behind a load balancer looks like this in practice:

```

# kubernetes deployment – 12-factor stateless app (Kubernetes v1.29)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  labels: { app: api }
spec:
  replicas: 6
  selector:
    matchLabels: { app: api }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 2, maxUnavailable: 1 }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: registry.internal/api:1.42.0
          ports: [{ containerPort: 8080 }]
          env:
            - name: REDIS_URL
              value: "redis://redis.internal:6379/0"
            - name: DATABASE_URL
              valueFrom: { secretKeyRef: { name: db-creds, key: url } }
            - name: PORT
              value: "8080"
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
            periodSeconds: 5
            failureThreshold: 3
          livenessProbe:
            httpGet: { path: /healthz, port: 8080 }
            periodSeconds: 10
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits: { cpu: "1000m", memory: "1Gi" }
---
apiVersion: v1
kind: Service
metadata: { name: api }
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api }

```



```
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: api }
  minReplicas: 6
  maxReplicas: 40
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization,
averageUtilization: 65 } }
    - type: Pods
      pods: { metric: { name: http_requests_per_second }, target: {
type: AverageValue, averageValue: "800" } }
```

With this manifest, `kubectl scale deployment api --replicas=12` or the HPA doubles capacity without touching application code — *because* no instance holds state the others need.

```
$ kubectl get hpa api --watch
NAME      REFERENCE          TARGETS          MINPODS  MAXPODS  REPLIC
AS
api       Deployment/api      62%/65%, 742/800  6         40       6
api       Deployment/api      88%/65%, 1180/800 6         40       6
api       Deployment/api      88%/65%, 1180/800 6         40       9 #
scaled up
api       Deployment/api      54%/65%, 610/800  6         40       9 #
load spread
```

Distributed-systems lens. Stateless services push the hard problems — consistency, durability, coordination — into the stateful tier (databases, caches, queues). That is deliberate. Databases have spent 40 years solving those problems; reimplementing them poorly inside application processes is how outages are made. The trade-off is network dependency: every request now needs at least one round-trip to the state tier, so latency, timeout, and retry semantics (Vol 3, Chapter 11) become first-class concerns.

Principle 2 — Partition everything that cannot fit on one machine

When data or load exceeds what a single node can handle — storage capacity, write throughput, memory for indexes — the answer is *partitioning*: split the dataset so each partition lives on a different node and most operations touch only one partition.

Partitioning strategies:

Strategy	How it maps keys → partitions	Strength	Weakness
Range	Key ranges (e.g., user_id 0–1M → shard 0)	Range scans are local	Hot ranges, rebalancing is expensive
Hash	hash(key) % N	Uniform distribution	Rehashing on N change moves ~all data
Consistent hash	Hash ring with virtual nodes	Minimal movement on membership change	Slight imbalance without enough vnodes
Directory	Lookup service (e.g., shard map in ZooKeeper)	Arbitrary placement, easy to move	Directory is a critical dependency

Consistent hashing is the default for caches and many KV stores because it minimizes disruption when nodes join or leave:

```

# consistent_hash.py - minimal consistent hash ring (Python 3.11)
import hashlib, bisect

class ConsistentHashRing:
    def __init__(self, nodes: list[str], vnodes: int = 150):
        self.vnodes = vnodes
        self.ring: dict[int, str] = {}
        self.sorted_keys: list[int] = []
        for node in nodes:
            self.add_node(node)

    def _hash(self, key: str) -> int:
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node: str) -> None:
        for i in range(self.vnodes):
            h = self._hash(f"{node}:{i}")
            self.ring[h] = node
            bisect.insort(self.sorted_keys, h)

    def remove_node(self, node: str) -> None:
        for i in range(self.vnodes):
            h = self._hash(f"{node}:{i}")
            self.ring.pop(h, None)
            idx = bisect.bisect_left(self.sorted_keys, h)
            if idx < len(self.sorted_keys) and self.sorted_keys[idx]
== h:
                self.sorted_keys.pop(idx)

    def get_node(self, key: str) -> str:
        if not self.ring:
            raise ValueError("empty ring")
        h = self._hash(key)
        idx = bisect.bisect_right(self.sorted_keys, h) % len(self.sorted_keys)
        return self.ring[self.sorted_keys[idx]]

# demo
ring = ConsistentHashRing(["cache-1", "cache-2", "cache-3"])
for k in ["user:42", "user:99", "order:1001", "session:abc"]:
    print(f"{k:15s} -> {ring.get_node(k)}")

```

```

user:42          -> cache-2
user:99          -> cache-1
order:1001       -> cache-3
session:abc      -> cache-2

```

Adding `cache-4` moves only $\sim 1/(N+1)$ of keys — roughly 25% when going from 3 to 4 nodes — versus $\sim 75\%$ with naive modulo hashing.

What partitioning costs. Every cross-partition operation (a query that touches two shards, a transaction that spans shards) is an order of magnitude more expensive and complex than a single-partition operation. The discipline is to choose a *partition key* that keeps the common case local. For a multi-tenant SaaS, `tenant_id` is usually correct; for a social feed, `user_id` is; for time-series, `metric + time bucket` is. Get the partition key wrong and the system will spend its life doing scatter-gather.

Boundary note. Physical partitioning mechanics (range vs. hash, rebalancing protocols, shard placement) overlap with Vol 5, Chapter 9 — Partitioning and Sharding, which treats storage-engine specifics. This chapter treats partitioning as an *architectural* choice; Vol 5 treats it as a *storage* implementation.

Principle 3 — Replicate for availability, and pay the consistency price consciously

If partitioning is about *capacity*, replication is about *survivability*. Every critical piece of data and every critical service should exist in more than one place so the failure of one replica does not become a user-visible outage.

Replication introduces the central tension of distributed systems: the more replicas you have, and the more available you want them to be during partitions, the harder it is to keep them consistent. This is the CAP/PACELC trade-off covered formally in Vol 6, Chapters 3-4. At the system-design level, the practical question is:

For this piece of data, what happens if two replicas diverge for 5 seconds?

- **If the answer is "nothing bad"** (session cache, precomputed recommendations, CDN content): use asynchronous, eventually consistent replication. Availability wins.
- **If the answer is "we double-charge or lose money"** (ledger, inventory, seat reservation): use synchronous replication or

consensus, and accept higher latency and lower availability during partitions.

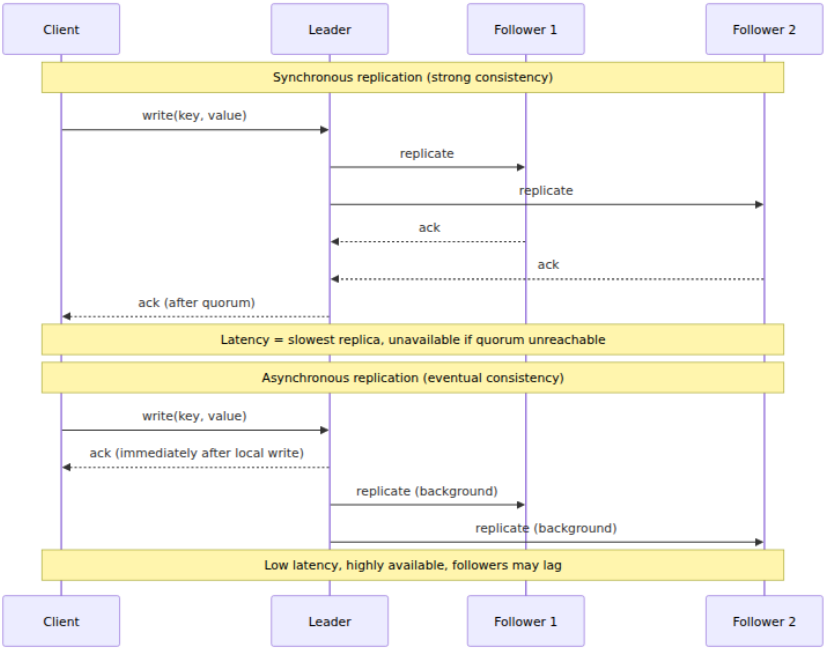
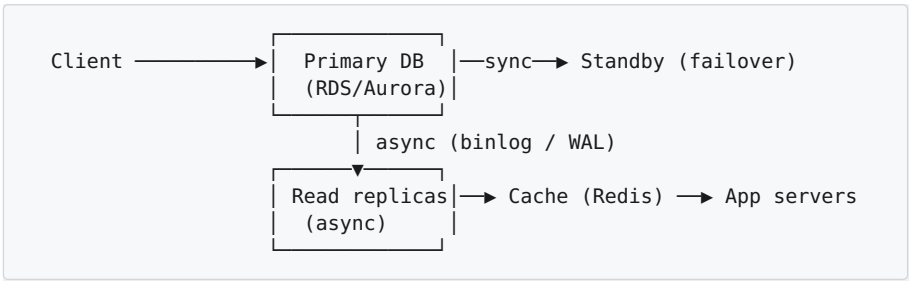


Figure 1-2: Synchronous versus asynchronous replication. The same three replicas make radically different trade-offs depending on when the client is acknowledged.

Real systems mix both. A common pattern:



Writes go to the primary and are synchronously replicated to a standby for failover (durability + availability). Reads fan out to asynchronous replicas and caches (scalability). The replication lag window — typically 10–100 ms for Aurora/MySQL, occasionally seconds under load — is the

inconsistency window the application must tolerate. If the application cannot tolerate it (e.g., "read your own writes" after a user updates their profile), it must read from the primary for that user's recent writes — a *read-after-write* routing rule.

```
# Aurora MySQL – observe replication lag (MySQL 8.0, Aurora 3.x)
$ mysql -h aurora-primary.internal -e "SHOW SLAVE STATUS\G" 2>/dev/
null | grep -E "Seconds_Behind|Replica_Lag"
Seconds_Behind_Master: 0
AuroraReplicaLag: 12 # milliseconds – healthy

$ mysql -h aurora-replica.internal -e "SELECT Replica_lag_in_msec FROM
information_schema.replica_host_status\G"
Replica_lag_in_msec: 47

# During a bulk load – lag spikes, application must handle staleness
$ mysql -h aurora-replica.internal -e "SELECT Replica_lag_in_msec FROM
information_schema.replica_host_status\G"
Replica_lag_in_msec: 1843
```

Principle 4 — Decouple in time and space

Synchronous calls couple caller and callee in *time* (both must be up simultaneously) and in *space* (caller must know callee's address). Each synchronous hop adds latency, reduces availability (availability multiplies: $99.9\% \times 99.9\% \times 99.9\% \approx 99.7\%$), and creates backpressure that propagates instantly.

Decoupling — via queues, event logs, or async workers — breaks one or both couplings:

Coupling	Synchronous	Decoupled
Time	Caller blocks until callee responds	Caller enqueues and continues; callee processes when ready
Space	Caller knows callee endpoint	Caller publishes to a topic; any subscriber can consume

A concrete example. An e-commerce checkout that synchronously calls inventory, payment, email, and analytics:

Checkout → Inventory (80 ms) → Payment (200 ms) → Email (50 ms) → Analytics (30 ms)
Total: ~360 ms, and any failure fails the checkout

Refactored with a queue:

Checkout → Inventory (80 ms) → Payment (200 ms) → Enqueue {order events}
Total user-visible: ~280 ms
Background: Email worker ← queue, Analytics worker ← queue (retries independently)

```
# producer – checkout service (Python 3.11, boto3 1.34)
import boto3, json, uuid, time

sqs = boto3.client("sqs", region_name="us-east-1")
QUEUE_URL = "https://sqs.us-east-1.amazonaws.com/123456789012/order-events"

def place_order(user_id: str, items: list[dict]) -> str:
    order_id = str(uuid.uuid4())
    # synchronous: only what the user must wait for
    reserve_inventory(items)
    charge_payment(user_id, items)

    # asynchronous: everything else
    sqs.send_message(
        QueueUrl=QUEUE_URL,
        MessageBody=json.dumps({
            "event": "order.placed",
            "order_id": order_id,
            "user_id": user_id,
            "items": items,
            "ts": int(time.time()),
        }),
        MessageGroupId=user_id,          # FIFO ordering per user if
        MessageDeduplicationId=order_id, # exactly-once enqueue with
        FIFO
    )
    return order_id
```

```

# consumer – email worker (long poll, visibility timeout handles
crashes)
import boto3, json

sqs = boto3.client("sqs", region_name="us-east-1")
QUEUE_URL = "https://sqs.us-east-1.amazonaws.com/123456789012/order-
events"

while True:
    resp = sqs.receive_message(
        QueueUrl=QUEUE_URL,
        MaxNumberOfMessages=10,
        WaitTimeSeconds=20,          # long poll – no busy loop
        VisibilityTimeout=30,        # if worker dies, message
        # reappears after 30s
    )
    for msg in resp.get("Messages", []):
        event = json.loads(msg["Body"])
        try:
            send_confirmation_email(event["user_id"], event["order_id"])
        except Exception as e:
            # don't delete – message becomes visible again for retry /
            # DLQ
            print(f"failed {event['order_id']}: {e}")

```

The queue absorbs bursts, survives consumer outages, and lets email and analytics scale independently of checkout. The cost is *eventual* side effects (email arrives seconds later) and the need to handle duplicate deliveries (SQS at-least-once; see Vol 10, Chapter 2 for delivery semantics).

Boundary note. Queue and log internals (Kafka partitions, SQS visibility timeout, exactly-once via idempotency keys and the outbox pattern) are in Vol 10 — Messaging, Streaming, and Event Systems. This chapter uses queues as an architectural primitive; Vol 10 explains how they work and fail.

Principle 5 — Design for failure as the normal case

In a system with hundreds of instances, failure is not exceptional — it is the steady state. Disks fail, nodes get preempted, deploys roll out bad

code, a downstream dependency times out. A scalable architecture treats failure handling as a first-class design activity, not an afterthought.

Three patterns recur:

1. **Timeouts everywhere, with budgets.** Every outbound call gets a timeout derived from the caller's SLO. If the caller's p99 budget is 500 ms and it makes two sequential calls, neither can have a 400 ms timeout. Budgets compose.
2. **Retries with backoff and jitter, but only for safe operations.** Retrying a non-idempotent `POST /charge` without an idempotency key doubles the charge. Retrying a `GET` is safe. Exponential backoff with jitter prevents thundering herds:

```
```python
```

## retry.py — exponential backoff with decorrelated jitter

---

```
import random, time
```

```
def retry_with_backoff(fn, max_attempts=5, base_ms=50,
 cap_ms=5000):
 delay_ms = base_ms
 for attempt in range(1, max_attempts + 1):
 try:
 return fn()
 except TransientError as e:
 if attempt == max_attempts:
 raise # decorrelated jitter: random
 between base and 3*current delay
 delay_ms = random.uniform(base_ms, min(cap_ms, delay_ms * 3))
 time.sleep(delay_ms / 1000.0)
```
```

3. **Circuit breakers and bulkheads.** When a downstream is failing, stop calling it for a while (circuit open) and shed load gracefully rather than queuing indefinitely. Bulkheads isolate failure domains so one slow dependency does not starve all threads.

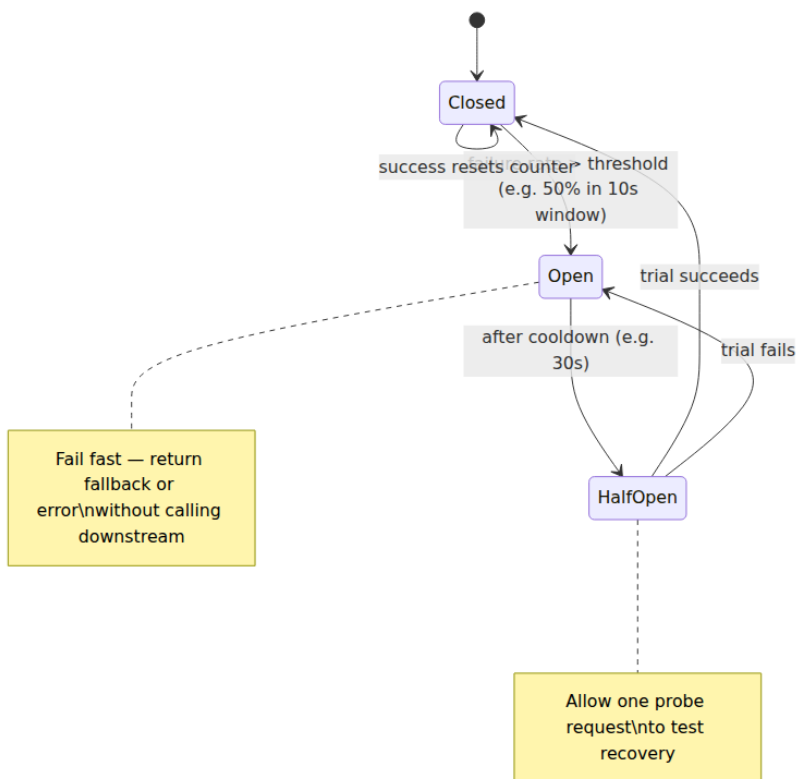


Figure 1-3: Circuit breaker state machine. Without it, a failing downstream turns every caller into a thread-pool exhaustion incident.

Real-world defaults that have survived production:

```
# resilience4j-style config (Spring Boot 3.2, resilience4j 2.1)
resilience4j.circuitbreaker:
  configs:
    default:
      slidingWindowType: COUNT_BASED
      slidingWindowSize: 100
      failureRateThreshold: 50
      slowCallRateThreshold: 80
      slowCallDurationThreshold: 2s
      permittedNumberOfCallsInHalfOpenState: 5
      waitDurationInOpenState: 30s
      minimumNumberOfCalls: 20
resilience4j.bulkhead:
  configs:
    default:
      maxConcurrentCalls: 40
      maxWaitDuration: 10ms
```

Principle 6 — Make the system observable and operable

A system that cannot be understood cannot be scaled. Observability is not "we have dashboards" — it is the ability to answer *why* the system behaved a certain way from its outputs, without needing to reproduce the behavior.

The three pillars, with backend-specific emphasis:

- **Metrics** (counters, gauges, histograms) — for alerting and capacity planning. Use histograms (Prometheus) or DDSketch, not averages. Alert on symptoms (latency, error rate) not causes (CPU).
- **Logs** (structured, sampled) — for forensics. Log request IDs, not just messages. At scale, log volume is a cost and performance concern; sample aggressively and index selectively.
- **Traces** (distributed, sampled) — for understanding fan-out. A single user request that touches 12 services is invisible without tracing.

Operability is the other half: can you *change* the system safely? That means:

- **Feature flags and progressive delivery** so new code can be enabled for 1% of traffic and rolled back without a deploy.

- **Runbooks and automation** for common operations (shard rebalancing, cache warming, failover) so they do not require tribal knowledge at 3 AM.
- **Capacity headroom and load shedding** so overload degrades gracefully rather than cascading.

```
# Prometheus – query p99 latency by service, the only latency worth
alerting on (PromQL)
$ curl -s 'http://prometheus.internal:9090/api/v1/query?
query=histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket{service="api"}[5m])))' |
jq .

# Jaeger – find traces where checkout exceeded 500ms
$ curl -s 'http://jaeger.internal:16686/api/traces?
service=checkout&minDuration=500ms&limit=20' | jq '.data[].traceID'

# kubectl – progressive delivery via Argo Rollouts (analysis template
checks error rate)
$ kubectl argo rollouts get rollout api --watch
Name:                api
Strategy:             Canary
  Step:              2/5
  Canary weight:     20%
Analysis:            successRate (current: 99.92%, threshold: 99.5%) – Succ
essful
```

Boundary note. Full treatment of SLIs/SLOs, metrics/tracing/logging, incident response, and deployment strategies is in Vol 11 — Reliability, Observability, and SRE. This chapter notes what must be designed in from day one; Vol 11 shows how to operate it.

Quantifying scale: Little's Law and queueing intuition

Intuition about scale is often wrong. Queueing theory gives two simple tools that are right often enough to be useful.

Little's Law

For any stable system:

$$L = \lambda \times W$$

- **L** = average number of items in the system (concurrency)
- **λ** = average arrival rate (throughput, requests/sec)
- **W** = average time an item spends in the system (latency, seconds)

Rearranged, it answers practical questions:

- "We handle $\lambda = 2,000$ req/s at $W = 150$ ms average latency. How many concurrent requests are in the system?" $\rightarrow L = 2000 \times 0.15 =$ **300 concurrent**.
- "We have a thread pool of 400. At 200 ms latency, what throughput can we sustain before queueing?" $\rightarrow \lambda = L / W = 400 / 0.2 =$ **2,000 req/s**. Beyond that, requests queue and latency rises.

```
# littles Law.py - capacity check
def max_throughput(concurrency: int, latency_s: float) -> float:
    return concurrency / latency_s

def required_concurrency(throughput_rps: float, latency_s: float) -> float:
    return throughput_rps * latency_s

print(f"400 threads @ 200ms -> {max_throughput(400, 0.2):.0f} rps")
print(f"Need for 5000 rps @ 100ms -> {required_concurrency(5000, 0.1):.0f} concurrent slots")
```

```
400 threads @ 200ms -> 2000 rps
Need for 5000 rps @ 100ms -> 500 concurrent slots
```

This is why adding threads is not free: each thread consumes memory (stack, buffers) and increases context-switching and GC pressure (Vol 1, Chapter 2; Vol 13). The scalable answer to higher λ is usually *lower W* (faster handlers, caching, async) or *more instances*, not a larger pool on one box.

Utilization and the knee

As utilization ($\rho = \lambda / \mu$, where μ is service rate) approaches 1, queue length and latency grow hyperbolically. The M/M/1 queue — the simplest model — gives mean queue length $= \rho / (1 - \rho)$:

| Utilization (ρ) | Mean queue length | What it feels like |
|------------------------|-------------------|------------------------------------|
| 0.50 | 1.0 | Comfortable |
| 0.70 | 2.3 | Warm — watch closely |
| 0.80 | 4.0 | Hot — tail latency rising |
| 0.90 | 9.0 | Saturated — p99 spiking |
| 0.95 | 19.0 | Overloaded — retries make it worse |

The practical rule: **target 60-70% utilization for latency-sensitive services**. The remaining headroom is not waste; it is the buffer that absorbs bursts without queueing. Autoscaling thresholds should fire at ~65% so new capacity is ready before the knee.

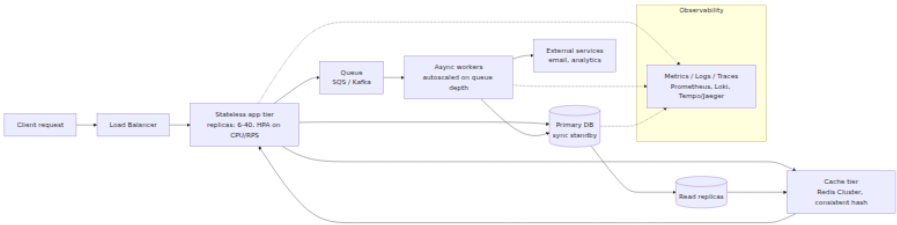


Figure 1-4: Reference architecture that embodies all six principles — stateless app tier, partitioned cache, replicated database with async read path, decoupled async workers, and observability as a sidecar concern.

Common antipatterns

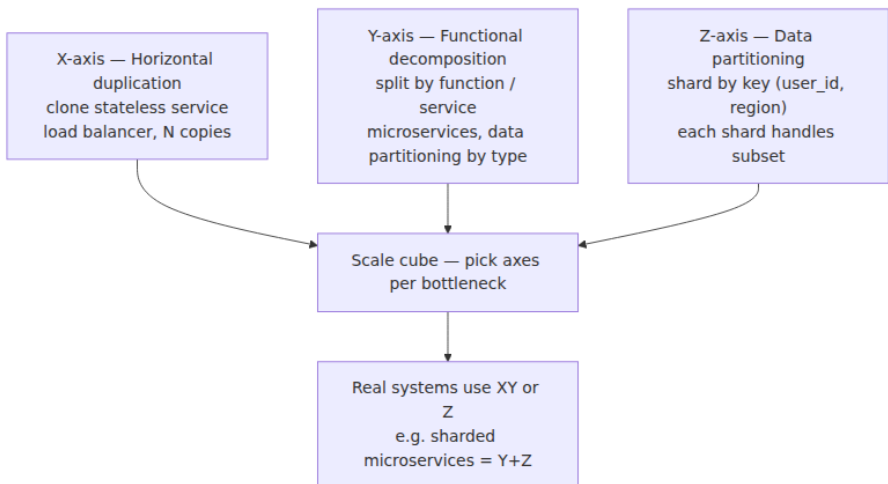
| Antipattern | Why it feels natural | How it breaks at scale | Fix |
|---|--------------------------|--|--|
| Stateful app servers (sticky sessions) | "Just keep it in memory" | Cannot scale or restart without user impact; LB becomes stateful | External session store or stateless tokens |

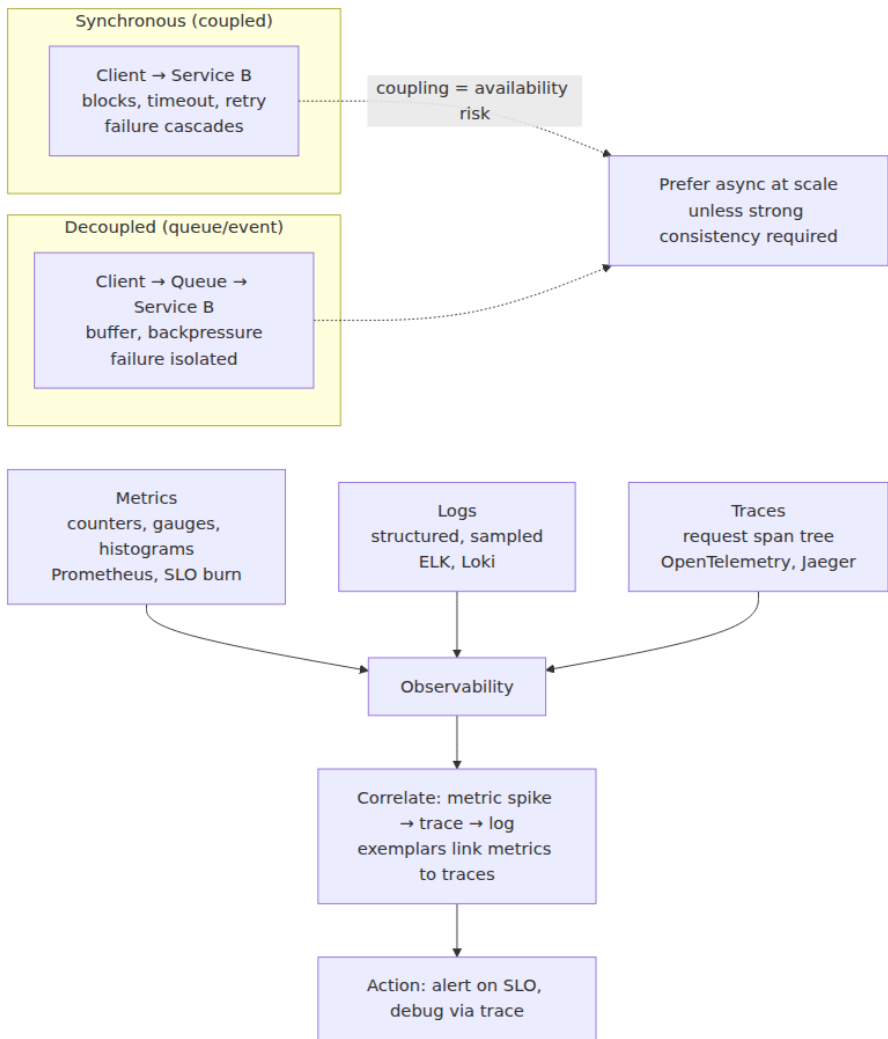
| Antipattern | Why it feels natural | How it breaks at scale | Fix |
|---|----------------------------------|--|--|
| Synchronous fan-out (one request → N sequential RPCs) | Simple to code | Latency adds, availability multiplies, one slow dep blocks all | Parallelize with timeouts, or make non-critical calls async |
| Single-writer bottleneck (all writes through one node/row) | Strong consistency is comforting | Write throughput capped at one machine | Partition by key, or use CRDTs / conflict resolution where allowed |
| Unbounded queues / thread pools | "Don't reject work" | OOM, GC storms, cascading latency | Bounded pools + backpressure + load shedding |
| Chatty clients (N+1 queries, large payloads) | ORM defaults | Network round-trips dominate latency | Batching, DataLoader, pagination, field selection |
| Shared database as integration (services join each other's tables) | "It's right there" | Coupling, contention, impossible to split | API or event-based integration; each service owns its data |

Key takeaways

- Scalability is not "fast" — it is the ability to handle growth along load, data, geography, and organizational axes at predictable marginal cost. Name the axis before proposing a solution.
- Six principles — stateless horizontal scaling, partitioning, replication with explicit consistency trade-offs, decoupling via queues, designing for failure, and observability/operability — are the recurring toolkit. Every large system is a composition of them.
- Horizontal scaling requires stateless services. Push session, cache, and queue state into infrastructure that is designed to manage it, and accept the network round-trip as the price.

- Partition to increase capacity, replicate to increase availability. Partitioning makes cross-partition operations expensive — choose the partition key that keeps the common case local. Replication forces a consistency choice — synchronous for correctness, asynchronous for scale.
- Decouple synchronous request paths from asynchronous side effects with durable queues. Durability and idempotency handling are the hard parts; the queue itself is simple.
- Failure is steady-state. Timeouts with budgets, retries only for idempotent operations with jittered backoff, circuit breakers, and bulkheads are not optional extras — they are the difference between a degraded dependency and a site-wide outage.
- Little's Law ($L = \lambda W$) and the utilization knee are the two quantitative intuitions that prevent most capacity misjudgments. Target 60-70% utilization and autoscale before the knee, not after.
- Know the boundaries: wire-level load balancing is Vol 3, consistency theory is Vol 6, observability and SRE practice is Vol 11, and API contracts are Vol 8. This chapter is the map; those volumes are the territories.





Further reading

- Kleppmann, M. *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapters 1-2, 5-6, 8-9, 12 — the most complete single treatment of partitioning, replication, and consistency for practitioners. <https://dataintensive.net/>
- Brewer, E. "CAP Twelve Years Later: How the 'Rules' Have Changed" (*Computer*, 45(2), 2012) — Brewer's own correction of CAP

oversimplifications and introduction of the harvest/yield framing.
<https://ieeexplore.ieee.org/document/6133253>

- Abadi, D. "Consistency Tradeoffs in Modern Distributed Database System Design" (*Computer*, 45(2), 2012) — the PACELC paper: what happens *else* when there is no partition. <https://cs-www.cs.yale.edu/homes/dna/papers/abadi-pacelc.pdf>
- Beyer, B., Jones, C., Petoff, J., Murphy, N. *Site Reliability Engineering* (O'Reilly, 2016), Chapters 2-3, 20-22 — Google's SRE framing of scalability as an economic and organizational problem. <https://sre.google/sre-book/table-of-contents/>
- Dean, J. and Barroso, L. A. "The Tail at Scale" (*CACM*, 56(2), 2013) — why p99 matters more than the mean and how to tame tail latency at scale. <https://research.google/pubs/the-tail-at-scale/>
- AWS Builders' Library — "Challenges with Distributed Systems" and "Timeouts, Retries and Backoff with Jitter" — short, experience-driven pieces from Amazon's internal practices. <https://aws.amazon.com/builders-library/>
- Nygard, M. *Release It!* 2nd ed. (Pragmatic Bookshelf, 2018), Chapters 4-5 — the original popularization of circuit breakers, bulkheads, and stability patterns. <https://pragprog.com/titles/mnee2/release-it-second-edition/>
- Little, J. D. C. "A Proof for the Queuing Formula: $L = \lambda W$ " (*Operations Research*, 9(3), 1961) — the three-page proof of Little's Law; worth reading once to see why it holds regardless of arrival distribution. <https://www.jstor.org/stable/167570>

Next: Chapter 2 — Back-of-the-Envelope Estimation and Capacity Planning — turns these principles into numbers: how to estimate QPS,

storage, bandwidth, and instance counts on a whiteboard, with enough accuracy to choose an architecture before writing code.

Chapter 2 — Back-of-the-Envelope Estimation and Capacity Planning

What this chapter covers. System design lives or dies on numbers. Before choosing databases, drawing boxes, or debating microservices, you must answer: how many requests per second, how many bytes stored, how much bandwidth, how many machines? Back-of-the-envelope estimation is the shared language that turns hand-waving into engineering. This chapter teaches the estimation toolkit — powers of ten, latency numbers every engineer should know, storage and bandwidth arithmetic — and applies it to five canonical workloads (read-heavy feed, write-heavy ingest, search, chat, and blob delivery). We then connect estimation to capacity planning: Little's Law and queueing models from Chapter 1 become provisioning formulas, and we show how to size instance fleets, database tiers, and cache layers with headroom, replication overhead, and failure domains included. The goal is fluency: you should be able to walk into a whiteboard interview or an architecture review and produce defensible numbers in under ten minutes.

Learning goals — after this chapter you should be able to:

- Recall latency, throughput, and capacity numbers to one significant figure — L1 cache through cross-region round-trips, disk and network bandwidth, and common request rates — and use them without looking them up.
- Decompose any estimation prompt into four quantities — QPS, storage, bandwidth, and instance count — and compute each from first principles with explicit assumptions.
- Apply powers-of-two and decimal approximations ($2^{10} \approx 10^3$, 1 KB $\approx 10^3$ bytes) fluently to do mental math at whiteboard speed.
- Run five canonical estimation templates (feed, ingest, search, chat, blob/CDN) and adapt them to novel products by swapping assumptions, not frameworks.

- Translate QPS and latency SLOs into fleet size via Little's Law, add headroom and replication factors, and produce a capacity plan that survives one AZ failure.
- Identify when estimation must defer to measurement — tail latency, GC pauses, and cache hit ratios cannot be estimated from first principles — and propose the load test that would close the gap.

Why estimation comes before architecture

A common failure mode in design discussions is to start with components: "We need Kafka, Cassandra, and Kubernetes." Without numbers, component choice is aesthetics. With numbers, it becomes constrained engineering:

- At 100 writes/sec, a single Postgres instance is the simplest correct answer. At 500,000 writes/sec, it is not an answer at all — you need partitioning or a log-structured store before the first line of code.
- At 10 GB of data, indexes fit in RAM on any machine. At 10 TB with random access, working-set size and IOPS dominate every decision.
- At 1,000 QPS with $p50 = 50$ ms, a single AZ of 8 app servers has headroom. At 80,000 QPS, you are reasoning about load balancer tiers, connection pooling, and whether your service discovery can handle the churn.

Estimation prunes the design space early. It also forces explicit assumptions — "we assume 20% DAU/MAU, average 3 requests per session, 400-byte average payload" — that reviewers can challenge. An estimate with stated assumptions is useful even when wrong; a design without estimates is unfalsifiable.

Boundary note. This chapter covers *pre-build* estimation and provisioning math — the whiteboard and spreadsheet phase. Continuous capacity management in production (autoscaling signals, burn-rate alerting, load testing at scale) is in Vol 11, Chapter 7 — Load Testing and Capacity Planning, and cloud cost and quota management is in Vol 12, Chapters 8–9. Algorithmic analysis of why certain data structures and rate limiters scale the way they do is in Vol 14. This chapter gives you the numbers to *choose* an architecture; those volumes show how to *operate* it.

The numbers every backend engineer should know

Memorize these to one significant figure. Precise values change yearly; orders of magnitude do not.

Latency

| Operation | Latency | Notes |
|--|-------------------|--|
| L1 cache reference | ~1 ns | Per core, no coherence miss |
| L2 cache reference | ~4 ns | |
| Main memory reference | ~100 ns | Random access, TLB hit |
| SSD random read (NVMe) | ~80-150 μ s | 4 KB, queue depth 1 |
| SSD sequential read | ~3-7 GB/s | PCIe 4.0 $\times$ 4 NVMe |
| HDD seek | ~5-10 ms | Mechanical; why HDDs are archival |
| Send 1 KB over 1 Gbps network | ~10 μ s | Serialization + wire; real RTT dominates |
| Round-trip within one AZ | ~0.3-0.6 ms | AWS us-east-1a intra-AZ p50 |
| Round-trip cross-AZ (same region) | ~0.8-2 ms | Depends on region and fiber path |
| Round-trip cross-region (US East $\rightarrow$ West) | ~60-75 ms | Speed of light in fiber |
| Round-trip US $\rightarrow$ Europe | ~90-120 ms | |
| Round-trip US $\rightarrow$ Asia | ~150-200 ms | |
| TLS 1.3 handshake (full, no resumption) | ~1-2 RTT + crypto | ~100-250 ms cross-continent |
| DNS lookup (cached at resolver) | ~1-10 ms | Uncached + recursive: 50-150 ms |

The cross-region numbers deserve emphasis: you cannot beat the speed of light. A system that requires a synchronous cross-region round-trip on

every user request has a hard floor of 60–150 ms before any application logic runs. Multi-region designs (Chapter 10) exist largely to avoid paying that tax on the hot path.

Throughput and bandwidth

| Resource | Throughput | Notes |
|----------------------------------|--|--|
| Single CPU core | ~3–4 GHz, ~10–30 GB/s memory bandwidth | Per core; Vol 1 for NUMA effects |
| 1 Gbps NIC | ~120 MB/s | Usable ~100 MB/s after overhead |
| 10 Gbps NIC | ~1.2 GB/s | Standard for cloud instances ≥ 4 vCPU |
| 25/100 Gbps NIC | ~3 / 12 GB/s | Common for storage-optimized instances |
| EBS gp3 (per volume) | 125–1000 MB/s, 3k–16k IOPS | Baseline 3k IOPS + burst |
| S3 single-object GET | ~50–100 MB/s per connection | Scales with parallel connections |
| Single Postgres instance (tuned) | ~5k–30k TPS (small rows) | Highly workload-dependent |
| Redis single shard | ~80k–120k ops/s | In-memory, pipelined |
| Kafka single broker | ~50–100 MB/s sustained write | Sequential I/O, zero-copy |
| Kafka single partition | ~10–30 MB/s | Limited by leader follower replication |

Powers of ten (decimal approximations — sufficient for estimation)

1 thousand = 10^3 $\approx$ 2^{10} 1 KB $\approx$ 10^3 bytes
1 million = 10^6 $\approx$ 2^{20} 1 MB $\approx$ 10^6 bytes
1 billion = 10^9 $\approx$ 2^{30} 1 GB $\approx$ 10^9 bytes
1 trillion = 10^{12} $\approx$ 2^{40} 1 TB $\approx$ 10^{12} bytes
1 quadrillion = 10^{15} $\approx$ 2^{50} 1 PB $\approx$ 10^{15} bytes

Time:
86,400 sec/day $\approx$ 10^5 sec/day (use 10^5 for mental math; real 86.4k)
30 days $\approx$ 2.6M sec $\approx$ 2.5 $\times$ 10^6
1 year $\approx$ 3.15 $\times$ 10^7 sec $\approx$ π $\times$ 10^7 (handy for annual projections)

Request math:
1M DAU $\times$ 10 req/user/day $\approx$ 10M req/day $\approx$ 115 req/s average
Peak is typically 3-10 $\times$ average (diurnal, flash crowd)
p99 capacity $\approx$ 3-5 $\times$ observed p50 for burstable services

The single most useful approximation for whiteboards: **86,400 $\approx$ 10^5** . It makes daily $\leftrightarrow$ per-second conversions trivial and the 15% error is smaller than your assumption uncertainty.

The four quantities

Every estimation decomposes into four quantities. Compute them in order — each constrains the next.

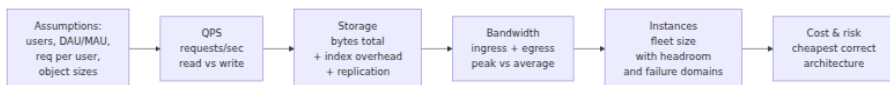


Figure 2-1: Estimation flow. Assumptions are explicit inputs; each quantity is derived, not guessed. Changing an assumption ripples forward automatically.

1. QPS (queries per second)

QPS = (DAU $\times$ avg requests per user per day) / 86,400
Peak QPS $\approx$ 3-10 $\times$ average QPS
Write QPS vs Read QPS $\approx$ read:write ratio drives store choice

Example: "30M MAU, 20% DAU, each active user makes ~15 API calls/day, 90% reads":

```
DAU = 30M × 0.20 = 6M
Total req/day = 6M × 15 = 90M
Average QPS = 90M / 86,400 ≈ 1,040
Peak QPS ≈ 5 × 1,040 ≈ 5,200
Read QPS ≈ 4,680    Write QPS ≈ 520
```

2. Storage

```
Total bytes = (objects per day × avg object size × retention days) ×
replication factor
Index overhead ≈ 10–30% for B-tree indexes; ~0 for pure log stores
```

Always state retention and replication explicitly. "We store 500 bytes of metadata per object for 3 years, replicated 3×" is verifiable; "we need a lot of storage" is not.

3. Bandwidth

```
Ingress (write) bandwidth = Write QPS × avg write payload
Egress (read) bandwidth = Read QPS × avg read payload
CDN/edge egress often dominates factor in cache hit ratio
```

Bandwidth determines NIC provisioning and, in the cloud, a surprising fraction of cost. Egress pricing (\$0.05–0.09/GB on AWS/GCP) makes a 10 Gbps sustained egress stream cost tens of thousands per month before any compute.

4. Instance count

```
Instances = ceil( Peak QPS × p99_latency / concurrency_per_instance )
Then add: headroom (30–40%), failure domain (N+1 AZ), replication
factor
```

This is Little's Law from Chapter 1, now used as a provisioning formula. `concurrency_per_instance` is threads, connections, or event-loop slots — whichever limits the runtime.

Five canonical templates

Template A — Read-heavy feed (Twitter/X home timeline, Instagram feed)

Assumptions:

MAU 300M, DAU 60M (20%), each user follows ~200 accounts
Each user reads feed 20×/day, posts 0.5×/day (read:write ≈ 40:1)
Timeline = 50 posts, each post metadata ~400 bytes + media pointers
Fan-out: naive push to all followers is infeasible at scale

| Quantity | Calculation | Result |
|----------------------------|--|-----------------------------------|
| Read QPS | $60M \times 20 / 86,400$ | ~13,900 avg, ~70k peak |
| Write QPS | $60M \times 0.5 / 86,400$ | ~350 avg, ~1,800 peak |
| Storage (new posts) | $30M \text{ posts/day} \times 400 \text{ B} \times 365 \text{ days}$ | ~4.4 TB/year (metadata only) |
| Media storage | $30M \times 40\% \text{ with media} \times 2 \text{ MB avg}$ | ~24 TB/day → ~8.8 PB/year |
| Bandwidth (timeline reads) | $13.9k \times 50 \times 400 \text{ B}$ | ~278 MB/s avg egress for metadata |

Architecture implication: the bottleneck is not write throughput but *fan-out*. Pushing each tweet to 200 followers' precomputed timelines costs 200 writes per post; at 350 posts/sec that is 70k timeline writes/sec. The standard answer is hybrid fan-out — push for normal users, pull (read-time merge) for celebrities with millions of followers — covered in Chapter 12's case study.

Template B — Write-heavy ingest (metrics, logs, IoT telemetry)

Assumptions:

10M devices, each sends 1 reading/sec, each reading 200 bytes
Retention: 30 days hot, 1 year cold
Read pattern: recent data (last hour) queried 100x/sec, historical rarely

| Quantity | Calculation | Result |
|--|--|-----------------------|
| Ingest QPS | $10\text{M} \times 1$ | 10M writes/sec |
| Ingress bandwidth | $10\text{M} \times 200\text{ B}$ | 2 GB/s |
| Storage (30 days) | $10\text{M} \times 200\text{ B} \times 86,400 \times 30$ | ~5.2 PB |
| Storage (1 year, cold with 5× compression) | $5.2\text{ PB} \times 12 / 5$ | ~12.5 PB |

Architecture implication: 10M writes/sec rules out any row-store OLTP database. The answer is a log-structured path — Kafka/Kinesis for ingest buffering (sequential I/O, ~100 MB/s per broker → ~20 brokers for 2 GB/s with replication), then tiered storage (Kafka → ClickHouse/TimescaleDB hot → S3/Parquet cold). See Vol 5, Chapter 2 for LSM-tree write amplification and Vol 10, Chapter 3 for Kafka sizing.

Template C — Search (full-text over 1B documents)

Assumptions:

1B documents, avg 5 KB text, inverted index 30% of corpus
 Query: 20k QPS, avg 2 terms, p99 latency SLO 200 ms
 Index updates: 5k docs/sec

| Quantity | Calculation | Result |
|---------------------|---|--------------------------------------|
| Corpus | $1\text{B} \times 5\text{ KB}$ | 5 TB raw |
| Index | $5\text{ TB} \times 30\%$ | ~1.5 TB (plus stored fields → ~3 TB) |
| Query QPS | 20k (given) | 20k |
| Bandwidth (queries) | $20\text{k} \times 1\text{ KB req} + 20\text{ KB resp}$ | ~420 MB/s egress |

Architecture implication: inverted indexes must be sharded (by document hash or term) and replicated. Each shard holds a Lucene/Elasticsearch segment; query fan-out across shards and merge is the latency-critical path. 20k QPS at 200 ms p99 with, say, 50 shards per query means each

shard must respond in single-digit milliseconds — hence extensive caching of posting lists and filter bitsets (Chapter 3).

Template D — Chat (WhatsApp/WeChat-style messaging)

Assumptions:

500M MAU, 100M DAU, avg user sends 20 messages/day, avg message 200 bytes
Group chats: 15% of messages, avg group size 12
Presence: each user has 50 contacts whose online status is tracked

| Quantity | Calculation | Result |
|----------------------------|---|--|
| Message QPS | $100\text{M} \times 20 / 86,400$ | ~23k avg, ~120k peak |
| With group fan-out | $23\text{k} \times (0.85 \times 1 + 0.15 \times 12)$ | ~60k deliveries/sec |
| Storage (messages, 1 year) | $2\text{B msgs/day} \times 200\text{ B} \times 365$ | ~146 TB/year |
| Bandwidth (deliveries) | $60\text{k} \times 200\text{ B}$ | ~12 MB/s (metadata only; media separate) |
| Presence QPS | $100\text{M online} \times \text{heartbeat}/30\text{s}$ | ~3.3M presence updates/sec |

Architecture implication: the hard part is *presence* and *delivery guarantees*. 3.3M presence updates/sec is higher than message throughput and requires an in-memory system (Redis, custom) with careful fan-out. Message delivery needs at-least-once with idempotent dedup on the client, and offline storage for disconnected recipients (mailbox per user). WebSocket connection count — 100M concurrent — dominates instance sizing (see below).

Template E — Blob / CDN delivery (YouTube thumbnails, image CDN)

Assumptions:

100M objects, avg object 800 KB, accessed 50x/day avg, Zipfian (top 10% gets 80% of reads)
 Origin → edge cache hit ratio 85%, edge → client hit ratio 90% (browser cache)

| Quantity | Calculation | Result |
|-------------------------|---|-----------------------------------|
| Total reads/day | $100\text{M} \times 50$ | 5B reads/day → ~58k reads/sec avg |
| Origin reads (15% miss) | $58\text{k} \times 15\%$ | ~8.7k origin fetches/sec |
| Origin egress | $8.7\text{k} \times 800\text{ KB}$ | ~7 GB/s |
| Edge egress | $58\text{k} \times 800\text{ KB}$ | ~46 GB/s |
| Storage (origin) | $100\text{M} \times 800\text{ KB} \times 3\times$ replication | ~240 TB |

Architecture implication: egress dominates cost. At \$0.08/GB, 46 GB/s = ~4 TB/s-hour? Let's compute properly:

46 GB/s x 86,400 s/day = 3.97 PB/day egress
 3.97 PB x \$0.08/GB ≈ \$317k/day without CDN

With a CDN (CloudFront/Fastly) caching 85%+ at the edge, origin egress drops to ~7 GB/s and CDN egress pricing (~\$0.02/GB blended) roughly halves the bill. The CDN is the architecture for this workload.

From estimation to fleet size

Estimation gives QPS, storage, and bandwidth. Capacity planning turns those into machines, with failure domains and headroom.

Step 1 — Per-instance concurrency

Determine how many concurrent requests one instance can handle. For thread-per-request runtimes (JVM, Python WSGI):

```
concurrency_per_instance = thread_pool_size
e.g., Tomcat maxThreads=200, pgbouncer pool=100
```

For event-loop runtimes (Go goroutines, Node.js, Netty):

```
concurrency_per_instance = limited by CPU or downstream connections,  
not threads  
e.g., Go service: ~5k  10k concurrent goroutines before scheduler pressure
```

For WebSocket / long-lived connections:

```
concurrency_per_instance = max_connections (file descriptors + memory  
per conn)  
e.g., 50k–100k conns per 8 GB instance with tuned ulimits (see Vol 2,  
Ch 7)
```

Check the real limit with a load test — do not trust defaults.

Step 2 — Apply Little's Law

```
instances_needed = ceil( peak_QPS × p99_latency /  
concurrency_per_instance )
```

Worked example — API tier:

```
Peak QPS: 20,000  
p99 latency (downstream included): 120 ms = 0.12 s  
Concurrency per instance (Tomcat, 200 threads, 70% usable to avoid  
queueing): 140  
  
instances_needed = 20,000 × 0.12 / 140 ≈ 17.1 → 18 instances
```

Step 3 — Add headroom, AZ redundancy, and replication

```
instances_with_headroom = ceil(18 / 0.65) ≈ 28    (target 65%  
utilization)  
instances_across_AZs    = ceil(28 / 2) × 3        (spread across 3 AZs,  
survive 1 AZ loss)      = 14 × 3 = 42            (but 28 active +  
spares; see below)
```

The precise AZ math depends on whether you require N+1 redundancy (survive one AZ failure without degradation) or merely survival (degraded but up). For N+1 across 3 AZs with 28 needed:

```
instances_per_AZ = ceil(28 / 2) = 14    (any 2 AZs must handle full
load)
total              = 14 x 3 = 42
active under normal = 28, spare capacity = 14 (one AZ worth)
```

For databases, multiply by replication factor (3× for quorum, 2× for primary-standby) and add backup storage.

Step 4 — Verify bandwidth and storage per instance

```
Per-instance egress = total_egress / instances
Per-instance IOPS   = total_IOPS / shards
Check: does per-instance demand exceed NIC, disk, or memory?
```

If it does, the tier needs sharding or a different instance family, not just more replicas.

Peak QPS + p99 latency

Little's Law:
$$\text{instances} = \text{QPS} \times W / \text{concurrency}$$

Headroom:
 $\div 0.65$ for 65% target

AZ redundancy:
 $\div 2 \times 3$ for 3-AZ N+1

Replication factor:
 $\times 3$ for quorum stores

Per-instance
bandwidth / IOPS /
memory OK?

Figure 2-2: Fleet sizing pipeline. Each step is multiplicative; skipping AZ redundancy or headroom is the most common capacity error in practice.

A complete worked example — URL shortener (interview-sized)

Prompt: "Design a URL shortener like TinyURL. 100M new URLs per month, 10B redirects per month, 500-byte average URL record, 5-year retention."

Assumptions

New URLs: 100M/month $\approx$ 3.3M/day $\approx$ 38 writes/sec avg, ~200 peak
Redirects: 10B/month $\approx$ 333M/day $\approx$ 3,850 reads/sec avg, ~20k peak
Read:write $\approx$ 100:1 $\square$ classic read-heavy, cache-friendly

Storage

New records: 100M $\times$ 500 B = 50 GB/month $\rightarrow$ 600 GB/year $\rightarrow$ 3 TB / 5 years
With 3 $\times$ replication (primary + 2 replicas or Dynamo-style): ~9 TB
Index on short code (8-byte hash): negligible vs data

Bandwidth

Write: 200 $\times$ 500 B $\approx$ 100 KB/s $\square$ trivial
Read: 20k $\times$ 500 B (redirect metadata) + HTTP 301 $\approx$ 10 MB/s avg, ~50 MB/s peak
With 80% cache hit ratio (top URLs are hot): origin read $\approx$ 4k/sec

Instances

App tier: 20k peak $\times$ 0.05 s (p99 with cache) / 140 concurrency $\approx$ 8 $\rightarrow$ 12 with headroom $\rightarrow$ 18 across 3 AZs
Cache: Redis cluster, 3 shards $\times$ 2 replicas, ~2 GB per shard for hot set (see Ch 3)
DB: Postgres with read replicas or DynamoDB on-demand; single primary handles 200 writes/sec easily

Cost sanity check (AWS us-east-1, 2024 pricing)

App: 18 $\times$ m7g.large (\$0.07/hr) $\approx$ \$900/month
Cache: 6 $\times$ cache.r7g.large (\$0.19/hr) $\approx$ \$820/month
DB: db.r7g.large Multi-AZ (\$0.29/hr) + storage $\approx$ \$260/month
Bandwidth: 10 MB/s $\times$ 2.6M sec/month $\approx$ 26 TB/month egress $\times$ \$0.09 $\approx$ \$2,340/month
 $\rightarrow$ Bandwidth dominates, as usual for read-heavy services

This is the level of detail an interviewer expects in 6–8 minutes: one line per quantity, explicit assumptions, a conclusion about which quantity drives the architecture (here: read cacheability and egress cost).

Latency budgeting

Every user-facing SLO implies a latency budget that must be *allocated* across tiers. If the SLO is p99 < 500 ms for a checkout flow that touches API → inventory → payment → queue, you cannot give each tier 400 ms.

```
SLO: p99 < 500 ms end-to-end
Budget allocation:
  Edge / CDN:          20 ms (TLS, Anycast)
  Load balancer:       5 ms
  API gateway + auth:   30 ms (JWT validation, rate limiting – Ch 8,
9)
  App logic:           80 ms
  Cache lookups (2×):   10 ms (2 × 5 ms Redis p99)
  Primary DB (1 query): 40 ms (p99 for indexed lookup)
  Payment RPC:         200 ms (external, with timeout + circuit break
er)
  Queue enqueue:       15 ms
  Margin:              100 ms
  _____
  Total:              500 ms
```

If payment's p99 is actually 350 ms under load, the budget is blown before any other tier misbehaves. The fix is either a higher SLO, a faster payment integration, or making payment asynchronous (Chapter 1, Principle 4) — not optimizing the cache from 5 ms to 3 ms.

```
# Measure p50/p90/p99 from access logs (OpenResty / nginx, 2024 format)
$ awk '{print $NF}' /var/log/nginx/access.log | sort -n | \
  awk '{a[NR]=$1} END {
    printf "p50  %.1f ms\n", a[int(NR*0.50)]*1000
    printf "p90  %.1f ms\n", a[int(NR*0.90)]*1000
    printf "p99  %.1f ms\n", a[int(NR*0.99)]*1000
    printf "p99.9 %.1f ms\n", a[int(NR*0.999)]*1000
  }'
p50  18.3 ms
p90  47.1 ms
p99  142.8 ms
p99.9 612.4 ms

# Prometheus histogram quantile – the production source of truth
(PromQL, Prometheus 2.50+)
# histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket[5m])))
```

Tail amplification matters: a request that fans out to 10 shards and waits for all of them has p99 ≈ (per-shard p99) amplified. If each shard has p99

= 20 ms and p90 = 8 ms, the fan-out p99 is not 20 ms — roughly, $1 - (1 - 0.01)^{10} \approx 9.6\%$ of fan-out requests hit at least one slow shard. This is why tail-at-scale techniques (hedged requests, tied requests) exist — Vol 3, Chapter 11.

What estimation cannot do

Estimation is powerful and limited. It cannot predict:

- **Tail latency** — GC pauses (Vol 13), lock contention (Vol 4), and noisy neighbors make p99 5–50× p50 in ways no whiteboard captures.
- **Cache hit ratios** — depend on access skew (Zipfian α), object size distribution, and eviction policy. A 90% hit ratio assumption is a guess until measured from production traces.
- **Database performance under contention** — B-tree page splits, WAL fsync stalls, and replication lag spikes are workload-specific.
- **Human factors** — deploy frequency, on-call load, and operational complexity do not appear in QPS math but dominate total cost of ownership.

The correct response is not to avoid estimation but to *bound* it and then measure:

```

# k6 load test – validate the app tier estimate (k6 v0.49, 2024)
$ cat load.js
import http from 'k6/http';
import { check } from 'k6';
export const options = {
  stages: [
    { duration: '2m', target: 5000 }, // ramp to expected peak
    { duration: '5m', target: 20000 }, // sustain peak
    { duration: '2m', target: 30000 }, // burst beyond peak – find
the knee
  ],
  thresholds: {
    http_req_duration: ['p(99)<200'], // SLO
    http_req_failed: ['rate<0.01'],
  },
};
export default function () {
  const r = http.get('https://api.staging.internal/v1/feed?
user_id=42');
  check(r, { 'status 200': (x) => x.status === 200 });
}

$ k6 run load.js
...
http_req_duration.....: avg=47ms p90=82ms p99=188ms
max=1.2s
http_reqs.....: 1847234 3078/s
iteration_duration.....: avg=58ms
x p(99)<200 – p99=188ms (pass, but near threshold)

# vegeta – quick validation of a single endpoint (vegeta 12.8)
$ echo "GET https://api.staging.internal/healthz" | vegeta attack
-rate=20000 -duration=60s | vegeta report
Requests [total, rate, throughput] 1200000, 20000, 19982
Duration [total, attack, wait] 60s, 60s, 12ms
Latencies [mean, 50, 95, 99, max] 11ms, 9ms, 18ms, 34ms, 210ms
Success [ratio] 99.97%
Status Codes [code:count] 200:1199640 503:360

```

The load test found the knee: at 20k RPS, p99 is 188 ms (healthy); at 30k it will likely breach. That validates headroom and tells you where to set the HPA threshold — information no whiteboard can provide.

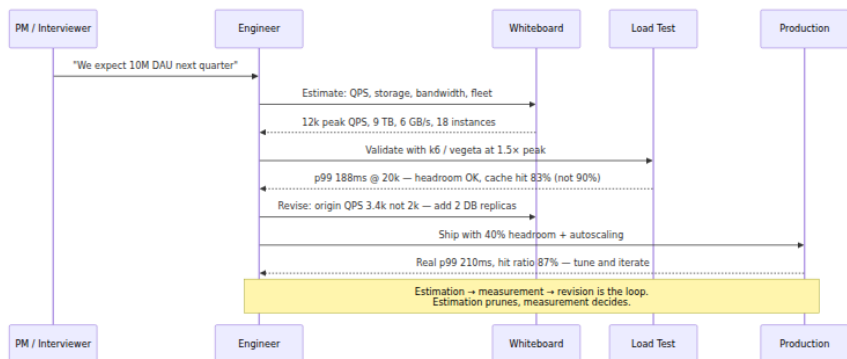


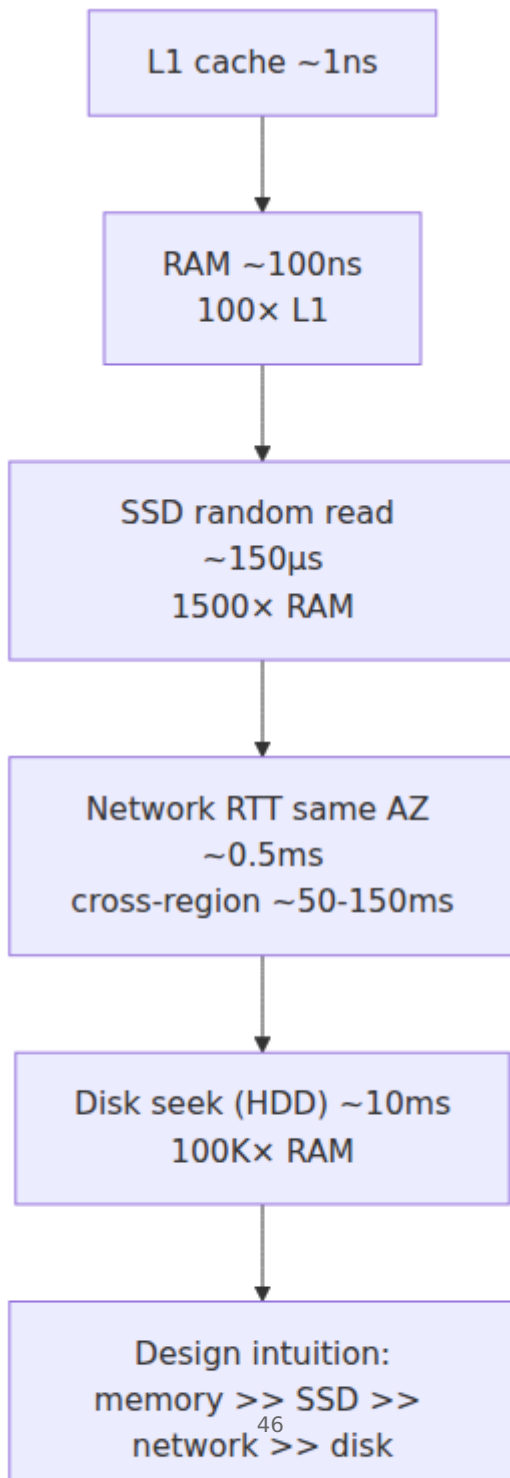
Figure 2-3: The estimation → validation → revision loop. Whiteboard numbers are hypotheses; load tests and production metrics are evidence.

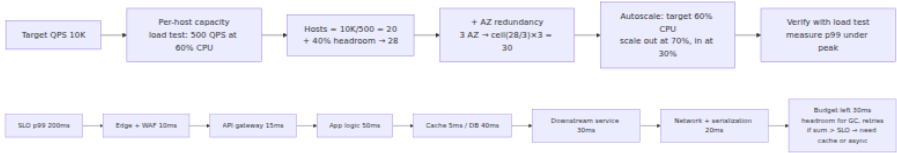
Key takeaways

- Estimation is the first design activity, not an afterthought. Without QPS, storage, bandwidth, and instance counts, component choices are unconstrained and architecture reviews become opinion contests.
- Memorize latency and throughput orders of magnitude — AZ round-trips, disk and network bandwidth, per-instance throughput — to one significant figure. They are the constants in every whiteboard equation.
- Every prompt decomposes into four quantities in order: QPS (from DAU × per-user rate), storage (objects × size × retention × replication), bandwidth (QPS × payload, with cache hit ratios), and instance count (Little's Law plus headroom and AZ redundancy). State assumptions explicitly so they can be challenged.
- Five templates — read-heavy feed, write-heavy ingest, search, chat, and blob/CDN — cover most interview and real-world workloads. Adapt them by swapping assumptions, not by inventing new frameworks.
- Fleet sizing is Little's Law (instances = QPS × latency / concurrency) multiplied by headroom (÷ 0.65), AZ redundancy (× 3/2 for 3-AZ N+1), and replication factor. Verify per-instance bandwidth and IOPS — if one dimension exceeds instance capacity, shard or choose a different family.

- Latency budgets compose: allocate an SLO across tiers and enforce timeout budgets that sum to less than the SLO. Fan-out amplifies tails — a request waiting for 10 shards has $\sim 10\times$ the chance of hitting a slow shard.
- Estimation cannot predict tail latency, hit ratios, or contention effects. Use it to prune the design space and produce a load-test plan, then let measurement close the gap. The loop is estimate $\rightarrow$ load test $\rightarrow$ revise $\rightarrow$ ship with headroom $\rightarrow$ observe $\rightarrow$ iterate.







Further reading

- Dean, J. "Designs, Lessons and Advice from Building Large Distributed Systems" (2009) — the original "numbers every engineer should know" talk and paper; latency and throughput constants with Google's production context. <https://research.google/pubs/designs-lessons-and-advice-from-building-large-distributed-systems/>
- Barroso, L. A., Clidaras, J., and Hölzle, U. *The Datacenter as a Computer* 3rd ed. (Morgan & Claypool, 2018), Chapter 1 — cost and provisioning models for warehouse-scale machines; how to reason about machine, power, and network as a unified resource. <https://www.datacenterasacputer.com/>
- Kleppmann, M. *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapters 1-2 — data systems framing of load, storage, and reliability numbers. <https://dataintensive.net/>
- AWS Builders' Library — "Ensuring Rollback Safety During Deployments" and "Avoiding Overload in Distributed Systems by Putting the Smaller Service in Control" — short, quantitative pieces on capacity and backpressure from Amazon's experience. <https://aws.amazon.com/builders-library/>
- Beyer, B., Jones, C., Petoff, J., Murphy, N. *Site Reliability Engineering* (O'Reilly, 2016), Chapters 25-27 — capacity planning as practiced at Google; demand forecasting, resource classes, and the role of headroom. <https://sre.google/sre-book/table-of-contents/>
- Gregg, B. *Systems Performance* 2nd ed. (Pearson, 2021), Chapters 2, 6 — USE and RED methods for connecting utilization, saturation, and errors to capacity decisions. <https://www.brendangregg.com/systems-performance-2nd-edition-book.html>
- Hellerstein, J. et al. "Serverless Capacity Planning" and related work on right-sizing — modern treatment of provisioning for autoscaled and serverless workloads where instance count is not static. <https://www.vldb.org/pvldb/vol13/p633-hellerstein.pdf>

- k6 and vegeta documentation — practical load-testing tools referenced in this chapter; their threshold and reporting models are worth studying before you need them under pressure. <https://k6.io/docs/> <https://github.com/tsenart/vegeta>
-

Next: Chapter 3 — Caching Strategies at Scale — takes the single most effective lever for read-heavy systems and shows how to use it without creating a consistency nightmare: eviction policies, write strategies, thundering herds, and distributed cache topologies.

Chapter 3 — Caching Strategies at Scale

What this chapter covers. Caching is the closest thing distributed systems has to a free lunch — and the most reliable source of subtle outages. For skewed workloads ($\alpha \approx 0.9$ – 1.0 — see Zipf analysis below), a well-placed cache can absorb roughly 80–90% of read traffic, cut p99 latency by an order of magnitude, and reduce database cost by more than half — but all three outcomes are workload-dependent, not universal. A poorly managed one can serve stale prices during a flash sale, amplify a cold start into a cascading failure, or silently corrupt data when an eviction policy interacts badly with access skew. This chapter builds caching from first principles: why it works (locality), where it lives (every layer from CPU to CDN), and the four decisions that define any caching strategy — what to cache, when to populate, when to invalidate, and what to do on a miss. We then cover eviction policies, write strategies, thundering-herd defenses, and distributed cache topologies (sidecar, external cluster, and CDN/edge), with failure modes and operational guidance for each. The chapter closes with cache sizing, hit-ratio math, and a checklist for cache reviews.

Learning goals — after this chapter you should be able to:

- Explain why caching works in terms of temporal and spatial locality and Zipfian access skew, and estimate hit ratios from access distributions.

- Name the caching layers in a modern backend stack — client, CDN/edge, load balancer, application, and database — and decide which layer to cache at for a given workload.
 - Choose among cache-aside, read-through, write-through, and write-behind strategies, and state the consistency and failure semantics of each.
 - Compare eviction policies (LRU, LFU, TinyLFU/W-TinyLFU, FIFO, random) and explain why modern caches use admission filters and segmented LRU rather than pure LRU.
 - Design defenses against thundering herds, cache stampedes, and cold starts using single-flight, probabilistic early expiration, and warming strategies.
 - Operate a distributed cache — Redis Cluster and Memcached — including consistent hashing, replication, persistence choices, and failure handling.
 - Size a cache tier from working-set estimates and explain why hit ratio, not cache size, is the metric that drives cost and latency.
-

Why caching works: locality and skew

Caching exploits two empirical properties of real workloads:

- **Temporal locality** — recently accessed items are likely to be accessed again soon. A user who viewed a product will view it again; a service that read a feature flag will read it again on the next request.
- **Spatial locality** — items near recently accessed items are likely to be accessed soon. Less relevant for KV caches than for CPU caches, but visible in range scans and prefix queries.

Together they produce **skew**: a small fraction of keys accounts for most accesses. Formally, many workloads follow a Zipfian distribution where the k -th most popular item's frequency is proportional to $1/k^\alpha$. For $\alpha \approx 0.9$ – 1.0 (common in web workloads), the top 10% of keys serves ~ 70 – 80% of requests. That is why a cache holding 10% of the dataset can achieve 80% hit ratio — and why hit ratio is non-linear in cache size.

Requests

```

^
| *
| **
| ***
| ****
| *****
| ****
| *****
| *****
| *****
+-----> Key rank (most  least popular)
|<-- 10% of keys -->|<-- 90% of keys -->|
|<-- 80% of reqs -->|<-- 20% of reqs -->|
      Hot set                Long tail

```

If accesses were uniform (every key equally likely), caching would be pointless — hit ratio would equal cache-size / dataset-size. Skew is what makes caching effective, and measuring your workload's skew is the first step in any caching decision.

```

# hit_ratio.py - estimate hit ratio from a Zipfian access trace (Python
3.11)
import random, collections

def zipf_keys(n_keys: int, alpha: float = 1.0) -> list[int]:
    weights = [1.0 / (k ** alpha) for k in range(1, n_keys + 1)]
    total = sum(weights)
    probs = [w / total for w in weights]
    # sample 1M requests
    return random.choices(range(n_keys), weights=probs, k=1_000_000)

def hit_ratio(trace: list[int], cache_size: int) -> float:
    # optimal offline: cache the hottest keys (upper bound for any
    policy)
    freq = collections.Counter(trace)
    hottest = {k for k, _ in freq.most_common(cache_size)}
    hits = sum(1 for k in trace if k in hottest)
    return hits / len(trace)

for cs in [100, 1000, 10000]:
    for alpha in [0.8, 1.0, 1.2]:
        trace = zipf_keys(100_000, alpha=alpha)
        print(f"cache={cs:5d} alpha={alpha:.1f}")
hit={hit_ratio(trace, cs):.1%})

```

```

cache= 100  alpha=0.8  hit=28.4%
cache= 100  alpha=1.0  hit=41.2%
cache= 1000 alpha=1.0  hit=58.7%
cache=10000 alpha=1.0  hit=74.3%
cache= 100  alpha=1.2  hit=56.1%
cache= 1000 alpha=1.2  hit=71.8%

```

Higher α (more skew) means a smaller cache achieves the same hit ratio. Measure α from production access logs before sizing — an assumption of $\alpha = 1.0$ when reality is 0.7 can halve your expected hit ratio.

Boundary note. CPU and OS page caches (Vol 1, Chapter 3; Vol 2, Chapters 4, 6) exploit the same locality principles in hardware and the kernel. CDN and edge caching overlap with Vol 3, Chapter 10 — Proxies, Service Mesh, and CDNs, which covers anycast, cache hierarchies, and invalidation at the network edge. This chapter focuses on *application and data* caching — the tier the backend team owns and operates.

Where caches live

A request traverses multiple caching layers. Each has different scope, coherence cost, and blast radius:

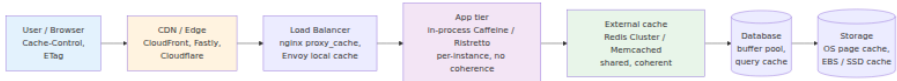


Figure 3-1: Caching layers from client to storage. Lower layers are larger and more shared; upper layers are smaller and closer to the user. Most latency wins come from the top; most hit-ratio wins come from the middle.

| Layer | Scope | Typical hit ratio | Invalidation cost |
|--------------------|-------------------|-------------------------------------|--------------------------------------|
| Browser / client | Per user | High for static assets, low for API | Cache-Control + ETag / If-None-Match |
| CDN / edge | Global, per PoP | 80-95% for cacheable content | Purge API (seconds-minutes) or TTL |
| LB / reverse proxy | Per PoP or per LB | 30-60% for API responses | Config reload or TTL |

| Layer | Scope | Typical hit ratio | Invalidation cost |
|-----------------------|---------------------|--|-------------------------------------|
| In-process (Caffeine) | Per instance | 20-50% (small, no coherence) | Local only — no coordination needed |
| External (Redis) | Shared across fleet | 70-90% (the main lever) | Explicit delete or TTL |
| DB buffer pool | Per DB node | 90-99% for working set (if it fits in RAM) | Automatic on write |

The right layer depends on the invalidation story:

- **Immutable or slowly changing** (images, JS bundles, product catalog snapshots) → CDN with long TTL + content-addressed URLs (change the URL, not the content).
- **Per-user, read-heavy, staleness-tolerant** (feed, recommendations, profile) → external cache with TTL + explicit invalidation on write.
- **Latency-critical, tiny working set** (feature flags, config, authZ decisions) → in-process cache with short TTL (seconds) and background refresh, backed by external cache.

Caching at multiple layers is multiplicative. A request that hits the CDN never reaches the app tier; one that hits the app's in-process cache never reaches Redis; one that hits Redis never reaches the database. But each layer adds staleness risk — a CDN serving a 5-minute-old price while Redis holds a 30-second-old price can produce inconsistencies visible to the user.

The four decisions

Every caching strategy answers four questions. Get any one wrong and the cache becomes a liability.

1. What to cache

Not everything benefits. Cache when:

- Reads dominate writes (read:write > 10:1 is a good heuristic).
- Computation or I/O cost per miss is high (DB query, aggregation, external API).

- Access skew is significant (Zipfian $\alpha > 0.7$).

Do not cache:

- Rapidly changing data where staleness is unacceptable (real-time inventory counts during a flash sale — cache with 1-second TTL and accept brief oversell, or do not cache at all).
- Per-request unique data (search results with many distinct queries — hit ratio will be near zero).
- Large objects that evict many small hot objects (consider a separate cache tier or size-aware admission).

2. When to populate (read path)

Three read strategies dominate:

Cache-aside (lazy loading) — the application manages the cache explicitly. On a read, check the cache; on a miss, read the database, populate the cache, and return.

```
# cache_aside.py — the most common application-level pattern
import redis, json

r = redis.Redis(host="redis.internal", port=6379,
decode_responses=True)

def get_user(user_id: str) -> dict | None:
    key = f"user:{user_id}"
    cached = r.get(key)
    if cached is not None:
        return json.loads(cached)          # hit

    # miss — load from database
    row = db.query("SELECT id, name, email FROM users WHERE id = %s",
(user_id,))
    if row is None:
        # cache negative result to avoid DB hammer on missing keys
        # (with short TTL)
        r.setex(f"user:{user_id}:miss", 30, "1")
        return None

    r.setex(key, 300, json.dumps(row))      # populate with 5-min TTL
    return row
```

Pros: only requested data is cached (efficient for sparse access); cache failures do not block reads (fall through to DB). Cons: first request after eviction always misses; application code is cache-aware.

Read-through — the cache sits in front of the database and loads on miss transparently. The application talks only to the cache.

```
App → Cache → (miss) → DB → Cache → App
```

Common with sidecar caches (e.g., `mcrouter` for Memcached) or database-integrated caches. Simpler application code, but the cache becomes a critical path component — if it is down, reads fail unless the cache proxies through.

Refresh-ahead / background refresh — a background process reloads data before it expires, so foreground reads never block on a miss. Best for latency-sensitive paths where even a cache miss is too slow. Implemented via probabilistic early expiration (see below) or explicit refresh workers.

3. When to invalidate (write path)

The two hard problems in computer science are cache invalidation, naming things, and off-by-one errors. Invalidation strategies:

| Strategy | How it works | Staleness | Complexity |
|---------------------------|--|------------------------------|--|
| TTL expiration | Entry expires after fixed time | Bounded by TTL | Trivial; no coordination |
| Explicit invalidation | Writer deletes/updates cache entry on write | Minimal (if delete succeeds) | Must handle delete failures |
| Write-through | Every write goes to cache and DB synchronously | Zero (if both succeed) | Higher write latency; cache is on write path |
| Write-behind (write-back) | Write to cache, async flush to DB | Window of durability risk | Complex; needs WAL and recovery |

TTL is the default for most systems because it bounds staleness without coordination. The choice of TTL is a direct staleness-vs-hit-ratio trade-off:

```
Hit ratio  $\approx 1 - \text{TTL\_miss\_rate}$   
Effective staleness = TTL (worst case)  
Short TTL (30s): low staleness, lower hit ratio, more DB load  
Long TTL (5 min): higher staleness, higher hit ratio, less DB load
```

Explicit invalidation reduces staleness but introduces a failure mode: if the cache delete fails (network blip, Redis down), stale data persists until TTL expiry. The safe pattern combines both — explicit delete *plus* a TTL as a safety net:

```
# write path – explicit invalidation with TTL safety net  
def update_user(user_id: str, fields: dict) -> None:  
    db.execute("UPDATE users SET name=%s WHERE id=%s",  
        (fields["name"], user_id))  
    try:  
        r.delete(f"user:{user_id}") # explicit invalidation  
    except redis.ConnectionError:  
        # delete failed – stale data will persist until TTL expiry (5  
        min)  
  
    # options: enqueue retry, or rely on TTL, or use cache stampede guard  
    enqueue_cache_invalidation(f"user:{user_id}")  
    logger.warning("cache delete failed, enqueued retry", extra={"u  
ser_id": user_id})
```

Write-through and write-behind place the cache on the write path, which couples cache availability to write availability. They are appropriate when the cache *is* the primary store (e.g., Redis as a session store) but not when the database is authoritative.

4. What to do on a miss

The miss path is where outages happen. A cache miss is not just "read the DB" — at scale, it is "thousands of concurrent requests all miss simultaneously and hammer the DB."

Eviction policies

When the cache is full, which entry to evict determines hit ratio. The optimal offline policy (Belady's — evict the entry whose next use is farthest in the future) is unimplementable; real policies approximate it.

| Policy | What it evicts | Strength | Weakness |
|------------------------------------|------------------------------------|---|--|
| LRU (least recently used) | Oldest access | Simple, good for recency | Scan pollution: a one-time scan evicts the hot set |
| LFU (least frequently used) | Lowest frequency | Good for skew | Stale frequency: old hot keys never decay |
| FIFO | Oldest insertion | Trivial | Ignores access pattern entirely |
| Random | Random entry | Surprisingly competitive at large sizes | No adaptivity |
| TinyLFU / W-TinyLFU | Admission filter + segmented LRU | Near-optimal, scan-resistant | More complex |
| ARC / LIRS | Adaptive recency/frequency balance | Self-tuning | Complex to implement |

Modern caches do not use pure LRU. Two improvements matter:

Segmented LRU (as in Caffeine/Ristretto). The cache is split into an admission window (small, LRU) and a main space (larger, frequency-biased). New entries enter the window; only entries that prove themselves (accessed again while in the window) are promoted to the main space. This resists scan pollution — a burst of cold keys fills the window but does not evict the hot set.

TinyLFU admission filter. Before admitting a new entry, estimate whether it is worth caching by comparing its frequency to the eviction candidate's frequency. A Count-Min Sketch tracks approximate frequencies with minimal memory. If the new entry is less frequent than the candidate, it is not admitted — the cache is not polluted.


```

New entry → TinyLFU filter → Admit? → Window (LRU, 1% of cache)
                                   No → discard
                                   Yes → Window → promoted to Main on re-
access
                                   Main = Eden (binned LRU)
+ Main probation

```

Caffeine (Java, widely used as an in-process cache) implements W-TinyLFU. Its hit ratio is within a few percent of optimal for most workloads and it is the default choice for JVM in-process caching:

```

// Caffeine – W-TinyLFU in-process cache (Java 17, Caffeine 3.1)
import com.github.benmanes.caffeine.cache.Caffeine;
import com.github.benmanes.caffeine.cache.Cache;
import java.time.Duration;

Cache<String, User> cache = Caffeine.newBuilder()
    .maximumSize(10_000) // entry count, or
    maximumWeight with weigher
    .expireAfterWrite(Duration.ofMinutes(5)) // TTL
    .expireAfterAccess(Duration.ofMinutes(2)) // idle expiration –
    evict if not read
    .recordStats() // for hit ratio
    monitoring
    .build();

// usage – explicit loading (cache-aside style)
User user = cache.get(userId, key -> db.findUser(key));

// metrics
System.out.println(cache.stats());
// CacheStats{hitCount=847231, missCount=48210, hitRate=0.946,
evictionCount=12043}

```

Redis and Memcached use simpler policies by default (Redis: `allkeys-lru` with sampled approximation; Memcached: LRU per slab class) but support LFU modes:

```
# Redis – eviction policy (redis.conf, Redis 7.2)
maxmemory 4gb
maxmemory-policy allkeys-lru           # or allkeys-lfu, volatile-lru,
noeviction
maxmemory-samples 5
# LRU approximation: sample 5 keys, evict least recent
# LFU mode – better for skewed workloads, decay over time
maxmemory-policy allkeys-lfu
lfu-log-factor 10
lfu-decay-time 1                       # minutes until counter decay

# Memcached – slab-aware LRU (Memcached 1.6)
# -M: return error on OOM instead of evicting (for critical caches)
# -o modern: enables segmented LRU + better memory efficiency
$ memcached -m 4096 -c 8192 -o modern -M
```

Thundering herds, stampedes, and cold starts

Three related failure modes share a cause: many concurrent requests miss the same cache entry simultaneously and all hit the backing store.

Thundering herd — a popular key expires and thousands of requests miss at once. The database receives a spike equal to the request rate for that key, not the normal miss rate.

Cache stampede — a variant where the recomputation is expensive (aggregation, report generation) and concurrent recomputations waste resources and contend.

Cold start — the cache is empty (deploy, failover, restart) and every request misses until the cache warms. Equivalent to a herd across all keys.

Defense 1 — Single-flight (request coalescing)

Only one request fetches the data; others wait for the result.

```

# singleflight.py – coalesce concurrent fetches for the same key
import threading

class SingleFlight:
    def __init__(self):
        self._lock = threading.Lock()
        self._inflight: dict[str, threading.Event] = {}
        self._results: dict[str, object] = {}

    def do(self, key: str, fn):
        with self._lock:
            if key in self._inflight:
                event = self._inflight[key]
            else:
                event = threading.Event()
                self._inflight[key] = event
                # we are the leader – fetch outside the lock
                do_fetch = True
                event = event
                # use a flag to track leadership
                self._results[key + ":leader"] = True
                do_fetch_key = key
                # release lock before I/O
                self._lock.release()
                try:
                    result = fn()
                    with self._lock:
                        self._results[key] = result
                        event.set()
                        del self._inflight[key]
                    return result
                except Exception as e:
                    with self._lock:
                        self._results[key + ":err"] = e
                        event.set()
                        del self._inflight[key]
                    raise
            finally:
                if 'do fetch' not in dir():
                    pass
                # non-leader path needs re-lock
                do_fetch = False

        # non-leader: wait for leader's result
        # (simplified – real impl uses double-check + event wait
        # outside lock)
        self._lock.release()
        event.wait(timeout=5.0)
        with self._lock:
            if key + ":err" in self._results:

```

```
        raise self._results[key + ":err"]
    return self._results.get(key)
```

```
# Go's singleflight is the canonical implementation (golang.org/x/sync/
singleflight)
# Python equivalent above; in production use cachetools + threading or
asyncio equivalent
```

In Go, this is a one-liner with `singleflight.Group`:

```
// singleflight in Go (Go 1.21, golang.org/x/sync/singleflight)
import "golang.org/x/sync/singleflight"

var g singleflight.Group

func getUser(userID string) (*User, error) {
    v, err, _ := g.Do(userID, func() (interface{}, error) {
        // only one goroutine executes this per userID; others wait
        if cached := redis.Get("user:" + userID); cached != nil {
            return cached, nil
        }
        user, err := db.FindUser(userID)
        if err == nil {
            redis.SetEx("user:"+userID, 300, user)
        }
        return user, err
    })
    if err != nil {
        return nil, err
    }
    return v.(*User), nil
}
```

Defense 2 — Probabilistic early expiration (x-fetch / early recomputation)

Instead of letting a key expire and causing a miss spike, refresh it *before* expiry with probability that increases as expiry approaches. Commonly called "x-fetch" or "probabilistic early expiration":

```

# probabilistic early expiration (based on nginx proxy_cache_use_stale
+ background update)
import time, random, math

def get_with_early_refresh(key: str, ttl_s: int = 300, beta: float = 1.
0):
    entry = r.get(key)
    if entry is None:
        return fetch_and_cache(key, ttl_s)

    # entry stores (value, expiry_timestamp)
    value, expiry = deserialize(entry)
    now = time.time()

    # gap = how close to expiry; larger gap → higher refresh
    probability
    gap = now - (expiry - ttl_s) # time since birth
    # x-fetch formula: refresh if now - delta*beta*log(rand) >= expiry
    # simplified: probability increases as we approach expiry
    if now + beta * math.log(random.random()) >= expiry:
        # background refresh – serve stale value immediately, refresh
    async
        enqueue_background_refresh(key, ttl_s)
        # optionally add jitter to expiry to desynchronize concurrent
    refreshes
    return value

```

Simpler variant — add jitter to TTL so keys do not expire simultaneously:

```

# TTL jitter – desynchronize expirations
base_ttl = 300
jitter = random.uniform(-0.1, 0.1) * base_ttl # ±10%
r.setex(key, int(base_ttl + jitter), value)
# 1000 keys that would have expired at T=300 now expire across
T=270..330

```

Defense 3 — Warming and stale-while-revalidate

- **Warming on deploy:** before cutting traffic to new instances, pre-populate their in-process caches from the external cache or a snapshot. Kubernetes `postStart` hooks or init containers can do this.
- **Warming after failover:** keep a standby cache (replica) warm via replication, not empty.
- **Stale-while-revalidate (SWR):** serve stale data while refreshing in the background. HTTP `Cache-Control: stale-while-revalidate=60` and Redis `GET` + `async SET` implement this.

```
# HTTP stale-while-revalidate – the CDN and browser do this natively
```

```
HTTP/1.1 200 OK
```

```
Cache-Control: max-age=60, stale-while-revalidate=30
```

```
ETag: "abc123"
```

```
# Client: within 60s → fresh (no origin fetch)
```

```
# Client: 60–90s → serve stale immediately, revalidate in background
```

```
# Client: after 90s → must revalidate before serving
```

```
# nginx – stale-while-revalidate for API responses (nginx 1.25)
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:100m inactive=5m;
server {
    location /api/ {
        proxy_cache api;
        proxy_cache_valid 200 60s;
        proxy_cache_use_stale error timeout updating;
# serve stale on origin error
        proxy_cache_background_update on;
# revalidate in background
        proxy_cache_lock on;
# single-flight at nginx layer
        proxy_pass http://app_backend;
    }
}
```

Defense 4 — Circuit breaker on the miss path

If the database is overloaded, cache misses should not add more load. A circuit breaker on the DB path (Chapter 1, Principle 5) that fails fast when the DB is slow prevents the cache miss storm from becoming a DB outage.

Distributed cache topologies

Memcached — simple, sharded, no replication

Memcached is a shared-nothing, sharded cache. Clients hash keys to servers (consistent hashing via client library like `mcrouter` or `ketama`); each key lives on exactly one server. If a server fails, its keys are simply missed — no failover, no replication.

```

Client (ketama ring) → memcached-1 (33% of keys)
                     → memcached-2 (33% of keys)
                     → memcached-3 (33% of keys)
One server dies → 33% miss spike until reshaped

```

Pros:极简, predictable, scales linearly by adding nodes, no replication lag.
 Cons: no persistence, no replication, rehashing on membership change causes a miss storm (mitigated by consistent hashing but not eliminated).
 Best for: large, transient caches where a miss is cheap (HTML fragments, session-adjacent data).

```

# mcrouter – Meta's Memcached router with consistent hashing and
# failover (mcrouter 2024)
$ cat /etc/mcrouter/config.json
{
  "pools": {
    "cache": {
      "servers": ["memcached-1:11211", "memcached-2:11211", "memcached-
3:11211"],
      "hash": "ketama"
    }
  },
  "route": {
    "type": "OperationSelectorRoute",
    "operation": "get",
    "default_policy": "RouteToAll",
    "policies": {
      "get": { "type": "HashRoute", "hash_func": "Ch3", "children": "PoolRoute|cache" },
      "set": { "type": "AllSyncRoute", "children": "PoolRoute|cache" }
    }
  }
}

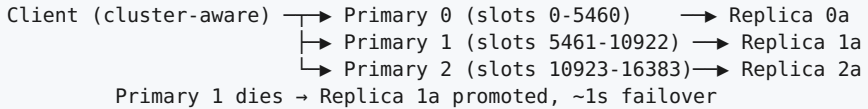
# Observe hit ratio
$ echo "stats" | nc memcached-1 11211 | grep -E "get_hits|get_misses|
evictions"
STAT get_hits 4823102
STAT get_misses 892341
STAT evictions 12043
# hit ratio = 4823102 / (4823102 + 892341) ≈ 84.4%

```

Redis Cluster — sharded with replication

Redis Cluster partitions data into 16,384 hash slots, each owned by a primary shard with 1-2 replicas. Failover is automatic via Raft-like

election among replicas. Persistence (RDB + AOF) is optional — most caches run without it for performance.



```
# Redis Cluster Setup (Redis 7.2, 2024)
$ redis-cli --cluster create 10.0.1.1:6379 10.0.1.2:6379 10.0.1.3:6379
\
  --cluster-replicas 1 --cluster-yes
>>> Performing hash slots allocation on 6 nodes...
M: 10.0.1.1:6379 slots:0-5460
M: 10.0.1.2:6379 slots:5461-10922
M: 10.0.1.3:6379 slots:10923-16383
R: 10.0.1.4:6379 replica of 10.0.1.1
R: 10.0.1.5:6379 replica of 10.0.1.2
R: 10.0.1.6:6379 replica of 10.0.1.3

# Observe cluster health and hit stats
$ redis-cli -h 10.0.1.1 -p 6379 cluster info
cluster_state:ok
cluster_slots_assigned:16384
cluster_known_nodes:6
cluster_size:3

$ redis-cli -h 10.0.1.1 info stats | grep -E "keyspace_hits|keyspace_misses|evicted_keys"
keyspace_hits:4823102
keyspace_misses:892341
evicted_keys:12043
# hit ratio ≈ 84.4% – same workload as Memcached above

# Latency check – Redis p99 should be < 5ms within AZ
$ redis-cli --latency-history -h 10.0.1.1
min: 0, max: 3, avg: 0.42 (435 samples) # ms – healthy
```

Redis Cluster handles slot migration for resharding (`CLUSTER SETSLOT ... MIGRATING`) with minimal disruption — keys move one at a time, not all at once.

Choosing between them

| Criterion | Memcached | Redis Cluster |
|-------------------|--------------------------------------|---|
| Data model | String only | Strings, hashes, sets, sorted sets, streams, JSON |
| Replication / HA | None (client must handle) | Built-in primary-replica + auto-failover |
| Persistence | None | RDB + AOF (optional) |
| Memory efficiency | Slab allocator, some waste | More overhead per key, but richer types |
| Operations | Simpler, fewer failure modes | More knobs, more to misconfigure |
| When to choose | Large transient cache, miss is cheap | Cache that is also a data store (sessions, leaderboards, rate-limit counters) |

Many teams run both: Memcached for large, low-value caches (rendered HTML, API responses) and Redis for small, high-value caches that need persistence or data structures (sessions, locks, counters for rate limiting – Chapter 9).

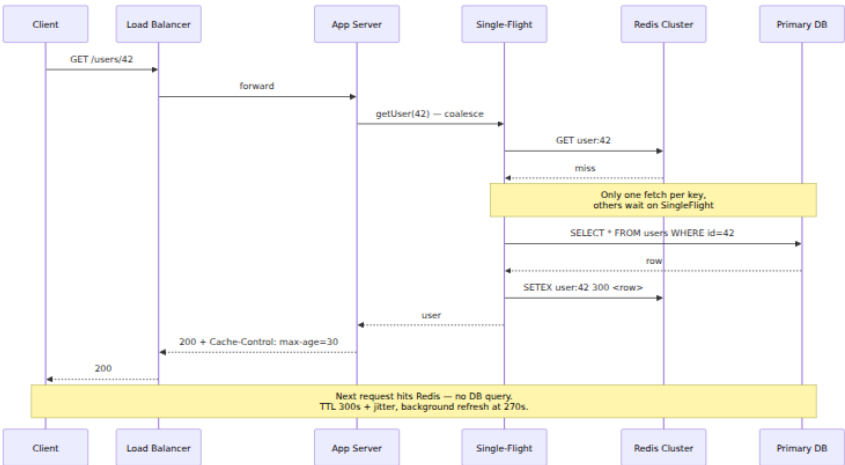


Figure 3-2: Read path with coalesced cache-aside, TTL with jitter, and background refresh. The single-flight prevents herds; the TTL bounds staleness; the jitter desynchronizes expirations.

Sizing and hit-ratio math

Cache sizing starts from the working set — the data that is hot enough to matter.

```
Working set = hot keys × avg value size
Cache size = working set / target_load_factor
target_load_factor ≈ 0.6 → 0.75 for Redis (hash table overhead)
For Memcached, usable ≈ configured × 0.85 (slab overhead)
```

Example:

```
Hot keys: 5M users × 1 KB profile = 5 GB working set
Target load factor: 0.65
Cache needed: 5 GB / 0.65 ≈ 7.7 GB
With 3 shards × 2 replicas (6 nodes): ~1.3 GB per node → r7g.large (16 GB) is ample
But with replication factor 2 (primary + replica): active data per shard = 7.7/3 ≈ 2.6 GB
Each primary holds 2.6 GB, each replica holds 2.6 GB → 15.6 GB total cluster memory
```

Hit ratio drives the economics. Suppose a database handles 10k QPS at p99 = 80 ms, and each query costs \$0.001 in provisioned capacity. With no cache, cost = 10k × \$0.001 = \$10/sec. With a cache at hit ratio h :

```
DB QPS with cache = (1 - h) × 10k
DB cost with cache = (1 - h) × $10/sec
Cache cost (Redis, 3 shards): ~$0.60/hr ≈ $0.00017/sec – negligible vs DB

h = 80% → DB QPS 2k → cost $2/sec (5× savings)
h = 90% → DB QPS 1k → cost $1/sec (10× savings)
h = 95% → DB QPS 500 → cost $0.50/sec (20× savings)
```

Each additional 5% of hit ratio beyond 80% halves the database load. That is why cache tuning — admission filters, proper TTLs, and avoiding scan pollution — has outsized ROI. A cache that achieves 95% instead of 85% does not save 10% of database cost; it saves 66%.

Monitor hit ratio continuously:

```
# Prometheus – Redis hit ratio (PromQL, redis_exporter 1.55)
$ curl -s 'http://prometheus.internal:9090/api/v1/query?
query=sum(rate(redis_keyspace_hits_total[5m])) /
(sum(rate(redis_keyspace_hits_total[5m])) +
sum(rate(redis_keyspace_misses_total[5m])))' | jq .
0.872    # 87.2% – healthy; alert if < 80% for 10m

# Application-level hit ratio (micrometer / Prometheus client)
# Counter: cache.gets{result="hit"} and cache.gets{result="miss"}
# hit_ratio = hit / (hit + miss)

# Caffeine stats via Micrometer (Spring Boot 3.2)
management.metrics.cache.enabled=true
# → /actuator/metrics/cache.gets?tag=result:hit
```

Cache review checklist

Before shipping or reviewing any caching layer, walk through this list:

- [] **Hit ratio target and measurement:** What hit ratio do you expect, how is it measured, and what alerts fire when it drops?
- [] **Staleness budget:** What is the maximum acceptable staleness for this data? Does the TTL + invalidation strategy respect it?
- [] **Invalidation path:** On every write, is the cache entry deleted or updated? What happens if the delete fails? Is there a TTL safety net?
- [] **Thundering herd:** Is there single-flight, TTL jitter, or probabilistic early expiration for hot keys? What happens during a cold start?
- [] **Negative cache:** Are misses for non-existent keys cached (with short TTL) to prevent DB hammer on enumeration attacks?
- [] **Eviction policy:** Is the policy appropriate for the access skew? Is the cache large enough that eviction is rare for the hot set?
- [] **Failure mode:** If the cache is down, does the system degrade to DB reads (with circuit breaker) or fail entirely? Has this been tested?
- [] **Sizing and cost:** Is the cache sized to the working set with headroom? Is the cost of the cache justified by the DB cost it saves?

- [] **Consistency:** If multiple caches hold the same data (in-process + Redis + CDN), what is the maximum divergence window? Is it acceptable?
- [] **Security:** Are cached entries scoped correctly (no cross-tenant leakage via shared cache keys)? Are cache keys constructed to prevent injection?

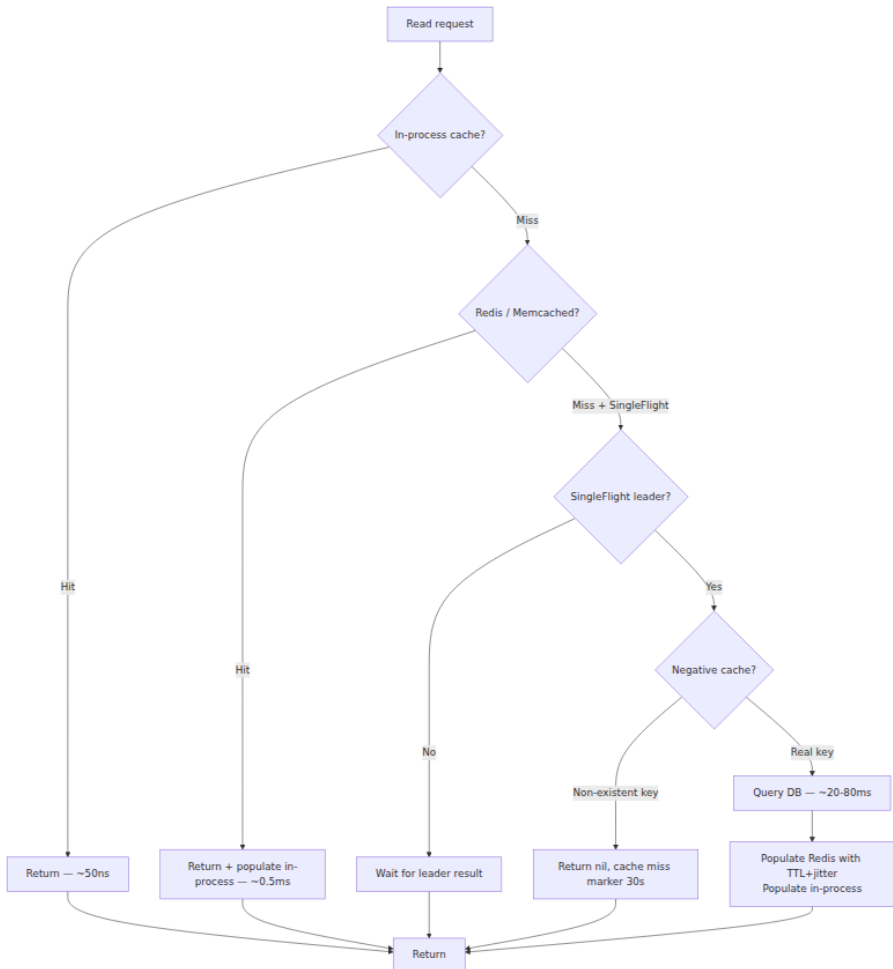
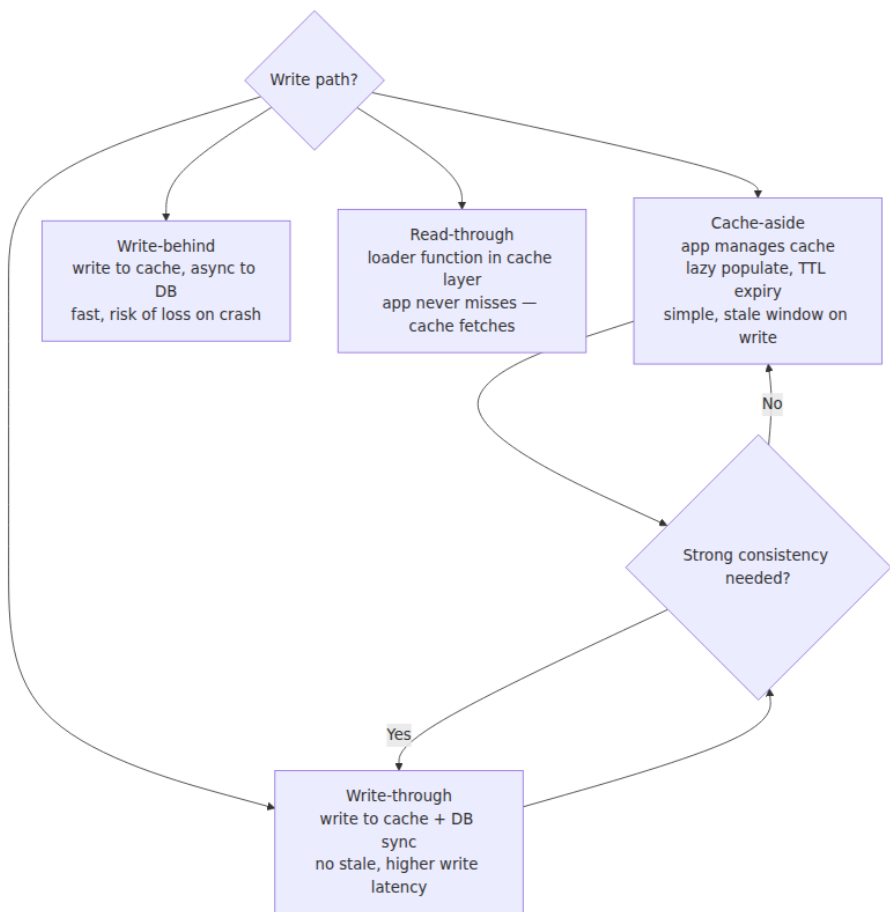
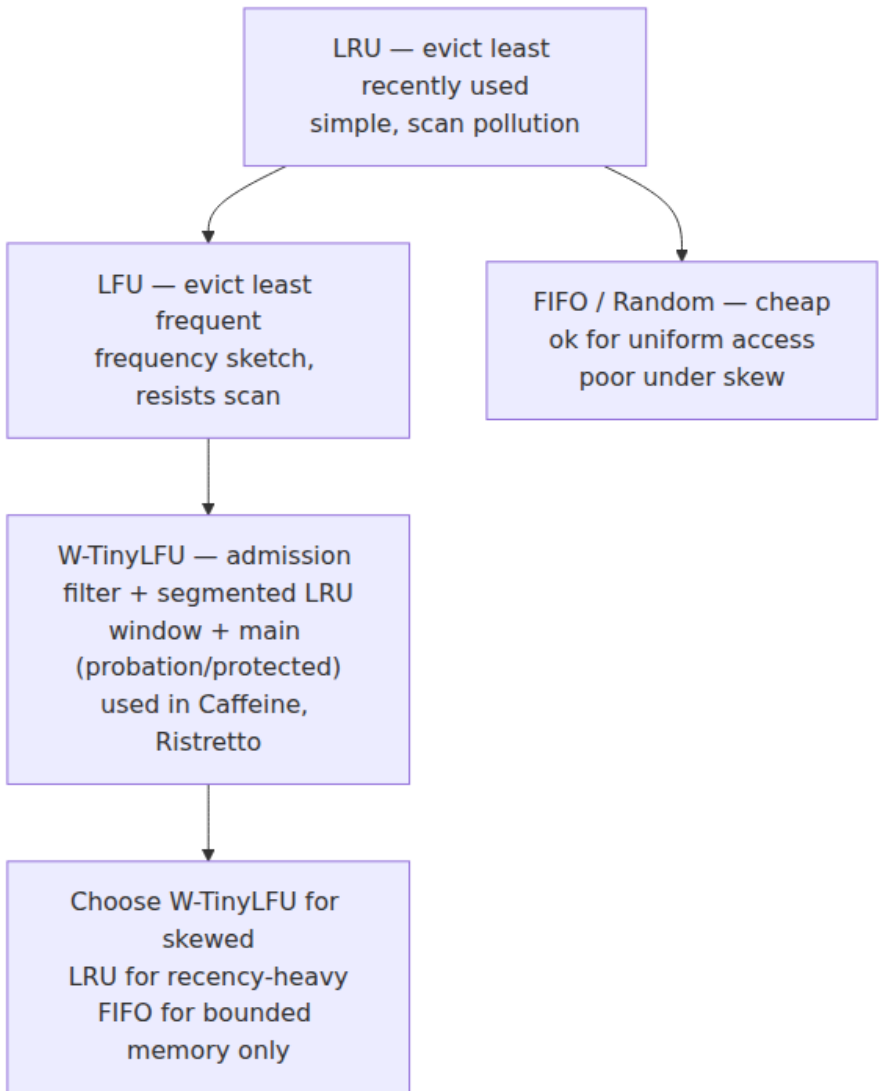


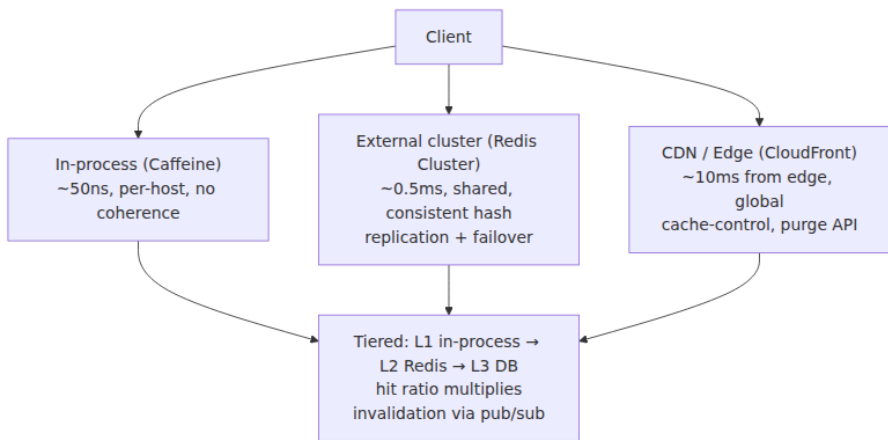
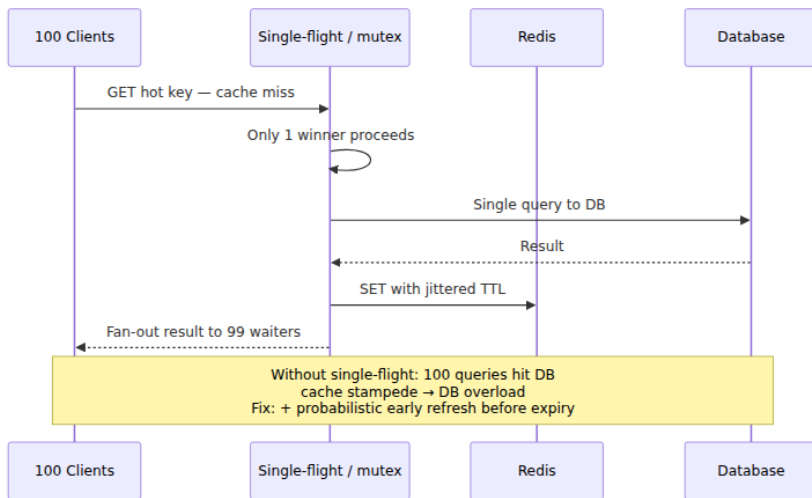
Figure 3-3: Complete read path with layered caching, single-flight, and negative caching. Every branch has bounded latency; no path hammers the database without coalescing.

Key takeaways

- Caching works because real workloads are skewed (Zipfian) — a small hot set serves most requests. Measure your workload's skew (α) before sizing; an assumption of high skew when reality is uniform leads to severe under-provisioning.
- Cache at the layer that matches the invalidation story: CDN for immutable/slow-changing content, external cache (Redis/Memcached) for shared application data, in-process (Caffeine) for tiny latency-critical working sets. Multiple layers multiply hit ratios but also multiply staleness windows.
- Four decisions define any caching strategy: what to cache (read-heavy, high-cost, skewed), when to populate (cache-aside for sparse access, read-through for simplicity, refresh-ahead for latency-sensitive), when to invalidate (TTL as default, explicit delete with TTL safety net for lower staleness), and what to do on a miss (single-flight, jitter, early expiration).
- Eviction policy matters more than cache size beyond a point. Modern caches use segmented LRU and TinyLFU admission filters to resist scan pollution and achieve near-optimal hit ratios. Do not use pure LRU for large caches with mixed hot/cold access.
- Thundering herds, stampedes, and cold starts are the primary cache-induced outage modes. Defend with single-flight coalescing, TTL jitter, probabilistic early expiration, warming, and stale-while-revalidate. A cache that is empty (deploy, failover) is a loaded gun pointed at the database.
- Distributed cache topology is a trade-off: Memcached is simpler and scales linearly but has no replication; Redis Cluster provides replication and richer data structures but has more operational complexity. Many teams run both for different tiers.
- Hit ratio is the metric that drives economics. Going from 85% to 95% hit ratio halves database load. Monitor it continuously, alert on drops, and treat cache tuning as high-ROI work. Size the cache to the working set with headroom; beyond that, policy matters more than bytes.







Further reading

- Caffeine cache design — Ben Manes, "Caffeine: A High Performance Caching Library for Java" — the W-TinyLFU paper and implementation that replaced Guava cache; explains segmented LRU and TinyLFU admission with benchmarks. <https://github.com/ben-manes/caffeine>
- Einziger, G., Friedman, R., and Manes, B. "TinyLFU: A Highly Efficient Cache Admission Policy" (*ACM Transactions on Storage*,

2017) — the TinyLFU paper; how a Count-Min Sketch admission filter achieves near-optimal hit ratios with minimal memory. <https://arxiv.org/abs/1511.03012>

- Redis documentation — "Redis Cluster Specification" and "Eviction Policies" (Redis 7.2, 2024) — authoritative reference for slot allocation, failover, and maxmemory policies. <https://redis.io/docs/reference/cluster-spec/> <https://redis.io/docs/reference/eviction/>
- Memcached documentation and mcrouter — Meta's mcrouter and Memcached slab allocator; practical guidance on sharding and consistent hashing at scale. <https://memcached.org/> <https://github.com/facebook/mcrouter>
- Nishtala, R. et al. "Scaling Memcache at Facebook" (NSDI 2013) — how Facebook scaled Memcached to billions of requests per second; consistent hashing, replication, and failure handling in production. <https://www.usenix.org/system/files/conference/nsdi13/nisthala.pdf>
- Huang, Q. et al. "An Analysis of Facebook Photo Caching" (SOSP 2013) — workload analysis of a petabyte-scale cache; Zipfian skew, TTL selection, and the cost of staleness. <https://www.cs.cmu.edu/~chensm/BigDataPage.html>
- Bronson, N. et al. "TAO: Facebook's Distributed Data Store for the Social Graph" (ATC 2013) — how Facebook layered caching (TAO) over MySQL for the social graph; cache consistency and invalidation at planetary scale. <https://www.usenix.org/conference/atc13/technical-sessions/presentation/bronson>
- Gabriel, S. "Stale-While-Revalidate, Stale-If-Error" (RFC 5861, 2010) — the HTTP cache extension that formalizes serving stale data during revalidation; basis for CDN and browser SWR behavior. <https://datatracker.ietf.org/doc/html/rfc5861>

Next: Chapter 4 — Load Balancing and Traffic Management — moves from caching (reducing load) to distributing it: L4/L7 load balancing at

system scale, traffic shaping, and the control plane that makes traffic policy safe to change.

Chapter 4 — Load Balancing and Traffic Management

What this chapter covers. Load balancing is where system design meets wire reality. Chapter 1 said "scale horizontally with stateless services" and Chapter 2 sized the fleet; this chapter decides how traffic *finds* those instances, how it *chooses* among them, and what happens when an instance is slow, failing, or warming up. We build the taxonomy — L3/L4/L7, client-side versus proxy, global versus local — and compare the algorithms that matter at scale (round-robin, least-request, EWMA/p2c, consistent hash, rendezvous). We then make those algorithms operational: health checking, outlier detection, slow-start, connection draining, and retry budgets. The second half is concrete: Envoy 1.30 listener/cluster/route configuration with outlier detection and circuit breaking, Kubernetes Service (ClusterIP, headless, topology-aware), Ingress NGINX and Gateway API (v1.1) HTTPRoute with traffic splitting, and global traffic management via DNS, anycast, and GSLB. The chapter closes with capacity-aware load balancing and the distributed-systems lens on why the load balancer is the most consequential single point of failure in a fleet.

Learning goals — after this chapter you should be able to:

- Distinguish L3, L4, and L7 load balancing, state when each is the right layer, and explain DSR versus proxy (NAT) return paths.
- Compare round-robin, least-request, EWMA, power-of-two-choices, consistent hash, and rendezvous hashing on uniformity, state, and behavior under heterogeneity.
- Configure Envoy (1.30) clusters with health checking, outlier detection, circuit breakers, and slow-start, and explain what each field does on the data path.
- Configure Kubernetes Services, Ingress, and Gateway API HTTPRoutes for traffic splitting, and explain kube-proxy iptables versus IPVS versus eBPF (Cilium) data paths.

- Reason about global load balancing — DNS TTL limits, anycast BGP, and GSLB health-driven steering — and its interaction with local balancing.
- Design capacity-aware and latency-aware routing (least-request with EWMA, panic thresholds, and retry budgets) that avoids amplifying tail latency.

The contract of a load balancer

A load balancer promises five things, rarely all perfectly: **distribution** (spread load evenly), **availability** (skip failed backends without client impact), **observability** (know where traffic went), **controllability** (shift traffic deliberately — canary, blue/green, drain), and **performance** (add negligible latency and handle failures faster than clients can). Every design choice trades among them.

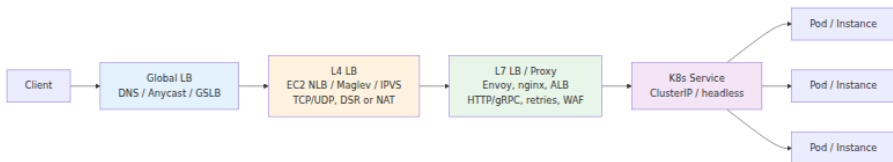


Figure 4-1: The load balancing stack. Global steering picks a region; L4 picks a node without parsing HTTP; L7 picks an instance with full request semantics; the Service abstraction picks a pod. Each layer sees less traffic and makes a more informed decision.

Boundary note. Wire-level mechanics (ECMP, Maglev consistent hashing, GUE/DSR encapsulation, QUIC connection migration) are covered in Vol 3, Chapter 9 — Load Balancing: L4, L7, and Algorithms and Chapter 10 — Proxies, Mesh, and CDNs. This chapter uses those mechanisms as *system-design primitives* and focuses on algorithm choice, configuration, and failure behavior at the fleet level.

Layers: L3, L4, L7 — and where the client sits

| Layer | Sees | Decision key | Typical product | Latency added |
|-----------------------------|---|-------------------------------|--|-----------------------------------|
| Global (DNS/anycast) | Hostname → region/PoP | Geography, health, latency | Route 53, Cloudflare, GSLB | 0 ms on data path (control plane) |
| L4 (transport) | TCP/UDP 5-tuple | Connection hash, least-conn | NLB, Maglev, IPVS, GCP ILB | < 0.1 ms (kernel/eBPF) |
| L7 (application) | HTTP method/path/headers, gRPC service/method | URL, header, cookie, latency | Envoy, nginx, ALB, Istio | 0.3–2 ms (parse + route) |
| Client-side | Full app context + history | Latency, error rate, locality | gRPC LB, Finagle, service mesh sidecar | 0 ms extra hop (in-process) |

Two return-path patterns determine L4 performance:

- **Proxy (NAT):** return traffic flows back through the LB. Simple, but the LB is bandwidth-bound.
- **Direct Server Return (DSR):** the backend replies directly to the client with the VIP as source, via tunneling (IPIP/GRE/GUE) or local VIP on loopback. The LB handles only ingress. Used by Google Maglev and Meta Katran; essential above ~100 Gbps per VIP.

Client-side balancing removes a hop entirely. A gRPC client with xDS or a mesh sidecar (Envoy) maintains its *own* view of the backend set and applies EWMA/least-request locally — lower latency, richer signal, but every client must get the right backend list.

Algorithms: what they optimize and what they cost

No single algorithm wins everywhere. The choice depends on request cost uniformity, backend heterogeneity, and whether affinity matters.

| Algorithm | State | Uniformity | Handles heterogeneity | Affinity | When to use |
|-----------------------------------|-----------------------|------------------------|--------------------------------|----------|---|
| Round-robin | None | Even by count | No — slow host gets same share | No | Uniform, CPU-bound, equally sized instances |
| Least-request / least-conn | Active count per host | Even by concurrency | Partially | No | Variable request cost, long-lived connections |
| EWMA + p2c | EWMA latency per host | Even by <i>latency</i> | Yes | No | Heterogeneous instances, noisy neighbors |
| Consistent hash (ketama) | Ring | By key, not by load | No | Yes | Cache locality, sharded state |
| Rendezvous (HRW) | None | By key, deterministic | No | Yes | Stateless consistent routing without a ring |
| Maglev / bounded-load CH | Ring + load cap | Bounded imbalance | Yes (cap) | Yes | Large L4 fleets needing minimal churn |

Round-robin and weighted round-robin

Trivial, stateless, and optimal when requests are uniform and backends are identical. Weighted round-robin assigns `weight  $\propto$  capacity`.

```

# weighted_round_robin.py - smooth weighted RR (nginx algorithm, Python
3.11)
class SmoothWRR:
    def __init__(self, backends: list[tuple[str, int]]):
        # backends: (name, weight)
        self.backends = [{"name": n, "weight": w, "current": 0} for n,
w in backends]
        self.total = sum(w for _, w in backends)

    def next(self) -> str:
        best = max(self.backends, key=lambda b: b["current"])
        best["current"] += best["weight"]
        for b in self.backends:
            b["current"] -= b["weight"] if b is best else 0
            # nginx variant: subtract total from winner only; above is
            # equivalent
            # corrected smooth algorithm (nginx): increment all by weight,
            # pick max, subtract total
            # simpler to implement as:
            for b in self.backends:
                b["current"] += b["weight"]
            chosen = max(self.backends, key=lambda b: b["current"])
            chosen["current"] -= self.total
            return chosen["name"]

```

Least-request and why least-conn lies

`least_conn` tracks open connections. For HTTP/1.1 with one request per connection it tracks load; for HTTP/2 or gRPC with multiplexed streams it does not — one connection can carry 100 streams while another carries 1. Envoy's `LEAST_REQUEST` therefore tracks *active requests* (streams), not connections, and optionally weights by response time. Without that, a slow backend that holds requests longer appears *more* loaded (more active requests) and correctly receives fewer new ones — a useful negative feedback loop.

EWMA and power of two choices (p2c)

EWMA tracks an exponentially weighted moving average of per-host latency; p2c samples two random hosts and picks the lower-EWMA one. Together (Finagle, Envoy's `LEAST_REQUEST` with `choiceCount: 2`) they approximate global least-latency routing with $O(1)$ state and no coordination.

```

pick = argmin( EWMA(host_a), EWMA(host_b) )   where a,b sampled
uniformly
EWMA_new =  $\alpha$  * observed_latency + (1 -  $\alpha$ ) * EWMA_old   #  $\alpha \approx 0.1$  to  $0.3$ 

```

Under heterogeneity (one AZ has noisy neighbors, one instance is on a degraded host), p2c + EWMA sheds load from slow hosts within seconds, while round-robin keeps hammering them.

Consistent hashing and bounded loads

When affinity matters — cache locality (Chapter 3), shard ownership — consistent hashing (Vol 1, Chapter 3) is correct. The operational concern is *imbalance under failure*: if a node dies, its keys redistribute to neighbors, spiking them. **Bounded-load consistent hashing** caps each node's share (e.g., at $1.25 \times \text{mean}$) and spills excess to the next node, trading a few extra cache misses for overload protection. Google's Maglev and Cloudflare's consistent hash both implement bounded variants.

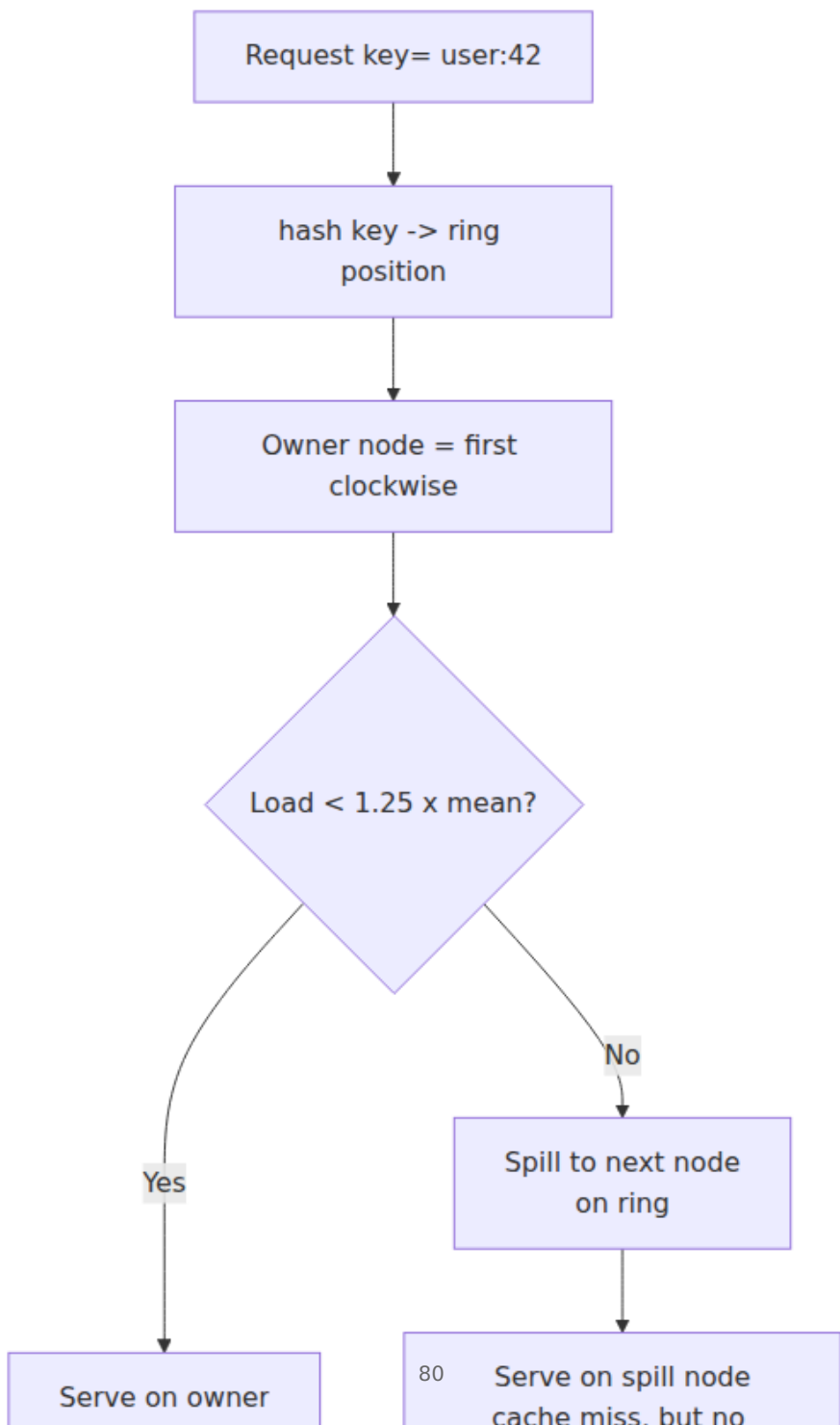


Figure 4-2: Bounded-load consistent hashing. Affinity is preserved when possible, but an explicit load cap prevents a hot shard from cascading into neighbor overload.

Health checking, outlier detection, and slow start

A load balancer that cannot detect failure is a failure amplifier. Three mechanisms compose:

Active health checking probes backends out-of-band (HTTP `/healthz`, TCP connect, gRPC health protocol). It removes dead hosts even with no traffic, but has a detection delay (interval $\times$ threshold).

Passive outlier detection watches *live traffic* for errors and ejections — 5xx, timeouts, or p99 spikes — and ejects the host from the pool without waiting for the next probe. It is faster and catches gray failures (slow, not dead) that active checks miss.

Slow start ramps traffic to a newly added or recently recovered host, avoiding a thundering herd that knocks it over again during warmup (JIT, cache cold, connection pool fill).

The interaction matters: active checks *re-add* hosts; outlier detection *removes* them; slow start *ramps* them. Misconfigure the timing and a flapping host oscillates in and out, each re-add spiking error rate.

Envoy in depth — the control plane's data plane

Envoy (CNCF, v1.30 — April 2024) is the reference L7 proxy: every major mesh (Istio, Cilium, App Mesh) and edge (Contour, Emissary) programs it via xDS. The configuration below wires a realistic service `api` with least-request, active health checking, outlier detection, circuit breaking, and slow start.

```

# envoy.yaml – Envoy 1.30 static config with least-request + outlier
# In production this is delivered via xDS (CDS/EDS/RDS/LDS); static
# here for clarity.
static_resources:
  listeners:
    - name: ingress
      address: { socket_address: { address: 0.0.0.0, port_value:
8080 } }
      filter_chains:
        - filters:
            - name: envoy.filters.network.http_connection_manager
              typed_config:
                "@type": type.googleapis.com/
envoy.extensions.filters.network.http_connection_manager.v3.HttpConnect
ionManager

                stat_prefix: ingress_http
                codec_type: AUTO
                route_config:
                  name: api_route
                  virtual_hosts:
                    - name: api
                      domains: ["api.internal", "api.internal:8080"]
                      routes:
                        - match: { prefix: "/" }
                          route:
                            cluster: api
                            timeout: 2s
                            retry_policy:
                              retry_on: "5xx,reset,connect-failure"
                              num_retries: 2
                              per_try_timeout: 1s
                              retry_back_off: { base_interval: 25ms,
max_interval: 250ms }

# retry budget: never retry more than 20% of active requests

# (Envoy >=1.28: retry_concurrency_window via runtime)
http_filters:
  - name: envoy.filters.http.router
    typed_config:
      "@type": type.googleapis.com/
envoy.extensions.filters.http.router.v3.Router

  clusters:
    - name: api
      connect_timeout: 200ms
      lb_policy: LEAST_REQUEST
      least_request_config: { choice_count: 2 } # p2c
      # slow start – ramp from 10% to 100% over 30s after host added
      slow_start_config:

```

```

    slow_start_window: 30s
    aggression: 1.0
    min_weight_percent: 10
  circuit_breakers:
    thresholds:
      - priority: DEFAULT
        max_connections: 8192
        max_pending_requests: 4096
        max_requests: 4096
        max_retries: 3
        track_remaining: true
  health_checks:
    - timeout: 2s
      interval: 5s
      unhealthy_threshold: 3
      healthy_threshold: 2
      interval_jitter_percent: 10
      http_health_check: { path: /healthz }
  outlier_detection:
    consecutive_5xx: 5
    interval: 5s
    base_ejection_time: 30s
    max_ejection_percent: 50           # never eject more than half
the cluster
    min_health_percent: 50           # panic threshold – stop
ejecting below this
    enforcing_consecutive_5xx: 100
    enforcing_success_rate: 100      # also eject on success-rate
outlier
    success_rate_minimum_hosts: 5
    success_rate_stale_time: 30s
  load_assignment:
    cluster_name: api
    endpoints:
      - lb_endpoints:
          - endpoint: { address: { socket_address: { address: 10.0.
1.10, port_value: 8080 } } }
          - endpoint: { address: { socket_address: { address: 10.0.
1.11, port_value: 8080 } } }
          - endpoint: { address: { socket_address: { address: 10.0.
2.10, port_value: 8080 } } }

```

Key fields and why they matter:

- `LEAST_REQUEST` with `choice_count: 2` implements p2c; without EWMA weighting it is still better than round-robin for variable-cost requests. Envoy's `ROUND_ROBIN` with `slow_start` is fine for uniform workloads.

- `max_ejection_percent: 50` prevents outlier detection from ejecting the entire cluster during a correlated failure (bad deploy).
`enforcing *` at 100 means ejections actually happen; lower values shadow-eject for canary analysis.
- `min_health_percent: 50` is Envoy's **panic threshold** — below this, Envoy ignores health and outlier state and sends to all hosts, preferring degraded service over black-holing.
- `retry_policy` with `per_try_timeout` bounds each attempt; without it, a slow upstream consumes the full `timeout` per retry.
- `circuit_breakers` are per-cluster, per-priority; they shed load *before* the upstream collapses, at the cost of `503` with `x-envoy-overloaded: true` to the caller.

```
# Envoy admin – inspect cluster health and outlier state (Envoy 1.30)
$ curl -s http://localhost:9901/clusters | grep -E "api::|health|eject"
api::10.0.1.10:8080::health_flags::/healthy
api::10.0.1.11:8080::health_flags::/failed_active_hc
api::10.0.1.11:8080::eject_active::true  ejection_time=27s
api::10.0.2.10:8080::health_flags::/healthy
api::circuit_breakers.default.rq_pending_open::0
api::circuit_breakers.default.rq_open::0
api::membership_healthy::2/3
```

Kubernetes: Service, kube-proxy, Ingress, and Gateway API

Kubernetes load balancing has four layers, each with a different data path.

Service — the in-cluster VIP

```
# k8s-service.yaml — Kubernetes v1.30
apiVersion: v1
kind: Service
metadata:
  name: api
  labels: { app: api }
spec:
  selector: { app: api }
  ports:
    - name: http
      port: 80
      targetPort: 8080
      protocol: TCP
  type: ClusterIP          # VIP only inside cluster; kube-proxy
                           # programs the datapath
---
# Headless service — DNS returns pod IPs directly; client or mesh does
# LB
apiVersion: v1
kind: Service
metadata: { name: api-headless }
spec:
  selector: { app: api }
  clusterIP: None
  ports: [{ port: 80, targetPort: 8080 }]
---
# Topology-aware routing — prefer same-zone endpoints (K8s 1.30,
# stable)
apiVersion: v1
kind: Service
metadata:
  name: api
  annotations:
    service.kubernetes.io/topology-mode: Auto # same-zone preferred,
    fallback cross-zone
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 8080 }]
  trafficDistribution: PreferClose             # KEP-4444, K8s 1.31+
  field (replaces hint)
```

The `Service` VIP is virtual — no process listens on it. `kube-proxy` programs the datapath:

| kube-proxy mode | Mechanism | Scale | Notes |
|--------------------------------|--|-----------------------------|--|
| iptables | DNAT rules, random per-packet | ~1k Services, ~5k endpoints | O(n) rule traversal; deprecated path |
| ipvs | Kernel IPVS (hash table, multiple schedulers: rr , lc , sh) | ~10k Services | O(1) lookup; supports sh (consistent hash) |
| nftables (KEP-3866, beta 1.31) | nft set/map | Similar to IPVS, cleaner | Replacement for iptables |
| eBPF (Cilium, no kube-proxy) | eBPF programs on cgroup/TC | 100k+ endpoints | Maglev consistent hash, socket LB, no iptables |

For gRPC or any L7-aware routing, a headless Service plus a mesh sidecar (Envoy/Istio) or a client-side LB is preferred — the Service VIP with iptables cannot do least-request or outlier detection.

Ingress and Gateway API — north-south

```
# ingress.yaml – Ingress NGINX 1.10 (ingress-nginx/controller v1.10,
K8s 1.30)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  annotations:
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "200ms"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "2s"
    nginx.ingress.kubernetes.io/upstream-hash-by: "$request_uri" #
consistent hash opt-in
    nginx.ingress.kubernetes.io/canary: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend: { service: { name: api, port: { number: 80 } } }
---
# Canary ingress – 10% to canary (NGINX canary annotations)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-canary
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10"
spec:
  ingressClassName: nginx
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend: { service: { name: api-canary, port: { number:
80 } } }
```

Ingress is effectively frozen; **Gateway API (v1.1, 2024)** is the replacement with typed, role-oriented resources:

```
# gateway-api.yaml – Gateway API v1.1 (gateway.networking.k8s.io/v1)
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata: { name: edge, namespace: default }
spec:
  gatewayClassName: istio          # or cilium, kong, gke-l7-glb
  listeners:
    - name: https
      port: 443
      protocol: HTTPS
      hostname: api.example.com
      tls: { mode: Terminate, certificateRefs: [{ name: api-tls }] }
      allowedRoutes: { namespaces: { from: Same } }
  ---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: api-route, namespace: default }
spec:
  parentRefs: [{ name: edge }]
  hostnames: ["api.example.com"]
  rules:
    - matches: [{ path: { type: PathPrefix, value: /v1 } }]
      backendRefs:
        - name: api
          port: 80
          weight: 90
        - name: api-canary
          port: 80
          weight: 10
      timeouts: { request: 2s, backendRequest: 1s }
      filters:
        - type: RequestHeaderModifier
          requestHeaderModifier: { add: [{ name: X-Route, value: stable
-or-canary }] }
        - type: ResponseHeaderModifier
          responseHeaderModifier: { add: [{ name: X-Served-By, value: e
dge }] }
```

Gateway API's advantages for traffic management: `weight` is first-class (no annotation hack), `HTTPRoute` can attach `BackendTSLSPolicy` for mTLS to backends, and `GRPCRoute` (v1.1) routes on gRPC service/method.

Global traffic management

Local balancing picks an instance; global balancing picks a *region*. Three mechanisms, often combined:

DNS-based GSLB (Route 53 latency/routing, GCP Cloud DNS routing policies, NS1). Health checks drive DNS answers; clients cache them for TTL seconds. TTL is the agility knob — 10 s gives fast failover but 10× more DNS QPS and resolver variance; 60 s is the common compromise. DNS cannot steer mid-connection and is blind to client-to-edge latency (it sees the resolver, not the client — EDNS Client Subnet partially fixes this).

Anycast advertises the same VIP from every PoP via BGP; the internet's routing picks the nearest (fewest AS hops). Latency-optimal and instant failover (BGP reconvergence in seconds), but all PoPs must serve the same content and capacity planning is harder — traffic follows BGP, not load.

GSLB with health and load feedback (Cloudflare Load Balancing, F5 BIG-IP DNS, Aviatrix). A control plane pushes region weights based on health, latency, and capacity; data-plane proxies or DNS answer accordingly. The distributed-systems concern is **split-brain steering** — if the GSLB control plane partitions, two regions may both believe they are primary.

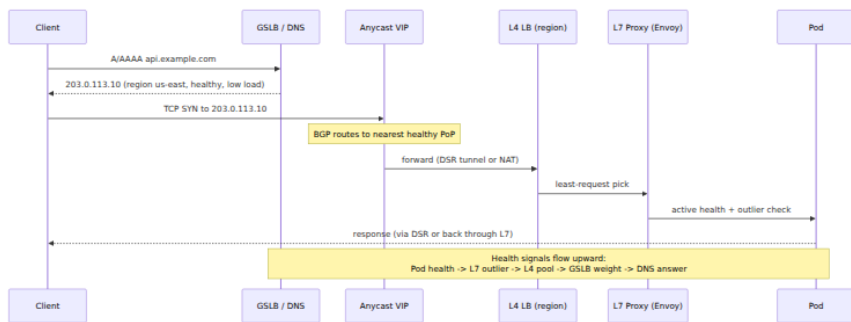


Figure 4-3: Global to local traffic path and health propagation. A failing pod is ejected locally (outlier), a failing region is drained globally (GSLB), and anycast provides sub-second PoP failover without DNS TTL delay.

Capacity-aware and latency-aware routing

Naive least-request still sends equal share to a host that is twice as slow (same active count, higher latency). Two refinements:

EWMA least-request (Envoy `LEAST_REQUEST` with `choiceCount: 2` plus success-rate outlier detection) approximates latency-aware routing without explicit weights.

Load-aware consistent hashing — when affinity is required but hot keys exist, combine consistent hashing with per-host load caps (bounded-load) or with a two-choice spillover: hash to primary, if overloaded try secondary hash.

Retry budgets prevent the LB from amplifying failure. Envoy's `retry_budget` (or gRPC's `retryThrottling` in service config) allows retries only when success rate justifies them — if the cluster is 95% failing, retries would double the load with near-zero benefit.

```
// gRPC service config – retry budget (gRFC A6, gRPC 1.65)
{
  "methodConfig": [{
    "name": [{ "service": "api.Search" }],
    "retryPolicy": {
      "maxAttempts": 3,
      "initialBackoff": "0.025s",
      "maxBackoff": "0.25s",
      "backoffMultiplier": 2,
      "retryableStatusCodes": ["UNAVAILABLE", "RESOURCE_EXHAUSTED"]
    }
  ]},
  "retryThrottling": {
    "maxTokens": 100,
    "tokenRatio": 0.1
  }
}
```

```
# Istio DestinationRule – outlier + circuit breaker + locality (Istio
1.22)
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: api }
spec:
  host: api.default.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      consistentHash: # or simple: LEAST_REQUEST
      httpHeaderName: x-user-id
      localityLbSetting:
        enabled: true
        failover:
          - from: us-east-1a
            to: us-east-1b
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 5s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
      minHealthPercent: 50
    connectionPool:
      tcp: { maxConnections: 8192 }
      http: { http1MaxPendingRequests: 4096, maxRequestsPerConnection:
1000 }
```

Distributed-systems lens

Load balancing is the fleet's nervous system, and like a nervous system it can cause the failure it is meant to prevent.

Herdling. A naive health check that marks a host healthy after one success, combined with no slow-start, produces a *thundering herd*: all LBs simultaneously return a recovering host to the pool, its queue fills in milliseconds, it fails health again, and every LB ejects it — oscillation forever. The fix is hysteresis (require N consecutive successes) plus slow-start ramping plus jittered health intervals.

Retry amplification. A 5% error rate with 2 retries and no budget becomes a 15% retry load; at 50% error rate it becomes 100% retry load — more load when the system is least able to handle it. Always bound retries as a *fraction of success*, not a multiple of failure.

Consistency of the backend set. Each LB has a slightly different view of which hosts are healthy (propagation delay from EDS/SDS). During membership churn, two LBs may disagree — one sends to a host the other has ejected. This is acceptable for stateless services but fatal for consistent-hash affinity: two LBs may route the same key to different hosts, doubling cache misses. Mitigate with a shared assignment (consistent ring versioned and distributed) or by accepting the miss.

The LB as a single point of failure. Every tier's LB is itself a fleet that must be load-balanced. The pattern is recursive — VIP → ECMP → L4 fleet → L7 fleet → Service — and each layer needs its own health checking, capacity headroom (N+1), and deployment strategy (rolling, with connection draining). Connection draining on shutdown is non-negotiable:

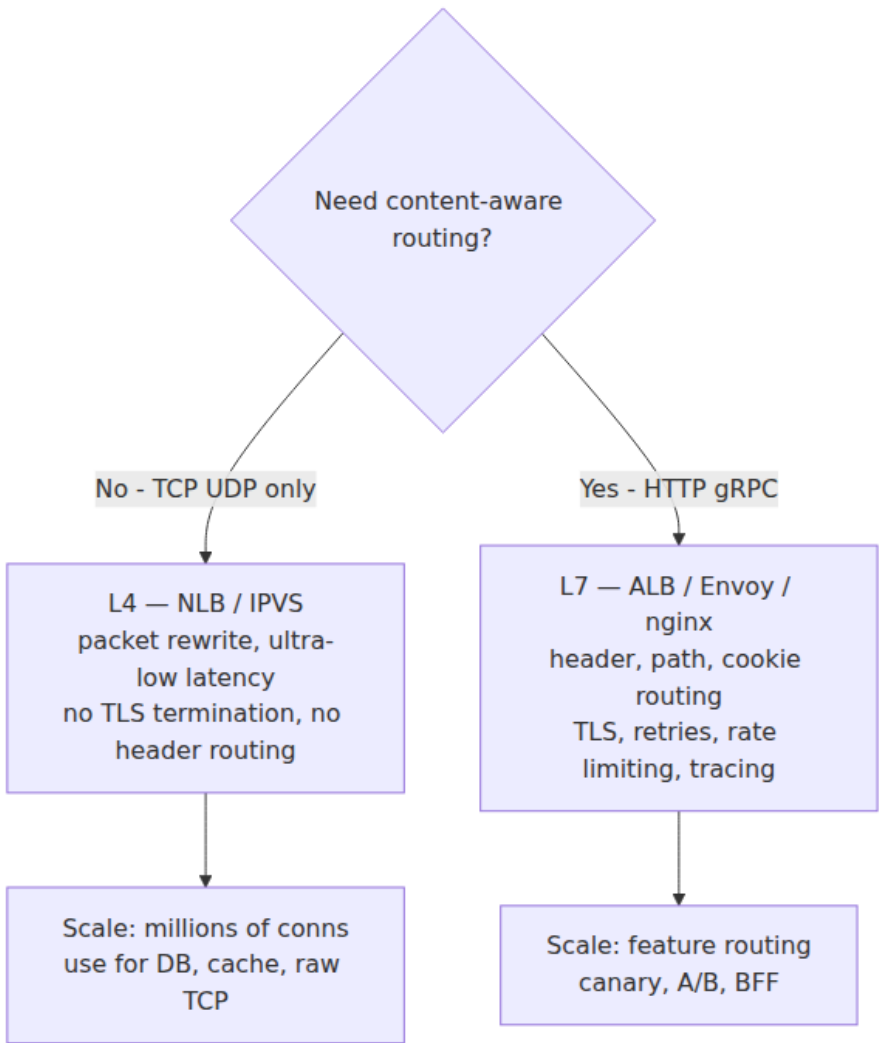
```
# Kubernetes – graceful termination with draining (K8s 1.30)
spec:
  terminationGracePeriodSeconds: 60
  containers:
    - name: api
      lifecycle:
        preStop:
          exec:
            command: ["/bin/sh", "-c", "sleep 5; /app/drain --grace 50s"]
            # 5s sleep lets the endpoint be removed from Service/EDS
            # before SIGTERM
      readinessProbe:
        httpGet: { path: /readyz, port: 8080 }
        periodSeconds: 5
        # Envoy sidecar drains via /healthcheck/fail then waits drain
        # duration
```

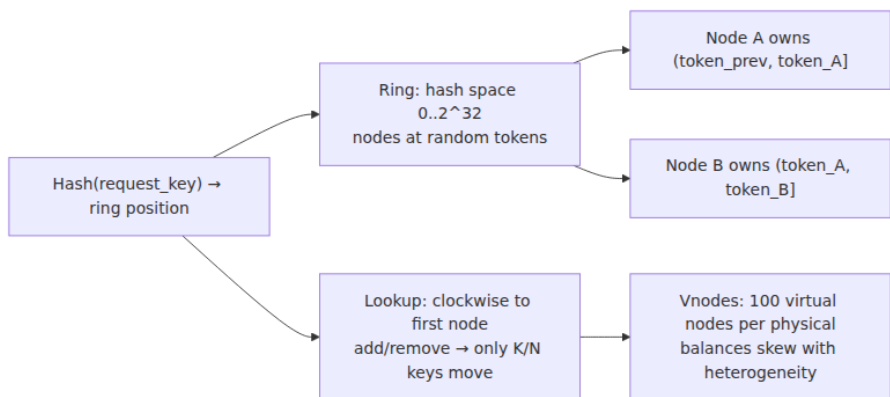
Without draining, a rolling deploy that restarts 10% of pods drops 10% of in-flight requests — a self-inflicted partial outage on every deploy.

Key takeaways

- Choose the layer deliberately: global (DNS/anycast) for region, L4 for node, L7 for instance, client-side/mesh for latency-optimal. Mixing layers without understanding the return path (NAT vs DSR) creates bandwidth or latency surprises.

- Algorithms trade uniformity for state and affinity. Round-robin is optimal for uniform work; least-request/EWMA+p2c for heterogeneous or variable-cost work; consistent hash/rendezvous for affinity; bounded-load variants for affinity under hot keys.
- Envoy is the lingua franca: `LEAST_REQUEST` with `choiceCount: 2`, `outlier_detection` with `max_ejection_percent` and `min_health_percent` (panic threshold), `circuit_breakers`, `health_checks`, and `slow_start_config` compose into a production-grade data path. Understand each field's effect on the packet, not just its name.
- Kubernetes Services are virtual; the real datapath is kube-proxy (iptables/IPVS/nftables) or eBPF (Cilium). For L7-aware routing, use a headless Service plus a mesh or Gateway API HTTPRoute with weighted `backendRefs`.
- Gateway API (v1.1) replaces Ingress for traffic management: typed `Gateway` / `HTTPRoute` / `GRPCRoute`, first-class `weight`, and `BackendTLSPolicy` for mTLS.
- Global balancing blends DNS GSLB (agile via TTL, but TTL-bound), anycast (instant, BGP-driven), and health-propagating control planes. Design for the case where the GSLB control plane partitions.
- Capacity-aware routing (EWMA, bounded-load hashing, retry budgets) and slow-start are not optimizations — they are stability requirements. Without them, a single slow host or a single bad deploy cascades.
- The load balancer fleet itself must be highly available (N+1, ECMP, draining). A deploy that does not drain connections is a partial outage.





Round-robin / weighted
RR
simple, ignores load



Least-connections
good for long-lived
conns
needs active conn
tracking



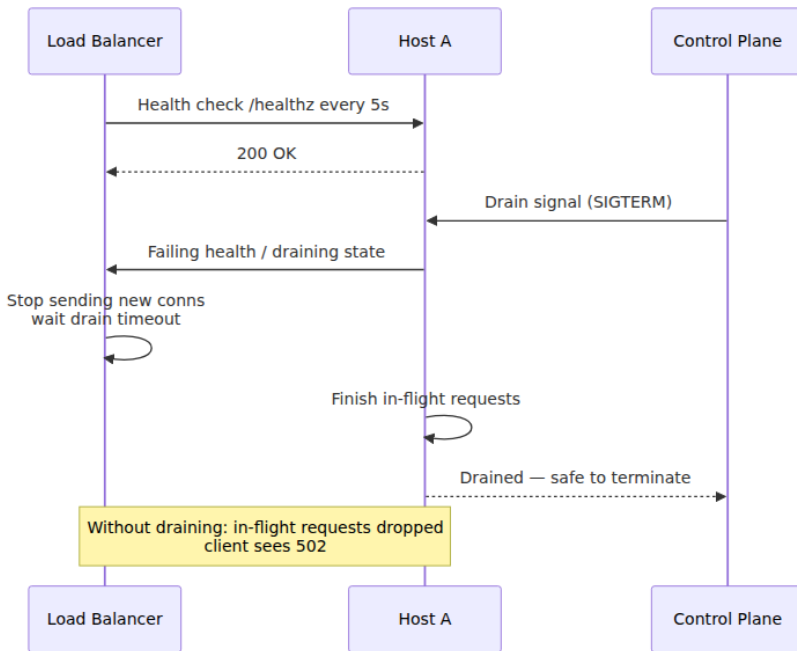
Consistent hash
affinity, cache-friendly
hot key risk



Power of two choices
pick 2 random, choose
less loaded
near-optimal with
minimal state



EWMA / p_{2c} + peak
Finagle, Envoy weighted



Further reading

- Envoy Proxy documentation — Load balancing, outlier detection, circuit breaking, health checking (Envoy 1.30 — April 2024). <https://www.envoyproxy.io/docs/envoy/v1.30/>
- Kubernetes SIG Network — Service, EndpointSlice, kube-proxy, Gateway API v1.1. <https://kubernetes.io/docs/concepts/services-networking/service/> and <https://gateway-api.sigs.k8s.io/>
- Google Maglev — Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer" (NSDI 2016). <https://research.google/pubs/maglev-a-fast-and-reliable-software-network-load-balancer/>
- Meta Katran — "Katran: A high performance layer 4 load balancer" and L4 DSR with eBPF/XDP. <https://github.com/facebookincubator/katran> and <https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/>

- Cilium — eBPF-based kube-proxy replacement, Maglev consistent hashing, socket LB. <https://docs.cilium.io/en/stable/network/kubernetes/kubeproxy-free/>
- gRPC load balancing — Pollard et al., gRFC A6 (retry throttling) and gRPC xDS LB policies. <https://github.com/grpc/proposal/blob/master/A6-client-retries.md>
- Dean, J. and Barroso, L. A. "The Tail at Scale" (CACM, 56(2), 2013) — why tail-aware balancing matters under fan-out. <https://research.google/pubs/the-tail-at-scale/>
- AWS Elastic Load Balancing — NLB/ALB/GWLB documentation, target health, slow start, cross-zone. <https://docs.aws.amazon.com/elasticloadbalancing/>

Next: Chapter 5 — Data Modeling for Scale — turns traffic estimates into shapes on disk and in memory: how to model data so that the common case is local, the hot path is indexed, and the schema can evolve without a flag-day migration.

Chapter 5 — Data Modeling for Scale

What this chapter covers. Chapter 2 asked how much data you have; Chapter 3 cached the hot working set; this chapter decides how that data is *shaped* — what the rows, documents, columns, and events look like, how they are partitioned and indexed, and how the shape evolves without breaking production. A model that is correct at 10 GB can become pathological at 10 TB: a missing partition key turns a point lookup into a scatter-gather, a chatty normalized schema turns one user request into twelve joins, and a schema change without compatibility discipline turns a rolling deploy into a deserialization outage. We build from first principles — entities, relationships, access patterns — through relational modeling for scale (normalization versus denormalization, indexing, partitioning), NoSQL families (document, wide-column, key-value, graph) and when each wins, time-series and search shapes, and the discipline of schema evolution (Avro/Protobuf compatibility, expand-contract, versioning). The second half is operational: secondary indexes, materialized views, CQRS read models, polyglot persistence, and the

distributed-systems lens on why data modeling is where consistency, availability, and team autonomy actually collide.

Learning goals — after this chapter you should be able to:

- Derive a data model from access patterns (not just entities), and explain why "model the queries, not just the domain" is the scaling rule.
- Choose normalization versus denormalization per workload, and quantify the join versus duplication trade-off.
- Select a partition key that keeps the common case local, explain hot-partition failure modes, and design composite keys that avoid scatter-gather.
- Compare relational, document, wide-column, key-value, graph, and search/time-series models on write path, read path, and evolution cost, and pick the right family per service.
- Design indexes (primary, secondary, covering, LSM-friendly) and know when to replace an index with a materialized view or CQRS read model.
- Evolve schemas safely via Avro/Protobuf compatibility rules and the expand-contract (parallel change) pattern without flag-day migrations.
- Apply polyglot persistence without creating a distributed monolith — one service, one primary store, explicit ownership.

Model the access patterns, not just the entities

Textbook ER modeling starts with entities and relationships. At scale the starting point is different: **what queries must be fast, at what QPS, with what latency SLO, and at what consistency**. The entity model follows; the query model leads. Two systems with identical entities (users, orders, messages) can have radically different models because their hot paths differ.

| Question | Why it drives the model |
|----------------------------|---|
| What is the p99 read path? | Determines partition key, covering indexes, denormalization |

| Question | Why it drives the model |
|--------------------------------------|---|
| What is the write:read ratio? | Write-heavy favors LSM/append; read-heavy favors B-tree + cache |
| What is the consistency requirement? | Strong consistency keeps normalized joins viable; eventual allows denormalized copies |
| What is the cardinality and skew? | High-cardinality keys partition well; skewed keys need salting or bounded-load hashing |
| What is the evolution rate? | Fast-evolving domains need schemaless or schema-on-read; stable domains benefit from strict schemas |

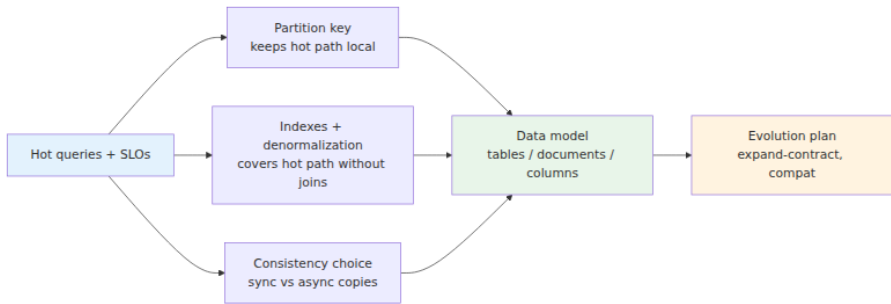


Figure 5-1: Access-pattern-driven modeling. The hot queries select the partition key and indexes; those choices select the store family and shape; the evolution plan protects the shape over time.

Boundary note. Storage-engine internals — B-tree versus LSM-tree, WAL, compaction, buffer pools, and physical partitioning mechanics — are in Vol 5 — Databases and Storage Systems. This chapter treats the *modeling* decision — what shape to store and how it maps to queries — and references those internals only as they constrain the model.

Relational modeling for scale

Normalization and when to violate it

Normalization (1NF-3NF/BCNF) eliminates redundancy and preserves invariants: one fact in one place, enforced by the database. At scale it has

a cost — every join is a potential cross-partition hop, and every foreign key is a write-time check that limits partition independence.

| Normal form | What it guarantees | Cost at scale |
|-------------|------------------------------------|-------------------------------------|
| 1NF | Atomic values, no repeating groups | None — always do this |
| 2NF | No partial-key dependencies | Slight — composite keys matter |
| 3NF/BCNF | No transitive dependencies | Join cost; denormalization pressure |

Rule of thumb: normalize until the hot read path hurts, then denormalize deliberately — with a plan for keeping copies consistent (change data capture, outbox, or application-level dual write with verification; see Vol 10, Chapter 6).

Example — an orders service. Normalized:

```

-- orders_normalized.sql – PostgreSQL 16, 3NF
CREATE TABLE users (
  user_id    BIGINT PRIMARY KEY,
  email      TEXT NOT NULL UNIQUE,
  name       TEXT NOT NULL
);

CREATE TABLE orders (
  order_id   BIGINT PRIMARY KEY,
  user_id    BIGINT NOTORGANICNULL REFERENCES users(user_id),
  status     TEXT NOT NULL CHECK (status IN
('pending','paid','shipped','cancelled')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- partition by range on created_at for retention and locality
  CONSTRAINT orders_created_check CHECK (created_at >= '2024-01-01')
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2024_q1 PARTITION OF orders
  FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE orders_2024_q2 PARTITION OF orders
  FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');

CREATE TABLE order_items (
  order_id   BIGINT NOT NULL REFERENCES orders(order_id),
  sku        TEXT NOT NULL,
  qty        INT NOT NULL CHECK (qty > 0),
  unit_price NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, sku)
);
CREATE INDEX ON order_items (sku); -- for "all orders containing sku
X"

-- Hot path: "recent orders for user 42 with items" – requires two
joins
-- At 20k QPS this join is the bottleneck

```

Denormalized for the hot path — pre-join into a read-optimized shape:

```

-- orders_denormalized.sql – same Postgres, read-optimized
CREATE TABLE user_orders_denorm (
  user_id    BIGINT NOT NULL,
  order_id   BIGINT NOT NULL,
  status     TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL,
  -- denormalized: embed items as JSONB to avoid second lookup
  items      JSONB NOT NULL, --
  [{"sku": "A123", "qty": 2, "unit_price": 19.99}]
  total      NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (user_id, created_at, order_id)
  -- partition key user_id keeps per-user timeline local; sort key
  created_at keeps range scans local
) PARTITION BY HASH (user_id);

-- Covering index for status-filtered timeline (avoids heap fetch)
CREATE INDEX ON user_orders_denorm (user_id, status, created_at DESC)
  INCLUDE (total) WHERE status IN ('pending', 'paid');

-- The write path now does: INSERT into orders + INSERT into
user_orders_denorm
-- Keep them consistent via transactional outbox (Vol 10, Ch 6), not
ad-hoc dual write

```

Trade-off made explicit: the denormalized table duplicates item data, costs more on write (two writes, CDC or outbox), but serves `GET /users/42/orders?status=paid` as a single partition-local range scan with no join — p99 8 ms versus 45 ms under load.

Partition keys and hot partitions

The partition key is the single most consequential modeling decision. A good key has three properties: **high cardinality** (many distinct values), **uniform distribution** (no single key dominates), and **query locality** (the hot query filters on it).

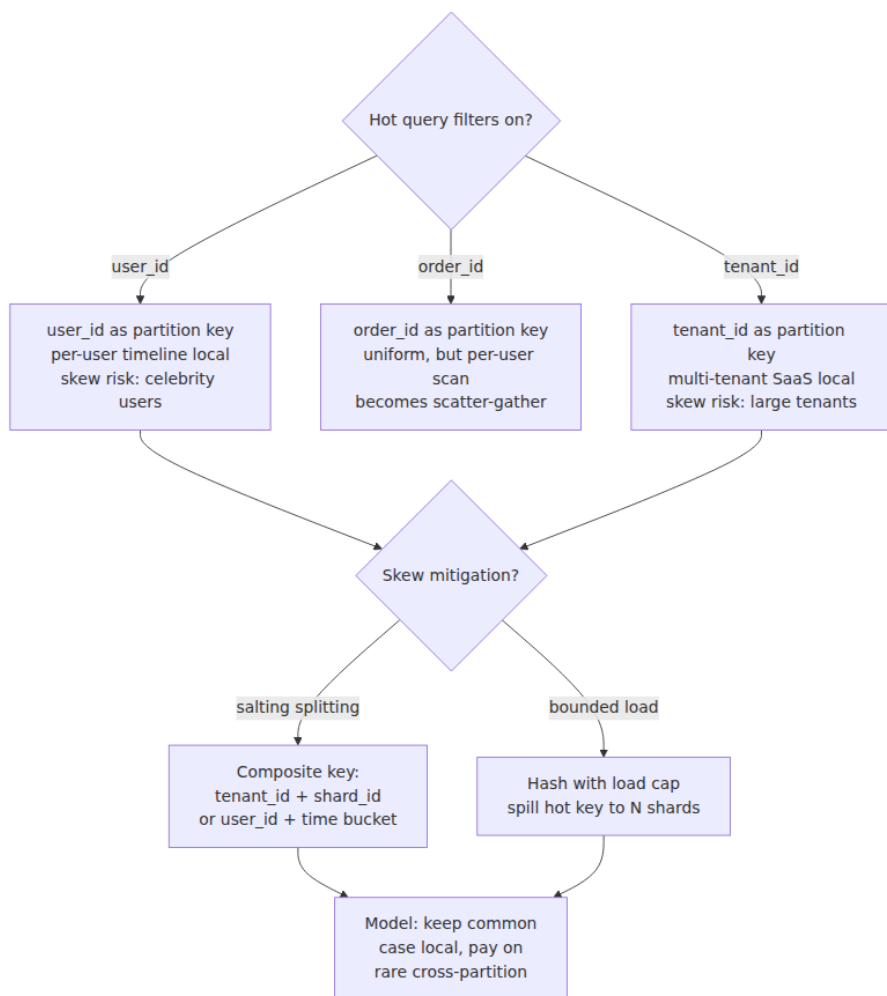


Figure 5-2: Partition key selection. The hot query's filter dictates the key; skew dictates the mitigation. A key that is perfect for one query is pessimal for another — that tension is why secondary indexes and CQRS exist.

Hot-partition antipattern — `status` as a partition key with three values (`pending`, `paid`, `shipped`) — puts one-third of all data on one node and makes every write contend on the same partition. Cardinality matters.

Indexing for the hot path

| Index type | When it wins | Cost |
|---------------------------------------|--|---|
| Primary (partition + sort key) | Point lookup and range scan on the hot path | None — it is the table order |
| Secondary (GSI/LSI) | Alternate access pattern without duplicating the table | Extra storage, write amplification, eventual lag on GSI |
| Covering (INCLUDE) | Avoids heap/clustered-index lookup for known projections | Larger index, faster reads |
| LSM-friendly (write-optimized) | Write-heavy, range-scan-heavy on LSM stores | Compaction cost; bloom filter tuning |

```
-- covering_index.sql – PostgreSQL 16
-- Without covering: index scan + heap fetch per row (random I/O)
-- With covering: index-only scan (sequential in index order)
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, total FROM user_orders_denorm
WHERE user_id = 42 AND status = 'paid' ORDER BY created_at DESC LIMIT 20;
-- Index Scan using user_orders_denorm_user_id_status_created_idx on
user_orders_denorm
--   Index Cond: (user_id = 42 AND status = 'paid')
--   Buffers: shared hit=4 (index-only; no heap hits due to INCLUDE)

-- DynamoDB GSI – alternate access pattern without full table scan
# DynamoDB ☐ orders table with GSI for "orders by sku"
# Primary: PK=user_id, SK=order_id#created_at (per-user timeline)
# GSI1:    PK=sku, SK=created_at (per-sku lookup ☐ pay write amplific
ation)
```

```

# dynamodb-orders.yaml – DynamoDB (AWS, 2024) with GSI
AWSTemplateFormatVersion: '2010-09-09'
Resources:
  OrdersTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: orders
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - { AttributeName: user_id, AttributeType: S }
        - { AttributeName: sk, AttributeType: S } #
order_id#created_at
        - { AttributeName: sku, AttributeType: S } # GSI PK
        - { AttributeName: created_at, AttributeType: S } # GSI SK
      KeySchema:
        - { AttributeName: user_id, KeyType: HASH }
        - { AttributeName: sk, KeyType: RANGE }
      GlobalSecondaryIndexes:
        - IndexName: gsil-sku-time
          KeySchema:
            - { AttributeName: sku, KeyType: HASH }
            - { AttributeName: created_at, KeyType: RANGE }
          Projection: { ProjectionType: INCLUDE, NonKeyAttributes: [total, status] }
          PointInTimeRecoverySpecification: { PointInTimeRecoveryEnabled: true }
      SSESpecification: { SSEEnabled: true }

```

GSI lag is a modeling consideration, not just an operational one — a GSI is eventually consistent (typically single-digit ms, occasionally seconds under burst). If the application requires read-after-write on the GSI path, it must read the primary or tolerate staleness.

NoSQL families — choosing the shape

No single store handles all shapes well. The family determines the write path, the read path, and the evolution cost.

| Family | Shape | Write path | Read path | Evolution | When it wins |
|--|---------------------|---------------------|--|------------------------------|---|
| Relational
(Postgres, MySQL) | Tables, rows, joins | B-tree or LSM, ACID | Joins, secondary indexes, window functions | Migrations (expand-contract) | Strong invariants, ad-hoc queries, transactions |

| Family | Shape | Write path | Read path | Evolution | When it wins |
|--|---|---------------------------------------|---|---------------------------------------|--|
| Document (MongoDB, DynamoDB) | JSON/ BSON per item | Per-document atomic | Key + secondary indexes, aggregation pipeline | Schemaless per document, app-enforced | Evolving schema, per-entity locality |
| Wide-column (Cassandra, ScyllaDB, Bigtable) | Partition key + clustering columns, wide rows | LSM, append-only, tunable consistency | Partition-local range scans; no joins | Add columns cheaply; no renames | Write-heavy, time series, partition-local scans |
| Key-value (Redis, DynamoDB KV mode) | Opaque value per key | In-memory or LSM | Point lookup only | Opaque — app owns shape | Cache, session, feature flags, counters |
| Graph (Neo4j, Neptune) | Nodes, edges, properties | Index on labels/edges | Traversals, path queries | Add labels/ props cheaply | Highly connected queries (fraud, recommendation) |
| Search (Elasticsearch, OpenSearch) | Inverted index + stored fields | Segment merge (LSM-like) | Full-text, faceting, geo | Mapping evolution is painful | Full-text, log analytics |
| Time-series (ClickHouse, TimescaleDB) | Metrics + tags + timestamp | Columnar LSM, compression | Range + aggregation over time | Add columns/ tags | Metrics, logs, telemetry |

Wide-column example — the shape that makes Cassandra fast is also the shape that makes it inflexible:

```

-- cassandra-time-series.cql  Cassandra 4.1 / ScyllaDB 5.4 (CQL)
-- Model the query: "all readings for device 42 in the last hour,
newest first"
CREATE TABLE device_readings (
    device_id uuid,
    -- bucket by hour to bound partition size (avoid unbounded
partitions)
    hour_bucket timestamp,          -- truncated to hour
    ts timestamp,
    metric text,
    value double,
    tags map<text,text>,
    PRIMARY KEY ((device_id, hour_bucket), ts, metric)
) WITH CLUSTERING ORDER BY (ts DESC, metric ASC)
    AND compaction = {'class': 'TimeWindowCompactionStrategy',
                      'compaction window unit': 'HOURS',
                      'compaction window size': '1'}
    AND gc_grace_seconds = 86400;

-- Point read: one partition, one range slice  fast (single replica s
et)
SELECT * FROM device_readings
WHERE device_id = ? AND hour_bucket = ? AND ts >= ? AND ts < ?;

-- Anti-pattern: WHERE metric = 'cpu' without device_id  scatter-
gather across all partitions
-- Fix: denormalize into a second table partitioned by (metric, hour_bu
cket) if that query is hot

```

Document example — flexible but with its own discipline:

```

// mongodb-orders.js – MongoDB 7.0 – document shape with schema
discipline
// One document per order; items embedded (no join), user denormalized
minimally
{
  _id: ObjectId("..."),
  orderId: "ord_9f3alc",
  userId: "user_42",
  status: "paid", // indexed
  createdAt: ISODate("2024-08-15T14:02:00Z"), // indexed
  items: [ // embedded – one fetch, no join
    { sku: "A123", qty: 2, unitPrice: 19.99 },
    { sku: "B456", qty: 1, unitPrice: 49.00 }
  ],
  total: 88.98,
  // schema version – application enforces evolution, not the database
  schemaVersion: 3
}
db.orders.createIndex({ userId: 1, createdAt: -1 });
db.orders.createIndex({ status: 1, createdAt: -1 }); // for
operational queries

// Query: per-user timeline – covered by first index, no aggregation
needed
db.orders.find({ userId: "user_42" }).sort({ createdAt: -1 }).limit(20)
;

```

The document trade-off: embedding avoids joins and keeps the hot path local, but unbounded arrays (e.g., embedding thousands of events per order) create large documents that are expensive to update (rewrite the whole document) and exceed the 16 MB limit. Rule: embed when the child is always fetched with the parent and is bounded; reference when it is large, shared, or independently queried.

Time-series and search shapes

Time-series data is append-only, naturally partitioned by time, and queried by range + aggregation. Columnar stores (ClickHouse, DuckDB, Redshift) compress it 5–20× and scan it 10–100× faster than row stores for analytical queries, at the cost of expensive point updates.

Search data is write-once, read-many, and queried by relevance, not key. Inverted indexes (Lucene/Elasticsearch) tokenize, stem, and index every

term — powerful for full-text, but mapping changes (e.g., changing a `text` field to `keyword`) require reindexing.

Both are typically *derived* stores — populated from the primary via CDC or a stream (Kafka → ClickHouse/Elasticsearch) — not primaries. The primary owns durability and transactions; the derived store owns query speed for a specific pattern. That separation is CQRS in practice.

Secondary indexes, materialized views, and CQRS

A secondary index answers "the same data, different key" without duplicating application logic. A materialized view pre-joins or pre-aggregates for a specific query. CQRS generalizes both: separate read models, each optimized for one query, populated asynchronously from the write model.

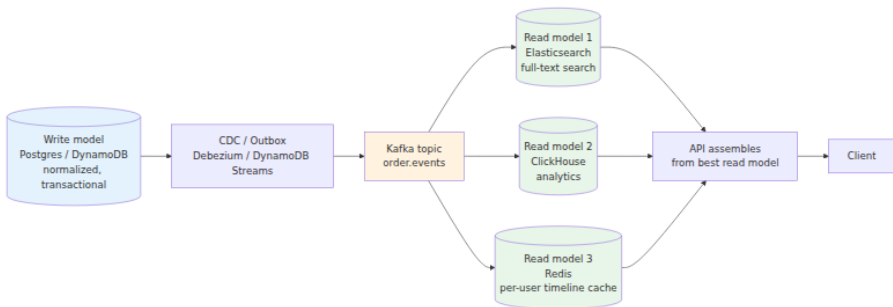


Figure 5-3: CQRS with CDC. The write model is normalized and transactional; each read model is denormalized for one hot query and lags the write model by milliseconds to seconds. The API picks the right read model per endpoint.

The cost is **lag and divergence**. Each read model is eventually consistent with the write model; a user who writes and immediately reads via a different model may see stale data. Mitigations: read-after-write routing (read the primary for the writer's own data for N seconds), version vectors, or `read-your-writes` via a cache that the writer populates.

Materialized views inside the database (Postgres `MATERIALIZED VIEW`, Cassandra MV — the latter notoriously problematic in production) are simpler but couple the read model to the primary's availability and

scaling. External read models scale independently at the cost of operational complexity.

Schema evolution — compatibility as a contract

Every long-lived system changes its schema while old and new code run simultaneously (rolling deploys, consumers lagging producers). Compatibility is not optional — it is a deployment invariant.

Avro and Protobuf compatibility rules

| Change | Avro | Protobuf (proto3) | Safe? |
|------------------------------------|--------------------------------------|------------------------------------|-----------------------------|
| Add optional field with default | Forward + backward | Add optional or with default | Yes — the compatible change |
| Remove optional field | Forward breaks if reader requires it | Reserved field number, never reuse | Conditional |
| Rename field | Breaks (name matters in Avro) | Safe (number matters, not name) | Avro no, Protobuf yes |
| Change type (int → string) | Breaks | Breaks (wire type changes) | No |
| Add required field without default | Breaks | Breaks | No |

```

// order.proto – Protobuf 3 (protoc 4.x), evolution-safe
syntax = "proto3";
package orders.v1;

message Order {
    string order_id = 1;
    string user_id = 2;
    // status as enum – adding values is safe; removing/renumbering is
    not
    enum Status {
        STATUS_UNSPECIFIED = 0;
        PENDING = 1;
        PAID = 2;
        SHIPPED = 3;
        CANCELLED = 4;
        // never reuse numbers; reserve removed values
        reserved 5 to 10;
    }
    Status status = 3;
    repeated Item items = 4;
    int64 created_at_ms = 5; // epoch millis – adding is safe

    // Added in v1.1 – optional with presence tracking
    optional string coupon_code = 6;
    // Added in v1.2 – new message, old readers ignore it
    Discount discount = 7;

    message Item {
        string sku = 1;
        int32 qty = 2;
        // Prices as string or int64 cents – never float (precision)
        int64 unit_price_cents = 3;
    }
    message Discount {
        int64 amount_cents = 1;
        string reason = 2;
    }
}

```



```
// Avro schema evolution – orders.avsc (Avro 1.11)
{
  "type": "record", "name": "Order", "namespace": "orders.v1",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "user_id", "type": "string"},
    {"name": "status", "type": {"type": "enum", "name": "Status",
      "symbols": ["PENDING", "PAID", "SHIPPED", "CANCELLED"]}},
    {"name": "items", "type": {"type": "array", "items": {
      "type": "record", "name": "Item",
      "fields": [
        {"name": "sku", "type": "string"},
        {"name": "qty", "type": "int"},
        {"name": "unit_price_cents", "type": "long"}
      ]}},
    {"name": "created_at_ms", "type": "long"},
    {"name": "coupon_code", "type": ["null", "string"], "default": null}
  ],
  {
    "name": "discount", "type": ["null", {
      "type": "record", "name": "Discount",
      "fields": [
        {"name": "amount_cents", "type": "long"},
        {"name": "reason", "type": "string"}
      ]}], "default": null}
  ]
}
```

Expand-contract (parallel change)

The only safe migration pattern for zero-downtime evolution. No flag day.

Phase 1  **Expand:** add new column/field, dual-write, backfill
 DB: ALTER TABLE orders ADD COLUMN coupon_code TEXT; -- nullable, no default rewrite
 App: write both old and new; read old
 Backfill: UPDATE orders SET coupon_code = ... WHERE coupon_code IS NULL (batched, throttled)

Phase 2  **Migrate:** switch reads to new
 App: read new, write both; verify parity (shadow read, compare)

Phase 3  **Contract:** remove old
 App: stop writing old
 DB: ALTER TABLE orders DROP COLUMN old_coupon; -- after no reader needs it

```

-- expand_contract.sql – PostgreSQL 16, zero-downtime column rename
(really: add + backfill + drop)
-- Step 1: expand – add new column, nullable, no table rewrite
(metadata-only in PG 11+)
ALTER TABLE orders ADD COLUMN coupon_code TEXT;

-- Step 2: backfill in batches (avoid long transactions that bloat WAL
and hold locks)
-- Run from a job, not a single UPDATE
-- pseudocode for batched backfill (Python, psycopg 3.1):
-- while True:
--     n = cur.execute("UPDATE orders SET coupon_code = legacy_coupon "
--                     "WHERE order_id IN (SELECT order_id FROM orders "
--                     "WHERE coupon_code IS NULL LIMIT 1000) RETURNING
1").rowcount
--     conn.commit()
--     if n == 0: break
--     time.sleep(0.05) -- throttle to avoid replica lag

-- Step 3: contract – after all readers use coupon_code, drop old
-- ALTER TABLE orders DROP COLUMN legacy_coupon;

```

For Protobuf/Avro over Kafka, the Schema Registry (Confluent 7.6, Apicurio 2.5) enforces compatibility at produce time — a producer cannot publish a schema that breaks the configured `BACKWARD` or `FORWARD` rule.

Polyglot persistence without a distributed monolith

Polyglot persistence — using the right store per service — is correct and dangerous. Correct because no single store is optimal for transactions, search, analytics, and caching simultaneously. Dangerous because each additional store is an operational burden and a consistency boundary, and "each service picks its store" without ownership discipline becomes a distributed monolith where every service joins every other service's tables.

The discipline:

- **One service, one primary store** it owns and is the sole writer to. Other services read via API or async events, not direct DB access.
- **Derived stores are populated via CDC/stream**, not dual writes. Dual writes without a transaction (or outbox) lose data on partial failure.

- **Shared-nothing by default.** If two services *must* share a database (rare, e.g., legacy), they share nothing at the table level — separate schemas, no cross-service joins, no foreign keys across service boundaries — so the split can be completed later.

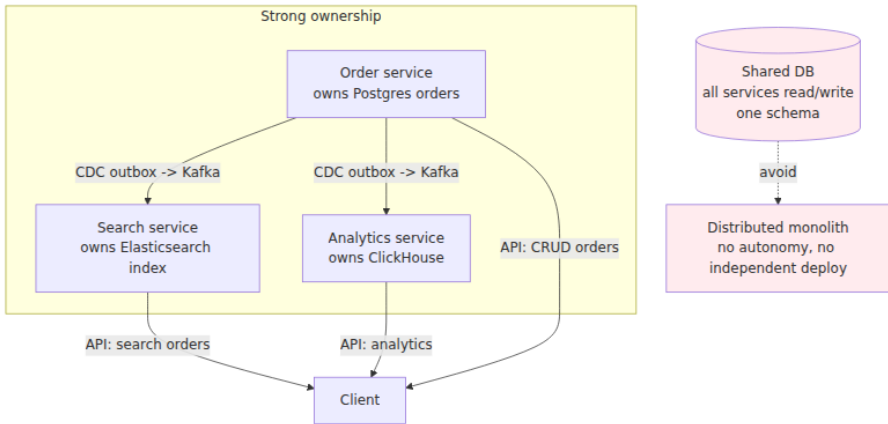


Figure 5-4: Polyglot persistence with strong ownership. Each service owns its store; derived stores are populated via streams, not shared tables. The anti-pattern — a shared database with cross-service joins — couples deploy, scale, and failure.

Distributed-systems lens

Data modeling is where distributed-systems trade-offs become schema.

Partitioning is a consistency boundary. A transaction that touches one partition is local and fast; one that touches N partitions requires 2PC or Saga and is slow and failure-prone (Vol 6, Chapters 5–6; Vol 5, Chapter 10). Every cross-partition query you model in is a distributed transaction you will operate at 3 AM. The model should make the common case single-partition and make cross-partition explicit and rare.

Denormalization is replication. Two copies of the same fact (normalized row plus denormalized read model) must be kept consistent — which is the replication problem (Vol 6, Chapter 3) at the application layer. CDC/outbox gives at-least-once delivery; consumers must be idempotent. Without idempotency, a retried CDC event doubles a counter.

Schema evolution is a rolling-deploy problem. Old and new code coexist for minutes to hours. A schema change that is not forward- and backward-compatible turns a rolling deploy into a deserialization outage — new producers write a field old consumers cannot read, or old producers omit a field new consumers require. Compatibility rules are not bureaucratic — they are the contract that makes independent deploys safe (Vol 11, Chapter 9).

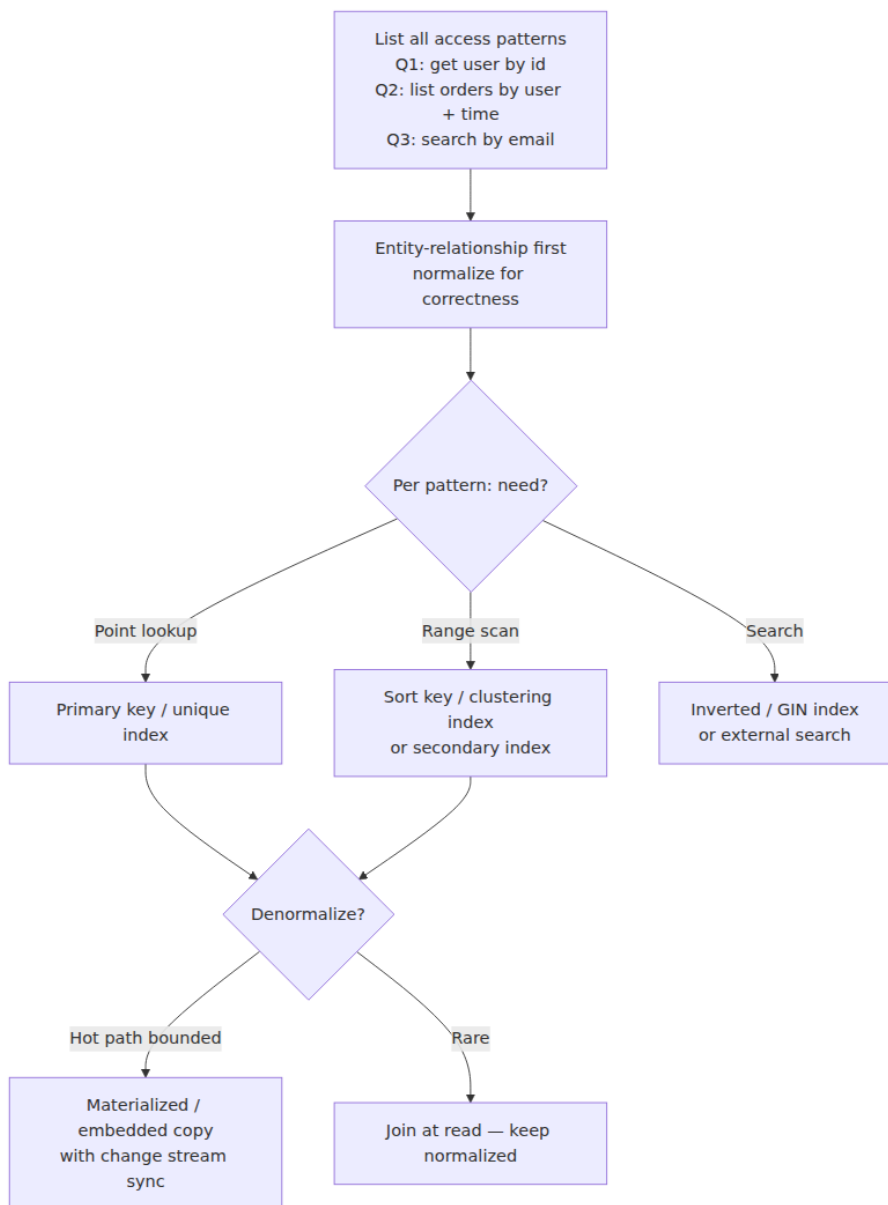
Ownership is a failure-domain boundary. A service that reads another service's tables directly inherits that service's availability, scaling, and migration constraints. When the owning service needs to re-partition, the reader breaks. API or event-based integration decouples fate at the cost of latency and staleness — a deliberate, per-endpoint trade-off.

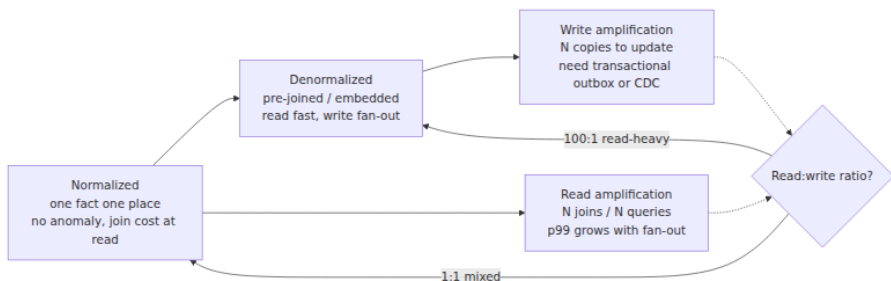
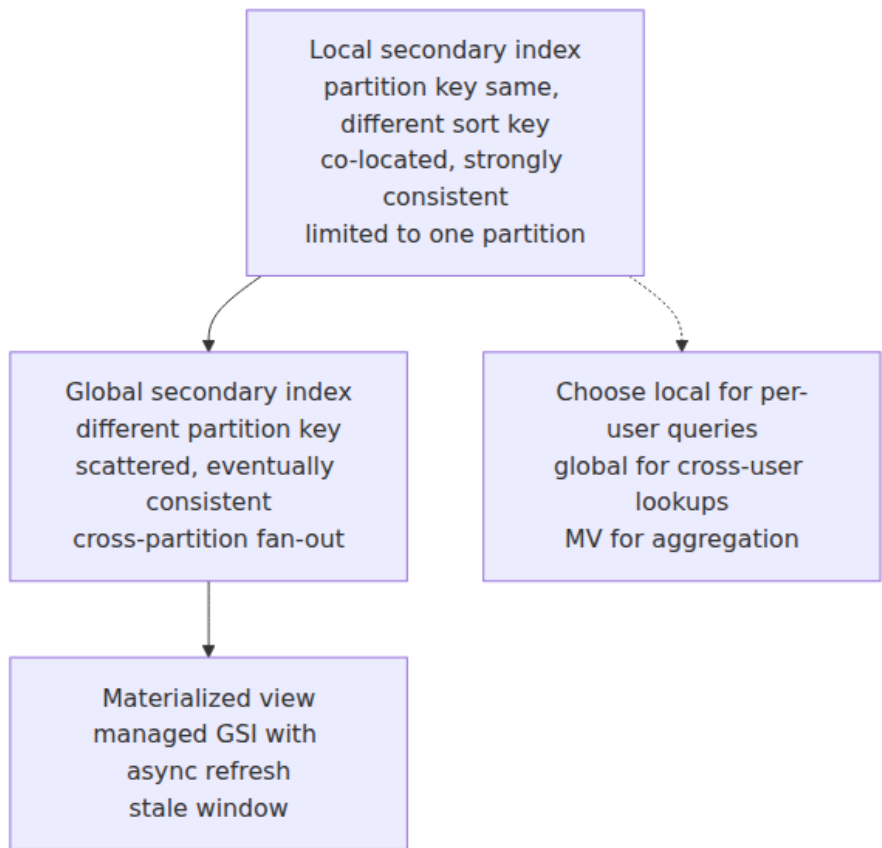
Key takeaways

- Model the hot queries first — their filters select the partition key, their projections select the indexes and denormalization, their consistency needs select the replication strategy. Entities alone do not determine the model.
- Normalize until the hot read path hurts, then denormalize deliberately with a CDC/outbox plan for keeping copies consistent. Every denormalized copy is a replication problem.
- The partition key must be high-cardinality, uniform, and query-local. Low-cardinality keys create hot partitions; keys that do not match the hot filter create scatter-gather. Mitigate skew with salting, time-bucketing, or bounded-load hashing.
- Pick the store family per service by write/read shape: relational for invariants and joins, document for evolving per-entity locality, wide-column for write-heavy partition-local scans, search for full-text, columnar for time-series analytics. Polyglot persistence is correct when each service owns one primary store.
- Secondary indexes, materialized views, and CQRS read models all answer "same data, different key" — each adds write amplification and lag. Design for the lag (read-after-write routing, staleness budgets) rather than pretending it does not exist.
- Evolve schemas via expand-contract with forward/backward compatibility (Avro/Protobuf rules, Schema Registry enforcement).

No flag-day migrations; old and new code must coexist during rolling deploys.

- One service, one primary store, sole writer. Derived stores via CDC/stream, not dual writes or shared tables. Cross-service table access is a distributed monolith.





Further reading

- Kleppmann, M. *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapters 2–4, 6 — data models, encoding/evolution, partitioning, replication. <https://dataintensive.net/>
- Sadalage, P. and Fowler, M. *NoSQL Distilled* (Addison-Wesley, 2012) — concise family comparison and modeling guidance.
- Confluent Schema Registry — Avro/Protobuf/JSON Schema compatibility rules (Confluent Platform 7.6). <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>
- Protobuf Language Specification — proto3, field numbers, reserved, compatibility (protoc 4.x). <https://protobuf.dev/programming-guides/proto3/>
- Fowler, M. "Parallel Change" (expand-contract) — the migration pattern. <https://martinfowler.com/bliki/ParallelChange.html>
- Kleppmann, M. et al. "Turning the Database Inside Out with Apache Samza" (2014) — CDC-driven CQRS and stream-table duality. <https://samza.apache.org/learn/documentation/0.7.0/container/streams-and-tables.html>
- DynamoDB Best Practices — partition keys, GSIs, adaptive capacity (AWS, 2024). <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key.html>
- Cassandra Data Modeling — CQL, partition keys, clustering order, compaction strategies (Cassandra 4.1 / ScyllaDB 5.4). https://cassandra.apache.org/doc/latest/cassandra/data_modeling/

Next: Chapter 6 — Monolith, Microservices, and Between — takes the data and traffic decisions from Chapters 4–5 and asks how to carve them

into deployable, team-owned units without creating a distributed monolith that is worse than the monolith it replaced.

Chapter 6 — Monolith, Microservices, and Between

What this chapter covers. Chapters 4 and 5 decided how traffic moves and how data is shaped; this chapter decides how those decisions are packaged into *deployable, team-owned units*. The monolith-versus-microservices debate is usually framed as a binary, which is why it produces bad decisions. In reality there is a spectrum — single-deployable monolith, modular monolith, service-oriented architecture, microservices, and the distributed monolith you get when you do microservices badly — and the right point on that spectrum is determined by team scale, domain boundaries, and operational maturity, not by fashion. We define each shape precisely, compare them on deploy, scale, failure, and team autonomy, then make the hard parts operational: how to carve boundaries (domain-driven design, bounded contexts), how to decompose without a big-bang rewrite (strangler fig, branch by abstraction, anti-corruption layer), how to own data per service (database per service, Saga versus 2PC), and how to observe and deploy a fleet of services (service mesh, contract testing, progressive delivery). The chapter closes with the distributed-systems lens on why the monolith is often the correct starting point and when it stops being so.

Learning goals — after this chapter you should be able to:

- Distinguish monolith, modular monolith, SOA, microservices, and distributed monolith on deploy, data, and team axes, and name the failure mode of each.
- Apply Conway's Law and bounded contexts to propose service boundaries from a domain model, and identify boundaries that will chatter.
- Plan a strangler-fig migration with branch-by-abstraction, anti-corruption layers, and expand-contract data splits — no big-bang rewrites.

- Choose per-service data ownership (database per service) and cross-service consistency (Saga/choreography/orchestration versus 2PC) per workflow, and explain the operational cost of each.
- Design inter-service communication — sync (gRPC/HTTP) versus async (events) — and apply idempotency, deadline propagation, and retry budgets at service boundaries.
- Operate a service fleet: service discovery, mesh (Envoy/Istio), contract testing, and progressive delivery with automated rollback.
- Argue when *not* to split — the organizational and operational preconditions for microservices and the cost of splitting too early.

The spectrum, not the binary

"Monolith versus microservices" as a binary hides the most useful options in the middle. Five shapes matter; most real systems live in the middle two and aspire badly to the last.

| Shape | Deploy units | Data | Team coupling | Scales to | Characteristic failure |
|-------------------------------|--|--|---------------------------------------|----------------------------------|--|
| Monolith | 1 | 1 DB, shared schema | All teams on one repo/ deploy | ~20–40 engineers, ~1k RPS per AZ | Merge conflicts, deploy queue, single failure domain |
| Modular monolith | 1 deploy, N modules with enforced boundaries | 1 DB, schema per module, no cross-module joins | Teams own modules, shared deploy | ~40–80 engineers | Boundary erosion without enforcement |
| Service-oriented (SOA) | ~5–20 services, coarse-grained | DB per service, sync calls | Teams own services, shared governance | ~80–200 engineers | Distributed monolith if boundaries are wrong |

| Shape | Deploy units | Data | Team coupling | Scales to | Characterist failure |
|-----------------------------|--------------------------------|--|--|-----------------------|--|
| Microservices | 20-200+ services, fine-grained | DB per service, events + APIs | Teams own services + data + deploy, fully autonomous | 200+ engineers | Operational complexity, observability, cost, eventual consistency everywhere |
| Distributed monolith | N services, tightly coupled | Shared DB or N DBs with distributed transactions | Teams cannot deploy independently | Any scale — but badly | Same coupling as monolith with added network latency and partial failure |

The distributed monolith is not a shape you choose — it is a shape you *get* when you split a monolith along the wrong boundaries, keep a shared database, or make every service synchronously call every other service on the hot path. It has the worst of both: the coupling of a monolith and the failure modes of a distributed system.

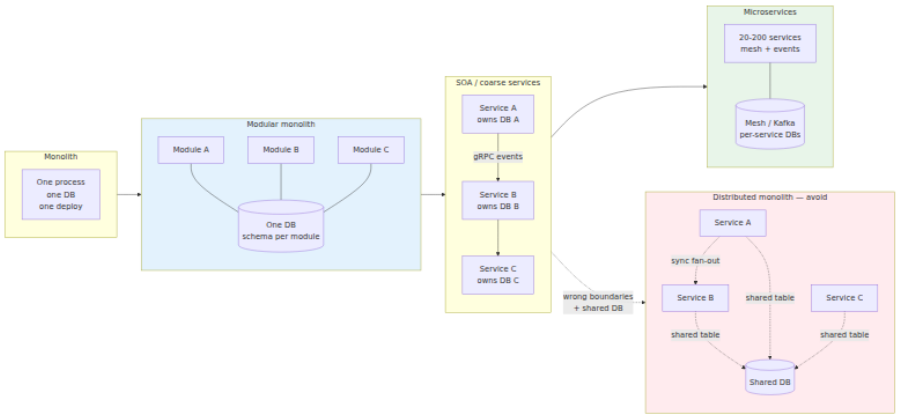


Figure 6-1: The architecture spectrum and the cliff. Movement right increases team autonomy but also operational cost; the distributed monolith is the failure mode of moving right without fixing data ownership or boundaries.

Boundary note. Service communication wire mechanics (HTTP/2, gRPC streaming, REST semantics, versioning) are in Vol 8 — APIs and Service

Design. Transactional outbox, CDC, and exactly-once stream processing are in Vol 10 — Messaging, Streaming, and Event Systems. This chapter treats the *packaging and ownership* decision; those volumes treat the *communication and consistency* mechanisms.

When to split — and when not to

Conway's Law — "organizations design systems that mirror their communication structures" — is usually quoted as a warning; it is better used as a planning tool. If three teams constantly merge-conflict on one codebase and queue behind one deploy, the system *already* wants three deployables — the architecture is lagging the organization. If one team of six owns the whole product, splitting it into twelve services creates twelve deploys for one team — pure overhead.

| Signal | Interpretation |
|---|--|
| Small team (< 20 engineers), few deploys/week, one domain | Stay monolith or modular monolith. Splitting adds latency, failure modes, and on-call load for no autonomy gain. |
| Merge queue > 1 day, deploy queue blocks hotfixes | Modular monolith first; extract the hottest contention point as a service if needed. |
| One component needs independent scaling (10× others) or independent failure domain (payments) | Extract that component — the scaling/failure boundary is a service boundary. |
| Multiple teams, independent roadmaps, need to deploy without coordinating | Services per bounded context — the team boundary is a service boundary. |
| Regulatory or blast-radius isolation (PII, payments) | Separate service + data store regardless of team size — the compliance boundary is a service boundary. |

Shopify, Stack Overflow, and Basecamp run monoliths or modular monoliths at very large scale — hundreds of engineers, millions of RPS — because their domains and teams allow it. Uber and Netflix run hundreds of microservices because their team scale and domain diversity require it.

Both are correct *for their context*. The error is cargo-culting one into the other's context.

Operational preconditions for microservices. Do not split until you have, at minimum: automated deploys with progressive delivery and rollback (Vol 11, Chapter 9), distributed tracing and correlated logs (Vol 11, Chapters 3-4), service discovery and health checking (Chapter 4), and per-service runbooks and SLOs. Without these, each new service *decreases* availability.

Finding boundaries — bounded contexts

Domain-driven design (DDD — Evans, 2003) gives the vocabulary: a **bounded context** is a domain boundary within which a single ubiquitous language and a single model are consistent. `Order` means one thing in the ordering context (a cart being built) and a different thing in the fulfillment context (a shipment to track) — that linguistic seam is a service seam.

Heuristics for boundaries that will *not* chatter:

- **Lifecycle cohesion** — entities created, updated, and deleted together belong together. Orders and order items share a lifecycle; orders and user profiles do not.
- **Invariant scope** — a constraint that must be transactional ("an order's total equals the sum of its items") must live inside one service. Constraints that can be eventual ("inventory eventually reflects orders") can span services.
- **Rate and scale mismatch** — a write-heavy ingest path and a read-heavy serving path want different stores and scaling policies — separate them even if the entities overlap.
- **Team ownership** — if two teams will own the data on either side of a boundary, make the boundary explicit and contractual (API + events), not a shared table.

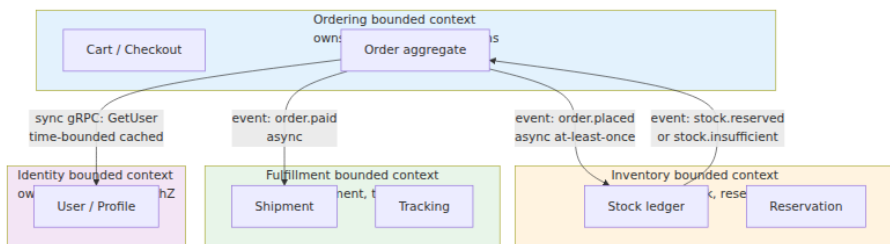


Figure 6-2: Bounded contexts for an e-commerce slice. Ordering, inventory, fulfillment, and identity each own their data and invariants; cross-context communication is via events (eventual) or bounded sync calls (with cache and timeout). The **Order** aggregate appears only in ordering — other contexts see projections.

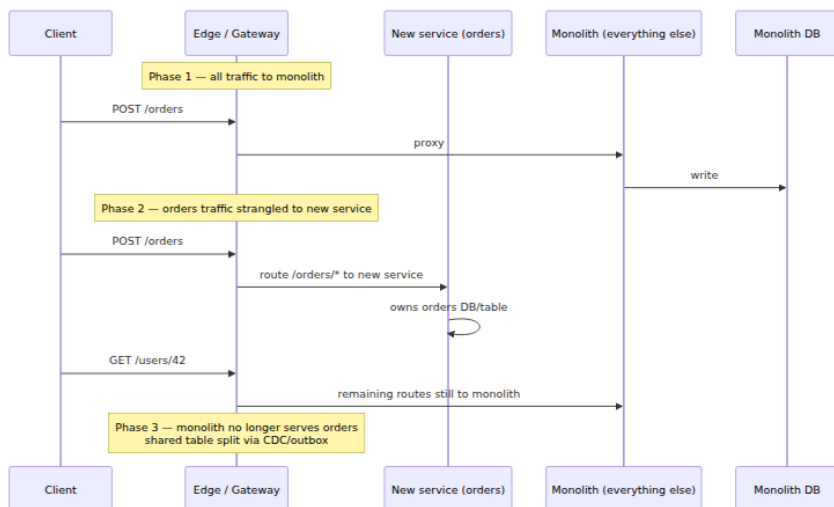
Boundaries that chatter — a service calling another in a tight loop per request — are a sign the boundary is wrong. If service A *always* calls service B to complete a request, they may be one service, or the data B owns should be replicated to A via CDC.

Decomposing without a big bang

Big-bang rewrites fail at a rate approaching certainty. Three patterns make strangulation incremental and reversible.

Strangler fig

Incrementally intercept traffic at the edge and route it to the new service, while the monolith continues to serve the rest. Over months, the monolith shrinks.



Edge routing for strangulation is a Gateway API or Envoy route — the same primitive as Chapter 4's traffic management:

```
# strangler-gateway.yaml – Gateway API v1.1, strangling /orders/* to
new service
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: orders-strangler, namespace: default }
spec:
  parentRefs: [{ name: edge }]
  hostnames: ["api.example.com"]
  rules:
    - matches: [{ path: { type: PathPrefix, value: /orders } }]
      backendRefs:
        - name: orders-service      # new service
          port: 80
          weight: 100
    - matches: [{ path: { type: PathPrefix, value: / } }]
      backendRefs:
        - name: monolith            # everything else still to monolith
          port: 80
          weight: 100
```

Branch by abstraction

When the extraction requires changing call sites *inside* the monolith, introduce an abstraction (interface) over the old implementation, add the

new implementation behind it, switch callers one by one, then remove the old.

```
// BranchByAbstraction.java – Java 17, strangling a monolith module
interface OrderRepository {
    Order findById(String id);
    void save(Order order);
}

// Old: direct DB access inside monolith
class MonolithOrderRepository implements OrderRepository {
    public Order findById(String id) { return jdbc.query("SELECT ...
WHERE id=?", id); }
    public void save(Order o) { jdbc.update("INSERT ...",
o.fields()); }
}

// New: calls extracted service via gRPC (behind same interface)
class ServiceOrderRepository implements OrderRepository {
    private final OrderServiceGrpc.OrderServiceBlockingStub stub;
    public Order findById(String id) {
        return fromProto(stub.getOrder(GetOrderRequest.newBuilder().set
Id(id).build()));
    }
    public void save(Order o) {
        stub.createOrder(toProto(o));
    }
}

// Wiring – feature-flagged switch, per-request or per-instance
OrderRepository repo = featureFlags.enabled("orders-service-
extraction", userId)
    ? new ServiceOrderRepository(stub)
    : new MonolithOrderRepository(jdbc);
```

Anti-corruption layer (ACL)

When the new service's model differs from the monolith's (and it should — that is the point of a bounded context), an ACL translates between them so neither model is polluted by the other. The ACL lives in the new service or as a sidecar, not in the monolith.

Monolith model: Order { id, userId, legacyCoupon, totalCents, statusInt }

 ACL translates

New model: Order { orderId, userId, couponCode, total { amount, cu
rrency }, status enum }

Data split — the hardest part

Code splits are reversible; data splits are not (easily). The safe sequence:

1. Monolith owns `table orders` (single DB)
 2. Add `new` service with its own `table orders_new` (same DB, separate schema) `→` dual-write via outbox
 3. Backfill `orders_new` from `orders` (batched, throttled)
 4. Switch `reads to new` service (shadow `read` + compare)
 5. Move `orders_new` `to` a separate `database` instance (logical split `→` physical split)
 6. `Stop` dual-write; monolith `table` becomes `read-only`, `then` is dropped

Each step is independently deployable and reversible until step 6. The outbox (Vol 10, Chapter 6) is load-bearing — without it, a crash between the two writes loses data.

Data per service and cross-service consistency

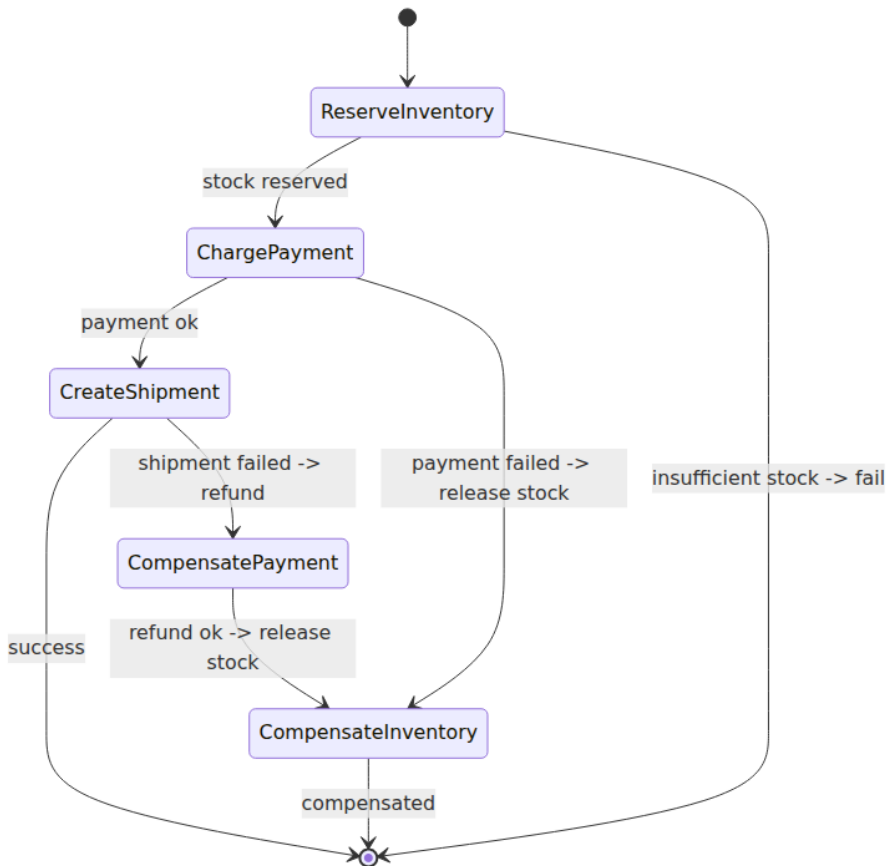
Database per service is the rule that makes services independently deployable and scalable. Shared databases reintroduce the coupling that services were meant to remove — a schema change in one service breaks another, and a slow query in one service contends with all.

Cross-service workflows then need a consistency strategy:

| Pattern | Consistency | Availability during partition | Complexity | When to use |
|---------------------|------------------------------|------------------------------------|-------------------------------------|---|
| 2PC / XA | Strong (atomic commit) | Blocks — coordinator is SPOF | High; rarely used in microservices | Only when strong cross-service atomicity is truly required (rare) |
| Saga (orchestrated) | Eventual, with compensations | Available — compensates on failure | Medium — orchestrator state machine | Multi-step workflows with clear compensations (order → payment → inventory) |

| Pattern | Consistency | Availability during partition | Complexity | When to use |
|--|----------------------|---------------------------------|---|-------------------------------------|
| Saga (choreographed) | Eventual, via events | Available — each service reacts | Low per service, high emergent complexity | Few participants, simple flows |
| Outbox + CDC + idempotent consumers | Eventual | Available | Low — the default for most flows | Any async cross-service side effect |

Saga example — orchestrated, with compensations:



```

# saga_orchestrator.py – orchestrated saga (Python 3.11, durable via DB
+ outbox)
import uuid, json, psycpg

def place_order_saga(user_id: str, items: list[dict]) -> str:
    order_id = str(uuid.uuid4())
    saga_id = str(uuid.uuid4())
    # orchestrator persists state – crash-recoverable
    db.execute("INSERT INTO saga_state (saga_id, order_id, step,
status) VALUES (%s,%s,'reserve','running'",
                (saga_id, order_id))

    try:
        inventory.reserve(order_id, items)                # step 1 –
        # compensable
        db.execute("UPDATE saga_state SET step='charge' WHERE saga_id=%s", (saga_id,))
        payment.charge(user_id, order_id, items)          # step 2 –
        # compensable (refund)
        db.execute("UPDATE saga_state SET step='ship' WHERE
saga_id=%s", (saga_id,))
        fulfillment.create_shipment(order_id)              # step 3
        db.execute("UPDATE saga_state SET status='succeeded' WHERE
saga_id=%s", (saga_id,))
        return order_id
    except InventoryError:
        db.execute("UPDATE saga_state SET
status='failed_insufficient_stock' WHERE saga_id=%s", (saga_id,))
        raise
    except PaymentError:
        inventory.release(order_id)
        # compensate step 1
        db.execute("UPDATE saga_state SET
status='compensated_payment_failed' WHERE saga_id=%s", (saga_id,))
        raise
    except FulfillmentError:
        payment.refund(order_id)
        # compensate step 2
        inventory.release(order_id)
        # compensate step 1
        db.execute("UPDATE saga_state SET
status='compensated_fulfillment_failed' WHERE saga_id=%s", (saga_id,))
        raise

```

Choreographed sagas (each service listens for events and acts) are simpler per service but harder to reason about — there is no single place that shows the workflow, and debugging a stuck saga requires correlating logs across N services. Prefer orchestration when the workflow has more than three steps or needs visible state for operations.

Communication — sync versus async at service boundaries

| Mode | Latency | Coupling | Failure handling | When |
|--|-------------------------|--------------------|--|--|
| Sync
gRPC/
HTTP | Low (one RTT + handler) | Temporal + spatial | Timeout, retry budget, circuit breaker | Read path, user-waiting writes |
| Async
events
(Kafka/
SQS) | Higher (enqueue + poll) | Decoupled | Retry, DLQ, idempotent consumer | Side effects, cross-context workflows, fan-out |

Both need the same three disciplines at every boundary:

Idempotency. Every handler must tolerate duplicate delivery — at-least-once is the only practical guarantee for queues and for retried RPCs. Key by business idempotency key (`order_id`, `Idempotency-Key` header), not by transport dedup.

```
# idempotent_handler.py – Python 3.11, Postgres-backed dedup
def handle_order_placed(event: dict) -> None:
    key = event["order_id"]
    # INSERT ... ON CONFLICT DO NOTHING is the idempotency gate
    n = db.execute(
        "INSERT INTO processed_events (event_id, at) VALUES
        (%s, now()) ON CONFLICT DO NOTHING",
        (key,)
    ).rowcount
    if n == 0:
        return # already processed – duplicate delivery, safe to
    ignore
    # ... actual work ...
    db.commit()
```

Deadline propagation. A user request with a 2 s SLO that fans out to three services cannot give each 2 s. Propagate remaining budget (gRPC `grpc-timeout`, or `x-request-timeout-ms` over HTTP) and enforce it server-side with `context.WithTimeout`.

Retry budgets. Never retry without a budget. Envoy/gRPC retry budgets (Chapter 4) apply at service boundaries too — retries that amplify failure into outage are the most common microservices failure mode.

Operating the fleet

Service discovery and mesh

In Kubernetes, discovery is DNS (`orders.default.svc.cluster.local`) plus EndpointSlices. A service mesh (Istio 1.22, Cilium Service Mesh, Linkerd 2.14) adds L7 policy — mTLS, retries, outlier detection, and traffic splitting — without per-service library changes, by injecting an Envoy sidecar or via eBPF.

```

# istio-traffic.yaml – Istio 1.22: canary + outlier + mTLS
(PeerAuthentication)
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: { name: orders }
spec:
  hosts: [orders.default.svc.cluster.local]
  http:
    - match: [{ headers: { x-canary: { exact: "true" } } }]
      route: [{ destination: { host: orders.default.svc.cluster.local,
subset: canary } }]
    - route:
      - destination: { host: orders.default.svc.cluster.local,
subset: stable }
        weight: 90
      - destination: { host: orders.default.svc.cluster.local,
subset: canary }
        weight: 10
      timeout: 2s
      retries: { attempts: 2, perTryTimeout: 1s, retryOn: "5xx,reset,co
nnect-failure" }
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: orders }
spec:
  host: orders.default.svc.cluster.local
  subsets:
    - name: stable
      labels: { version: stable }
    - name: canary
      labels: { version: canary }
  trafficPolicy:
    loadBalancer: { simple: LEAST_REQUEST }
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 5s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
    connectionPool:
      tcp: { maxConnections: 8192 }
      http: { http1MaxPendingRequests: 4096 }
      tls: { mode: ISTIO_MUTUAL } # mTLS – identity via SPIFFE (Vol 9,
Ch 10)

```

Contract testing

In a fleet, a breaking change in one service breaks its consumers — often silently until production. Consumer-driven contract testing (Pact, Spring

Cloud Contract) makes the contract explicit and verifiable in CI, without requiring integrated end-to-end environments per PR.

```
Consumer (orders) defines expectation: "GET /users/{id} returns {id, email, name}"  
  → Pact file committed  
Provider (users) verifies: "can I still satisfy this Pact?" in CI  
  → break the contract → PR fails before merge
```

Progressive delivery

Every service deploy is a partial outage if done as a rolling restart without draining (Chapter 4). Progressive delivery — canary or blue/green with automated analysis — limits blast radius:

```

# rollout.yaml – Argo Rollouts 1.7 (Kubernetes, progressive delivery)
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata: { name: orders }
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - analysis:
            templates: [{ templateName: success-rate }]
            args: [{ name: service, value: orders }]
        - setWeight: 50
        - pause: { duration: 10m }
        - analysis:
            templates: [{ templateName: success-rate }]
  trafficRouting:
    istio:
      virtualService: { name: orders }
      destinationRule: { name: orders, canarySubsetName: canary,
stableSubsetName: stable }
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: orders
          image: registry.internal/orders:1.43.0
          ports: [{ containerPort: 8080 }]
          lifecycle:
            preStop: { exec: { command: ["/bin/sh", "-c", "sleep 5; /app/
drain --grace 50s"] } }
            readinessProbe: { httpGet: { path: /readyz, port: 8080 },
periodSeconds: 5 }
      ---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: { name: success-rate }
spec:
  metrics:
    - name: success-rate
      successCondition: result[0] >= 0.995
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            sum(rate(http_requests_total{service="{{args.service}}",cod
e=~"2.."}[2m]))
            /

```



```
sum(rate(http_requests_total{service="{{args.service}}"}[2m]
))
```

If success rate drops below 99.5% during the canary, Argo Rollouts automatically rolls back — no human in the loop for the common case.

Distributed-systems lens

Splitting a system changes its failure modes from local to distributed, which is a strict superset.

Partial failure. A monolith fails as a unit — up or down, easy to reason about. A fleet of services fails *partially* — orders is up but inventory is slow, so checkouts degrade but browsing works. Handling partial failure requires the patterns from Chapter 1, Principle 5 at every boundary: timeouts with budgets, retries with budgets, circuit breakers, bulkheads, and fallbacks. Without them, one slow service cascades via thread-pool exhaustion into a site-wide outage.

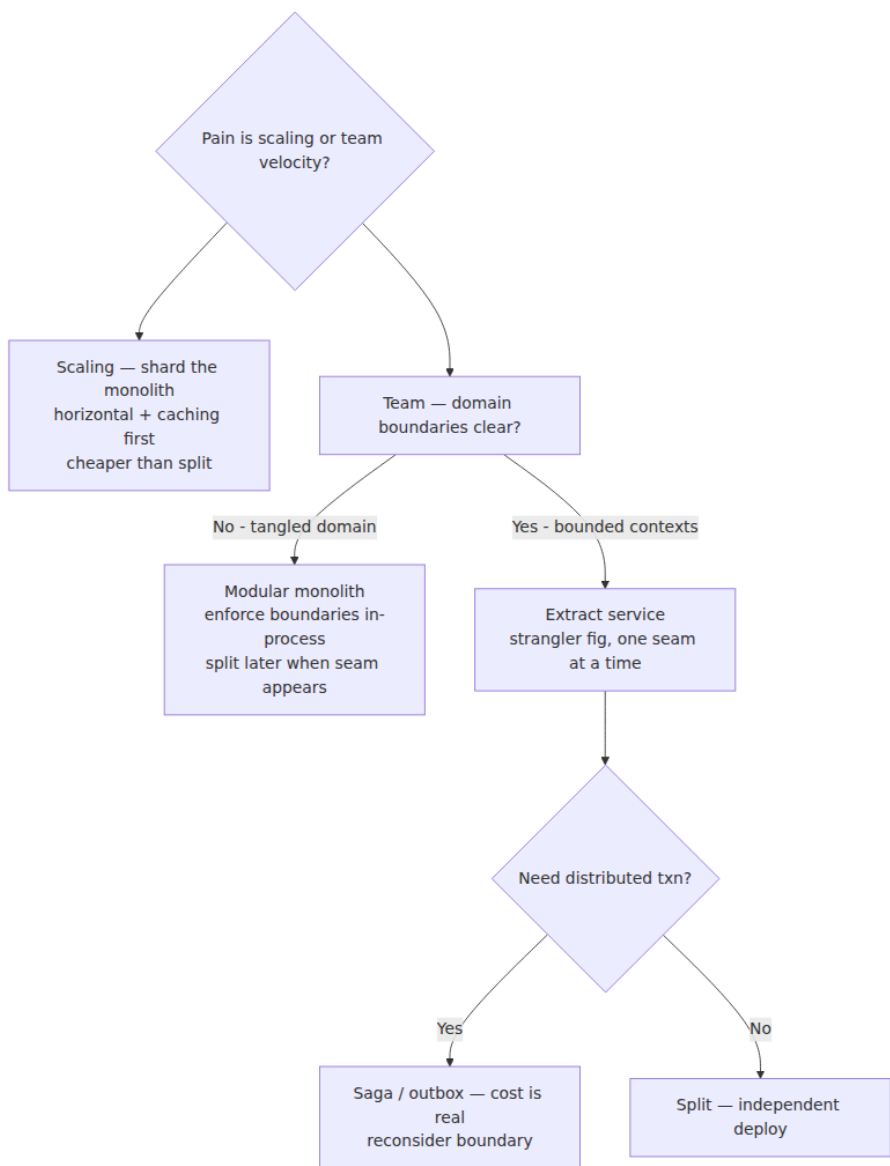
Consistency scope. A monolith's single database gives ACID transactions for free across the whole domain. A service fleet gives ACID only *within* one service; cross-service workflows are eventually consistent and require Sagas or outbox/CDC. The modeling decision (Chapter 5) determines how often cross-service transactions are needed — good boundaries minimize them, bad boundaries make every write a distributed transaction.

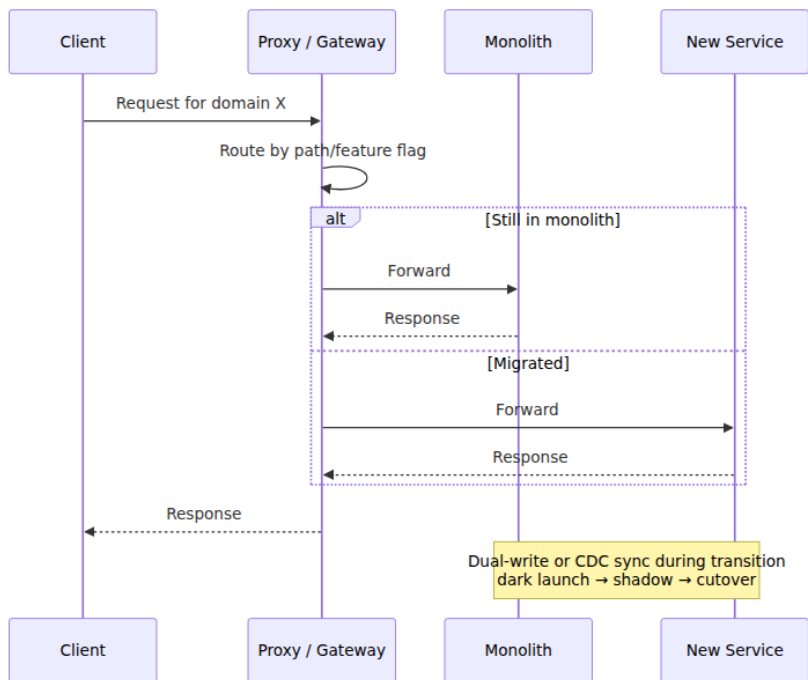
Observability cost. A request that touched one process and one database now touches five services, two queues, and three data stores. Without distributed tracing (trace ID propagated via `traceparent` / `x-request-id` at every hop) and correlated logs (Vol 11, Chapters 3–4), that request is undebuggable. The cost of observability in a service fleet is not optional tooling — it is a prerequisite for operating the system at all.

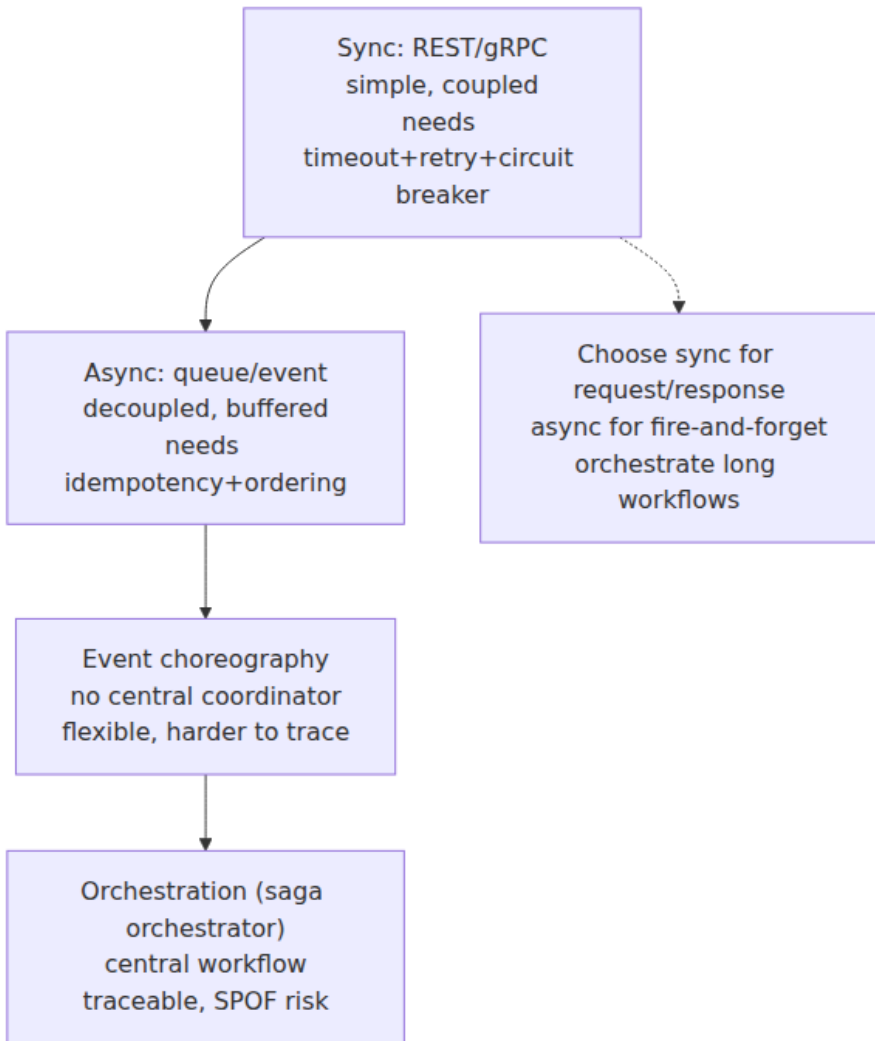
Team autonomy versus coordination cost. Services give teams independent deploys, but every service boundary is a contract that must be versioned, tested, and evolved (Vol 8). The coordination cost grows with the number of boundaries. A modular monolith with enforced module boundaries and a single deploy can give 80% of the autonomy with 20% of the operational cost — which is why it is the right answer for most teams under ~80 engineers.

Key takeaways

- There is a spectrum — monolith, modular monolith, SOA, microservices, distributed monolith — not a binary. The distributed monolith (tightly coupled services with shared data or sync fan-out) is the failure mode to avoid, not a shape to choose.
- Split when team, scaling, or failure-domain boundaries demand it — not before. The preconditions are automated deploys, tracing, discovery, and per-service SLOs; without them, each new service reduces availability.
- Boundaries come from bounded contexts — lifecycle cohesion, invariant scope, and rate/scale mismatch. A boundary that chatters (tight sync loop per request) is the wrong boundary; replicate data via CDC instead.
- Decompose incrementally: strangler fig at the edge, branch by abstraction inside the monolith, anti-corruption layers between models, and expand-contract for data splits. No big-bang rewrites; each step independently reversible until the final contract.
- One service, one primary store, sole writer. Cross-service workflows use orchestrated Sagas with compensations or outbox/CDC with idempotent consumers — not 2PC, not shared tables.
- At every service boundary enforce idempotency, deadline propagation, and retry budgets. Retries without budgets turn partial failure into cascading failure.
- Operate the fleet with a mesh (mTLS, outlier detection, traffic splitting), contract testing (Pact), and progressive delivery (Argo Rollouts with automated analysis). A deploy without draining and canary analysis is a partial outage.
- The monolith is often the correct starting point. A modular monolith with enforced boundaries is the pragmatic default for most teams; services are extracted when a boundary earns its operational cost.







Further reading

- Evans, E. *Domain-Driven Design* (Addison-Wesley, 2003), Chapters 1-3, 14 — bounded contexts, ubiquitous language, anti-corruption layers.

- Newman, S. *Monolith to Microservices* 2nd ed. (O'Reilly, 2024) — strangler fig, branch by abstraction, data splits, with worked examples.
- Fowler, M. "Strangler Fig Application" and "Branch by Abstraction" — the incremental decomposition patterns. <https://martinfowler.com/bliki/StranglerFigApplication.html> and <https://martinfowler.com/bliki/BranchByAbstraction.html>
- Richardson, C. *Microservices Patterns* (Manning, 2018), Chapters 4–6, 9 — Saga, outbox, CDC, and service discovery patterns.
- Istio documentation — VirtualService, DestinationRule, PeerAuthentication, mTLS (Istio 1.22). <https://istio.io/latest/docs/>
- Argo Rollouts — progressive delivery with analysis templates (Argo Rollouts 1.7). <https://argoproj.github.io/argo-rollouts/>
- Pact — consumer-driven contract testing. <https://docs.pact.io/>
- Vol 8 — APIs and Service Design and Vol 10 — Messaging, Streaming, and Event Systems — companion volumes for contract and event mechanics referenced here.

Next: Chapter 7 — Event-Driven Architecture — takes the async side of Chapter 6's communication and makes it the primary design paradigm: when to model the system as events and streams rather than requests and responses, and how to do it without losing ordering, exactly-once, or sanity.

Chapter 7 — Event-Driven Architecture

What this chapter covers. Chapters 4–6 separated how traffic moves and how data is owned; this chapter replaces synchronous call chains with *events* as the connective tissue. Event-driven architecture (EDA) promises decoupling in time, space, and version — producers do not know who consumes, consumers do not need producers to be alive, and both can evolve independently — but it trades that flexibility for a harder consistency model, operational complexity in ordering and delivery, and a new category of failure modes where "it was delivered" does not mean "it

was processed correctly." We define the vocabulary precisely (event vs. command vs. message, queue vs. log vs. pub/sub), show when events beat synchronous RPC and when they do not, then make EDA operational: Apache Kafka 3.7 topic and producer configuration with replication and exactly-once semantics, Change Data Capture with Debezium 2.5 and the transactional outbox pattern that solves the dual-write problem, choreography versus orchestration (Saga) for cross-service workflows, event schema evolution with Avro and Confluent Schema Registry 7.6, and the non-negotiables for production events — idempotency, partitioning for ordering, dead-letter handling, and observability with trace propagation through events. The chapter closes with the distributed-systems lens on why events are the right default for state propagation but the wrong default for request-response.

Learning goals — after this chapter you should be able to:

- Distinguish events, commands, and domain events, and decide per interaction whether synchronous RPC, asynchronous events, or a hybrid is appropriate.
- Compare queues, logs, and pub/sub on durability, ordering, fan-out, and replay, and select the right primitive for a given workflow.
- Configure a production Kafka topic and producer (replication factor, `acks`, `enable.idempotence`, `transactional.id`) and explain what each setting guarantees after a broker or producer crash.
- Implement Change Data Capture with Debezium and the transactional outbox pattern to eliminate dual writes, with real configuration and failure analysis.
- Choose between choreography and orchestration (Saga) for multi-service workflows, name the failure mode of each, and design compensation and observability for both.
- Design event schemas for evolution (Avro + Schema Registry compatibility modes) and enforce idempotent, ordered consumption with partition keys and consumer group semantics.
- Reason about the distributed-systems trade-offs of EDA: eventual consistency windows, ordering vs. availability, replay as a recovery tool and a hazard, and when to keep a synchronous path.

Events are facts, commands are requests

The most common misuse of "event-driven" is to emit events that are actually commands with a different name. The distinction matters because it determines who owns the decision and what coupling remains.

| Concept | Meaning | Who decides | Example | Coupling |
|---------------------|---|--------------------------------|---|--|
| Event | A fact about something that <i>has happened</i> , immutable | Producer (source of truth) | <code>order.placed</code>
<code>{order_id, user_id, total, placed_at}</code> | Producer knows nothing about consumers |
| Command | A request for something <i>to happen</i> , may be rejected | Consumer (recipient validates) | <code>ReserveInventory</code>
<code>{sku, qty, order_id}</code> | Sender expects a specific handler and a result |
| Domain event | An event with business meaning inside a bounded context (DDD) | Owning aggregate | <code>payment.authorized</code> ,
<code>shipment.dispatched</code> | Ubiquitous language boundary — see Chapter 6 |
| Message | Generic envelope — may carry event or command | Depends on payload | Any pub/sub payload | Transport concept, not a domain concept |

An event is a *notification* — "this happened." A command is an *instruction* — "please do this." Conflating them creates hidden coupling: if `OrderService` emits `SendEmail {to, template}` it is commanding the email service through an event bus, and the ordering service now breaks

when the email service changes its contract. The correct event is `order.placed` ; the email service decides independently to react to it.

Three dimensions of decoupling explain why teams adopt EDA:

- **Temporal decoupling** — producer and consumer need not be alive at the same time. A `user.created` event written to a log at 02:00 is consumed at 02:15 after the consumer deploys.
- **Spatial decoupling** — producers address a *topic*, not a host. Adding a new consumer (fraud detection, analytics, search index) requires zero changes to the producer.
- **Version decoupling** — with schema evolution rules, producers can add optional fields without breaking consumers — if the contract discipline holds.

The cost is that the system becomes *eventually consistent* by construction. After `order.placed` is emitted, inventory, payments, recommendations, and the read model each converge at different times. Clients must be taught that "order placed" does not mean "order ready" — a UX and API contract change, not just an infrastructure change.

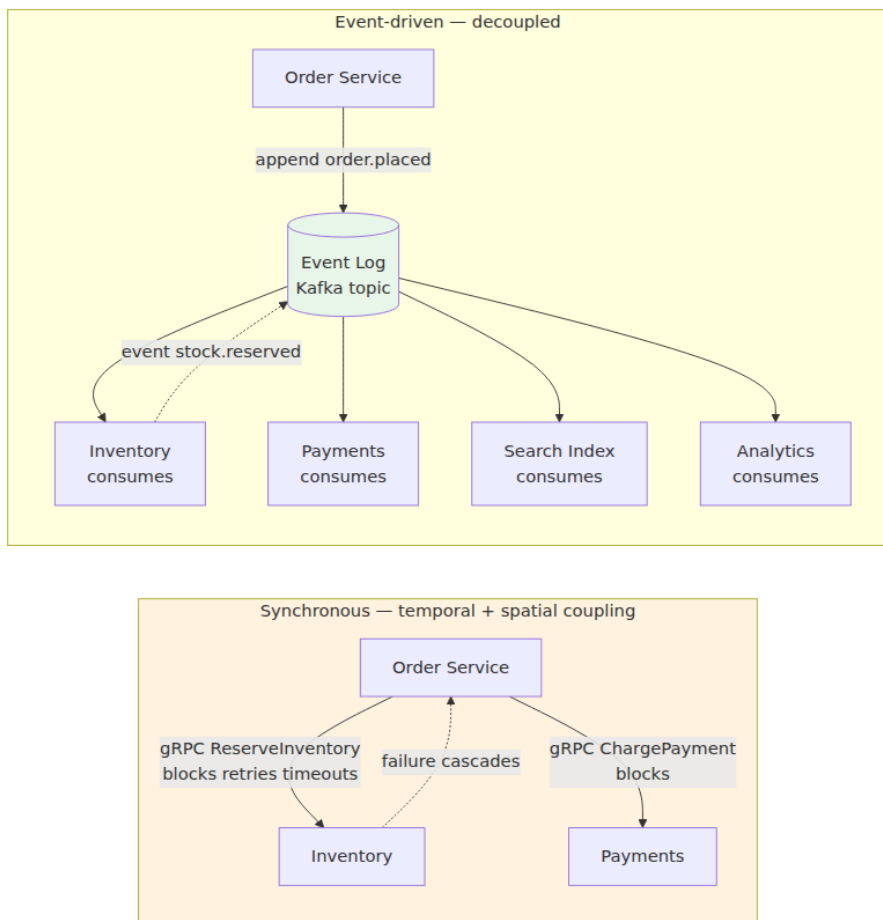


Figure 7-1: Synchronous fan-out couples availability and latency across services; event-driven fan-out decouples them but introduces eventual consistency and consumer autonomy.

Boundary note. This chapter treats EDA as a *system-design pattern* — when to use events, how to scope them to service boundaries, and how to choose choreography vs. orchestration. The *mechanisms* underneath — queue vs. log internals, delivery semantics (at-most/at-least/exactly-once), Kafka broker architecture, stream processing, the outbox pattern in depth, and backpressure — are the subject of Volume 10 — Messaging, Streaming, and Event Systems. Consistency models that constrain what

events can guarantee are in Volume 6. Wire-level reliability primitives (timeouts, retries, hedging) are in Volume 3, Chapter 11.

Building blocks: queues, logs, and pub/sub

Three primitives are routinely conflated under "message queue." They differ on the questions that matter at scale.

| Primitive | Durability | Ordering | Fan-out | Replay | Competing consumers | Representative |
|---------------------------------------|-----------------------------------|------------------------------|--|---------------------|---|--|
| Queue (point-to-point) | Durable until ack | Per-queue FIFO (usually) | One consumer gets each message | No (deleted on ack) | Yes — work queue | SQS Standard Queue, RabbitMQ, ActiveMQ |
| Log (partitioned, append-only) | Durable, retained by time/size | Per-partition total order | Each consumer group gets every message | Yes — rewind offset | Yes — within a group, partitions assigned | Kafka, Redpanda, Pulsar (partitioned), Kinesis |
| Pub/sub topic (broadcast) | Broker-dependent; often ephemeral | Best-effort or per-publisher | Every subscriber gets every message | Usually no | Depends — push vs pull | SNS, Redis Pub/Sub, NATS, Google Pub/Sub |

The decision tree:

- Need **work distribution** where exactly one worker handles each task (send email, resize image)? Use a **queue** with competing consumers and visibility timeouts.
- Need **state propagation** where multiple services react to the same fact and must be able to re-process after a deploy or bug (order placed → inventory, payments, search, analytics)? Use a **log** with retention and replay.
- Need **ephemeral broadcast** with minimal operational overhead and no replay (cache invalidation ping, presence)? Use **pub/sub**.

Most backend systems that claim to be "event-driven" actually need a **log** but adopt a queue — and then rebuild replay, ordering, and fan-out poorly

on top. If more than one consumer needs the same message, or if you will ever want to re-process history, start with a log.

The envelope matters

Every event should carry metadata that makes it operable without parsing the payload:

```
{
  "event_id": "evt_01HQ8X2K3F9A2B7C6D5E4F3G2H",
  "event_type": "order.placed",
  "event_version": "1.2",
  "occurred_at": "2026-08-20T14:32:11.142Z",
  "producer": "order-service",
  "partition_key": "order_9f3a2b1c",
  "correlation_id": "req_7e1d4c8a",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "payload": {
    "order_id": "ord_9f3a2b1c",
    "user_id": "usr_42",
    "total_cents": 4999,
    "currency": "USD",
    "items": [{"sku": "sku_123", "qty": 2}]
  }
}
```

`event_id` enables idempotent deduplication. `partition_key` pins ordering. `correlation_id` and `traceparent` (W3C Trace Context) stitch distributed traces across async hops — without them, an event bus is an observability black hole.

Kafka as the event log — configuration that matters

Apache Kafka 3.7 is the de facto log for event-driven systems at scale. Understanding its replication and producer semantics is non-optional; misconfiguring them is how teams lose events while believing they are durable.

Topic and broker layout

A topic is partitioned; each partition is replicated across `replication.factor` brokers. One replica is the leader; followers replicate via the ISR (in-sync replica) set. Only ISR members are eligible

to become leader without data loss when `unclean.leader.election.enable=false` (which it must be).

```
# kafka-topics.sh – production topic creation (Kafka 3.7, KRaft mode)
# Run against a KRaft cluster (no ZooKeeper since 3.3+)
bin/kafka-topics.sh --create \
  --topic order.events \
  --partitions 24 \
  --replication-factor 3 \
  --config min.insync.replicas=2 \
  --config retention.ms=604800000 \
  --config retention.bytes=107374182400 \
  --config max.message.bytes=1048576 \
  --config compression.type=lz4 \
  --config cleanup.policy=delete \
  --bootstrap-server kafka-1:9092,kafka-2:9092,kafka-3:9092

# Compacted topic for changelog / KTable source (retains latest key)
bin/kafka-topics.sh --create \
  --topic user.profile \
  --partitions 12 \
  --replication-factor 3 \
  --config cleanup.policy=compact \
  --config min.insync.replicas=2 \
  --config segment.ms=600000 \
  --bootstrap-server kafka-1:9092,kafka-2:9092,kafka-3:9092
```

Key choices:

- **Partitions = parallelism ceiling.** A consumer group can have at most one active consumer per partition. 24 partitions → at most 24 consumers in the group scale out. Pick enough for peak throughput but not so many that rebalances become expensive (hundreds is fine; thousands needs tuning).
- **min.insync.replicas=2 with replication.factor=3** tolerates one replica down while still accepting writes. With `min.insync.replicas=1`, a single remaining replica could ack and then be lost.
- **retention.ms=7 days** is the replay window. For audit or reprocessing, 7–30 days is typical; for compacted changelogs, retention is "forever latest value."

Producer guarantees — the three settings that determine durability

```
# producer.properties – safe producer (Kafka clients 3.7.x)
bootstrap.servers=kafka-1:9092,kafka-2:9092,kafka-3:9092
acks=all
enable.idempotence=true
max.in.flight.requests.per.connection=5
retries=2147483647
delivery.timeout.ms=120000
request.timeout.ms=30000
linger.ms=10
batch.size=65536
compression.type=lz4
transactional.id=order-service-tx-01
# For exactly-once across consume-transform-produce (read-process-
write):
# use initTransactions() + begin/commitTransaction around the loop
```

What each does on the failure path:

- **acks=all** — leader waits for all ISR replicas to ack before responding. Without this, **acks=1** or **acks=0** can ack before replication — a leader crash loses the message silently. Latency cost is one network RTT to followers (~1–3 ms intra-AZ); the durability cost of not paying it is data loss.
- **enable.idempotence=true** — producer gets a PID and sequence numbers; the broker deduplicates retries. Without it, a retried `send()` after a timeout can duplicate the event. Requires **acks=all**, **retries>0**, and **max.in.flight ≤ 5**.
- **transactional.id** — enables the transactional API for atomic multi-partition writes and consume-transform-produce (the building block for exactly-once stream processing). Transactions add a transaction coordinator and require **isolation.level=read_committed** on consumers to avoid reading aborted messages.

```

// Exactly-once consume-transform-produce (Kafka 3.7, Java client)
// OrderService consumes order.placed, emits stock.reserve commands
atomically
Properties props = new Properties();
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "order-service-
tx-01");
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
props.put(ProducerConfig.ACKS_CONFIG, "all");
KafkaProducer<String, byte[]> producer = new KafkaProducer<>(props);
producer.initTransactions();

KafkaConsumer<String, byte[]> consumer = new KafkaConsumer<>(consumerPr
ops);
consumer.subscribe(List.of("order.events"));

while (true) {
    ConsumerRecords<String, byte[]> records = consumer.poll(Duration.of
Millis(500));
    if (records.isEmpty()) continue;
    producer.beginTransaction();
    for (ConsumerRecord<String, byte[]> r : records) {
        // transform: produce derived events
        ProducerRecord<String, byte[]> out =
            new ProducerRecord<>("inventory.commands", r.key(), buildRe
serve(r));
        producer.send(out);
    }
    // atomically commit consumed offsets with produced records
    producer.sendOffsetsToTransaction(
        Map.of(new TopicPartition("order.events", 0),
            new OffsetAndMetadata(consumer.position(new TopicPartiti
on("order.events", 0)))),
        consumer.groupMetadata());
    producer.commitTransaction();
}

```

Consumer isolation:

```

# consumer.properties
bootstrap.servers=kafka-1:9092,kafka-2:9092,kafka-3:9092
group.id=inventory-service
isolation.level=read_committed
enable.auto.commit=false
auto.offset.reset=earliest
max.poll.interval.ms=300000
session.timeout.ms=10000

```

`read_committed` hides aborted transactional messages. `enable.auto.commit=false` with manual `commitSync()` (or transactional commit as above) prevents committing an offset before processing succeeded — otherwise a crash after commit but before processing loses the event.

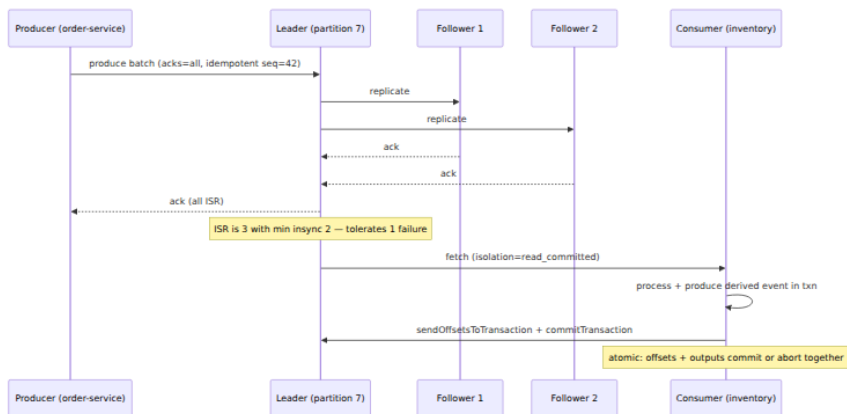


Figure 7-2: Kafka replication and transactional consume-transform-produce. The `acks=all` + idempotence + transaction trio is the only path to no-loss, no-duplicate semantics without application-level deduplication.

Change Data Capture and the outbox — eliminating dual writes

The most common source of lost events in production is not Kafka — it is the **dual write**: `UPDATE orders SET status='placed'` and `producer.send(orderPlaced)` as two separate operations with no atomicity. If the DB commits but the send fails (process crash between the two), the event is lost. If the send succeeds but the DB rolls back, a phantom event is emitted. Retrying naively doubles one side.

The transactional outbox

Write the event to an `outbox` table in the same DB transaction as the business update. A relay publishes outbox rows to Kafka, then marks them sent. The DB transaction is the atomic boundary; at-least-once delivery to Kafka follows.


```

-- Postgres 16 – outbox table co-located with business tables
CREATE TABLE outbox (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type    TEXT NOT NULL,          -- 'order'
    aggregate_id      TEXT NOT NULL,          -- 'ord_9f3a2b1c'
    event_type        TEXT NOT NULL,          -- 'order.placed'
    event_version     TEXT NOT NULL DEFAULT '1.0',
    payload           JSONB NOT NULL,
    headers           JSONB NOT NULL DEFAULT '{}',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at      TIMESTAMPTZ,
    -- idempotency: one row per business fact (unique on aggregate +
event)
    UNIQUE (aggregate_type, aggregate_id, event_type, event_version)
);
CREATE INDEX outbox_unpublished_idx ON outbox (created_at) WHERE published_at IS NULL;

-- Business transaction – single atomic commit
BEGIN;
    INSERT INTO orders (id, user_id, total_cents, status) VALUES ('ord_9f3a2b1c', 'usr_42', 4999, 'placed');
    INSERT INTO outbox (aggregate_type, aggregate_id, event_type, payload, headers)
    VALUES ('order', 'ord_9f3a2b1c', 'order.placed',
        '{"order_id":"ord_9f3a2b1c","user_id":"usr_42","total_cents":4999}',
        '{"traceparent":"00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"}');
COMMIT;

```

Relay options, in order of preference:

1. **Debezium CDC (log-based)** — tails the Postgres WAL / MySQL binlog; no polling, no extra write path, survives relay restarts. Correct choice at scale.
2. **Polling publisher** — `SELECT ... WHERE published_at IS NULL FOR UPDATE SKIP LOCKED` in a loop. Simple, but adds read load and has a polling delay. Acceptable for low throughput.
3. **Triggers / logical decoding plugins** — avoid; operational burden and version coupling.

Debezium 2.5 with Postgres 16 — outbox relay

```
# debezium-connector-outbox.json — Kafka Connect REST (Debezium 2.5,
Kafka 3.7)
{
  "name": "orders-outbox-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnec
tor",
    "database.hostname": "pg-primary.internal",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "${file:/secrets/pg.properties:password}",
    "database.dbname": "orders",
    "topic.prefix": "orders",
    "table.include.list": "public.outbox",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_orders_outbox",
    "publication.name": "debezium_pub",
    "tombstones.on.delete": "false",
    "transforms": "outbox",
    "transforms.outbox.type": "io.debezium.transforms.outbox.EventRoute
r",
    "transforms.outbox.table.field.event.id": "id",
    "transforms.outbox.table.field.event.key": "aggregate_id",
    "transforms.outbox.table.field.event.payload": "payload",
    "transforms.outbox.table.fields.additional.placement": "headers:hea
der:traceparent",
    "transforms.outbox.route.by.field": "event_type",
    "transforms.outbox.route.topic.replacement": "order.events",
    "key.converter":
"org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://schema-
registry:8081",
    "snapshot.mode": "initial",
    "snapshot.locking.mode": "none"
  }
}
```

```
# Alternative: Debezium Server (standalone, no Kafka Connect) –
debezium-server.properties
debezium.source.connector.class=io.debezium.connector.postgresql.PostgresConnector
debezium.source.database.hostname=pg-primary.internal
debezium.source.plugin.name=pgoutput
debezium.source.topic.prefix=orders
debezium.source.table.include.list=public.outbox
debezium.sink.type=kafka
debezium.sink.kafka.producer.bootstrap.servers=kafka-1:9092,kafka-2:9092
debezium.sink.kafka.producer.acks=all
debezium.transforms=outbox
debezium.transforms.outbox.type=io.debezium.transforms.outbox.EventRouter
```

Operational notes:

- **WAL retention** — Debezium holds a replication slot; if the connector is down, WAL accumulates on the primary. Monitor `pg_replication_slots.active` and `pg_wal_lsn_diff`; alert if lag exceeds retention.
- **Exactly-once to Kafka** — Debezium's Kafka sink is at-least-once by default; consumers must be idempotent (see below). Debezium with Kafka transactions is possible but adds coordinator overhead; most teams accept at-least-once + idempotent consumers.
- **Ordering** — CDC preserves commit order per table; the `aggregate_id` as Kafka key preserves per-aggregate order end-to-end if the producer partitions by that key.

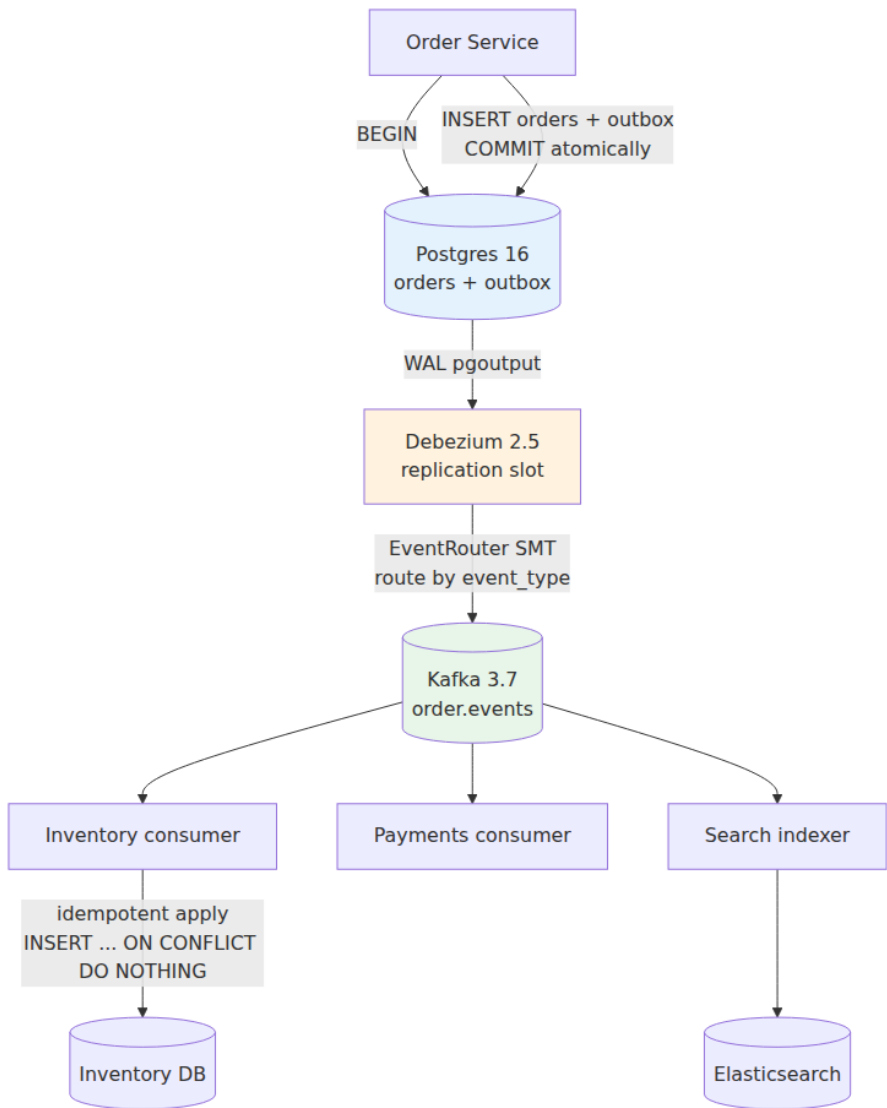


Figure 7-3: Transactional outbox with Debezium CDC. The database transaction is the atomic boundary; WAL tailing provides at-least-once, ordered relay to Kafka without polling or dual writes.

Boundary note. Full CDC operational detail — snapshot modes, schema history topics, handling DDL, backfilling, and the interaction with logical

replication slots — is in Volume 10, Chapter 6 — The Outbox Pattern and the Dual-Write Problem. Volume 5, Chapter 8 covers physical vs. logical replication as a storage-engine primitive.

Choreography vs. orchestration — coordinating workflows without distributed transactions

A single business workflow often spans services: `place order` → `reserve inventory` → `charge payment` → `schedule shipment`. Each step can fail and must be compensatable. Two coordination styles exist; most mature systems use both, per workflow.

Choreography — each service reacts to events independently

No central coordinator. `OrderService` emits `order.placed`; `InventoryService` consumes it and emits `stock.reserved` or `stock.insufficient`; `PaymentService` reacts to `stock.reserved`; and so on. Each service knows only its predecessor's events.

Pros: fully decoupled, no single point of coordination, easy to add new reactors. Cons: workflow is implicit — spread across N services' handlers. Debugging "why did order_123 stall?" requires correlating logs across N consumers. Failure handling is distributed; compensations must be designed per service.

Orchestration — a coordinator drives the workflow (Saga orchestration)

A dedicated orchestrator (Temporal/Cadence, AWS Step Functions, or a bespoke saga coordinator) holds workflow state, invokes services (via commands or events), and runs compensations on failure. The workflow is explicit and visible in one place.

Pros: workflow is a first-class artifact — versioned, visualized, replayable. Compensation and timeout policy live together. Easier to reason about and to operate. Cons: orchestrator is a stateful component to run. At very high scale, it can become a bottleneck or a coupling point if every workflow routes through one cluster.

| Criterion | Choreography | Orchestration (Saga) |
|---------------------|---|---|
| Workflow visibility | Distributed — must be reconstructed from logs | Central — state machine / workflow definition |
| Coupling | Low — services know only events they care about | Higher — orchestrator knows service interfaces |
| Failure handling | Each service compensates locally | Coordinator triggers compensations |
| When to use | Simple fan-out (notify N consumers), no rollback needed | Multi-step with compensations, timeouts, human approval |

Rule of thumb: fan-out notifications → choreography; multi-step transactions that must be rolled back → orchestration.

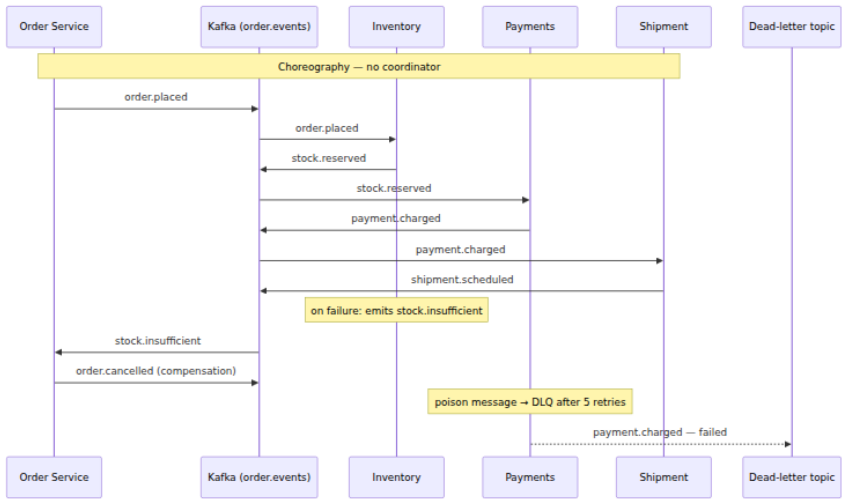


Figure 7-4: Choreographed workflow with compensation events. Each service is autonomous but the end-to-end trace must be reconstructed from correlated events; a poison message is isolated to a dead-letter topic rather than blocking the partition.

Compensation example (orchestrated saga with Temporal — Go SDK):

```

// saga_orchestrator.go - Temporal workflow (temporal.io/sdk-go v1.30)
func PlaceOrderWorkflow(ctx workflow.Context, order Order) error {
    ao := workflow.ActivityOptions{
        StartToCloseTimeout: 30 * time.Second,
        RetryPolicy: &temporal.RetryPolicy{MaximumAttempts: 3},
    }
    ctx = workflow.WithActivityOptions(ctx, ao)

    // Step 1 - reserve inventory (compensatable)
    if err := workflow.ExecuteActivity(ctx, ReserveInventory, order).Get(
t(ctx, nil); err != nil {
        return err // nothing to compensate yet
    }
    // ensure compensation runs if later steps fail
    defer func() {
        if err != nil {
            _ = workflow.ExecuteActivity(ctx, ReleaseInventory,
order).Get(ctx, nil)
        }
    }()

    // Step 2 - charge payment (compensatable)
    if err := workflow.ExecuteActivity(ctx, ChargePayment, order).Get(c
tx, nil); err != nil {
        return err // defer releases inventory
    }
    defer func() {
        if err != nil {
            _ = workflow.ExecuteActivity(ctx, RefundPayment,
order).Get(ctx, nil)
        }
    }()

    // Step 3 - schedule shipment (final)
    if err := workflow.ExecuteActivity(ctx, ScheduleShipment, order).Ge
t(ctx, nil); err != nil {
        return err // defers refund + release
    }
    return workflow.ExecuteActivity(ctx, ConfirmOrder, order).Get(ctx,
nil)
}

```

Temporal persists workflow state in its own store (backed by Cassandra/Postgres/MySQL), retries activities with backoff, and replays deterministically after worker crashes — the orchestrator itself is fault-tolerant. The alternative without an orchestrator is to hand-roll state machines in each service and reconcile via events, which is where most homegrown saga implementations become unmaintainable.

Schema, versioning, and idempotency — the non-negotiables

Events are an API. Without schema discipline, "decoupled" becomes "unknowingly coupled to whatever JSON the producer happened to emit last Tuesday."

Schema Registry and compatibility

Confluent Schema Registry 7.6 (or Apicurio 2.6) stores Avro/Protobuf/JSON Schema versions and enforces compatibility on produce. Producers register schemas; consumers fetch them by ID embedded in the message.

```
// Avro schema – order.placed v1.1 (backward compatible: adds optional field)
{
  "type": "record",
  "name": "OrderPlaced",
  "namespace": "com.example.orders",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "user_id", "type": "string"},
    {"name": "total_cents", "type": "int"},
    {"name": "currency", "type": "string", "default": "USD"},
    {"name": "coupon_code", "type": ["null", "string"], "default": null}
  ]
}
```

Compatibility modes:

- **BACKWARD** — new schema can read old data (consumers can upgrade before producers). Required default for most topics: add only optional fields with defaults.
- **FORWARD** — old schema can read new data (producers can upgrade before consumers). Useful when you control consumer rollouts tightly.
- **FULL** — both. Safest, most restrictive.
- **NONE** — no checks. Do not use in production.


```
# Register and set compatibility (Confluent Schema Registry 7.6)
curl -s -X POST http://schema-registry:8081/subjects/order.events-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d '{"schema": "{\"type\": \"record\", \"name\": \"OrderPlaced\", ...}\"}'

curl -s -X PUT http://schema-registry:8081/config/order.events-value \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d '{"compatibility": "BACKWARD"}'
```

Producer and consumer wiring (Confluent Kafka Avro serializer, v7.6.x):

```
// Producer — Avro + Schema Registry
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaAvroSerializer.class);
props.put("schema.registry.url", "http://schema-registry:8081");
props.put("auto.register.schemas", "false"); // CI registers schemas, not prod

// Consumer — specific Avro record
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, KafkaAvroDeserializer.class);
props.put("schema.registry.url", "http://schema-registry:8081");
props.put(KafkaAvroDeserializerConfig.SPECIFIC_AVRO_READER_CONFIG, true);
```

Idempotency and ordering

At-least-once delivery means consumers *will* see duplicates (retries, rebalances, transactional aborts visible before `read_committed` filtering in some clients). Every consumer must be idempotent.

Patterns:

- **Natural idempotency** — `PUT /inventory/{sku}` with absolute quantity or `SET status='reserved' WHERE status='pending'` is idempotent by semantics.
- **Idempotency key table** — `INSERT INTO processed_events(event_id) VALUES ($1) ON CONFLICT DO NOTHING;` if inserted, process; if conflict, skip.
- **Deduplication window** — Redis `SET event_id 1 NX EX 86400` for 24-hour dedup where DB writes are expensive.

```

# idempotent_consumer.py – Python 3.11, confluent-kafka 2.4, psycopg
3.1
import psycopg, redis
from confluent_kafka import Consumer

r = redis.Redis(host="redis", decode_responses=True)
conn = psycopg.connect("postgres://app@pg-primary/orders")

consumer = Consumer({
    "bootstrap.servers": "kafka-1:9092",
    "group.id": "inventory-service",
    "isolation.level": "read_committed",
    "enable.auto.commit": False,
})

consumer.subscribe(["order.events"])
while True:
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        continue
    event_id = msg.headers().get("event_id")
    # 24h dedup window in Redis before hitting Postgres
    if not r.set(f"dedup:{event_id}", "1", nx=True, ex=86400):
        consumer.commit(msg) # duplicate – skip but advance offset
        continue
    with conn.transaction():
        # business logic – also guarded by DB unique constraint as
        # fallback
        process(msg)
        conn.execute("INSERT INTO processed_events(event_id) VALUES
(%s) ON CONFLICT DO NOTHING", (event_id,))
    consumer.commit(msg)

```

Ordering: Kafka orders only *within a partition*. To guarantee that all events for `order_9f3a2b1c` are processed in order, produce them with the same key so they land on the same partition:

```

ProducerRecord<String, byte[]> rec =
    new ProducerRecord<>("order.events", /* key= */ orderId, /* value=
*/ avroBytes);
// DefaultPartitioner hashes key → partition deterministically
producer.send(rec);

```

Consumers must not parallelize *across* events for the same key if order matters. Within a partition, a single consumer thread processes sequentially; scaling out means more partitions, not more threads per partition.

Putting it together — when to use events and when not to

Events are not a universal replacement for RPC. The decision hinges on whether the caller needs a timely answer.

| Need | Pattern | Example |
|---|--|--|
| "Do this and tell me if it succeeded, now" | Synchronous RPC (gRPC/HTTP) with timeout + retry budget | Charge payment, check authZ, fetch user profile |
| "This happened; others may care, eventually" | Event to log, fan-out to consumers | Order placed → inventory, search, analytics |
| "Do this workflow with compensations" | Orchestrated Saga (Temporal/Step Functions) | Checkout spanning 4 services with rollback |
| "Propagate state change to many readers" | CDC / event log (Kafka + Debezium) | DB write → cache invalidation, read-model projection |
| "Need request-response but decoupled in time" | Request-reply over events (correlation ID + reply topic) | Rarely — prefer RPC; async reply adds latency and complexity |

The distributed-systems lens:

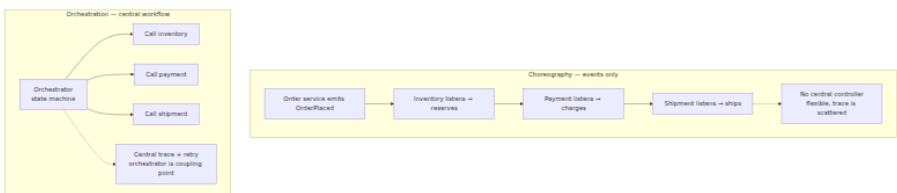
- **Consistency window** — after `order.placed`, there is a window where the order exists but inventory has not been reserved. Clients reading their order must handle `status=pending` explicitly. The window's p99 is a function of consumer lag (`consumer_group_lag` metric), not producer latency.
- **Backpressure** — slow consumers cause lag growth, not producer throttling (unless you add it). Monitor `kafka_consumer_lag` and `kafka_log_end_offset - kafka_consumer_offset` per partition; autoscale consumers or shed low-priority event types before the log retention window expires.
- **Replay is a superpower and a hazard.** Replaying `order.events` from 7 days ago rebuilds a projection after a bug — but also re-emits

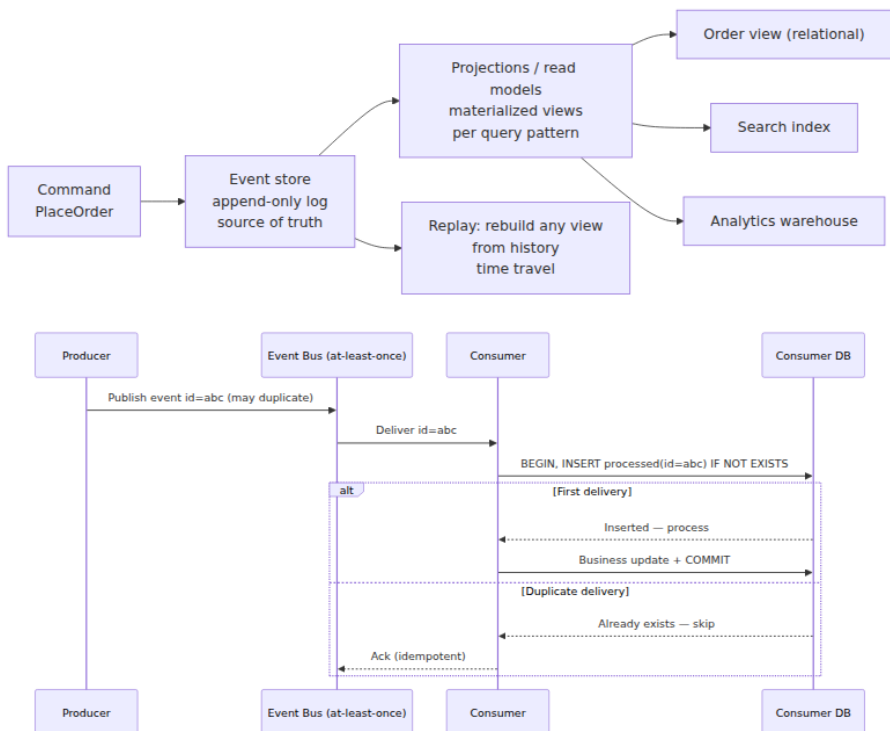
side effects if consumers are not idempotent. Separate *state-rebuilding* replays (to new consumer groups or new topics) from *side-effecting* replays.

- **Schema is the contract.** A breaking schema change is a distributed outage that deploys gradually. Enforce registry compatibility in CI; never set `auto.register.schemas=true` in production.

Key takeaways

- Events are immutable facts about the past; commands are requests that may be rejected — naming them correctly prevents hidden coupling through the event bus.
- Queues distribute work to one consumer, logs propagate state to many consumers with replay, pub/sub broadcasts ephemerally — choose the primitive by fan-out and replay needs, not by fashion.
- Kafka durability depends on `acks=all` + `min.insync.replicas≥2` + `replication.factor=3`; producer idempotence prevents duplicates on retry and transactions make consume-transform-produce atomic.
- The dual-write problem (DB commit + event send) is solved by the transactional outbox with CDC (Debezium tailing the WAL) — never attempt two independent commits and hope they align.
- Choreography suits fan-out notifications; orchestration (Saga via Temporal/Step Functions) suits multi-step workflows with compensations — most systems need both.
- At-least-once delivery is the reality; every consumer must be idempotent (key table or dedup window). Ordering is per-partition only — partition by aggregate key when order matters.
- Schema evolution with a registry and backward compatibility is the only way to keep decoupled services actually decoupled — breaking schema changes are silent distributed outages.





Further reading

- Kleppmann, M. *Designing Data-Intensive Applications*, Chapters 11–12 (stream processing, future of data systems) — O'Reilly, 2017.
- Apache Kafka 3.7 documentation — replication, producer configs, transactions: <https://kafka.apache.org/documentation/#replication> and <https://kafka.apache.org/documentation/#producerconfigs>
- Debezium 2.5 documentation — Postgres connector, outbox SMT, replication slots: <https://debezium.io/documentation/reference/2.5/connectors/postgresql.html>
- Confluent Schema Registry 7.6 — compatibility types and Avro integration: <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>
- Richardson, C. *Microservices Patterns*, Chapter 3 (inter-service communication) and Chapter 4 (Saga): <https://microservices.io/patterns/communication-style/event-driven.html>

- Temporal documentation — saga and workflow patterns: <https://docs.temporal.io/workflows>
- W3C Trace Context — `traceparent` propagation for async traces: <https://www.w3.org/TR/trace-context/>

Chapter 8 — API Gateways, BFF, and Edge

What this chapter covers. Chapter 4 placed load balancers in the traffic path and Chapter 6 split the monolith into services; this chapter puts a *facade* in front of that fleet and pushes logic to where users actually are. Without a gateway, every client must discover, authenticate to, and handle failures for N services — coupling clients to the service topology and leaking internal boundaries. An API gateway centralizes cross-cutting concerns (routing, authentication, rate limiting, transformation, observability) so that services stay focused on business logic. The Backend-for-Frontend (BFF) pattern refines the facade per client class so that mobile, web, and partner APIs each get the shape they need without forcing a lowest-common-denominator contract. Edge computing extends the facade to the CDN PoP, where requests can be authenticated, personalized, and even served without ever reaching the origin. We make each layer concrete: Envoy 1.30 and Kong Gateway 3.7 gateway configurations with JWT validation and request transformation, a BFF aggregation service with deadline-propagated fan-out, and edge logic with Cloudflare Workers and Fastly Compute — then close with the distributed-systems lens on why the gateway is simultaneously the best place to enforce policy and the most dangerous single point of failure.

Learning goals — after this chapter you should be able to:

- Explain what an API gateway owns versus what belongs in services or in the mesh, and name the failure mode when the boundary is wrong.
- Configure Envoy (1.30) and Kong (3.7) as gateways for routing, JWT authentication, header transformation, and rate-limit integration, and explain each field on the data path.

- Compare gateway, sidecar/mesh, and library (client-side) enforcement on latency, blast radius, and deploy coupling, and choose per concern.
 - Design a BFF layer that aggregates, filters, and reshapes service responses per client without becoming a distributed monolith or a bottleneck.
 - Implement edge logic (auth, A/B, personalization, caching) with Cloudflare Workers and Fastly Compute, reason about cold starts and consistency at the edge, and decide what must stay at the origin.
 - Operate gateways at scale: canary routing, version-aware routing, observability (access logs, tracing, metrics), and the high-availability and multi-region deployment model that prevents the gateway from being a single point of failure.
-

Why a gateway — and what it should not do

A gateway is a *reverse proxy with policy*. It terminates client connections, applies cross-cutting concerns once, and forwards to the appropriate backend — ideally without containing business logic.

What belongs at the gateway:

- **Routing** — path/host/header → upstream cluster. `/api/v1/orders/*` → order-service, `/api/v1/users/*` → user-service. Version-aware routing, canary splits, and header-based traffic steering (see below).
- **Authentication and coarse authorization** — validate JWT / mTLS client certificate, attach `x-user-id` / `x-tenant-id` headers for backends to trust. Fine-grained authorization (can this user edit this order?) stays in the owning service.
- **Rate limiting and quotas** — enforce per-client, per-route, per-tenant ceilings before traffic reaches backends. The algorithm lives here; the quota state may be local or in Redis (Chapter 9).
- **Transformation** — header injection, CORS, request/response rewriting, protocol translation (HTTP/1.1 → gRPC via `grpc-web`, REST → gRPC transcoding).
- **Observability** — access logs, metrics (requests, latency, error rate per route), distributed trace initiation (inject `traceparent`), and WAF / bot signals.

- **Resilience primitives** — timeouts, retries with budgets, circuit breaking, and outlier detection at the edge of the system (complements mesh-level resilience inside).

What does *not* belong at the gateway:

- Business logic, workflow orchestration, or data aggregation beyond thin BFF shaping (that is a service).
- Stateful session handling beyond token validation.
- Long-lived business transactions.

The anti-pattern is the "gateway monolith" — a gateway that accumulates business logic because it was the easiest place to add a new endpoint. It becomes the most coupled, most frequently deployed, and most fragile component in the system. The test: if removing the gateway (routing directly to services with mesh auth) would break business invariants, logic has leaked into the wrong layer.

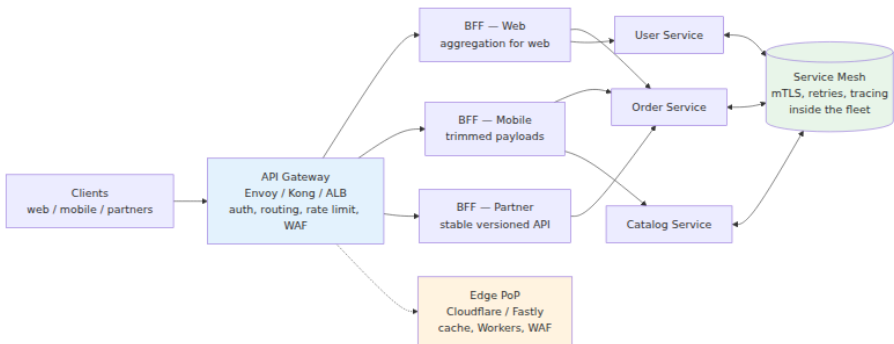


Figure 8-1: Gateway, BFF, mesh, and edge. The gateway enforces policy once at the perimeter; BFFs reshape per client; the mesh enforces policy inside; the edge enforces and serves before traffic reaches the origin.

Boundary note. Wire-level proxying, L4/L7 algorithms, and service mesh data-plane mechanics (Envoy sidecar, xDS, mTLS) are in Volume 3, Chapters 9–10 and Volume 12. API contract design, versioning, and schema evolution are in Volume 8. Rate-limiting algorithms at system scale are in Chapter 9 of this volume; algorithmic analysis of those same algorithms is in Volume 14, Chapter 8. This chapter treats the *gateway/BFF/edge* as *system-design components* — their responsibilities, configuration, and failure modes.

Envoy as a gateway — configuration that matters

Envoy 1.30 is both the canonical gateway data plane and the sidecar inside Istio/Linkerd. Running it as an edge gateway means configuring listeners, routes, clusters, and filters explicitly.

Static gateway with JWT auth, routing, and transformation

```

# envoy-gateway.yaml – Envoy 1.30 as edge gateway (static config; xDS
via control plane in prod)
admin:
  address:
    socket_address: { address: 127.0.0.1, port_value: 9901 }

static_resources:
  listeners:
    - name: https_ingress
      address:
        socket_address: { address: 0.0.0.0, port_value: 443 }
      filter_chains:
        - transport_socket:
            name: envoy.transport_sockets.tls
            typed_config:
              "@type": type.googleapis.com/
envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
            common_tls_context:
              tls_certificates:
                - certificate_chain: { filename: "/etc/envoy/tls/tls.crt" }
                  private_key: { filename: "/etc/envoy/tls/tls.key" }
              filters:
                - name: envoy.filters.network.http_connection_manager
                  typed_config:
                    "@type": type.googleapis.com/
envoy.extensions.filters.network.http_connection_manager.v3.HttpConnect
ionManager
                    stat_prefix: ingress_http
                    access_log:
                      - name: envoy.access_loggers.file
                        typed_config:
                          "@type": type.googleapis.com/
envoy.extensions.access_loggers.file.v3.FileAccessLog
                        path: /var/log/envoy/access.log
                        log_format:
                          json_format:
                            start_time: "%START_TIME%"
                            method: "%REQ(:METHOD)%"
                            path: "%REQ(X-ENVOY-ORIGINAL-PATH?:PATH)%"
                            route: "%ROUTE_NAME%"
                            upstream: "%UPSTREAM_HOST%"
                            duration_ms: "%DURATION%"
                            response_code: "%RESPONSE_CODE%"
                            trace_id: "%REQ(X-REQUEST-ID)%"
                        http_filters:
                          # 1 – JWT validation (before routing)
                          - name: envoy.filters.http.jwt_authn
                            typed_config:
                              "@type": type.googleapis.com/
envoy.extensions.filters.http.jwt_authn.v3.JwtAuthentication

```

```

    providers:
      cognito:
        issuer: https://cognito-idp.us-east-1.amazonaws.com/
us-east-1_abc123
        audiences: [api.example.com]
        remote_jwks:
          http_uri:
            uri: https://cognito-idp.us-east-1.amazonaws.com/
us-east-1_abc123/.well-known/jwks.json
            cluster: jwks_cluster
            timeout: 5s
            cache_duration: { seconds: 300 }
            forward: true
            payload_in_metadata: jwt_payload
        rules:
          - match: { prefix: /api/ }
            requires: { provider_name: cognito }
          - match: { prefix: /health }
            requires: { allow_missing: true }
    # 2 – rate limiting (delegated to external service – see
Chapter 9)
    - name: envoy.filters.http.ratelimit
      typed_config:
        "@type": type.googleapis.com/
envoy.extensions.filters.http.ratelimit.v3.RateLimit
        domain: api_gateway
        request_type: external
        rate_limit_service:
          grpc_service:
            envoy_grpc: { cluster_name: ratelimit }
            transport_api_version: V3
        failure_mode_deny: false # fail-open on rate-limit
service outage
    # 3 – router (must be last)
    - name: envoy.filters.http.router
      typed_config:
        "@type": type.googleapis.com/
envoy.extensions.filters.http.router.v3.Router
      route_config:
        name: api_routes
        virtual_hosts:
          - name: api
            domains: ["api.example.com"]
            # CORS for browser clients
            cors:
              allow_origin_string_match: [{ safe_regex: { regex: "htt
ps://.*\\.example\\.com" } }]
              allow_methods: "GET, POST, PUT, DELETE, OPTIONS"
              allow_headers: "authorization,content-type,x-request-
id,traceparent"
              max_age: "600"

```

```

    routes:
      # Version-aware routing – header pins canary
      - match: { prefix: /api/v1/orders, headers: [{ name: x-
canary, string_match: { exact: "1" } }] }
        route:
          cluster: orders_canary
          timeout: 3s
          retry_policy:
            retry_on: 5xx
            num_retries: 1
            per_try_timeout: 2s
            retry_host_predicate: [{ name: envoy.retry_host_pre
dicates.previous_hosts }]
          host_rewrite_literal: orders.internal
          request_headers_to_add:
            - header: { key: x-gateway-route, value: "orders-
canary" }
      - match: { prefix: /api/v1/orders }
        route:
          cluster: orders
          timeout: 3s
          retry_policy: { retry_on: 5xx, num_retries: 1,
per_try_timeout: 2s }
          request_headers_to_add:
            - header: { key: x-user-id, value: "%DYNAMIC_METADATA(e
nvoy.filters.http.jwt_authn.jwt_payload:sub)%" }
            - match: { prefix: /api/v1/users }
              route: { cluster: users, timeout: 2s }
            - match: { prefix: /health }
              direct_response: { status: 200, body: { inline_string:
"ok\n" } }

clusters:
- name: orders
  connect_timeout: 0.5s
  type: STRICT_DNS
  lb_policy: LEAST_REQUEST
  health_checks:
    - timeout: 2s
      interval: 10s
      unhealthy_threshold: 3
      healthy_threshold: 2
      http_health_check: { path: /healthz }
  circuit_breakers:
    thresholds:
      - max_connections: 4096
        max_pending_requests: 1024
        max_requests: 2048
        max_retries: 3
  outlier_detection:
    consecutive_5xx: 3

```

```

    interval: 10s
    base_ejection_time: 30s
    max_ejection_percent: 50
  load_assignment:
    cluster_name: orders
    endpoints:
      - lb_endpoints:
          - endpoint: { address: { socket_address: { address: orders.inte
rnal, port_value: 8080 } } }
      - name: orders_canary
        connect_timeout: 0.5s
        type: STRICT_DNS
        lb_policy: LEAST_REQUEST
        load_assignment:
          cluster_name: orders_canary
          endpoints:
            - lb_endpoints:
                - endpoint: { address: { socket_address: { address: orders-
canary.internal, port_value: 8080 } } }
            - name: users
              connect_timeout: 0.5s
              type: STRICT_DNS
              lb_policy: LEAST_REQUEST
              load_assignment:
                cluster_name: users
                endpoints:
                  - lb_endpoints:
                      - endpoint: { address: { socket_address: { address: users.inter
nal, port_value: 8080 } } }
                  - name: jwks_cluster
                    connect_timeout: 1s
                    type: LOGICAL_DNS
                    lb_policy: ROUND_ROBIN
                    load_assignment:
                      cluster_name: jwks_cluster
                      endpoints:
                        - lb_endpoints:
                            - endpoint: { address: { socket_address: { address: cognito-
idp.us-east-1.amazonaws.com, port_value: 443 } } }
                    transport_socket:
                      name: envoy.transport_sockets.tls
                      typed_config:
                        "@type": type.googleapis.com/
envoy.extensions.transport_sockets.tls.v3.UpstreamTlsContext
                  - name: ratelimit
                    connect_timeout: 0.25s
                    type: STRICT_DNS
                    lb_policy: ROUND_ROBIN
                    typed_extension_protocol_options:
                      envoy.extensions.upstreams.http.v3.HttpProtocolOptions:
                        "@type": type.googleapis.com/

```

```

envoy.extensions.upstreams.http.v3.HttpProtocolOptions
  explicit_http_config: { http2_protocol_options: {} }
  load_assignment:
    cluster_name: ratelimit
    endpoints:
      - lb_endpoints:
          - endpoint: { address: { socket_address: { address: ratelimit.i
nternal, port_value: 8081 } } }

```

What each section does on the failure path:

- **jwt_authn before routing** — unauthenticated requests are rejected at the edge with 401 before any upstream sees them.
`cache_duration: 300s` avoids fetching JWKS on every request.
`forward: true` propagates the validated claims downstream if needed; `payload_in_metadata` lets `request_headers_to_add` inject `sub` as `x-user-id`.
- **failure_mode_deny: false** on the rate-limit filter — if the rate-limit service is down, traffic passes rather than black-holing the entire API. The alternative (`true`) is fail-closed — correct for auth, wrong for rate limiting in most products.
- **Outlier detection + circuit breakers** — Envoy ejects hosts that return `consecutive_5xx: 3` within `interval: 10s` for `base_ejection_time: 30s`, up to `max_ejection_percent: 50`. This is local to the gateway instance; for cross-instance ejection, use the mesh or a shared outlier store.
- **Retry predicate previous_hosts** — retries never hit the same host that just returned 5xx, avoiding amplification to a single failing pod.

Kong Gateway 3.7 — declarative config with the same policy

Kong is a common alternative where teams want an admin API, plugin ecosystem, and declarative configuration without writing Envoy xDS control planes.

```

# kong.yaml – Kong Gateway 3.7 declarative config (DB-less)
_format_version: "3.0"
services:
- name: orders
  url: http://orders.internal:8080
  routes:
  - name: orders-v1
    paths: [/api/v1/orders]
    strip_path: false
    plugins:
    - name: jwt
      config:
        uri_param_names: [jwt]
        key_claim_name: kid
        run_on_preflight: false
    - name: rate-limiting
      config:
        minute: 600
        hour: 10000
        policy: redis
        redis_host: redis.internal
        redis_port: 6379
        redis_timeout: 2000
        fault_tolerant: true
        hide_client_headers: false
        limit_by: consumer
  - name: cors
    config:
      origins: ["https://*.example.com"]
      methods: [GET, POST, PUT, DELETE, OPTIONS]
      headers: [authorization, content-type, x-request-id, traceparen
t]
      max_age: 600
  - name: request-transformer
    config:
      add:
        headers: ["x-gateway-route:orders", "x-request-id:$uuid"]
  - name: orders-canary
    paths: [/api/v1/orders]
    headers: { x-canary: ["1"] }
    strip_path: false
    service: orders-canary

- name: orders-canary
  url: http://orders-canary.internal:8080

consumers:
- username: mobile-app
  jwt_secrets:
  - key: mobile-app-key

```



```

    algorithm: RS256
    rsa_public_key: "-----BEGIN PUBLIC KEY-----\nMIIBIjANB...\n-----
END PUBLIC KEY-----"

plugins:
- name: prometheus
  config:
    per_consumer: true
    status_code_metrics: true
    latency_metrics: true
- name: opentelemetry
  config:
    endpoint: http://otel-collector:4318/v1/traces
    resource_attributes:
      service.name: kong-gateway

```

Trade-offs: Kong's plugin model is simpler to operate than Envoy's filter chain, but less expressive for header-based routing and outlier detection. Both delegate rate-limit state to Redis or an external gRPC service — local counters diverge across gateway replicas.

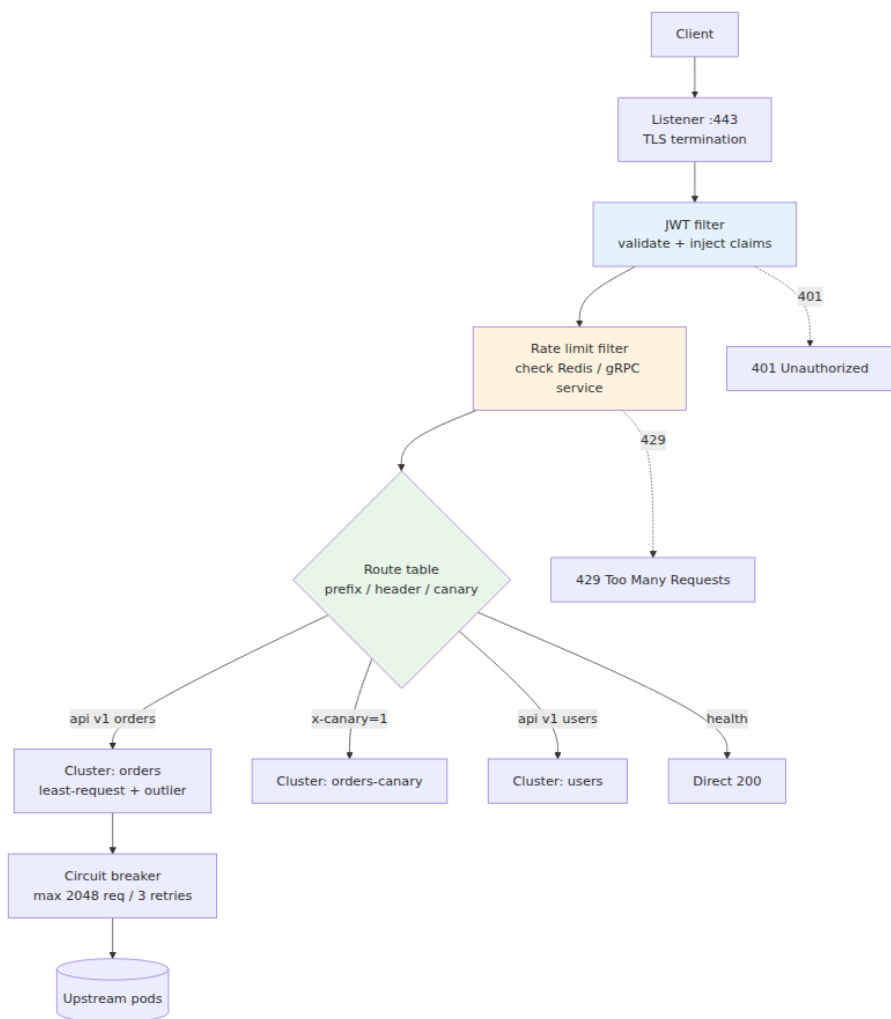
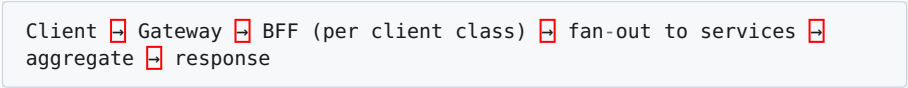


Figure 8-2: Gateway filter chain. Authentication runs first (fail fast, no upstream cost), rate limiting second (protect backends), routing last. Each filter can short-circuit with an error response without reaching upstream.

Backend for Frontend — one facade per client, not one facade for all

The BFF pattern (SoundCloud, 2015; popularized by Sam Newman) observes that a single "one size fits all" API forces every client to over-fetch, under-fetch, or work around a contract designed for someone else. A mobile app on a constrained network needs trimmed payloads and fewer round trips; a web app needs richer payloads with server-rendered fragments; a partner API needs a versioned, minimal, heavily rate-limited surface. One gateway route table cannot serve all three well.

A BFF is a thin, client-specific *aggregation and shaping layer* that sits behind the gateway (or is itself gateway-routed) and fans out to services. It owns no business invariants — it orchestrates reads.



What a BFF does and does not do

| BFF does | BFF does not |
|--|--|
| Aggregate <code>GET /orders/{id}</code> + <code>GET /users/{id}</code> + <code>GET /catalog/{sku}</code> into one <code>GET /bff/mobile/order-view/{id}</code> | Own order or user business logic |
| Filter fields per client (mobile gets 12 fields, web gets 48) | Own the canonical schema (services do) |
| Batch and deduplicate downstream calls (DataLoader pattern) | Become a shared library imported by services |
| Enforce client-specific deadlines and fallbacks (return partial on timeout) | Implement distributed transactions |

The defining constraint: a BFF is *owned by the client team*, not the service team. The mobile BFF is owned by the mobile team, deploys with the mobile release cadence, and can be broken by service changes only through versioned contracts — the same boundary that Chapter 6 draws for services.

BFF aggregation with deadline propagation

The hardest part of a BFF is not the fan-out — it is the failure handling when one of N backends is slow. The BFF must enforce a *client deadline*, propagate it downstream, and return a partial or degraded response rather than failing the entire aggregation because one dependency timed out.

```

// bff/mobile/order_view.go - Go 1.22, fan-out with deadline
propagation
package mobile

import (
    "context"
    "golang.org/x/sync/errgroup"
    "google.golang.org/grpc/metadata"
)

type OrderView struct {
    Order *Order `json:"order"`
    User *User `json:"user,omitempty"`
    Catalog *Catalog `json:"catalog,omitempty"`
    Warnings []string `json:"warnings,omitempty"`
}

func (h *Handler) GetOrderView(ctx context.Context, orderID string) (*OrderView, error) {
    // Client deadline is the budget; BFF reserves 50ms for
    // serialization
    // Downstream calls share the remaining deadline via gRPC
    // metadata / HTTP header
    ctx, cancel := context.WithTimeout(ctx, 800*time.Millisecond)
    defer cancel()

    var view OrderView
    g, ctx := errgroup.WithContext(ctx)

    g.Go(func() error {
        o, err := h.orders.GetOrder(ctx, orderID) // propagates ctx
        // deadline via gRPC
        if err != nil {
            return err // order is required - fail the whole view
        }
        view.Order = o
        return nil
    })

    g.Go(func() error {
        // user and catalog are best-effort - degrade gracefully
        u, err := h.users.GetUser(ctx, extractUserID(ctx))
        if err != nil {
            view.Warnings = append(view.Warnings, "user unavailable")
            return nil // do not fail the group
        }
        view.User = u
        return nil
    })

    g.Go(func() error {
        c, err := h.catalog.GetCatalog(ctx, extractSKU(ctx))

```

```

        if err != nil {
            view.Warnings = append(view.Warnings, "catalog
unavailable")
            return nil
        }
        view.Catalog = c
        return nil
    })

    if err := g.Wait(); err != nil {
        return nil, err // only hard failures (order) reach here
    }
    return &view, nil
}

// Downstream gRPC clients propagate deadline automatically via
context;
// for HTTP, inject as header so the callee can enforce it too:
func injectDeadline(req *http.Request, ctx context.Context) {
    if dl, ok := ctx.Deadline(); ok {
        ms := time.Until(dl).Milliseconds()
        if ms > 0 {
            req.Header.Set("x-deadline-ms", fmt.Sprint(ms))
        }
    }
    if tp := trace.SpanContextFromContext(ctx); tp.IsValid() {
        req.Header.Set("traceparent", fmt.Sprintf("00-%s-%s-01", tp.TraceID(), tp.SpanID()))
    }
}

```

Key decisions:

- **errgroup with selective failure** — required data (`order`) fails the view; enrichment data (`user` , `catalog`) degrades. This is the difference between a BFF and a naive fan-out that returns 500 when any dependency is slow.
- **Deadline propagation** — without it, downstream services use their own default timeouts (often 5-30s) and the BFF waits long after the client has given up. gRPC propagates deadlines automatically via `grpc-timeout`; HTTP requires an explicit header and callee cooperation.
- **No shared BFF** — if one BFF serves web, mobile, and partners, it becomes the gateway monolith again. Split by client class; share nothing except observability and deploy tooling.

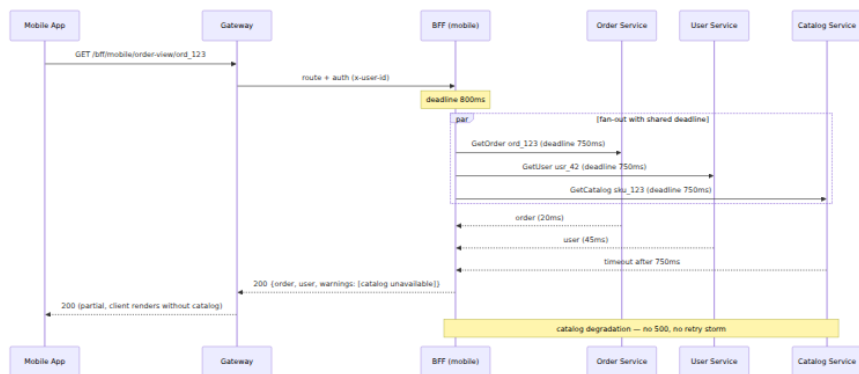


Figure 8-3: BFF fan-out with deadline propagation and graceful degradation. The BFF's deadline is the client's budget; downstream calls share it and degrade individually rather than failing the entire aggregation.

Edge — serve and protect before the origin

The edge is the CDN PoP — Cloudflare (300+ cities), Fastly, Akamai, CloudFront — where the request arrives before it traverses the internet to your origin. Running logic at the edge cuts latency (no origin round trip), reduces origin load (cache hits and early rejects), and enforces policy closest to the attacker.

What to run at the edge, in order of safety:

1. **Cache and serve** — static assets, API responses with `Cache-Control`, stale-while-revalidate. No code, just headers.
2. **Reject** — WAF, bot detection, rate limiting, IP allow/deny. Fail fast before origin cost.
3. **Authenticate and personalize** — validate JWT at the edge, inject `x-user-id`, route to the right origin shard, A/B bucket, and set `Vary` correctly.
4. **Compute** — render, transform, or aggregate at the edge (Workers/Compute@Edge) when the working set fits in the PoP's memory and isolation budget.

What *not* to run at the edge: authoritative writes, cross-request state that must be strongly consistent, or logic that needs secrets you cannot scope to the edge trust boundary.

Cloudflare Workers — auth and routing at the edge

Cloudflare Workers (V8 isolates, not containers — sub-millisecond cold starts, 128 MB per worker, CPU time limits) run on every PoP.


```

// edge/auth-router.js - Cloudflare Worker (workerd, 2024+)
// Validates JWT at the edge, routes to origin shard, injects headers
export default {
  async fetch(request, env, ctx) {
    const url = new URL(request.url);

    // 1 - health and static bypass auth
    if (url.pathname === "/health" || url.pathname.startsWith("/static/")) {
      return fetch(request);
    }

    // 2 - validate JWT at the edge (no origin round trip for bad tokens)
    const auth = request.headers.get("authorization");
    if (!auth || !auth.startsWith("Bearer ")) {
      return new Response("missing token", { status: 401 });
    }
    const token = auth.slice(7);
    let claims;
    try {
      claims = await verifyJWT(token, env.JWKS_URL); // cached JWKS,
see below
    } catch (e) {
      return new Response("invalid token", { status: 401 });
    }

    // 3 - route to origin shard by tenant (header-based sharding)
    const tenant = claims["custom:tenant_id"] || "default";
    const originHost = tenantShard(tenant); // e.g., "orders-us-east-1.internal"

    // 4 - A/B bucket at the edge (deterministic, no origin call)
    const bucket = hashBucket(claims.sub, "checkout-v2", 100);
    const variant = bucket < 10 ? "v2" : "v1";

    // 5 - forward with enriched headers, preserve tracing
    const outReq = new Request(request);
    outReq.headers.set("x-user-id", claims.sub);
    outReq.headers.set("x-tenant-id", tenant);
    outReq.headers.set("x-variant", variant);
    outReq.headers.set("x-edge-pop", request.cf.colo);
    // traceparent already present from gateway - pass through
    const outUrl = new URL(request.url);
    outUrl.hostname = originHost;

    // 6 - cache GETs at the edge (respect origin Cache-Control)
    if (request.method === "GET") {
      const cache = caches.default;
      let resp = await cache.match(outReq);

```

```

    if (resp) {
      resp = new Response(resp.body, resp);
      resp.headers.set("x-cache", "HIT");
      return resp;
    }
    const originResp = await fetch(new Request(outUrl, outReq));
    // only cache 200s with explicit cache-control
    if (originResp.ok && originResp.headers.get("cache-control")) {
      ctx.waitUntil(cache.put(outReq, originResp.clone()));
    }
    const r = new Response(originResp.body, originResp);
    r.headers.set("x-cache", "MISS");
    return r;
  }

  return fetch(new Request(outUrl, outReq));
},
};

async function verifyJWT(token, jwksUrl) {
  // Minimal – in production use `jose` or `cloudflare/workers-jwt`
  // JWKS is cached in Workers KV / Cache API with 5m TTL
  const jwks = await fetch(jwksUrl, { cf: { cacheTtl:
300 } }).then((r) => r.json());
  // ... signature verification, exp/nbf/aud checks ...
  return claims;
}

function tenantShard(tenant) {
  const shards = { "tenant-a": "orders-us-east-1.internal", "tenant-
b": "orders-eu-west-1.internal" };
  return shards[tenant] || "orders.internal";
}

function hashBucket(key, experiment, mod) {
  // FNV-1a or similar – deterministic per user per experiment
  let h = 2166136261;
  const s = `${key}:${experiment}`;
  for (let i = 0; i < s.length; i++) { h ^= s.charCodeAt(i); h = Math.i
mul(h, 16777619); }
  return (h >>> 0) % mod;
}

```

```
# wrangler.toml - Cloudflare Workers (Wrangler 3.x)
name = "auth-router"
main = "edge/auth-router.js"
compatibility_date = "2024-08-20"
route = { pattern = "api.example.com/*", zone_name = "example.com" }

[vars]
JWKS_URL = "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_abc123/.well-known/jwks.json"

[observability]
enabled = true
head_sampling_rate = 0.1
```

Fastly Compute — Rust at the edge with stronger isolation

Fastly Compute runs WebAssembly (Wasmtime) with a Rust/JS SDK, 128 MB, and no outbound connection pooling surprises.

```
// edge/fastly_auth/src/main.rs - Fastly Compute (Rust, fastly =0.10)
use fastly::{Request, Response, http::StatusCode};

#[fastly::main]
fn main(mut req: Request) -> Result<Response, fastly::Error> {
    // 1 - bypass for health
    if req.get_path() == "/health" {
        return
    }
    Ok(Response::from_status(StatusCode::OK).with_body("ok\n"));
    // 2 - JWT validation (JWKS cached in Config Store / KV)
    let token = req.get_header_str("authorization")
        .and_then(|v| v.strip_prefix("Bearer "))
        .ok_or_else(|| fastly::Error::msg("missing bearer"))?;
    let claims = verify_jwt(token)?; // uses jsonwebtoken crate, JWKS
    from Config Store

    // 3 - inject headers and route to origin shard
    let tenant = claims.tenant_id.as_deref().unwrap_or("default");
    let backend = match tenant {
        "tenant-a" => "orders_us_east_1",
        "tenant-b" => "orders_eu_west_1",
        _ => "orders_origin",
    };
    req.set_header("x-user-id", claims.sub);
    req.set_header("x-tenant-id", tenant);
    req.set_header("x-edge-pop", std::env::var("FASTLY_POP").unwrap_or_default());

    // 4 - stale-while-revalidate at the edge
    if req.get_method() == fastly::http::Method::GET {
        req.set_header("fastly-ttl", "60"); // cache 60s at edge
        req.set_header("fastly-stale-while-revalidate", "300");
    }

    // 5 - forward - Fastly handles connection pooling to named backend
    Ok(req.send(backend?))
}
```

Edge vs. origin — the consistency and trust boundary

The edge is eventually consistent with the origin by construction: cache entries are stale until revalidated, Config Store / KV writes propagate in

seconds to minutes, and a PoP that has not seen a JWKS rotation will accept a revoked token until its cache TTL expires. Design for it:

- **Auth at the edge** is best-effort fast path; the origin must re-validate for authoritative decisions. The edge rejects the obviously bad; the origin is the source of truth.
 - **Writes never complete at the edge** — they proxy to the origin. The edge can validate, rate-limit, and enrich, but the commit happens at the origin. Patterns like Cloudflare Durable Objects and CRDTs at the edge exist but are not a substitute for origin consensus.
 - **Cache keys must include Vary dimensions** — Vary: Authorization, Accept-Language, x-variant or the edge serves one user's personalized response to another.
-

Operating gateways at scale — availability, observability, canary

A gateway is a single point of failure if deployed as one. Production gateways are a *fleet* behind anycast or DNS, with independent failure domains per AZ and region.

Checklist for a gateway fleet that does not become the outage:

- **Anycast or GSLB in front** — one IP advertised from multiple PoPs/regions via BGP anycast (Cloudflare, GCP Cloud Load Balancing) or DNS with health checks (Route 53). No single LB node is special.
- **Per-AZ gateway replicas** — at least 2 per AZ, 3 AZs per region. Rolling deploys one replica at a time; health checks drain before termination.
- **No shared mutable state on the data path** — rate-limit counters in Redis (with fail-open), JWKS cached locally with TTL, route table from xDS/config reload — no DB query per request.
- **Progressive delivery for route changes** — canary gateway config (new route, new plugin) to 1% of traffic via header or weight, with automatic rollback on error-rate SLO breach (see Volume 11, Chapter 9).
- **Gateway is stateless** — any instance can handle any request. Session affinity at the gateway defeats horizontal scaling and creates hot shards.

Observability per route (not per gateway instance):

```
# Prometheus recording rules – gateway SLOs per route (Prometheus 2.53)
groups:
- name: gateway_slo
  interval: 30s
  rules:
  - record: gateway:request_rate:by_route_5m
    expr: sum by (route) (rate(envoy_cluster_upstream_rq_total[5m]))
  - record: gateway:error_rate:by_route_5m
    expr: |
      sum by (route)
      (rate(envoy_cluster_upstream_rq_xx{response_code_class="5xx"}[5m]))
      / sum by (route) (rate(envoy_cluster_upstream_rq_total[5m]))
  - record: gateway:p99_latency:by_route_5m
    expr: histogram_quantile(0.99, sum by (route, le)
(rate(envoy_cluster_upstream_rq_duration_bucket[5m])))
  - alert: GatewayRouteHighErrorRate
    expr: gateway:error_rate:by_route_5m > 0.01
    for: 5m
    labels: { severity: page }
    annotations:
      summary: "Route {{ $labels.route }} 5xx > 1% for 5m"
  - alert: GatewayRateLimitServiceDown
    expr: increase(envoy_cluster_circuit_breakers_default_cx_open[5m])
    > 0
    labels: { severity: ticket }
    annotations:
      summary: "Rate-limit service circuit breaker opened – gateway is
fail-open"
```

Access logs must be structured JSON with `route`, `upstream_host`, `duration`, `response_code`, `trace_id`, and `x-user-id` (if authenticated) — otherwise debugging "why did tenant-a see 429?" requires correlating N log formats.

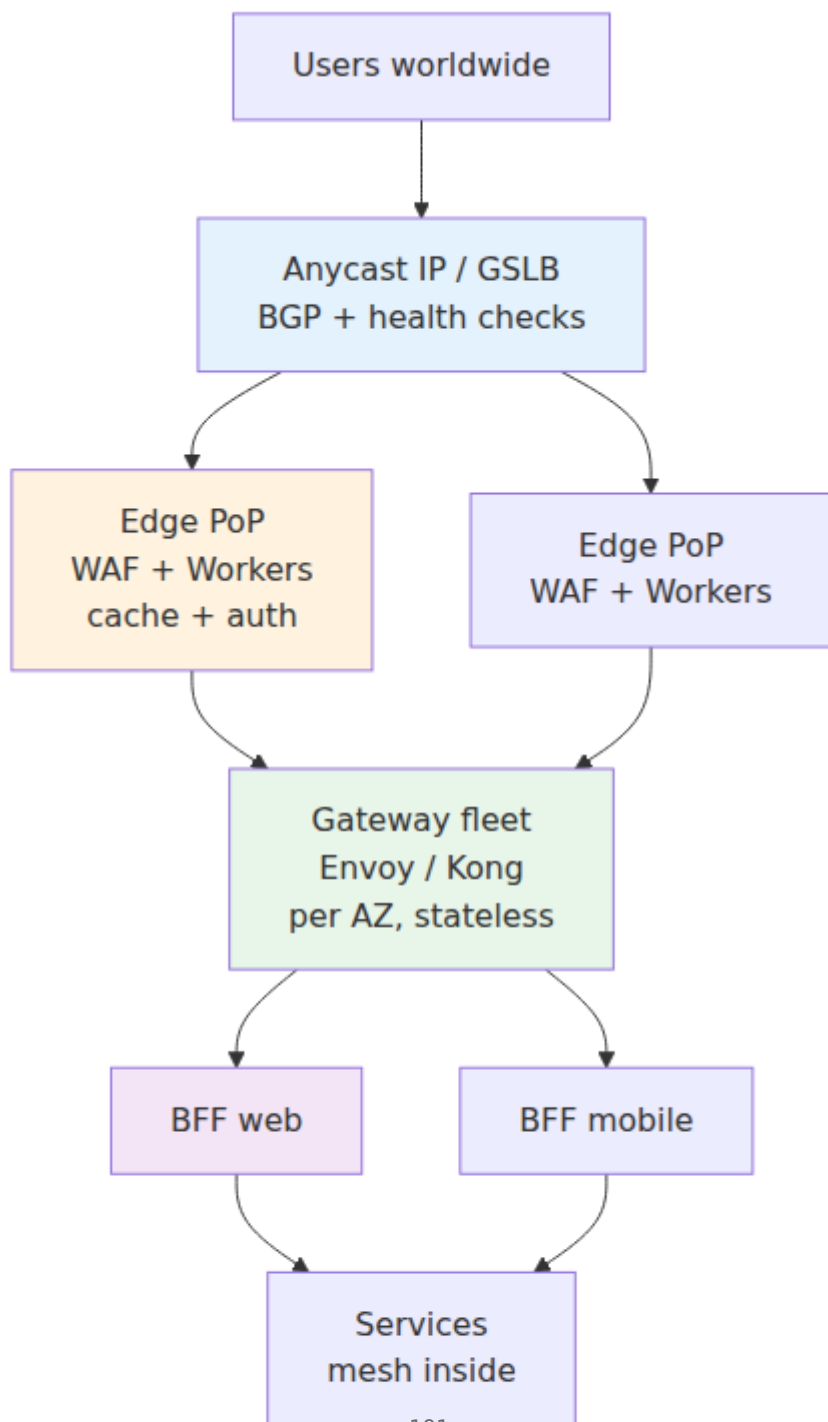


Figure 8-4: From users to origin through edge and gateway. Anycast/GSLB picks the PoP; the edge caches and authenticates; the gateway fleet (per AZ, stateless) routes to BFFs and services. No single box is special; every layer is horizontally scaled and independently deployable.

The distributed-systems lens:

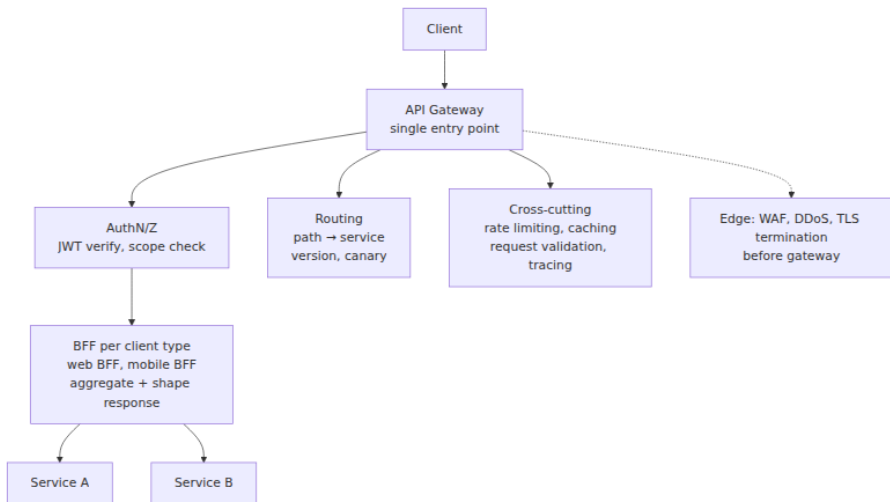
- **Policy placement is a consistency trade-off.** Gateway-enforced policy (auth, rate limits, WAF) is consistent at the perimeter but stale if the gateway's view of truth (JWKS, quota counters) lags the origin. Mesh-enforced policy is fresher (closer to the service) but multiplies enforcement points. Most systems enforce *coarsely* at the gateway (is this token valid? is this tenant over quota?) and *finely* at the service (can this user mutate this resource?).
 - **The gateway amplifies failures.** A bad gateway config (route loop, auth regression, rate-limit misconfiguration) affects *every* route simultaneously. Treat gateway config changes with the same rigor as database migrations: canary, automatic rollback, and a break-glass bypass (direct-to-service via mesh for operators).
 - **Edge and origin form a split-brain system.** The edge's cache and KV are eventually consistent with the origin. Never treat an edge write (cache purge, KV update) as synchronous — design idempotent purges and tolerate stale reads within the TTL window.
-

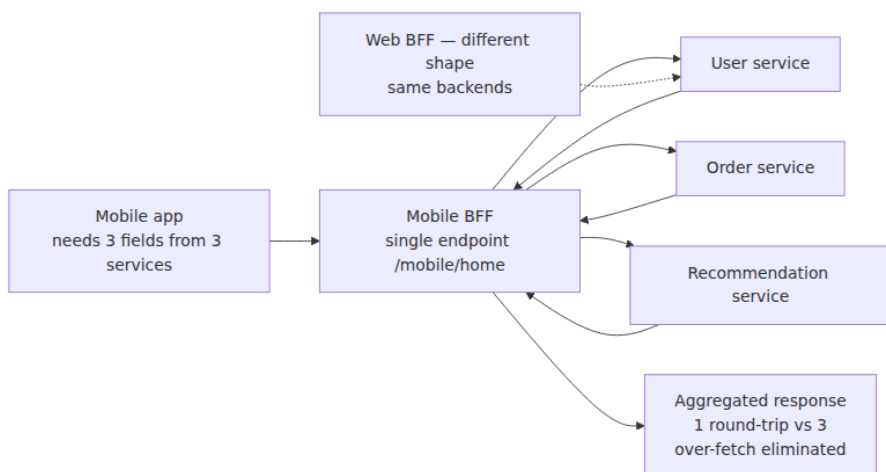
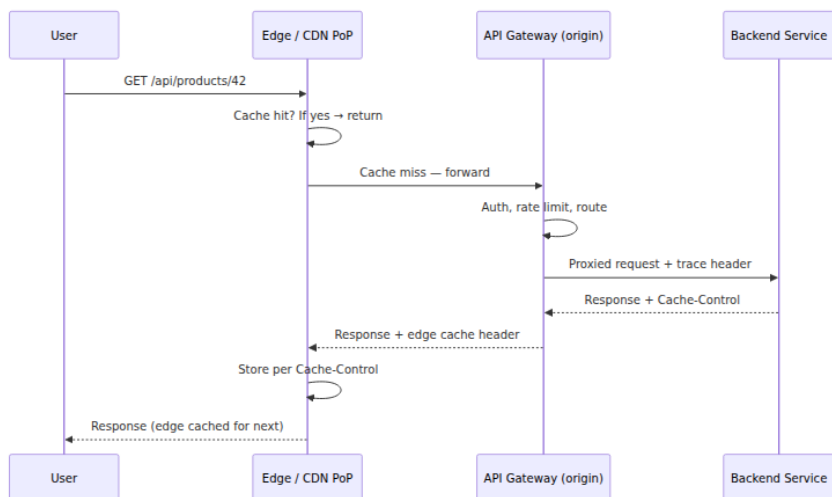
Key takeaways

- A gateway centralizes cross-cutting concerns (auth, routing, rate limiting, transformation, observability) so services stay focused on business logic — but it must not accumulate business logic itself or it becomes the most coupled component in the system.
- Envoy as a gateway composes listeners, filter chains (JWT, rate limit, router), and clusters with outlier detection and circuit breakers; Kong provides the same policy via declarative config and plugins — both delegate quota state to an external store.
- Filter ordering matters: authenticate first (fail fast, no upstream cost), rate-limit second (protect backends), route last — each filter can short-circuit without reaching upstream.
- A BFF is a per-client-class aggregation layer owned by the client team, not a shared service — it fans out with deadline propagation

and degrades gracefully when enrichment dependencies are slow, rather than failing the entire view.

- Edge computing (Cloudflare Workers, Fastly Compute) cuts latency and origin load by authenticating, routing, and caching at the PoP — but the edge is eventually consistent with the origin, so writes still commit at the origin and cache keys must include all Vary dimensions.
- Gateway fleets must be stateless, per-AZ, behind anycast/GSLB, with no per-request DB on the data path, progressive delivery for config changes, and per-route SLOs — otherwise the gateway is the single point of failure it was meant to eliminate.
- Policy placement is a consistency trade-off: enforce coarsely at the gateway (valid token? over quota?) and finely at the owning service (can this user mutate this resource?).





Further reading

- Envoy 1.30 documentation — listeners, filter chains, JWT and rate-limit filters, outlier detection: <https://www.envoyproxy.io/docs/envoy/v1.30.0/>
- Kong Gateway 3.7 — declarative configuration, JWT and rate-limiting plugins: <https://docs.konghq.com/gateway/3.7.x/>

- Newman, S. *Building Microservices* (2nd ed.), Chapter 8 — API gateways and BFF: https://samnewman.io/books/building_microservices_2nd_edition/
- Cloudflare Workers documentation — isolates, routing, caching, and observability: <https://developers.cloudflare.com/workers/>
- Fastly Compute documentation — Rust SDK, backends, and edge caching: <https://developer.fastly.com/learning/compute/>
- Google Cloud Load Balancing — anycast and GSLB architecture: <https://cloud.google.com/load-balancing/docs/load-balancing-overview>
- W3C Trace Context — `traceparent` propagation through gateway and BFF hops: <https://www.w3.org/TR/trace-context/>

Chapter 9 — Rate Limiting, Quotas, and Fairness

What this chapter covers. Chapters 4 and 8 can route and authenticate every request, but without limits a single misbehaving client — a runaway cron job, a scrape loop, a compromised API key — can consume the entire fleet's capacity and violate every tenant's SLO. Rate limiting is the mechanism that turns "best effort for everyone" into "guaranteed fair share for each." This chapter builds the primitives from first principles, makes them operational at scale, and shows how quotas (business-level budgets) and fairness policies (who gets what when capacity is scarce) compose on top. We compare the four algorithms that actually matter in production — fixed window, sliding window (log and counter), token bucket, and leaky bucket — on burst handling, memory cost, and behavior under clock skew, then implement them where they run: local in-process limiters (Go `x/time/rate`, Guava), distributed limiters backed by Redis 7.2 with atomic Lua, and Envoy's external `ratelimit` gRPC service as the gateway integration point. The second half covers quota design (per-tenant, per-route, tiered plans), fairness strategies (weighted fair queuing, priority lanes, and shed-vs-queue choices), and the operational realities of global limits — why local counters diverge, how to bound the error with probabilistic reconciliation, and when to accept eventual enforcement rather than pay the coordination cost. The chapter

closes with the distributed-systems lens on why rate limiting is a consistency-availability trade-off in disguise.

Learning goals — after this chapter you should be able to:

- Compare fixed window, sliding window log/counter, token bucket, and leaky bucket on burst allowance, memory, precision at window boundaries, and sensitivity to clock skew, and choose per use case.
- Implement a correct token-bucket and sliding-window counter in application code and as an atomic Redis 7.2 Lua script, explain the failure mode if the script is not atomic, and handle Redis unavailability with fail-open vs. fail-closed semantics.
- Configure Envoy 1.30's external rate-limit service (Lyft `ratelimit` with Redis), Kong Gateway 3.7, and NGINX `limit_req` for gateway-level enforcement with per-route and per-tenant descriptors.
- Design a quota system that composes per-second rate limits, per-hour/day quotas, and burst allowances into tiered plans, with headers (`RateLimit-*` , `Retry-After`) and over-quota UX that does not create retry storms.
- Apply fairness strategies — weighted limits, priority lanes, and load shedding vs. queuing — to protect SLOs when the system is saturated, and explain when each harms tail latency.
- Reason about distributed rate limiting's fundamental trade-off: local counters are available but imprecise ($N\times$ over-allowance), global counters are precise but add latency and become a single point of failure — and choose the reconciliation strategy (sync interval, probabilistic allowance, or hierarchical caps) that bounds the error for your SLO.

Why "just add more capacity" does not solve fairness

Rate limiting is often framed as protection against abuse. In practice it protects against four distinct failure modes, only one of which is malicious:

| Threat | Example | What fails without limits |
|----------------------------------|---|---|
| Buggy client | Deploy pushes a tight retry loop (1k RPS per instance × 500 instances) | Backend thread pools exhaust; latency spikes for all tenants |
| Noisy neighbor | Tenant A backfills 10M rows through the same API tenant B uses interactively | Tenant B's p99 doubles; SLO breach despite no code change |
| Cost and quota exhaustion | LLM or search API billed per token/query; one key burns a month's budget in an hour | Business cost, downstream provider throttles everyone |
| Intentional abuse / DoS | Credential stuffing, scraping, volumetric flood | Availability loss, WAF bypass if rate limiting is the only line |

The distributed-systems consequence: without per-client limits, the system's *observed* capacity is the capacity of its *most aggressive client*. Every SLO becomes "as good as the worst tenant's behavior." Rate limiting makes each tenant's experience independent of others' load — the isolation primitive that lets Chapter 4's load balancing and Chapter 8's gateway actually deliver on their promises.

Quotas and fairness refine the guarantee:

- **Rate limit** — "at most R requests per second, with burst B." A *mechanism* enforced on the data path.
- **Quota** — "at most Q requests per hour/day/month." A *business contract* that may span multiple rate-limit windows and may be enforced asynchronously.
- **Fairness** — "when demand exceeds capacity, who is throttled first and who is protected." A *policy* that composes limits with priority.

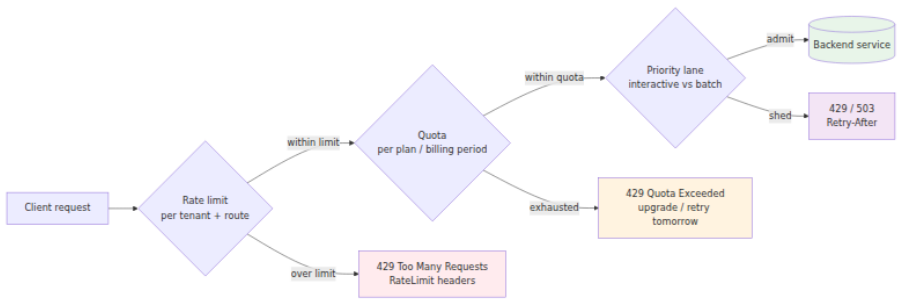


Figure 9-1: The enforcement stack. Rate limits protect per-second capacity, quotas enforce business budgets over longer windows, and fairness decides who is shed when both are saturated. Each layer returns distinct headers so clients can distinguish "slow down" from "out of budget."

Boundary note. This chapter treats rate limiting as a *system-design mechanism* — algorithm choice, gateway/service placement, quota composition, and fairness policy at fleet scale. The *algorithmic analysis* of the same structures — amortized cost, probabilistic structures for sliding windows, consistent hashing for sharded limits, and scheduling theory (WFQ, deficit round-robin) — is in Volume 14, Chapter 8 — Rate-Limiting and Scheduling Algorithms. Wire-level reliability that interacts with throttling (timeouts, retries, backoff, hedging) is in Volume 3, Chapter 11 — Network Reliability. Client-side retry discipline after receiving 429/503 is covered there; this chapter covers the server-side enforcement and header contract.

The four algorithms — and when each is the right one

Every production limiter is a variant of one of four ideas. Understanding their burst behavior and window-boundary precision is what separates a limiter that protects backends from one that merely counts.

1. Fixed window counter

Count requests in the current window (e.g., 60 seconds). Reset to zero at the boundary. Allow if `count < limit`.

Pros: trivial, $O(1)$ memory (one counter per key), easy to implement in Redis with `INCR` + `EXPIRE`. Cons: **boundary spike** — 100 requests at 00:59 and 100 at 01:01 both pass a "100/min" limit but deliver 200 in 2 seconds. Under bursty traffic the effective limit is $2\times$ the configured one.

```
Window 1 (00:00-01:00): ██████████ 100 at 00:59 — allowed (count=100)
Window 2 (01:00-02:00): ██████████ 100 at 01:01 — allowed (count reset
to 0 → 100)
Actual in 2s window:      200 — double the intended rate
```

Never use fixed window alone for protecting latency-sensitive backends; the spike *is* the failure mode you were trying to prevent.

2. Sliding window log and sliding window counter

Sliding window log — store a timestamp per request, evict entries older than the window, count the remainder. Precise (true sliding window), but $O(N)$ memory per key — prohibitive for high-cardinality keys (per-user, per-IP) at high rates.

Sliding window counter (hybrid) — keep two fixed-window counters (previous and current) and weight the previous window by overlap. $\text{effective} = \text{prev_count} \times (1 - \text{overlap_fraction}) + \text{current_count}$. $O(1)$ memory, $\sim 1\%$ error vs. true sliding window, and the standard choice for gateway-level limits.

Example: limit 100/min, at 01:15 (25% into current window), previous window had 80, current has 30. $\text{Effective} = 80 \times 0.75 + 30 = 90 \rightarrow \text{allow}$. At 01:45 with 60 in current, $\text{effective} = 80 \times 0.25 + 60 = 80 \rightarrow \text{allow}$. Smooths the boundary without per-request storage.

Cloudflare, Kong, and the Envoy `ratelimit` service all implement this variant by default.

3. Token bucket

A bucket holds at most `capacity` tokens, refilled at `refill_rate` per second. Each request consumes one token; if the bucket is empty, the request is throttled. Buckets can be described with two numbers: *sustained rate* and *burst*.

Pros: allows bursts up to `capacity` while enforcing a long-term rate — matches how backends actually behave (they can absorb a short burst but not a sustained overload). $O(1)$ memory. The canonical choice for API

rate limits ("100 RPS, burst 200"). Cons: burst size must be tuned — too large and bursts still overwhelm backends; too small and legitimate spikes are throttled.

4. Leaky bucket (and GCRA)

Requests enter a queue drained at a fixed rate; if the queue is full, requests are rejected. Equivalent to token bucket with the burst constrained to one — it *smooths* rather than *allows* bursts. GCRA (Generic Cell Rate Algorithm) is the formalization used in telecom and in some Redis modules.

Use leaky bucket when you need *shaping* (smooth the output to a constant rate — e.g., webhook delivery, outbound email) rather than *policing* (allow bursts but cap the average). For inbound API protection, token bucket is almost always the better fit.

| Algorithm | Burst | Memory/
key | Precision | When to use |
|-------------------------------|----------------------|----------------|---------------------|---|
| Fixed window | 2× spike at boundary | O(1) | Poor at boundaries | Never alone for latency-sensitive paths; OK for coarse daily quotas |
| Sliding window log | None | O(N) | Exact | Low-cardinality keys where precision justifies cost |
| Sliding window counter | ~1% overshoot | O(1) | Approximate, smooth | Gateway per-route/ per-tenant limits — best general default |
| Token bucket | Bounded by capacity | O(1) | Exact per token | API rate limits with explicit burst ("100/s, burst 200") |
| Leaky bucket / GCRA | None (smooths) | O(1) | Exact per drain | Outbound shaping (webhooks, queue consumers) |



Figure 9-2: Burst behavior of the four algorithms. Fixed window's boundary spike is the hazard; sliding-window counter smooths it; token bucket explicitly bounds bursts; leaky bucket eliminates them.

Local limiter — correct token bucket in application code

Every service should have a local limiter as the last line of defense, independent of the gateway. It protects against gateway bypass (internal callers), gateway failure (fail-open), and per-instance overload that a global counter cannot see.

Go's `golang.org/x/time/rate` (the standard library's rate limiter, used inside Kubernetes and many backends) is a token-bucket with a clean API:

```
// local_limiter.go - Go 1.22, golang.org/x/time/rate
package limits

import (
    "context"
    "golang.org/x/time/rate"
    "time"
)

// Per-tenant limiters - one bucket per tenant, bounded map with TTL
type TenantLimiter struct {
    limiters map[string]*rate.Limiter
    mu       sync.RWMutex
    r        rate.Limit // tokens per second
    burst    int
}

func NewTenantLimiter(rps int, burst int) *TenantLimiter {
    return &TenantLimiter{
        limiters: make(map[string]*rate.Limiter),
        r:        rate.Limit(rps),
        burst:    burst,
    }
}

func (t *TenantLimiter) Allow(tenant string) bool {
    t.mu.RLock()
    lim, ok := t.limiters[tenant]
    t.mu.RUnlock()
    if !ok {
        t.mu.Lock()
        // double-check after acquiring write lock
        if lim, ok = t.limiters[tenant]; !ok {
            lim = rate.NewLimiter(t.r, t.burst)
            t.limiters[tenant] = lim
        }
        t.mu.Unlock()
    }
    return lim.Allow()
}

// Allow with context - waits up to deadline for a token (for non-
// interactive work)
func (t *TenantLimiter) Wait(ctx context.Context, tenant string) error {
    t.mu.RLock()
    lim, ok := t.limiters[tenant]
    t.mu.RUnlock()
    if !ok {
        t.mu.Lock()

```

```

    if lim, ok = t.limiters[tenant]; !ok {
        lim = rate.NewLimiter(t.r, t.burst)
        t.limiters[tenant] = lim
    }
    t.mu.Unlock()
}
return lim.Wait(ctx) // blocks until token available or ctx
cancelled
}

// Evict idle tenants to bound memory – call periodically
func (t *TenantLimiter) EvictIdle(ttl time.Duration) {
    // In production, track last-access per limiter (e.g., sync.Map
    with timestamp)
    // and delete entries idle > ttl. Omitted for brevity – use
    github.com/hashicorp/golang-lru
    // or a TTL cache like github.com/jellydator/ttlcache for high-
    cardinality keys.
}

```

Python equivalent for scripting and data-plane helpers:

```

# token_bucket.py – Python 3.11, no dependencies
import time
import threading

class TokenBucket:
    def __init__(self, rate: float, capacity: int):
        self.rate = rate           # tokens per second (sustained)
        self.capacity = capacity   # max burst
        self.tokens = float(capacity)
        self.updated_at = time.monotonic()
        self.lock = threading.Lock()

    def allow(self, tokens: int = 1) -> bool:
        with self.lock:
            now = time.monotonic()

# refill based on elapsed time – monotonic clock, immune to NTP skew
            elapsed = now - self.updated_at
            self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)

            self.updated_at = now
            if self.tokens >= tokens:
                self.tokens -= tokens
                return True
            return False

    def retry_after(self, tokens: int = 1) -> float:
        with self.lock:
            deficit = tokens - self.tokens
            return max(0.0, deficit / self.rate) if deficit > 0 else 0.0

```

Critical detail: **use `time.monotonic()` / `clock_gettime(CLOCK_MONOTONIC)`**, never wall-clock time. Wall clocks jump on NTP correction and leap seconds; a backward jump refills the bucket spuriously or, worse, a forward jump empties it and throttles all traffic.

Distributed limiter — Redis 7.2 with atomic Lua

Local buckets diverge: 10 gateway replicas each allowing 100 RPS actually allow 1000 RPS globally. For per-tenant or global limits that must be precise, the counter must be shared. Redis 7.2 is the standard shared store — single-threaded execution makes Lua scripts atomic without distributed locks.

Token bucket in Redis — atomic Lua

```
-- token_bucket.lua — Redis 7.2 Lua, atomic token bucket
-- KEYS[1] = bucket key (e.g., "ratelimit:tenant:{id}:orders")
-- ARGV[1] = capacity (burst)
-- ARGV[2] = refill_rate (tokens per second)
-- ARGV[3] = requested tokens (usually 1)
-- ARGV[4] = now_ms (client-supplied monotonic-ish millis; see note)
-- Returns: { allowed (1/0), remaining, retry_after_ms }

local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local requested = tonumber(ARGV[3])
local now_ms = tonumber(ARGV[4])

local data = redis.call("HMGET", key, "tokens", "updated_at")
local tokens = tonumber(data[1])
local updated_at = tonumber(data[2])

if tokens == nil then
    tokens = capacity
    updated_at = now_ms
else
    local elapsed_ms = now_ms - updated_at
    if elapsed_ms < 0 then elapsed_ms = 0 end -- clock skew guard
    local refill = elapsed_ms * refill_rate / 1000.0
    tokens = math.min(capacity, tokens + refill)
    updated_at = now_ms
end

local allowed = 0
local retry_after_ms = 0
if tokens >= requested then
    tokens = tokens - requested
    allowed = 1
else
    local deficit = requested - tokens
    retry_after_ms = math.ceil(deficit / refill_rate * 1000)
end

-- TTL: time to fully refill + 60s slack, so idle keys expire
local ttl_ms = math.ceil(capacity / refill_rate * 1000) + 60000
redis.call("HMSET", key, "tokens", tokens, "updated_at", updated_at)
redis.call("PEXPIRE", key, ttl_ms)

local remaining = math.floor(tokens)
return {allowed, remaining, retry_after_ms}
```

Invoke from application or gateway helper:

```
# Test with redis-cli (Redis 7.2)
redis-cli --eval token_bucket.lua ratelimit:tenant:42:orders , 200 100
1 1710000000000
# → 1) (integer) 1    -- allowed
#    2) (integer) 199  -- remaining
#    3) (integer) 0    -- retry_after_ms

# Under contention – 201st request in same millisecond
redis-cli --eval token_bucket.lua ratelimit:tenant:42:orders , 200 100
1 1710000000001
# → 1) (integer) 0    -- throttled
#    2) (integer) 0
#    3) (integer) 10  -- retry after 10ms (1 token / 100 per sec)
```

```
# redis_limiter.py – Python 3.11, redis-py 5.0, atomic Lua
import time, redis

r = redis.Redis(host="redis.internal", port=6379,
decode_responses=True)
with open("token_bucket.lua") as f:
    script = f.read()
token_bucket = r.register_script(script)

def allow(tenant: str, route: str, capacity=200, rate=100) -> dict:
    key = f"ratelimit:tenant:{tenant}:{route}"
    now_ms = int(time.time() * 1000) # wall clock here is OK – Lua
guards skew
    allowed, remaining, retry_after_ms = token_bucket(
        keys=[key], args=[capacity, rate, 1, now_ms]
    )
    return {
        "allowed": bool(allowed),
        "remaining": remaining,
        "retry_after_ms": retry_after_ms,
    }
```

Sliding window counter in Redis — the gateway default

More precise at window boundaries than token bucket for fixed "N per minute" semantics; preferred for tiered plan quotas.

```

-- sliding_window.lua - Redis 7.2, sliding window counter (2-window
hybrid)
-- KEYS[1] = current window key, KEYS[2] = previous window key
-- ARGV[1] = limit, ARGV[2] = window_ms, ARGV[3] = now_ms
-- Returns: { allowed, remaining, retry_after_ms }

local cur_key = KEYS[1]
local prev_key = KEYS[2]
local limit = tonumber(ARGV[1])
local window_ms = tonumber(ARGV[2])
local now_ms = tonumber(ARGV[3])

local cur_count = tonumber(redis.call("GET", cur_key) or "0")
local prev_count = tonumber(redis.call("GET", prev_key) or "0")

-- overlap fraction of previous window still in the sliding window
local elapsed_in_cur = now_ms % window_ms
local overlap = (window_ms - elapsed_in_cur) / window_ms
local effective = prev_count * overlap + cur_count

if effective >= limit then
    local retry_after = window_ms - elapsed_in_cur
    return {0, 0, retry_after}
end

-- allow - increment current window
cur_count = redis.call("INCR", cur_key)
if cur_count == 1 then
    redis.call("PEXPIRE", cur_key, window_ms * 2)
end
-- ensure prev key has TTL if it exists
redis.call("PEXPIRE", prev_key, window_ms * 2)

local remaining = math.max(0, limit - math.ceil(effective) - 1)
local retry_after = 0
return {1, remaining, retry_after}

```

Production note: for high-cardinality keys (per-IP, per-user at millions of distinct keys), storing two keys per limiter doubles memory. Alternatives: Redis Cell module (`CL.THROTTLE` — GCRA-based, single key, used by Lyft's `ratelimit` service) or a single sorted-set sliding log with `ZREMRANGEBYSCORE` (precise but heavier). Evaluate memory with `MEMORY USAGE` and `INFO memory` before choosing.

Handling Redis failure — fail-open vs. fail-closed

Redis is now on the data path. When it is down, two choices:

- **Fail-open** (allow traffic) — preserves availability; risks backend overload during the Redis outage. Correct for most user-facing APIs where a brief over-allowance is preferable to a global 503.
- **Fail-closed** (reject traffic) — preserves backend safety; turns a Redis outage into a full API outage. Correct only for cost-bound or safety-critical limits (billing, abuse).

Implement both: fail-open for per-tenant rate limits, fail-closed for global abuse caps — and alert on `redis_up == 0` so the window of imprecision is short.

```
def allow_with_fallback(tenant, route):
    try:
        return allow(tenant, route)
    except redis.ConnectionError:
        # metric + alert — we are now imprecise
        metrics.incr("ratelimit.redis_unavailable")
        # Fail-open for rate limits, fail-closed for abuse caps
        if route in ABUSE_CAPS:
            return {"allowed": False, "remaining": 0,
                "retry_after_ms": 1000}
        return {"allowed": True, "remaining": -1, "retry_after_ms": 0}
```

Gateway integration — Envoy external rate limiting

Envoy delegates rate-limit decisions to an external gRPC service so the data plane stays fast and the policy lives in one place. The standard implementation is Lyft's `ratelimit` (Go, Redis-backed), which speaks Envoy's `RateLimitService` API.

Envoy 1.30 — route descriptors

Building on the gateway from Chapter 8, add per-route and per-tenant descriptors:


```

# envoy-gateway-ratelimit.yaml – additions to the Chapter 8 gateway
static_resources:
  listeners:
    - name: https_ingress
      filter_chains:
        - filters:
            - name: envoy.filters.network.http_connection_manager
              typed_config:
                "@type": type.googleapis.com/
envoy.extensions.filters.network.http_connection_manager.v3.HttpConnect
ionManager
          http_filters:
            - name: envoy.filters.http.ratelimit
              typed_config:
                "@type": type.googleapis.com/
envoy.extensions.filters.http.ratelimit.v3.RateLimit
                domain: api_gateway
                request_type: external
                rate_limit_service:
                  grpc_service:
                    envoy_grpc: { cluster_name: ratelimit }
                    transport_api_version: V3
                failure_mode_deny: false
                enable_x_ratelimit_headers: DRAFT_VERSION_03 #
RateLimit-Limit/Remaining/Reset
                rate_limited_as_resource_exhausted: true # 429, not
500
          route_config:
            virtual_hosts:
              - name: api
                domains: ["api.example.com"]
                routes:
                  - match: { prefix: /api/v1/orders }
                    route: { cluster: orders, timeout: 3s }
                    typed_per_filter_config:
                      envoy.filters.http.ratelimit:
                        "@type": type.googleapis.com/
envoy.extensions.filters.http.ratelimit.v3.RateLimitPerRoute
                      rate_limits:
                        # Descriptor: per-tenant (header x-tenant-id) +
per-route
                  - actions:
                      - request_headers: { header_name: x-tenant-id,
descriptor_key: tenant_id }
                      - generic_key: { descriptor_value: orders_write }
                        # Global abuse cap – per-IP
                  - actions:
                      - remote_address: {}
                      - generic_key: { descriptor_value: global_ip }

```

Lyft ratelimit — configuration

```
# ratelimit-config.yaml — Lyft ratelimit (v2, Redis 7.2)
domain: api_gateway
descriptors:
  # Per-tenant orders write: 100/s burst 200, 10000/hour
  - key: tenant_id
    rate_limit:
      requests_per_unit: 100
      unit: second
  - key: tenant_id
    value: tenant_a          # override for high-tier tenant
    rate_limit:
      requests_per_unit: 1000
      unit: second
  - key: tenant_id
    rate_limit:
      requests_per_unit: 10000
      unit: hour
  # Per-route — orders_write sub-key
  - key: orders_write
    rate_limit:
      requests_per_unit: 200
      unit: second
  # Global per-IP abuse cap
  - key: global_ip
    rate_limit:
      requests_per_unit: 50
      unit: second

# ratelimit deployment — Envoy cluster points here
# ratelimit.yaml — Kubernetes Deployment (ratelimit v2, image:
# envoyproxy/ratelimit:v1.6.0)
# Redis backend — single primary with replica; ratelimit handles
# failover
storage:
  type: redis
  redis:
    address: redis.internal:6379
    pool_size: 20
```

The `ratelimit` service implements GCRA/sliding-window internally and returns `OVER_LIMIT` with `RateLimit` headers; Envoy translates that to 429 with `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After`. No Lua needed — but Redis is still on the path, with the same fail-open/fail-closed choice (`failure_mode_deny`).

Kong and NGINX — alternatives at the gateway

```
# kong.yaml – Kong Gateway 3.7, per-consumer rate limiting (DB-less,
Redis)
services:
- name: orders
  url: http://orders.internal:8080
  routes:
  - name: orders-v1
    paths: [/api/v1/orders]
    plugins:
    - name: rate-limiting
      config:
        minute: 600
        hour: 10000
        policy: redis
        redis_host: redis.internal
        redis_port: 6379
        fault_tolerant: true      # fail-open when Redis down
        hide_client_headers: false
        limit_by: consumer
# per-consumer (JWT claim) – also: ip, header, service
```

```
# nginx.conf – NGINX 1.25, local token bucket (no Redis – per-worker,
imprecise across replicas)
limit_req_zone $http_x_tenant_id zone=tenant_orders:10m rate=100r/s;
limit_req_zone $binary_remote_addr zone=ip_global:10m rate=50r/s;
limit_req_status 429;
limit_req_log_level notice;

server {
  listen 443 ssl;
  location /api/v1/orders {
    limit_req zone=tenant_orders burst=200 nodelay;
    limit_req zone=ip_global burst=50 nodelay;
    proxy_pass http://orders.internal;
    add_header RateLimit-Limit $limit_req_remaining always;
    add_header Retry-After $limit_req_retry_after always;
  }
}
```

NGINX's `limit_req` is per-worker, not shared — with 8 workers, the effective limit is 8× the configured value unless `limit_req_dry_run` and shared memory are tuned. For precise multi-replica limits, delegate to Redis or the Envoy external service rather than relying on `limit_req` alone.

Quotas, tiers, and the header contract

Rate limits protect per-second capacity; quotas enforce business budgets over hours, days, or months. A sane plan composes both:

| Tier | Per-second (burst) | Per-minute | Per-hour | Per-day | Over-quota behavior |
|------------|--------------------|------------|-----------|----------|----------------------|
| Free | 10/s (20) | 200/min | 5k/hour | 50k/day | 429 + upgrade CTA |
| Pro | 100/s (200) | 2k/min | 50k/hour | 500k/day | 429 + Retry-After |
| Enterprise | 1000/s (2000) | 20k/min | 500k/hour | 5M/day | 429, dedicated shard |

Implementation: hourly/daily quotas are counters in Redis or the billing DB, incremented per request and reset on a schedule. They need not be on the synchronous data path — an async counter (Kafka → aggregator → Redis) with a short staleness window (5-10s) is often sufficient, trading immediate precision for lower per-request latency. For billing-critical quotas (paid API with hard caps), enforce synchronously.

The header contract — IETF RateLimit fields (RFC draft, widely adopted)

Every throttled or near-limit response should include:

```
HTTP/1.1 429 Too Many Requests
RateLimit-Limit: 100
RateLimit-Remaining: 0
RateLimit-Reset: 42
Retry-After: 42
Content-Type: application/json

{"error":"rate_limited","message":"per-tenant limit 100/s exceeded","retry_after_ms":42}
```

And successful responses near the limit should warn:

```
HTTP/1.1 200 OK
RateLimit-Limit: 100
RateLimit-Remaining: 3
RateLimit-Reset: 12
```

Envoy's `enable_x_ratelimit_headers: DRAFT_VERSION_03` emits these automatically; Kong's `hide_client_headers: false` does the same. Clients must respect `Retry-After` with jittered backoff — see Volume 3, Chapter 11 for the retry discipline that prevents thundering herds after a 429.

Over-quota UX must not create a retry storm: return 429 (not 503), include `Retry-After`, use exponential backoff with jitter on the client, and consider a 429 response cache at the edge so repeated over-quota requests from the same client do not each hit the origin.

Fairness — who gets throttled when everyone is over

When the fleet is saturated, naive "first-come first-served" throttling punishes well-behaved tenants for noisy neighbors' load. Fairness policies decide the shedding order.

Weighted limits and priority lanes

- **Weighted limits** — each tenant's limit is proportional to their tier weight. Enterprise tenants get 10× the tokens of free tenants from the same shared bucket. Implement as per-tenant buckets with tier-proportional `capacity / rate` (simplest) or as a single WFQ scheduler (more precise, more complex).
- **Priority lanes** — classify requests by criticality: `priority: interactive` (user-facing), `priority: batch` (backfill, analytics), `priority: internal` (cron). When saturated, shed batch first, then internal, then interactive last. Envoy supports this via `priority` on routes and `overload_manager` with shedding thresholds.
- **Queue vs. shed** — queuing (buffer and drain at capacity) trades latency for throughput; shedding (immediate 429) trades throughput for latency. For user-facing APIs, shedding is almost always correct — a queued request that takes 8 seconds is worse than a fast 429 the client can retry. For background work (webhooks, queue consumers), queuing with bounded depth and backpressure (Volume 10, Chapter 7) is appropriate.

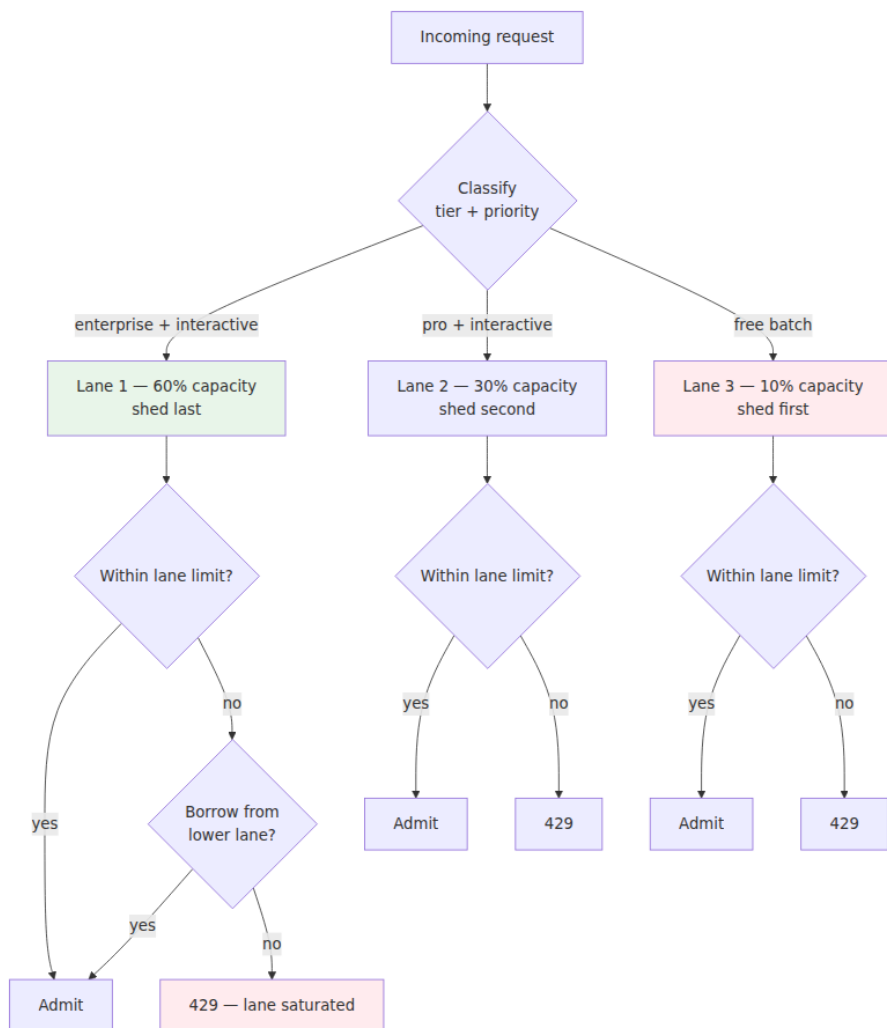


Figure 9-3: Priority lanes. Each lane has a weighted share of capacity; borrowing allows high-priority lanes to use idle capacity from lower lanes, but not vice versa. Shedding order is the inverse of priority.

Global limits — the consistency-availability trade-off

A truly global limit ("at most 1000 RPS across all 20 gateway replicas") is a distributed consensus problem. Options and their error bounds:

| Strategy | Precision | Latency per request | Availability when coordinator down | When to use |
|--|--|---------------------|--|--|
| Local counters only | $N \times$ over-allowance (N = replicas) | 0 ms | Always available | Limits where $2-5 \times$ overshoot is tolerable; high RPS where per-request coordination is too expensive |
| Synchronous global (Redis INCR per request) | Exact | +1-3 ms (Redis RTT) | Fail-open or fail-closed — your choice | Precise per-tenant / abuse caps at moderate RPS |
| Periodic sync (local + flush every 100ms) | Bounded by <code>sync_interval</code> $\times$ RPS | 0 ms (async flush) | Degrades to local | High-RPS APIs needing bounded error without per-request coordination |
| Probabilistic (allow with p = remaining/limit) | Statistical, converges over window | 0 ms | Always available | Very high cardinality (per-IP at millions of IPs) where exact is infeasible |
| Hierarchical (gateway cap + service cap) | Two-level bound | 0 ms at service | Degrades gracefully | Defense in depth — gateway protects fleet, service protects itself |

For most products, **local with periodic sync** hits the sweet spot: each replica allows `limit/N` locally and reconciles every 100ms via Redis. The

worst-case over-allowance is `sync_interval × RPS` — at 100 RPS and 100ms, at most 10 extra requests per window, negligible for capacity protection. At 10k RPS, the error grows and synchronous global may be warranted for the tighter abstraction.

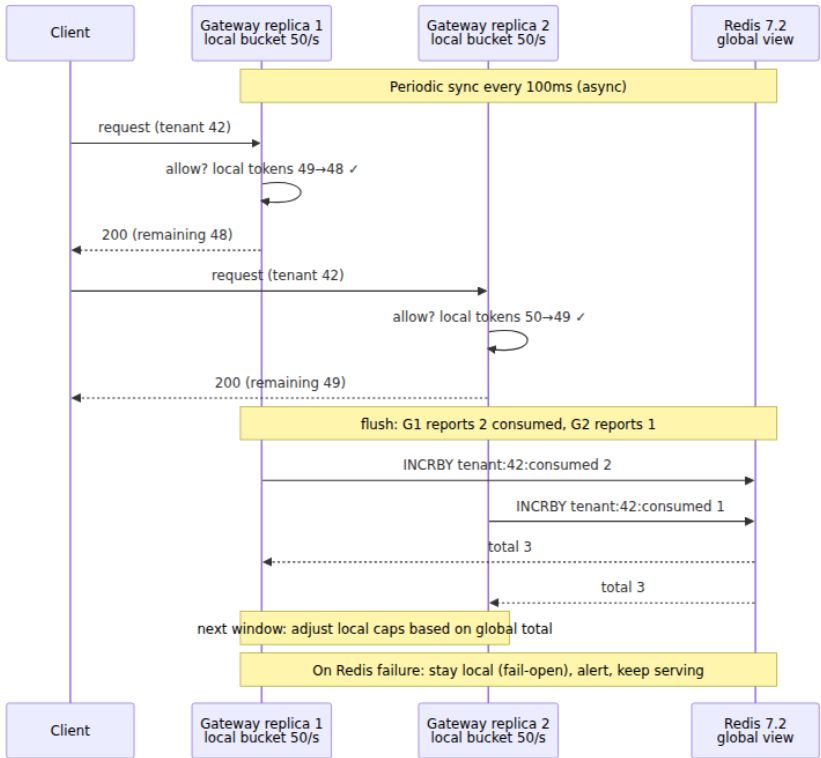


Figure 9-4: Hierarchical rate limiting with periodic reconciliation. Local buckets admit with zero added latency; an async flush reconciles the global view and adjusts the next window's local caps. Redis failure degrades to local enforcement — availability over precision.

Observability and tuning — what to measure and alert on

A limiter you cannot observe is a limiter that silently throttles legitimate traffic or silently over-allows abuse. Minimum per-route, per-tenant metrics:

- **Throttle rate** — `rate(http_requests_throttled_total[5m]) / rate(http_requests_total[5m])` — alert if > 1-5% for enterprise tenants (indicates under-provisioned limit or client bug).
- **Remaining distribution** — histogram of `RateLimit-Remaining` at admit time — if p50 remaining is near zero, the limit is too tight; if p99 remaining is near capacity, the limit is unused.
- **Redis error rate and p99 latency** — `rate(redis_errors_total[5m])` and `histogram_quantile(0.99, redis_command_duration_seconds)` — alert if error rate > 0 or p99 > 5ms (limiter is becoming the bottleneck).
- **Over-allowance estimate** — `sum(local_allowed) - sum(redis_global_count)` over the window — tracks divergence for local/periodic-sync strategies.
- **Retry-after compliance** — share of clients that retry within `Retry-After` vs. immediately — non-compliant clients amplify load; consider edge caching of 429s for repeat offenders.

```
# Prometheus alerts – rate limiting (Prometheus 2.53)
groups:
- name: rate_limiting
  interval: 30s
  rules:
  - alert: TenantThrottledHighRate
    expr: |
      sum by (tenant) (rate(http_requests_throttled_total[5m]))
      / sum by (tenant) (rate(http_requests_total[5m])) > 0.05
    for: 10m
    labels: { severity: ticket }
    annotations:
      summary: "Tenant {{ $labels.tenant }} throttled > 5% for 10m – li
mit may be too tight or client buggy"
  - alert: RateLimitRedisDown
    expr: up{job="redis"} == 0
    for: 2m
    labels: { severity: page }
    annotations:
      summary: "Redis down – rate limiting is fail-open and imprecise"
  - alert: RateLimitRedisSlow
    expr: histogram_quantile(0.99,
rate(redis_command_duration_seconds_bucket[5m])) > 0.005
    for: 5m
    labels: { severity: ticket }
    annotations:
      summary: "Redis p99 > 5ms – limiter latency may affect gateway p9
9"
```

Tuning loop:

1. Start with token bucket `rate = p95 legitimate RPS × 1.5`, `burst = rate × 2`. Measure throttle rate for legitimate traffic — target < 0.1%.
2. Set quota windows (hourly/daily) at 10× the per-second allowance scaled to the window — quotas should not throttle normal usage, only abuse.
3. Simulate noisy neighbor: one tenant at 10× its limit while others are at p50 — verify that other tenants' p99 latency and throttle rate are unchanged (isolation).
4. Chaos-test Redis failure — kill Redis, verify fail-open preserves availability and alert fires, restore Redis, verify global counts reconverge within one sync interval.

The distributed-systems lens:

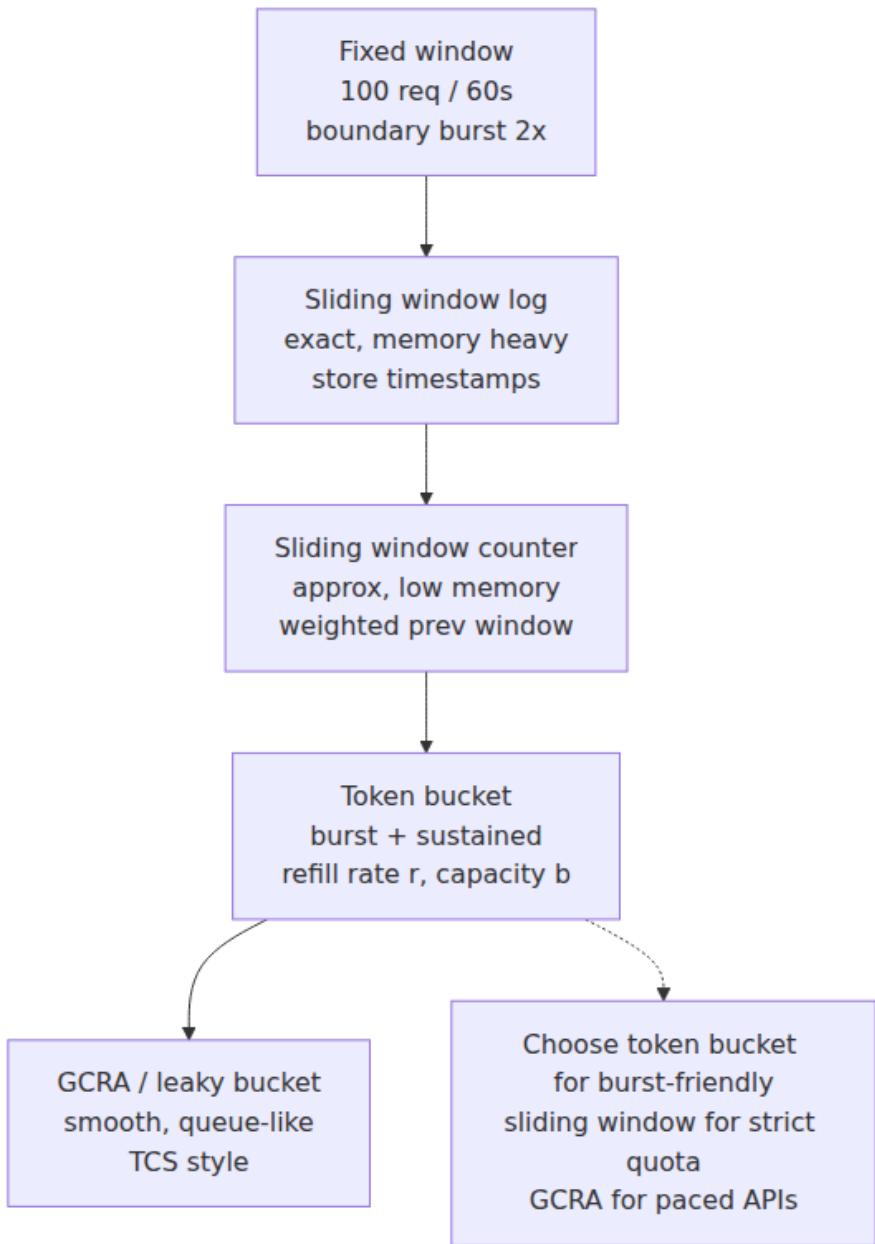
- **Rate limiting is consistency vs. availability in disguise.** A precise global limit is a strongly consistent counter — it needs coordination (Redis, consensus) and pays latency and availability cost. A local limit is an eventually consistent counter — available and fast but imprecise. The reconciliation strategy is your consistency model.
 - **Throttling is load shedding, not backpressure.** Backpressure (Volume 10, Chapter 7) asks the producer to slow down cooperatively; throttling rejects uncooperative producers. Use backpressure inside the fleet (queues, flow control) and throttling at the perimeter (gateway, edge) — mixing them (throttling internal services hard) creates cascading 429s that are harder to debug than explicit queue depth signals.
 - **The header contract is the API.** A 429 without `Retry-After` and `RateLimit-*` is indistinguishable from a transient 503 to a client — it will retry immediately and amplify the overload. The headers turn throttling into a cooperative protocol.
-

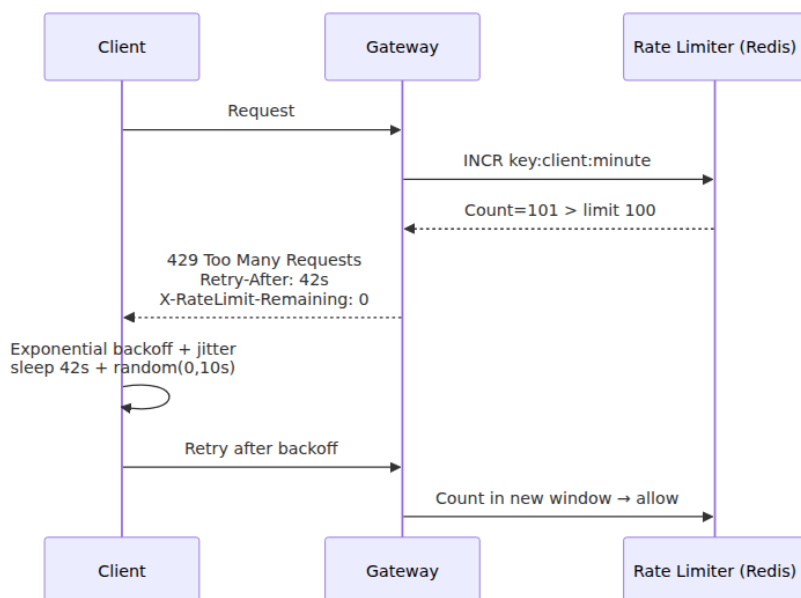
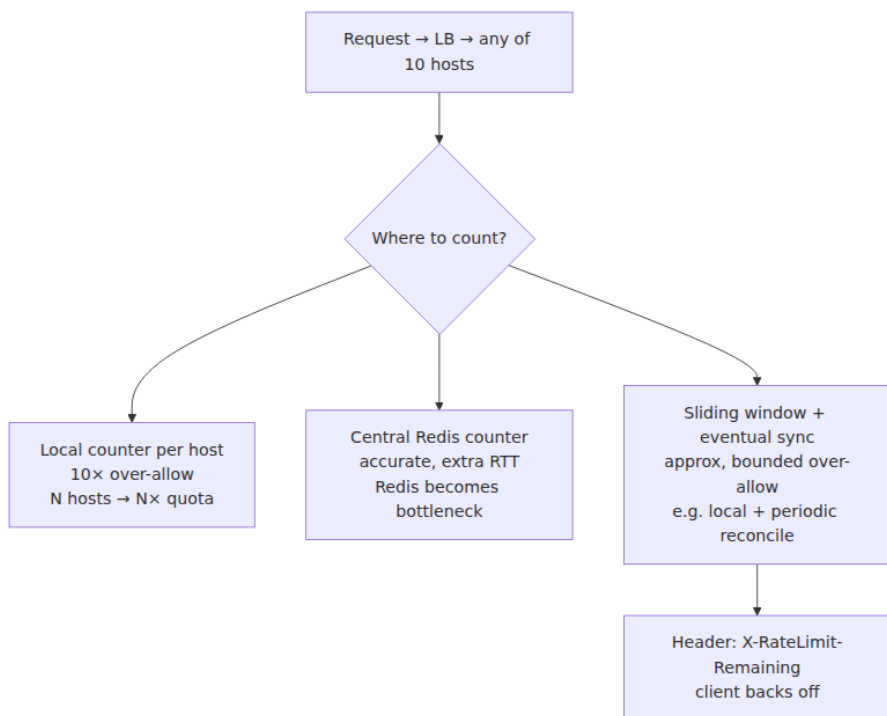
Key takeaways

- Rate limits protect per-second capacity, quotas enforce business budgets over longer windows, and fairness decides who is shed first — three layers that compose, not one mechanism.
- Fixed window spikes to 2× at boundaries; sliding-window counter smooths with ~1% error and O(1) memory (the gateway default); token bucket bounds bursts explicitly ("100/s, burst 200"); leaky bucket smooths for outbound shaping.
- Local limiters (per-instance token bucket) are the last line of defense and must use monotonic clocks; distributed limiters (Redis + atomic Lua) are needed for precise per-tenant/global caps and add 1–3 ms per request plus a Redis availability dependency.
- Redis Lua scripts are atomic because Redis is single-threaded — non-atomic read-then-write races over-count under contention; always use `EVAL` with all keys/args in one script.
- Gateway enforcement (Envoy external `ratelimit`, Kong, NGINX) keeps policy out of services but must handle Redis failure explicitly

— fail-open for availability, fail-closed only for cost/safety-critical caps — and emit `RateLimit-*` / `Retry-After` headers so clients can cooperate.

- Fairness requires weighted limits, priority lanes, and a shed-vs-queue decision — shed for user-facing (fast 429 beats slow queue), queue with bounded depth for background work.
- Global limits are a consistency-availability trade-off: local is available but $N\times$ imprecise, synchronous global is precise but adds latency and a single point of failure, periodic sync bounds the error to `sync_interval × RPS` and is the right default for most fleets.





Further reading

- Envoy 1.30 — external rate limiting and `enable_x_ratelimit_headers` : https://www.envoyproxy.io/docs/envoy/v1.30.0/configuration/http/http_filters/rate_limit_filter
- Lyft `ratelimit` service — configuration and GCRA semantics: <https://github.com/envoyproxy/ratelimit>
- Redis 7.2 — `EVAL` , Lua scripting, and `redis-cell (CL.THROTTLE)` : <https://redis.io/docs/latest/commands/eval/> and <https://github.com/brandur/redis-cell>
- Kong Gateway 3.7 — rate-limiting plugin (Redis policy): <https://docs.konghq.com/hub/kong-inc/rate-limiting/>
- NGINX `limit_req` — `limit_req_zone` , `burst` , `nomodel` : https://nginx.org/en/docs/http/nginx_http_limit_req_module.html
- IETF `RateLimit` header fields — `RateLimit-Limit/Remaining/Reset` and `Retry-After` (draft-ietf-httpapi-ratelimit-headers): <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>
- Volume 14, Chapter 8 — algorithmic analysis of rate-limiting and scheduling (WFQ, DRR, consistent hashing for sharded limits).
- Volume 3, Chapter 11 — client-side retry discipline after 429/503 (backoff, jitter, retry budgets, hedging).

Chapter 10 — Multi-Region and Geo-Distributed Systems

What this chapter covers. Chapters 1–9 built a system that scales within a single region — load balancing, caching, data modeling, and traffic policy that survive data-center failures. A regional system, even a well-run one, still has a single fate domain: a region-wide network partition, a control-plane outage, or a compliance mandate that data stay in-country is enough to take it offline or make it illegal to operate. This chapter moves the failure domain to the planet. We make explicit why teams go multi-region — latency, survival, and sovereignty — and the price they pay: the speed of light is not negotiable, and consistency across a WAN is a choice with a latency tax. We compare deployment topologies on a spectrum from active-passive through active-active to

cell-based architectures, show how global traffic is actually steered (GeoDNS, anycast, and global L7 load balancers with health-checked failover) with real Route 53 and GCP configurations, then turn to the hard problem — data. Synchronous cross-region replication buys strong consistency and sells availability and tail latency; asynchronous buys the inverse and forces conflict resolution. We ground that trade-off in production systems — Spanner and CockroachDB for externally consistent SQL, DynamoDB Global Tables and Cassandra for tunable/eventual models, Aurora Global Database for single-writer relational — and in replication machinery you can operate (Postgres logical replication, Kafka MirrorMaker 2). We cover federation patterns (control plane vs. data plane separation, regional cells, data-residency sharding), consistency choices that determine where reads and writes may go, and the operational realities that determine whether a multi-region system actually survives the failure it was built for: deploy coordination, clock discipline, split-brain prevention, and failover drills that avoid thundering herds. The chapter closes with a distributed-systems lens on why multi-region is a CAP/PACELC exercise at 50-200 ms round trips.

Learning goals — after this chapter you should be able to:

- Name the three drivers of multi-region (latency, availability/survival, sovereignty) and quantify the latency budget imposed by the speed of light between common region pairs.
- Compare active-passive, active-active, active-active sharded, and cell-based topologies on write locality, failover RTO/RPO, operational complexity, and blast radius, and choose per workload.
- Configure global traffic steering with Route 53 (latency + failover + health checks), anycast, and GCP Global Load Balancing, and explain failover behavior under partial degradation and DNS caching.
- Decide per dataset between synchronous and asynchronous cross-region replication, name the consistency/latency/availability consequence of each, and select a conflict-resolution strategy (last-write-wins, vector clocks, CRDTs) when async is required.
- Operate multi-region data planes for four reference systems — Spanner/CockroachDB, DynamoDB Global Tables, Aurora Global Database, Cassandra multi-DC — including DDL/replication configuration and failure semantics.
- Design a cell-based architecture and a control-plane/data-plane split that isolates regional failures and supports data-residency

constraints, and describe the deploy, observability, and failover-drill machinery that makes it actually work.

- Reason about multi-region through PACELC: what you give up on every write, what you gain on partition, and why read locality and write locality cannot both be optimal.

Why one region is not enough

A single region — three availability zones, redundant power and networking inside one geography — survives a zone failure. It does not survive three classes of reality:

| Driver | What forces it | What breaks without a second region |
|--------------------|--|---|
| Latency | Users are global; light in fiber is ~200,000 km/s, so New York → Singapore is ~210 ms RTT before any processing. Interactive SLOs (p99 < 300 ms) are impossible from one origin. | Every distant user pays the RTT; edge caching (Chapter 8) helps reads but not writes. |
| Survival | Region-wide events — network backbone cuts, power grid failures, control-plane outages (us-east-1, December 2021, took down DynamoDB, EC2, and the console that operates them). | Total outage; RTO is however long the provider needs to restore the region. |
| Sovereignty | GDPR, data-residency laws (Russia FZ-152, China DSL/PIPL, Saudi PDPL), and contractual commitments require data to stay or be servable from a jurisdiction. | Non-compliance; in some markets, inability to operate at all. |

The cost of going multi-region is coordination over a wide-area network. Inside a region, p99 RTT between AZs is 1–3 ms; across regions it is 20–200 ms. That two-order-of-magnitude gap is where consistency, latency, and availability trade-offs live, and where naive "just replicate everything synchronously" designs collapse.

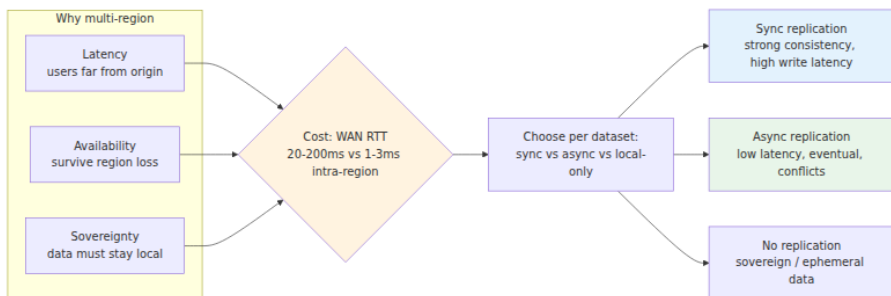


Figure 10-1: The three drivers and the per-dataset decision they force. Most systems use all three strategies for different datasets in the same architecture.

Boundary note. Consistency models, linearizability, and consensus (Paxos/Raft) are in Volume 6, Chapters 3–6. Caching and edge delivery are in Chapter 3 and Chapter 8 (this volume). Replication mechanics inside a single region (leader-follower, quorum, WAL shipping) are in Volume 5, Chapter 8. This chapter treats those primitives as building blocks and asks what changes when the replication link is a WAN: latency budgets, conflict resolution at distance, and global traffic steering. Sovereignty as a security/compliance control is referenced here for architecture; the regulatory detail is in Companion Book 8 and Volume 9, Chapter 11. Capacity math for cross-region bandwidth is introduced here and analyzed formally in Volume 14.

Topologies: from passive standby to cells

There is no single "multi-region architecture." There is a spectrum, and the choice is per service and per dataset.

Active-passive (primary-standby)

One region serves all traffic; a second region has a warm standby that takes over on failure. Writes go to the primary; data is replicated asynchronously (occasionally synchronously for critical metadata).

- **RTO:** minutes (DNS failover + warm-up); **RPO:** seconds to minutes (async lag).
- **Pros:** Simple; no conflicts; strong consistency in the primary.

- **Cons:** Passive region is idle cost; failover is a rare code path that rots; thundering herd when traffic shifts.

Good for: relational primary with Aurora Global Database, control planes that can tolerate minutes of failover, and teams taking their first step multi-region.

Active-active (multi-primary, shared dataset)

Every region serves reads and writes; each write is visible everywhere. Requires either synchronous replication (Spanner, CockroachDB) or async with conflict resolution (DynamoDB Global Tables, Cassandra).

- **RTO:** zero for surviving regions (no failover — traffic already there);
RPO: zero if sync, bounded lag if async.
- **Pros:** Lowest latency for nearby writes; no idle capacity.
- **Cons:** Sync path adds 50–150 ms to every write; async path forces application-level conflict handling.

Active-active sharded (partitioned by locality)

Each region owns a shard — e.g., users homed to their nearest region. Writes for a shard go to its home region (single writer per key); reads are local when possible. Cross-shard operations are rare and explicitly routed.

- **Pros:** No cross-region write latency on the hot path; no conflicts (single writer per key); sovereignty falls out naturally.
- **Cons:** Cross-shard transactions are expensive; re-homing a user is a migration; hot shard imbalance.

This is the dominant pattern for user-facing data at scale (Google, Meta, Uber) and the basis for cells.

Cell-based architecture

A *cell* is a self-contained regional stack (compute + storage + async workers) that serves a subset of traffic. Cells are replicated across regions, and a global control plane assigns tenants/users to cells. Failure is contained to a cell; rollout is cell-by-cell.

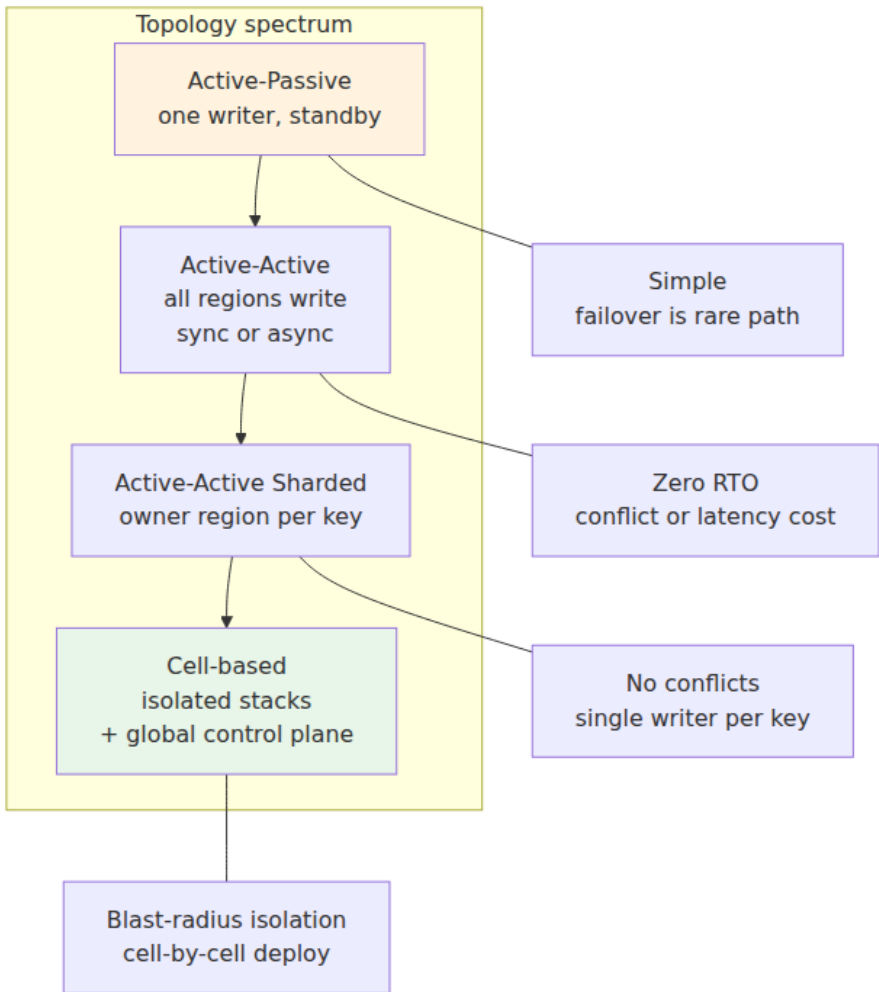


Figure 10-2: The topology spectrum. Complexity and isolation increase to the right; most mature platforms converge on sharded active-active or cells.

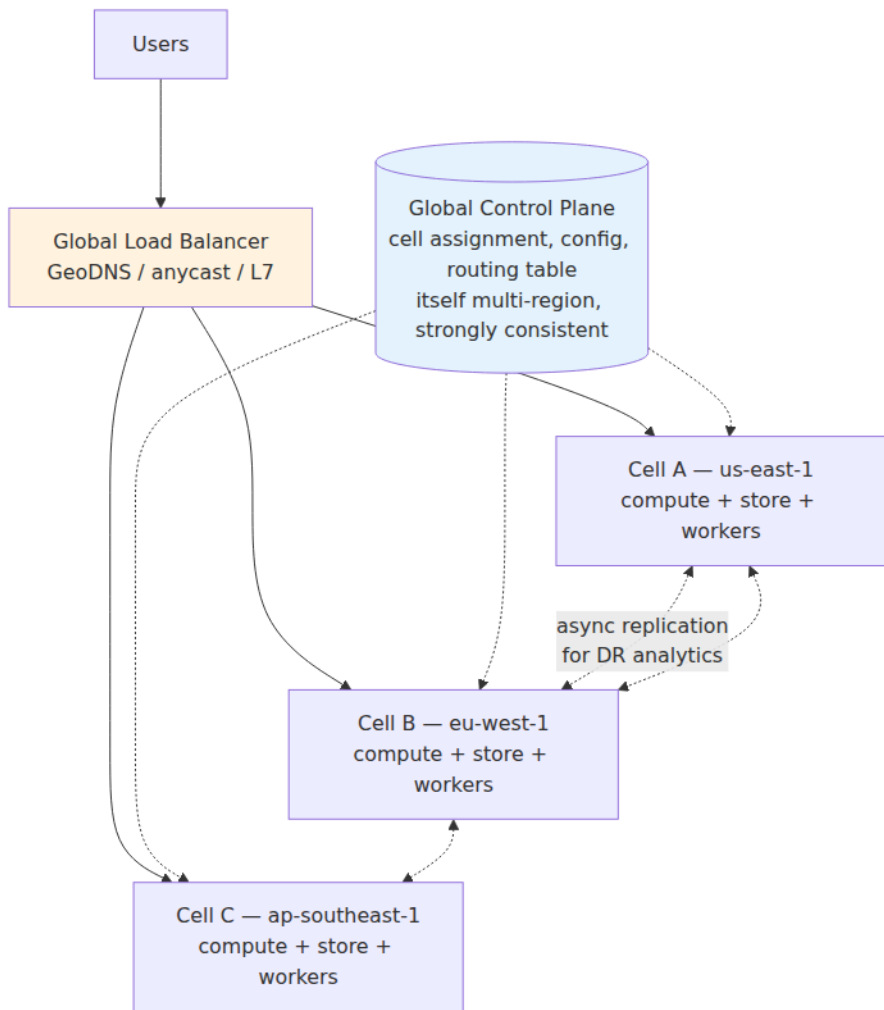


Figure 10-3: Cell-based architecture. The control plane (strongly consistent, multi-region) assigns each tenant/key to a cell; data planes are isolated so a cell failure does not cascade. Cross-cell replication is async and limited to DR/analytics — not on the write path.

The control plane / data plane split is the key insight: the thing that decides *where* traffic goes must itself be strongly consistent and survive a region loss, but it is small and low-write-rate. Data planes can then be

simple, regional, and eventually consistent. AWS, Google, and Meta all converge on this shape.

The WAN changes everything: latency budgets and PACELC

The speed of light in fiber caps what any protocol can do. Useful RTTs to internalize:

| Pair | Fiber distance | RTT (typical) | What fits in a 300 ms SLO? |
|----------------------------|----------------|---------------|---|
| us-east-1 ↔ us-west-2 | ~4,000 km | 60-70 ms | One cross-region write + processing |
| us-east-1 ↔ eu-west-1 | ~6,500 km | 70-80 ms | Same |
| us-east-1 ↔ ap-northeast-1 | ~11,000 km | 150-170 ms | Barely one write; reads must be local |
| eu-west-1 ↔ ap-southeast-1 | ~11,000 km | 160-180 ms | Same |
| us-east-1 ↔ ap-southeast-1 | ~16,000 km | 200-230 ms | No cross-region round trip fits without blowing the SLO |

PACELC makes the trade-off precise: **if Partitioned, choose Availability vs. Consistency; Else, choose Latency vs. Consistency.** Across a WAN, "Else" dominates — even without a partition, synchronous replication pays latency on every write.

- **PA/EL** (Spanner, CockroachDB): Consistent when partitioned; consistent (and slower) even when healthy. Every write waits for a cross-region quorum or TrueTime commit wait.
- **PA/EC** (DynamoDB Global Tables, Cassandra): Consistent when partitioned (by being available and eventually consistent); consistent path is *not* taken when healthy — writes are local and replicate async.

Most user-facing systems choose **PA/EC for data-plane writes** (local latency matters) and **PA/EL for the control plane** (correctness matters more than latency, and write rate is low).

Global traffic steering

DNS-based steering (GeoDNS + health checks)

The oldest mechanism: authoritative DNS returns different answers by client geography and liveness.

```
# Route 53 – latency-based routing with health-checked failover
# Two records for the same name; Route 53 picks the lowest-latency
healthy endpoint
```

```
resource "aws_route53_record" "api_latency_usel" {
  zone_id = aws_route53_zone.example.zone_id
  name     = "api.example.com"
  type     = "A"
  set_identifier = "usel"
  latency_routing_policy { region = "us-east-1" }

  alias {
    name           = aws_lb.usel_global.dns_name
    zone_id        = aws_lb.usel_global.zone_id
    evaluate_target_health = true
  }
  health_check_id = aws_route53_health_check.usel.id
}

resource "aws_route53_record" "api_latency_euw1" {
  zone_id = aws_route53_zone.example.zone_id
  name     = "api.example.com"
  type     = "A"
  set_identifier = "euw1"
  latency_routing_policy { region = "eu-west-1" }
  alias {
    name           = aws_lb.euw1_global.dns_name
    zone_id        = aws_lb.euw1_global.zone_id
    evaluate_target_health = true
  }
  health_check_id = aws_route53_health_check.euw1.id
}

resource "aws_route53_health_check" "usel" {
  type           = "HTTPS"
  fqdn           = "api-us-east-1.example.com"
  port           = 443
  resource_path  = "/healthz"
  failure_threshold = 3
  request_interval = 10
}
```

```
# Route 53 failover variant – primary + secondary with active-passive
semantics
# api.example.com -> primary (us-east-1) when healthy, else secondary
(eu-west-1)
# Useful when you cannot serve from both regions for a given dataset.
```


DNS failover is simple and provider-independent, but `TTL` governs failover speed and many resolvers ignore low TTLs. Expect 30-120 s effective failover, not milliseconds.

Anycast and global L7 load balancers

Anycast announces the same IP from multiple PoPs; BGP routes each client to the nearest healthy PoP. Cloudflare, Google Cloud Global LB, and AWS Global Accelerator use this to steer before DNS is even involved.

```
Client (Berlin) --BGP--> nearest PoP (Frankfurt) --private
backbone--> origin region (eu-west-1)
Client (Tokyo) --BGP--> nearest PoP (Tokyo) --private backbone-
-> origin region (ap-northeast-1)
```

Global L7 LBs (GCP Global External LB, Cloudflare, AWS ALB + Global Accelerator) add health checking, failover, and L7 policy at the edge, with failover in seconds and no DNS TTL dependency. The trade-off is provider coupling and that failover policy lives in the provider's control plane — which itself must be multi-region.

Sticky routing and locality-aware backends

Inside the mesh, locality matters too. Envoy's locality-aware routing keeps requests inside the region/AZ when possible and only spills over when local capacity is degraded.

```
# Envoy – locality weighted load balancing + outlier detection for
regional spillover
# Cluster with two localities (us-east-1a, eu-west-1a); Envoy prefers
local, fails over.
cluster:
  name: api_upstream
  type: EDS
  eds_cluster_config: { eds_config: { ads: {} } }
  common_lb_config:
    locality_weighted_lb_config: {}
  outlier_detection:
    consecutive_5xx: 5
    interval: 10s
    base_ejection_time: 30s
    max_ejection_percent: 50
  load_assignment:
    endpoints:
      - locality: { region: "us-east-1", zone: "us-east-1a" }
        weight: 100
        lb_endpoints: [{ endpoint: { address: { socket_address: {
address: 10.0.1.10, port_value: 8080 }}}}]
      - locality: { region: "eu-west-1", zone: "eu-west-1a" }
        weight: 100
        lb_endpoints: [{ endpoint: { address: { socket_address: {
address: 10.1.2.10, port_value: 8080 }}}}]
```

Data replication across the WAN

The decision per dataset

Not every dataset needs the same guarantee. A mature multi-region system classifies each dataset:

| Dataset | Typical choice | Why |
|---------------------------------|---|--|
| User profile, auth, billing | Sync (Spanner/CockroachDB) or sharded single-writer | Strong consistency; losing a write is worse than extra latency |
| Orders, payments | Sync or sharded single-writer per tenant | Correctness over latency |
| Feed, timeline, recommendations | Async (Cassandra/DynamoDB Global Tables) | Eventual is fine; latency matters |

| Dataset | Typical choice | Why |
|---|---|--------------------------------|
| Analytics, logs, audit | Async (Kafka MM2, Kinesis) | Throughput; replay covers loss |
| Sovereign data (EU PII) | Local-only, no cross-region replication | Compliance boundary |
| Ephemeral (sessions, rate-limit counters) | Local or async best-effort | Loss is tolerable |

The mistake is choosing one replication mode for the whole system.

Synchronous: Spanner, CockroachDB, Aurora Global Database

Spanner (Google) uses TrueTime (GPS + atomic clocks) to assign globally meaningful timestamps and runs Paxos per shard across regions. Writes wait for `commit wait` (~5-10 ms) plus cross-region quorum. Reads are strongly consistent and can be served from the nearest read-only replica without cross-region RTT if they use bounded staleness.

```
-- Spanner DDL - multi-region instance + interleaved tables for
locality
-- Instance config: nam-eur-asia1 (multi-region, 3 read-write + 3 read-
only replicas)
CREATE INSTANCE app_prod
WITH
  CONFIG = "nam-eur-asia1",
  NODES = 3;

CREATE TABLE Users (
  user_id      STRING(36) NOT NULL,
  home_region  STRING(16) NOT NULL,
  email        STRING(320) NOT NULL,
  created_at   TIMESTAMP OPTIONS (allow_commit_timestamp=true),
) PRIMARY KEY (user_id);

CREATE TABLE Orders (
  user_id      STRING(36) NOT NULL,
  order_id     STRING(36) NOT NULL,
  total        NUMERIC NOT NULL,
  status       STRING(16) NOT NULL,
) PRIMARY KEY (user_id, order_id),
  INTERLEAVE IN PARENT Users ON DELETE CASCADE;
-- Interleaving co-locates a user's orders with the user row → single-
shard transaction.
```

CockroachDB is wire-compatible with Postgres and uses Raft per range. Leaseholder handles reads locally; writes replicate to a quorum.

```
-- CockroachDB – multi-region database with survival goals and table
locality
CREATE DATABASE app;

ALTER DATABASE app SET PRIMARY REGION "us-east-1";
ALTER DATABASE app ADD REGION "eu-west-1";
ALTER DATABASE app ADD REGION "ap-southeast-1";
ALTER DATABASE app SURVIVE REGION FAILURE; -- tolerate one region loss

-- Table homed to a region: all leaseholders (and thus low-latency
reads/writes) stay there
ALTER TABLE app.users SET LOCALITY REGIONAL BY ROW;
ALTER TABLE app.users ALTER COLUMN crdb_region TYPE STRING DEFAULT "us-
east-1";

-- Global table: every region has a replica, strongly consistent
ALTER TABLE app.feature_flags SET LOCALITY GLOBAL;
```

Aurora Global Database keeps a single primary region writable; secondaries are read-only with <1 s lag and can be promoted in <1 minute (RPO ~1 s, RTO ~60 s). Good when you need relational strong consistency on writes but can serve reads globally.

```
aws rds create-global-cluster \
  --global-cluster-identifier app-global \
  --source-db-cluster-identifier arn:aws:rds:us-
east-1:123456789:cluster:app-primary

aws rds create-db-cluster \
  --db-cluster-identifier app-secondary-eu \
  --engine aurora-postgresql --engine-version 15.4 \
  --global-cluster-identifier app-global \
  --region eu-west-1
```

Asynchronous: DynamoDB Global Tables, Cassandra, Postgres logical, Kafka

DynamoDB Global Tables replicate async with last-write-wins (vector-clock-like reconciliation under the hood). Writes are local (<10 ms); replication lag is typically <1 s.

```

# DynamoDB Global Tables – create + write with version for application-
level conflict handling
import boto3

client = boto3.client("dynamodb", region_name="us-east-1")
client.create_global_table(
    GlobalTableName="events",
    ReplicationGroup=[{"RegionName": "us-east-1"}, {"RegionName": "eu-
west-1"}],
)

# Write with a conditional version – application resolves conflicts,
not just LWW
table = boto3.resource("dynamodb", region_name="us-east-1").Table("even
ts")
table.put_item(
    Item={"pk": "user#42", "sk": "order#99", "status": "placed", "v":
3, "updated_at": "2026-08-20T12:00:00Z"},
    ConditionExpression="attribute_not_exists(v) OR v < :v",
    ExpressionAttributeValues={":v": 3},
)

```

Cassandra multi-DC lets each DC have its own replication factor and uses tunable consistency per operation.

```

# cassandra.yaml – multi-DC snitch + replication
endpoint_snitch: GossipingPropertyFileSnitch
# cassandra-rackdc.properties
dc=us-east
rack=rack1

# CQL – keyspace replicated to two DCs, RF 3 each
CREATE KEYSPACE app WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'us-east': '3',
    'eu-west': '3'
};

-- Write local, read local – no WAN on either path; async repair
handles divergence
INSERT INTO app.timeline (user_id, post_id, ts) VALUES ('u42', 'p99',
timestamp(now()));
SELECT * FROM app.timeline WHERE user_id='u42' LIMIT 50; --
LOCAL_QUORUM

```

```
# Kafka MirrorMaker 2 – async topic replication between regional
clusters
# Replicates topic `orders` from us-east cluster to eu-west with offset
translation.
clusters: us-east, eu-west
us-east.bootstrap.servers: kafka-us-east.example.com:9092
eu-west.bootstrap.servers: kafka-eu-west.example.com:9092

mirrors:
- source: us-east
  target: eu-west
  topics: "orders, payments.*"
  replication.factor: 3
  sync.group.offsets.enabled: true
  emit.heartbeats.enabled: true
```

Postgres logical replication for selective cross-region tables (not whole-cluster physical streaming):

```
-- Primary region (us-east-1)
CREATE PUBLICATION app_pub FOR TABLE users, orders;
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '...';

-- Secondary region (eu-west-1)
CREATE SUBSCRIPTION app_sub
  CONNECTION 'host=pg-us-east.example.com dbname=app user=replicator
password=...'
  PUBLICATION app_pub
  WITH (copy_data = true, create_slot = true);
-- Lag: SELECT now() - pg_last_xact_replay_timestamp();
```

Conflict resolution when async is unavoidable

If two regions accept writes to the same key concurrently, replication must reconcile. Options, in order of increasing sophistication:

| Strategy | How it resolves | When to use | Cost |
|------------------------------------|--|--|--------------------------|
| Last-write-wins (LWW) | Highest timestamp wins; other write is lost | Counters, session state, cache — loss is tolerable | Simple; silent data loss |
| Version + application merge | App stores <code>v</code> or vector clock; conflict handler merges | Orders, carts — merge is domain-specific | App complexity |

| Strategy | How it resolves | When to use | Cost |
|------------------------------|--|---|-----------------------------------|
| CRDTs | Commutative operations (G-Counter, OR-Set, LWW-Register) converge without coordination | Counters, sets, collaborative state | Limited data types (Vol 6, Ch 11) |
| Single writer per key | Shard by owner region — no concurrent writes to same key | User data, tenant data — dominant pattern | Requires routing + re-homing |

The single-writer-per-key rule eliminates conflicts by construction and is why sharded active-active is preferred whenever feasible. Reserve CRDTs/LWW for data where true multi-writer is required.

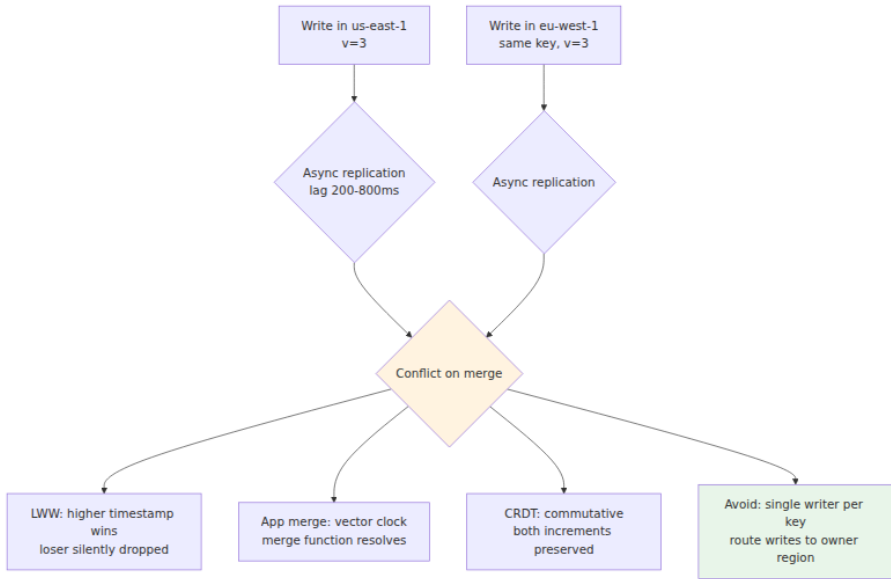


Figure 10-4: Conflict resolution options when async replication allows concurrent writes. The cheapest is to avoid the conflict by routing.

Sharding and placement for the global dataset

For user- or tenant-scoped data, the pattern is: **home region per principal, local reads/writes, async for DR.**

```
user_id --hash--> home_region (us-east-1)
|
+--> All writes for user_id route to us-east-1 (single writer)
+--> Reads from nearest region: if local replica is stale-but-accepta
ble, serve locally;
|   if strong read required, proxy to home region.
+--> Re-homing: copy data, flip routing table atomically (control pla
ne transaction).
```

Key count matters: with millions of principals, consistent hashing spreads load; with thousands of large tenants, explicit assignment with capacity-weighted placement avoids hot regions. Re-homing must be online — double-write, backfill, cutover — and is itself a distributed transaction.

Data residency adds a placement constraint: EU users' rows must have `region = eu-west-1` and must not replicate outside the EU. That is a placement policy enforced in the control plane and in replication filters (e.g., Postgres publication filtered by `WHERE region = 'eu'` or Spanner row-level locality). Audit it.

Operating multi-region: the parts that actually fail

Deploy coordination. Deploying to all regions at once couples their failure. Cell-by-cell rollout (one region, bake, next) with automatic rollback on SLO burn is the standard. The control plane that orchestrates deploys must itself be multi-region.

Clock discipline. Spanner depends on TrueTime; CockroachDB on `max_offset` (default 500 ms) with clock-offset monitoring; Cassandra on NTP. Monitor `clock_offset` as an SLO — a drifting clock causes lease violations and stale reads.

Split-brain prevention. Two regions must not both believe they are primary for the same shard. Use a strongly consistent control plane (Spanner, etcd, or Route 53 health checks with quorum) to elect/assign primaries. Fencing tokens (monotonic epoch per shard) reject stale writers after failover.

Failover drills and thundering herds. A failover that has never been exercised will not work. Run quarterly region-evacuation drills (drain, fail over, measure RTO/RPO, fail back). Pre-warm the receiving region (autoscaling, cache, DB connections) — a cold standby that receives

100% of global traffic will collapse. Shed or degrade non-critical traffic during evacuation.

Observability across regions. Trace context must propagate across regions (W3C `traceparent` over replication and RPC). Correlate replication lag, error rate, and latency per region pair. Alert on `replication_lag > SLO` before clients notice stale reads.

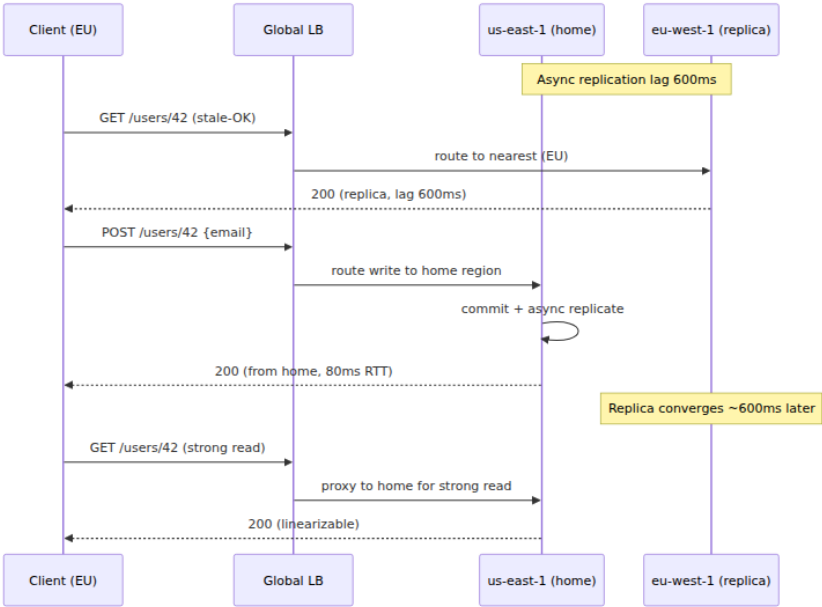


Figure 10-5: Read locality vs. write locality. Stale-tolerant reads stay local; writes and strong reads go to the home region. The routing policy is per request, not per service.

Choosing per workload

| Question | Leads to |
|--|---|
| Can you tolerate 60-150 ms extra on every write? | If yes, consider sync (Spanner/CockroachDB). If no, async or sharded single-writer. |
| Is loss of a concurrent write acceptable? | If no, single writer per key — do not rely on LWW. |

| Question | Leads to |
|---|--|
| Must data stay in a jurisdiction? | Placement policy + filtered replication; audit the replication path, not just the primary store. |
| What is the real RTO/RPO? | Measure it with drills; DNS TTL, warm-up, and replication lag dominate, not the database's claimed failover. |
| Is the control plane itself multi-region? | If the thing that decides failover is single-region, the system is single-region. |

Distributed-systems lens

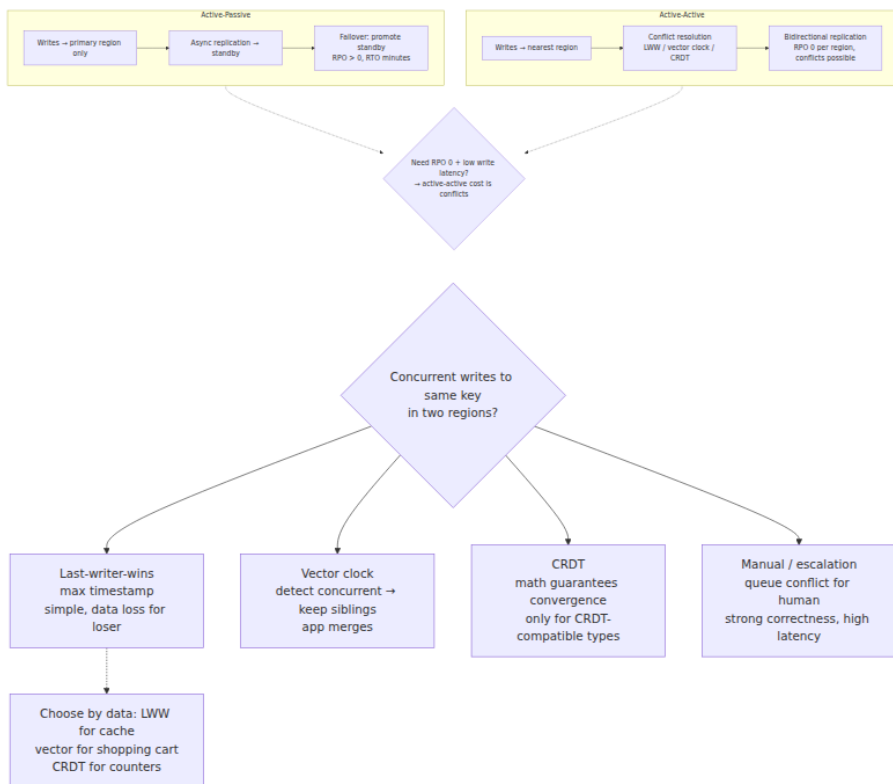
Multi-region makes PACELC tangible. Every synchronous write pays latency even when the network is healthy; every async write risks conflict or staleness. There is no topology that gives you single-region latency, strong consistency, and survival of a region — you are choosing which dataset gets which property. The mature answer is to make that choice explicitly, per dataset, and to isolate the strongly consistent sliver (the control plane) so the rest of the system can be fast and eventually consistent. Cells are that isolation made operational: blast radius, deploy, and failure domain all aligned.

Key takeaways

- Multi-region is driven by latency, survival, and sovereignty — each forces a different replication choice, and most systems need all three modes for different datasets.
- The topology spectrum (active-passive → active-active → sharded → cell-based) trades simplicity for isolation; cells with a strongly consistent control plane are the end state for large platforms.
- WAN RTT (20–230 ms) dominates SLOs; PACELC is lived on every write — PA/EL for the control plane, PA/EC for data-plane writes in most user-facing systems.
- Global traffic steering is a stack (anycast/Global LB at L4/L7 plus GeoDNS/latency routing at DNS plus locality-aware mesh inside);

DNS TTL and health-check intervals bound failover speed — measure effective RTO, not claimed.

- Data replication is per-dataset: sync (Spanner, CockroachDB, Aurora Global) for correctness-critical data; async (DynamoDB Global Tables, Cassandra, Kafka MM2, Postgres logical) for latency-sensitive data, with explicit conflict resolution; local-only for sovereign/ephemeral data.
- Avoid conflicts by construction with single writer per key (home region per principal); reserve LWW/CRDTs for data where true multi-writer is required.
- Operability determines survival: cell-by-cell deploys, clock-offset SLOs, fencing tokens against split-brain, pre-warmed failover targets, and quarterly region-evacuation drills that measure real RTO/RPO.



Further reading

- Google Spanner — TrueTime and externally consistent transactions. <https://research.google/pubs/pub39966/>
- CockroachDB — Multi-region patterns, survival goals, and table localities. <https://www.cockroachlabs.com/docs/stable/multiregion-overview>
- DynamoDB Global Tables — How it works and conflict handling. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>
- Marc Brooker (AWS) — Cell-based architecture and AZ/region isolation. <https://brooker.co.za/blog/2021/12/06/region.html> and <https://aws.amazon.com/builders-library/static-stability-using-availability-zones/>
- AWS Route 53 — Latency-based and failover routing, health checks. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>
- Pat Helland — Life beyond distributed transactions (at-most-once, idempotence). <https://queue.acm.org/detail.cfm?id=3025012>
- Kleppmann — Designing Data-Intensive Applications, Ch. 5 (Replication) and Ch. 8 (Distributed Systems Troubles).
- CRDTs and eventual consistency — see Volume 6, Chapter 11 and Shapiro et al., A comprehensive study of CRDTs. <https://arxiv.org/abs/1603.01529>

Chapter 11 — Designing for Failure: Bulkheads, Circuit Breakers, and Load Shedding

What this chapter covers. Chapters 1-10 showed how to make a system fast; this chapter makes it survive when fast is not an option. In a fleet of dozens of services, failure is not an exception — it is the steady state: a downstream deploys bad code, a database replica lags, a dependency's p99 triples, a retry storm amplifies a blip into an outage. Without explicit failure design, one slow dependency cascades into every

caller, every caller exhausts its threads, and the entire graph collapses — even though most services are healthy. We build the resilience toolkit from first principles: timeouts and deadline propagation that bound waiting, retries with budgets that help without amplifying, hedging that trades duplicate work for tail-latency reduction, bulkheads that partition failure, circuit breakers that stop calling what is already down, and load shedding that chooses *which* work to drop when there is more demand than capacity. Each pattern is made concrete — Go `context` and gRPC deadlines, Resilience4j and Envoy circuit-breaker configuration, bulkhead thread-pool sizing, and priority-aware shedding with real code — and we show how they compose into a coherent posture rather than a pile of knobs. The chapter closes with the distributed-systems lens on why resilience is about bounding blast radius and preserving partial availability, not preventing failure.

Learning goals — after this chapter you should be able to:

- Classify failure modes (fail-stop, fail-slow, overload, partial degradation) and explain why fail-slow is the most dangerous for cascading failures.
- Set timeouts and propagate deadlines correctly across a call graph (Go `context`, gRPC deadline, Envoy `timeout / retry`), and explain why timeouts without deadlines still cascade.
- Implement retries with exponential backoff, jitter, idempotency, and a retry budget that prevents amplification, and decide when *not* to retry.
- Implement the circuit-breaker state machine (closed → open → half-open) with Resilience4j/Envoy/istio, tune thresholds, and explain what breaks if the breaker is per-instance vs. per-cluster.
- Apply bulkheads — thread-pool, connection-pool, and semaphore isolation — to partition failure, size them from Little's Law, and avoid the "bulkhead that is just a smaller thread pool" anti-pattern.
- Design load shedding that protects SLOs under overload: priority lanes, shed-at-the-edge, queue-vs-drop, and graceful degradation with feature flags.
- Reason about hedging (hedged requests) and when duplicate work reduces p99 without collapsing the downstream.
- Compose timeouts, retries, breakers, bulkheads, and shedding into a single call-site policy and verify it with fault injection.

Failure is the steady state

A useful taxonomy:

| Mode | What happens | Why it cascades | Example |
|----------------------------|---|---|--|
| Fail-stop | Process crashes or network partitions; callers get an error quickly | Least dangerous — callers learn fast and can fail over | Pod OOMKilled, AZ network cut |
| Fail-slow | Dependency is alive but p99 goes 10–100× (GC pause, lock contention, downstream overload) | Callers block, exhaust threads/connections, become fail-slow themselves | Database replica lag, downstream thread-pool exhaustion |
| Overload | Demand exceeds capacity; latency rises, errors climb, retries amplify | Retry storm: $1\% \text{ error} \times 3 \text{ retries} \times N \text{ callers} = 3 \times \text{load}$ on an already overloaded dependency | Flash sale, scrape loop, thundering herd after failover |
| Partial degradation | Subset of shards/regions/replicas unhealthy | Without isolation, unhealthy subset poisons healthy callers that happen to hit it | One Cassandra replica slow, one Kafka partition leader stalled |

Fail-slow is the killer. A dependency that fails fast costs one timeout; a dependency that responds in 5 s instead of 50 ms holds every caller's thread for 5 s. With a 200-thread pool and 100 RPS, 10 concurrent slow calls consume 5% of the pool for 5 s — and the next 500 requests queue behind them. This is *cascading failure*: healthy services become unhealthy because they wait on an unhealthy one.

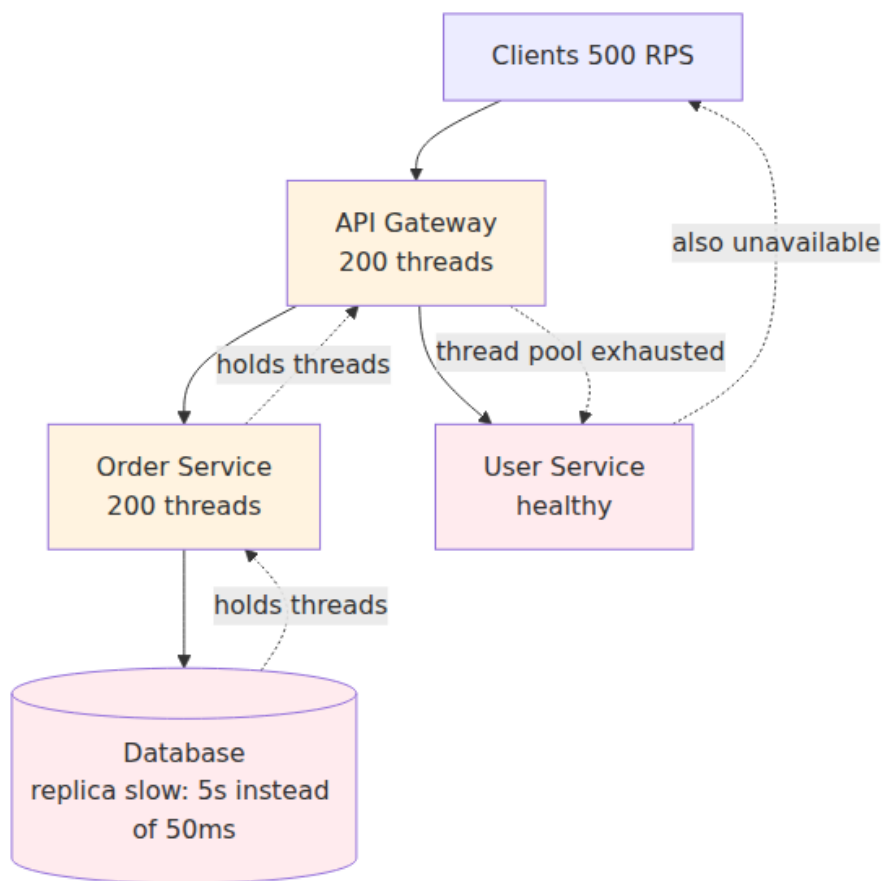


Figure 11-1: Cascading failure via fail-slow. A slow replica holds Order Service threads, which hold Gateway threads, which starve even healthy paths (User Service). One slow dependency makes the whole graph unavailable.

Resilience patterns all answer one question: **how do you bound the blast radius of a slow or failing dependency?**

Boundary note. Wire-level primitives (timeouts, retries, backoff, hedging) at the TCP/HTTP layer are introduced in Volume 3, Chapter 11 — Network Reliability. This chapter treats them as *system-design patterns* composed with bulkheads, breakers, and shedding at service and mesh scale, with sizing, tuning, and interaction. Rate limiting and quotas (backpressure at the edge) are in Chapter 9. Chaos and fault-

injection as *operational practice* are in Volume 11, Chapter 8; Jepsen-style correctness testing is in Volume 6, Chapter 12. SLOs and error budgets that define "how much failure is acceptable" are in Volume 11, Chapter 1.

Timeouts and deadline propagation — the first line of defense

A timeout says "do not wait longer than X." A deadline says "this entire causal chain must finish by wall-clock time T." Without deadlines, timeouts compose badly.

Suppose Gateway calls Order (timeout 500 ms), Order calls Inventory (timeout 500 ms) and Payments (timeout 500 ms) sequentially. Gateway's timeout is 1 s, but Order's two calls can take up to 1 s total — Gateway times out while Order is still waiting, and Order's work is wasted (and may have side effects). Worse, retries from Gateway create duplicate work in Order that is still running.

Deadline propagation fixes this: Gateway sets `deadline = now + 800 ms` and passes it to Order; Order passes `deadline - elapsed` to each downstream, so the whole tree respects one budget.

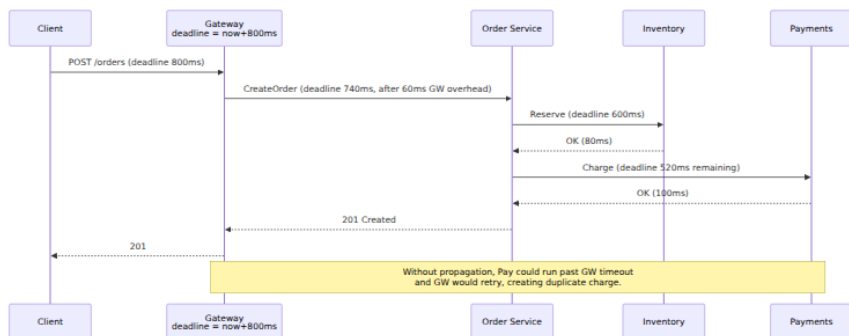


Figure 11-2: Deadline propagation. Each hop forwards the remaining budget so no downstream outlives its caller.


```

// Go – deadline propagation via context (server side)
func (s *OrderService) CreateOrder(ctx context.Context, req *CreateOrderRequest) (*Order, error) {
    // ctx already carries the deadline from gRPC (grpc-timeout header).
    // Enforce it on every downstream call; do not use context.Background().
    invCtx, cancel := context.WithTimeout(ctx, 300*time.Millisecond)
    defer cancel()
    if err := s.inventory.Reserve(invCtx, req.Items); err != nil {
        return nil, fmt.Errorf("inventory: %w", err)
    }

    payCtx, cancel2 := context.WithTimeout(ctx, 300*time.Millisecond)
    defer cancel2()
    if err := s.payments.Charge(payCtx, req.Payment); err != nil {
        return nil, fmt.Errorf("payments: %w", err)
    }
    return s.store.Create(ctx, req)
}

// gRPC client – set deadline at the edge
ctx, cancel := context.WithTimeout(context.Background(), 800*time.Millisecond)
defer cancel()
order, err := orderClient.CreateOrder(ctx, req)

```

```

# Envoy – timeout and retry that respects the deadline (x-envoy-expected-rq-timeout-ms)
route_config:
  virtual_hosts:
    - name: api
      routes:
        - match: { prefix: "/orders" }
          route:
            cluster: order_service
            timeout: 0.8s # hard cap; Envoy cancels upstream after this
            retry_policy:
              retry_on: "5xx,reset,connect-failure"
              num_retries: 2
              per_try_timeout: 0.35s # each try bounded; total still capped by route timeout
              retry_back_off: { base_interval: 25ms, max_interval: 200ms }
  s }

```

Rules:

- **Every outbound call gets a timeout derived from the incoming deadline**, never a fixed constant that ignores it.
 - Use `context (Go)`, `Deadline (gRPC)`, `timeout (Envoy/HTTP)` — never `context.Background()` on a downstream call.
 - **Timeout < SLO budget / depth of call graph**. If the call graph is 3 deep and the SLO is 500 ms, per-hop timeout cannot be 500 ms.
 - **Hedge vs. retry interacts with deadlines** — see Hedging below.
-

Retries — help that hurts

Retries turn transient failures into successes — and turn overload into collapse if unbounded. Three disciplines:

1. Retry only what is safe and likely to succeed

| Retry? | Condition |
|--------|---|
| Yes | Idempotent operations, transient errors (503 , 429 with <code>Retry-After</code> , connect failures), safe to duplicate |
| No | Non-idempotent writes without idempotency key, 400 / 422 (client error), 429 without backoff, overloaded downstream (retry amplifies) |

Idempotency keys (Volume 6, Chapter 9; Volume 8, Chapter 6) make retries safe for writes: `Idempotency-Key: <uuid>` + server-side dedup by key.

2. Backoff with jitter — de-synchronize the herd

```
// Exponential backoff with full jitter (decorrelated jitter variant is
also common)
func backoff(attempt int) time.Duration {
    const base = 50 * time.Millisecond
    const cap_ = 2 * time.Second
    exp := base * time.Duration(1<<attempt) // 50, 100, 200, 400, ...
    if exp > cap_ {
        exp = cap_
    }
    // Full jitter: random in [0, exp) — prevents synchronized retries
    return time.Duration(rand.Int63n(int64(exp)))
}

// Retry with idempotency key and budget check
func doWithRetry(ctx context.Context, op func(context.Context) error) error {
    var last error
    for attempt := 0; attempt < 3; attempt++ {
        if attempt > 0 {
            select {
            case <-time.After(backoff(attempt)):
            case <-ctx.Done():
                return ctx.Err()
            }
        }
        if err := op(ctx); err == nil {
            return nil
        } else if !isRetryable(err) {
            return err
        } else {
            last = err
        }
        if !retryBudget.Allow() { // see below
            return fmt.Errorf("retry budget exhausted: %w", last)
        }
    }
    return last
}
```

Without jitter, 1,000 clients that fail at the same instant retry at the same instant — a thundering herd that re-overloads the dependency at the exact moment it is recovering.

3. Retry budgets — bound amplification

A retry budget caps the fraction of calls that may be retries (typical: 20-30% of total requests). Borrowed from Finagle/Envoy:

```
budget = min(0.2 * total_requests_in_window, max_retries)
```

If 1,000 RPS is flowing and 10% fails, a 20% budget allows 200 retries/s — enough to handle transients, not enough to 3× the load. When the budget is exhausted, fail fast instead of retrying.

```
# Envoy retry budget (circuit-breaker-adjacent)
circuit_breakers:
  thresholds:
    - max_retries: 50          # cap concurrent retries
      retry_budget:
        budget_percent: 20
        min_retry_concurrency: 5
```

Circuit breakers — stop calling what is already down

A circuit breaker wraps a downstream call and tracks its health. Three states:

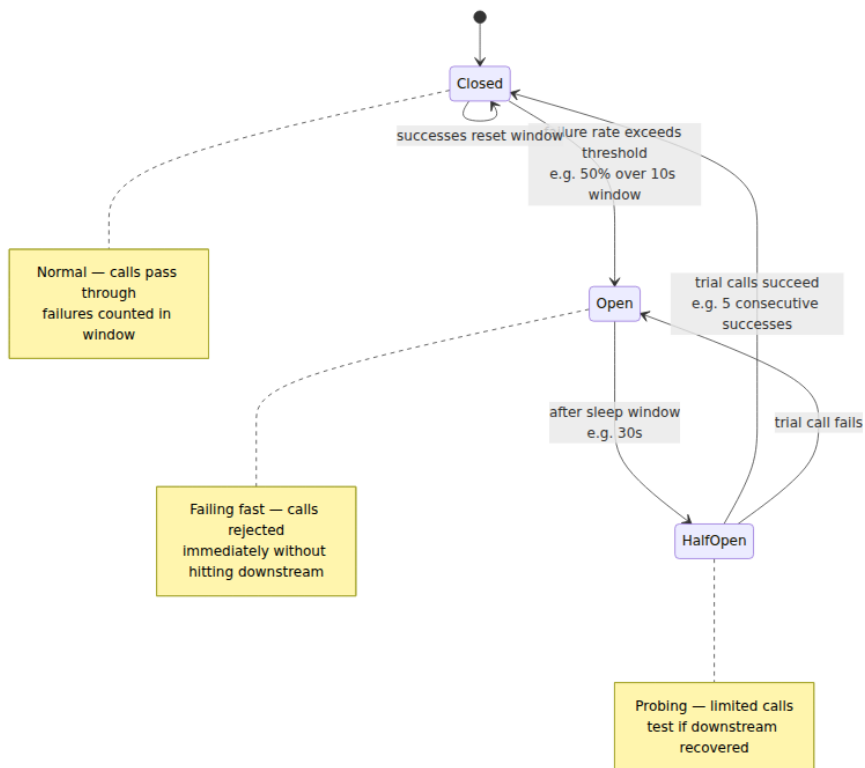


Figure 11-3: Circuit-breaker state machine. The breaker fails fast in Open, giving the downstream time to recover and the caller time to shed or fallback.

Why not just rely on timeouts? A timeout still pays the timeout duration and still sends load to an overloaded dependency. An open breaker rejects in microseconds and sheds load — the downstream gets breathing room, the caller preserves threads.

```

// Resilience4j – circuit breaker around a downstream call (Java)
CircuitBreakerConfig cfg = CircuitBreakerConfig.custom()
    .failureRateThreshold(50)                // open if >=50% failures
    .slidingWindowSize(100)                 // over last 100 calls
    .minimumNumberOfCalls(20)               // need 20 calls before
evaluating
    .waitDurationInOpenState(Duration.ofSeconds(30))
    .permittedNumberOfCallsInHalfOpenState(5)
    .recordExceptions(IOException.class, TimeoutException.class)
    .ignoreExceptions(BusinessException.class) // don't count 4xx as
failure
    .build();

CircuitBreaker cb = CircuitBreaker.of("inventory", cfg);
Supplier<Order> decorated = CircuitBreaker.decorateSupplier(cb, () -> i
nventory.reserve(items));
Try<Order> result = Try.ofSupplier(decorated)
    .recover(CircuitBreakerOpenException.class, e -> fallbackOrder());

// Metrics – export to Prometheus via Micrometer
// circuitbreaker_calls_total{state="open"} / {state="closed"} /
{state="half_open"}

```

```

# Envoy – outlier detection as a distributed circuit breaker (per-host
ejection)
outlier_detection:
  consecutive_5xx: 5
  interval: 10s
  base_ejection_time: 30s
  max_ejection_percent: 50
  failure_percentage_threshold: 50
  enforcing_consecutive_5xx: 100
  enforcing_failure_percentage: 100

# Istio DestinationRule – circuit breaker at mesh level
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: inventory-breaker }
spec:
  host: inventory.default.svc.cluster.local
  trafficPolicy:
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
    connectionPool:
      tcp: { maxConnections: 200 }
      http: { http1MaxPendingRequests: 100, maxRequestsPerConnection: 1
0 }

```

Tuning guidance:

- **Window size** must be large enough to distinguish noise from failure (100 calls), small enough to react quickly (10 s).
- **Half-open probe count** low (3–5) — a recovering downstream should not be hit by full traffic.
- **Per-instance vs. per-cluster breaker:** local breakers trip independently (one slow pod does not open the breaker for all pods); cluster breakers (Envoy outlier detection) are needed when the downstream is a shared database. Use both: local breaker for thread protection, outlier ejection for host-level isolation.
- **Do not count business errors** (4xx, validation) as failures — they indicate caller bugs, not downstream health.

Bulkheads — partition failure

A bulkhead isolates failure to a compartment. Two forms:

Thread-pool / semaphore bulkheads

Give each downstream its own concurrency limit so a slow dependency cannot consume the caller's entire pool.

```
// Resilience4j – bulkhead (semaphore) per downstream
BulkheadConfig bhCfg = BulkheadConfig.custom()
    .maxConcurrentCalls(30)           // at most 30 concurrent calls to
inventory
    .maxWaitDuration(Duration.ofMillis(50)) // queue briefly, then
reject
    .build();
Bulkhead inventoryBulkhead = Bulkhead.of("inventory", bhCfg);

// Compose with breaker: bulkhead -> breaker -> call
Supplier<Order> guarded = Bulkhead.decorateSupplier(inventoryBulkhead,
    CircuitBreaker.decorateSupplier(cb, () -> inventory.reserve(items))
);
```

```
// Go – semaphore bulkhead via weighted semaphore + context
var invSem = semaphore.NewWeighted(30)

func callInventory(ctx context.Context, req any) error {
    if err := invSem.Acquire(ctx, 1); err != nil {
        return fmt.Errorf("inventory bulkhead full: %w", err)
    }
    defer invSem.Release(1)
    return inventory.Reserve(ctx, req)
}
```

Sizing from Little's Law: `concurrency = throughput × latency`. If Inventory handles 100 RPS at p50 50 ms, steady-state concurrency is 5. A bulkhead of 30 allows 6× burst and bounds the blast radius to 30 threads. Size per downstream from its SLO, not from the caller's pool size.

Connection-pool bulkheads

Separate pools per downstream (or per criticality tier) so a slow downstream does not exhaust the shared HTTP client.


```
# Envoy – connection pool per cluster is already a bulkhead
clusters:
  - name: inventory
    connect_timeout: 0.25s
    circuit_breakers:
      thresholds: [{ max_connections: 100, max_pending_requests: 50,
max_requests: 200 }]
  - name: payments # separate pool – inventory slowness cannot
exhaust payments connections
    connect_timeout: 0.25s
    circuit_breakers:
      thresholds: [{ max_connections: 100, max_pending_requests: 50,
max_requests: 200 }]
```

The anti-pattern: creating one bulkhead for "all external calls" — that is just a smaller global pool, not isolation. Bulkheads must be per dependency and per criticality.

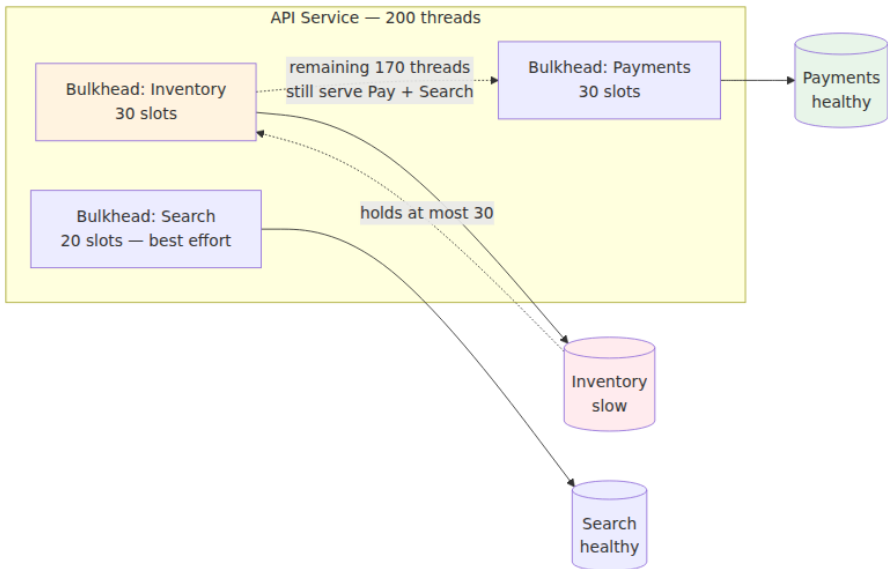


Figure 11-4: Bulkheads partition the caller's concurrency. Inventory slowness can at most consume 30 slots; Payments and Search remain available.

Load shedding — choose what to drop

When demand exceeds capacity, every system sheds load — the question is whether it does so deliberately or by collapsing. Shedding is admission control at the server.

Where to shed

| Layer | Mechanism | Granularity |
|----------------|--|--|
| Edge / Gateway | Envoy
circuit_breakers.max_requests ,
token-bucket admission | Per-route, per-tenant — best place to shed (closest to client, cheapest) |
| Service | Priority lanes, queue + drop, concurrency caps | Per-handler, per-priority |
| Downstream | DB max_connections , queue depth, backpressure signals | Coarse — already too late if shedding here |

Shed as early as possible. A request rejected at the gateway costs microseconds; a request that traverses three services before being rejected at the database costs milliseconds and holds resources in every hop.

Priority lanes

Not all requests are equal. Interactive reads > writes > batch/backfill > health checks that are not readiness probes.

```
// Priority-aware handler – shed low priority first under load
type Priority int

const (
    PrioInteractive Priority = iota // never shed until last
    PrioWrite
    PrioBatch                      // shed first
)

var (
    inflight = make(map[Priority]*int32)
    limits   = map[Priority]int32{PrioInteractive: 100, PrioWrite:
80, PrioBatch: 20}
)

func handle(w http.ResponseWriter, r *http.Request) {
    prio := priorityOf(r) // from header / route / tenant tier
    n := inflight[prio]
    if atomic.AddInt32(n, 1) > limits[prio] {
        atomic.AddInt32(n, -1)
        // Shed low priority first: if batch is full, try shedding
batch to admit interactive
        if prio == PrioInteractive && tryShedBatch() {
            // admitted by shedding batch
        } else {
            http.Error(w, "overloaded", http.StatusServiceUnavailable)
            w.Header().Set("Retry-After", "1")
            return
        }
    }
    defer atomic.AddInt32(n, -1)
    serve(w, r)
}
```

```
# Envoy – priority routing + circuit breakers per priority
# High-priority cluster gets higher caps
clusters:
- name: api_high
  circuit_breakers: { thresholds: [{ max_requests: 500 }] }
- name: api_low
  circuit_breakers: { thresholds: [{ max_requests: 50 }] }

# Gateway sheds low-priority cluster first via rate limit / concurrency
cap
```

Queue vs. drop

| Strategy | Behavior | When to use |
|-------------------------|---|---|
| Drop (fail fast) | Reject immediately with 503 / 429 + Retry-After | Interactive path — caller can retry or fallback; queuing would blow latency SLO |
| Bounded queue | Buffer briefly (50-200 ms) then drop | Short bursts where a small queue absorbs jitter without violating SLO |
| Unbounded queue | Never — queue grows until OOM or latency is minutes | Never in production |

Graceful degradation composes with shedding: when shedding, serve a degraded response (cached, stale, partial) rather than an error where possible. Degradation is a product decision — "show stale feed with a banner" vs. "show error" — but the mechanism is a feature flag / fallback branch.

```
func getFeed(ctx context.Context, userID string) (*Feed, error) {
    feed, err := feedService.Get(ctx, userID)
    if err != nil {
        if errors.Is(err, ErrOverloaded) || errors.Is(err, ErrCircuitOp
en) {
            // Degrade: serve stale cached feed rather than error
            if cached := cache.Get("feed:" + userID); cached != nil {
                return cached, nil // caller adds Degraded: true header
            }
        }
        return nil, err
    }
    return feed, nil
}
```

Hedging — duplicate work to cut tail latency

A hedged request sends the same RPC to two replicas (or retries quickly to a second replica) and takes the first response, cancelling the other. It trades extra load for lower p99.

```

// Hedged request – send to replica 2 if replica 1 hasn't responded in
hedgeDelay
func hedgedGet(ctx context.Context, key string) (Value, error) {
    const hedgeDelay = 30 * time.Millisecond // p90 of the downstream
    ctx1, cancel1 := context.WithCancel(ctx)
    defer cancel1()

    type result struct { v Value; err error }
    ch := make(chan result, 2)

    go func() { v, err := replica1.Get(ctx1, key); ch <- result{v,
err} }()

    select {
    case r := <-ch:
        return r.v, r.err // first replica won
    case <-time.After(hedgeDelay):
        // Hedged call to second replica
        ctx2, cancel2 := context.WithCancel(ctx)
        defer cancel2()
        go func() { v, err := replica2.Get(ctx2, key); ch <- result{v,
err} }()

        r := <-ch // first of the two to respond
        // The slower goroutine's context is cancelled when we return
        return r.v, r.err
    case <-ctx.Done():
        return nil, ctx.Err()
    }
}

```

Hedging helps when downstream p99 >> p50 (GC pauses, noisy neighbor) and hurts when the downstream is overloaded — duplicate requests increase load. Guard hedging with the same budget as retries, and only hedge idempotent reads.

Composing the policy — a call-site template

A single outbound call should compose all patterns in order:

```

incoming deadline
→ bulkhead (admit? else 503)
→ breaker (open? else fail fast)
→ timeout (per-try, derived from deadline)
→ retry/hedge (bounded by budget, only idempotent)
→ actual RPC
← fallback / degrade on failure

```

```

// Composed call site – bulkhead + breaker + timeout + retry + fallback
func getInventory(ctx context.Context, items []Item) (*Reservation, err
or) {
    // Bulkhead
    if err := invSem.Acquire(ctx, 1); err != nil {
        return fallbackReservation(items)
    }
    defer invSem.Release(1)

    // Breaker
    if !invBreaker.Allow() {
        return fallbackReservation(items)
    }

    var res *Reservation
    err := retry.Do(ctx, func(ctx context.Context) error {
        // Per-try timeout derived from incoming deadline
        tryCtx, cancel := context.WithTimeout(ctx,
200*time.Millisecond)
        defer cancel()
        var err error
        res, err = inventoryClient.Reserve(tryCtx, items)
        return err
    }, retry.WithBudget(retryBudget), retry.RetryIf(isRetryable))

    if err != nil {
        invBreaker.RecordFailure()
        return fallbackReservation(items)
    }
    invBreaker.RecordSuccess()
    return res, nil
}

```

```
# Envoy – same composition declaratively
clusters:
  - name: inventory
    connect_timeout: 0.2s
    circuit_breakers:
      thresholds: [{ max_connections: 100, max_pending_requests: 50,
max_requests: 200, max_retries: 20 }]
    outlier_detection: { consecutive_5xx: 5, interval: 10s,
base_ejection_time: 30s, max_ejection_percent: 50 }
    common_http_protocol_options: { idle_timeout: 60s }
routes:
  - match: { prefix: "/orders" }
    route:
      cluster: inventory
      timeout: 0.6s
      retry_policy: { retry_on: "5xx,reset", num_retries: 2,
per_try_timeout: 0.25s }
```

Verifying resilience — fault injection

A resilience policy that has never been exercised is a hypothesis. Verify continuously:

- **Unit / integration:** inject latency (`tc qdisc`), errors (proxy that returns 503), and slow downstream (sleep) in tests; assert breaker opens, bulkhead rejects, fallback serves.
- **Staging chaos:** latency injection at the mesh (Istio `fault.delay`, Envoy `fault filter`), pod kills, partition via `iptables`.
- **Production:** controlled fault injection (Chaos Mesh, Litmus, AWS FIS) behind feature flags, during business hours, with automatic rollback on SLO burn — see Volume 11, Chapter 8.

```
# Istio – fault injection for resilience testing (staging)
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: { name: inventory-fault }
spec:
  hosts: [inventory.default.svc.cluster.local]
  http:
    - fault:
        delay: { percentage: { value: 5 }, fixedDelay: 2s } # 5% of
requests delayed 2s
        abort: { percentage: { value: 1 }, httpStatus: 503 }
        route: [{ destination: { host: inventory.default.svc.cluster.local } }]
    }
```

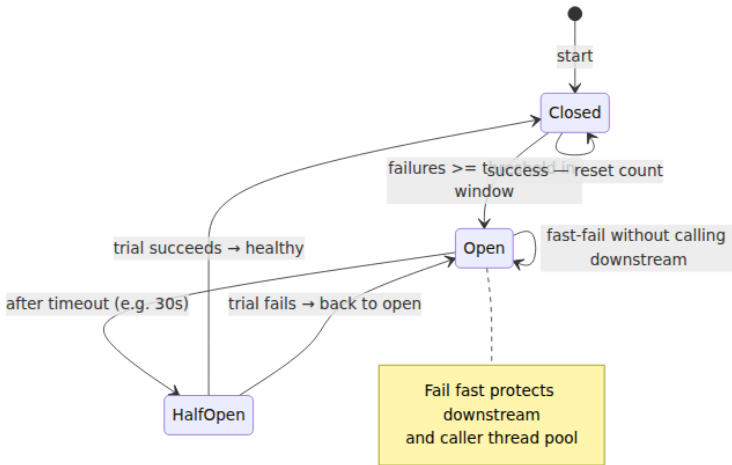
Distributed-systems lens

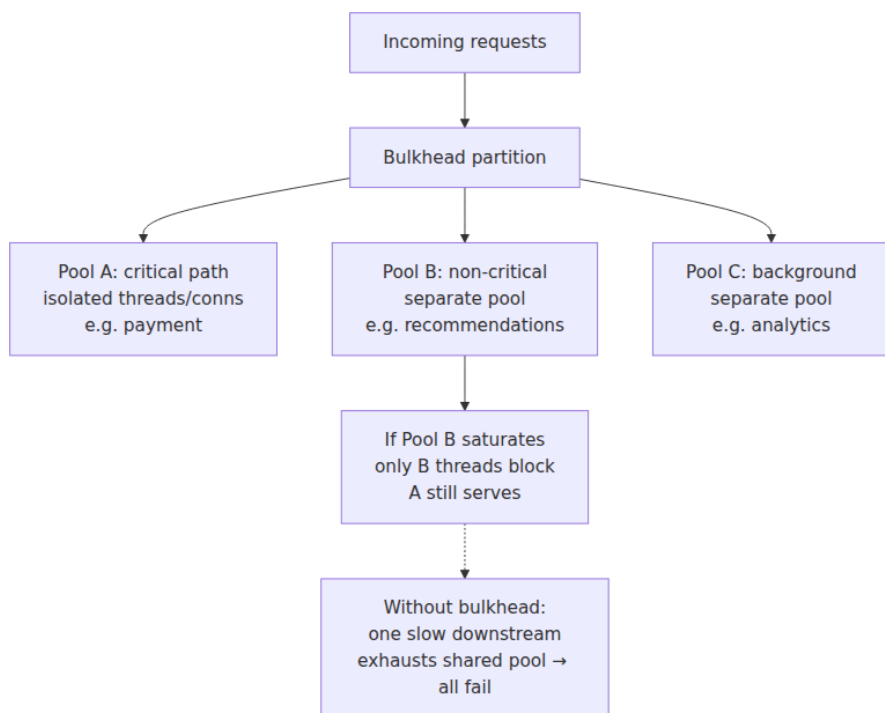
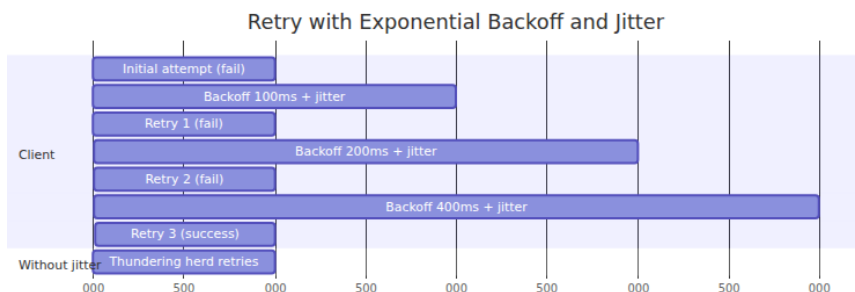
Resilience is not "make failures go away" — it is "make failures local and partial." Timeouts bound waiting; breakers bound calling; bulkheads bound sharing; shedding bounds demand. Together they turn a graph where one slow node makes all nodes slow into a graph where one slow node makes only its callers degrade, and only for the blast radius you chose. The price is deliberate: some requests are rejected or degraded that could have succeeded if you had waited longer. That is the availability/consistency/latency trade-off made operational — preferring partial availability and bounded latency over unbounded waiting for strong consistency.

Key takeaways

- Fail-slow is the most dangerous mode — it cascades by holding threads/connections; fail-stop is cheap by comparison. Design for fail-slow.
- Always propagate deadlines — a timeout without a deadline that covers the whole causal chain still cascades and wastes work; derive per-hop timeouts from the incoming deadline.
- Retry only idempotent operations with exponential backoff + jitter and a retry budget (20–30%) that bounds amplification; otherwise fail fast.

- Circuit breakers (closed/open/half-open) fail fast in microseconds and give overloaded downstreams breathing room; tune window size, half-open probes, and per-instance vs. per-cluster scope separately.
- Bulkheads partition concurrency per downstream (and per criticality) — size from Little's Law, not from the caller's pool; one shared bulkhead is not a bulkhead.
- Shed load at the edge, by priority (interactive > write > batch), with explicit queue-vs-drop policy and `Retry-After`; degrade to cached/stale responses where product allows.
- Hedging trades duplicate load for lower p99 — only for idempotent reads, guarded by a budget, with hedge delay $\approx$ p90.
- Compose at the call site (bulkhead $\rightarrow$ breaker $\rightarrow$ timeout $\rightarrow$ retry/hedge $\rightarrow$ fallback) and verify with fault injection from unit test through production chaos.





Further reading

- Nygard — Release It! (2nd ed.), Stability patterns (Circuit Breaker, Bulkhead, Timeout).
- Netflix — Hystrix (historical) and resilience4j docs. <https://resilience4j.readme.io/>

- Envoy — Circuit breaking, outlier detection, retry budgets, fault injection. https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/fault_filter
- Google SRE Book — Handling overload, load shedding, cascading failure. <https://sre.google/sre-book/handling-overload/>
- Finagle — Retry budgets and hedging. <https://twitter.github.io/finagle/guide/Resilience.html>
- Brooker et al. — Timeout and retry guidance (AWS Builders' Library). <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>
- Volume 3, Chapter 11 — Network reliability primitives; Volume 11, Chapters 1 and 8 — SLOs and chaos engineering.

Chapter 12 — Design Case Studies: Feed, Chat, Notifications, and a Global KV Store

What this chapter covers. Chapters 1–11 built the system-design vocabulary — scaling principles, estimation, caching, traffic management, data modeling, service decomposition, events, gateways, rate limiting, multi-region, and resilience. This chapter uses that vocabulary to design four systems that appear in almost every senior backend interview and, more importantly, in almost every large product. Each case is treated as a real build, not a whiteboard sketch: requirements with scale numbers, the trade-off that defines the architecture, a concrete component design with real configuration and code, storage and streaming choices with failure semantics, and the operational realities that determine whether the design survives contact with production. The four are chosen to cover the design space: **Feed** (read-heavy fan-out and ranking), **Chat** (real-time messaging with ordering and presence), **Notifications** (fan-out to heterogeneous channels with preferences and batching), and **Global KV** (the storage substrate that the other three could be built on). Together they exercise every pattern from the volume and show how the same primitives compose into very different systems when the requirements shift.

Learning goals — after this chapter you should be able to:

- Design a social feed that serves personalized, ranked timelines at 300M DAU / 100K RPS, compare fan-out-on-write vs. fan-out-on-read vs. hybrid on cost and tail latency, and choose per user class (normal, celebrity, cold).
 - Design a 1:1 and group chat system with presence, typing indicators, and ordered, exactly-once delivery over WebSockets, with message storage, pagination, and multi-device sync that does not lose or duplicate messages.
 - Design a notification platform that fans out events to push, email, and SMS with per-user preferences, batching/digest, priority lanes, rate limits, and idempotent delivery, and explain where it sheds or degrades under overload.
 - Design a globally distributed key-value store with tunable consistency, consistent hashing, quorum replication, hinted handoff, and cross-region replication, and explain the CAP/PACELC trade-off per operation.
 - Apply estimation (Chapter 2), caching (Chapter 3), traffic management (Chapter 4), data modeling (Chapter 5), events (Chapter 7), gateways/edge (Chapter 8), rate limiting (Chapter 9), multi-region (Chapter 10), and resilience (Chapter 11) inside each case, naming which pattern dominates where.
 - Name the failure mode of each case (hot celebrity, thundering herd on reconnect, notification storm, ring imbalance) and the mitigation that bounds it.
-

How to read these cases

Each case follows the same shape so you can compare them:

1. **Requirements and scale** — functional/invariants and back-of-envelope numbers.
2. **The defining trade-off** — the one decision that shapes the architecture.
3. **Architecture and data flow** — components, storage, streaming, and a Mermaid diagram.

- 4. **Deep dive** — the hard part (ranking, ordering, batching, consistency) with real config/code.
- 5. **Failure and operations** — what breaks at scale and how it is bounded.

A note on numbers: every scale estimate states its assumptions. Change the assumptions and the design may change — that is the point of Chapter 2's estimation discipline.

Boundary note. These cases are *system-design compositions* of primitives defined elsewhere. Algorithmic depth (consistent hashing, probabilistic structures, scheduling) is in Volume 14. Queue/log/stream internals and delivery semantics are in Volume 10. Database engine internals are in Volume 5. Distributed-systems theory (consistency, consensus, quorums) is in Volume 6. This chapter shows how to *compose* those primitives into product systems and where to look up the mechanism underneath.

Case 1 — Feed: the timeline problem

Requirements and scale

Functional: users follow other users/topics; each user has a personalized home timeline (ranked) and a profile timeline (chronological); posts contain text, media refs, and engagement counts (likes, reposts, replies); clients paginate with cursor; edits/deletes propagate; ranking considers recency, engagement, and affinity.

Non-functional: 300M DAU, 500M posts/day, average 200 follows per user, p99 home-timeline read < 200 ms, write (post) p99 < 500 ms, available during AZ/region loss, strong enough consistency that a user sees their own post immediately (read-your-writes) but followers may see it within seconds.

| Quantity | Assumption | Result |
|----------------------|--------------------|---|
| Writes | 500M posts/day | ~5.8K posts/s avg, ~25K peak |
| Fan-out work (naive) | 200 followers/post | ~1.16M timeline inserts/s avg, ~5M peak |

| Quantity | Assumption | Result |
|----------------|------------------------------------|--|
| Reads | 300M users × 20 timeline reads/day | ~70K reads/s avg, ~200K peak |
| Read:write | ~12:1 | Read-heavy — cache and precomputation dominate |
| Storage (post) | 1 KB metadata + media refs | ~500 GB/day, ~180 TB/year before replication |

The defining trade-off: fan-out on write vs. on read

| Strategy | Write path | Read path | Cost | Tail latency | When it wins |
|--------------------------------|--|--|-------------------------------------|---|-------------------------------------|
| Fan-out on write (push) | On post, append post ID to each follower's timeline (materialized) | Read pre-materialized timeline — $O(1)$ | Write amplification $200\times$ avg | Low, bounded | Read-heavy, normal users |
| Fan-out on read (pull) | Write only to author's outbox | On read, merge outboxes of all followees, rank, paginate | No write amplification | High, fan-in over 200 outboxes + ranking per read | Write-heavy celebrities, cold users |
| Hybrid | Push to active followers; pull for celebrities + inactive | Push path for most, pull/merge for the long tail | Balanced | Bounded with fallback | Production default |

Pure push collapses on celebrities (a 100M-follower account would require 100M inserts per post, $100M \times 25K$ peaks is impossible). Pure pull collapses on reads (merging 200 outboxes per read at 200K RPS is 40M outbox reads/s). Every real feed is hybrid.

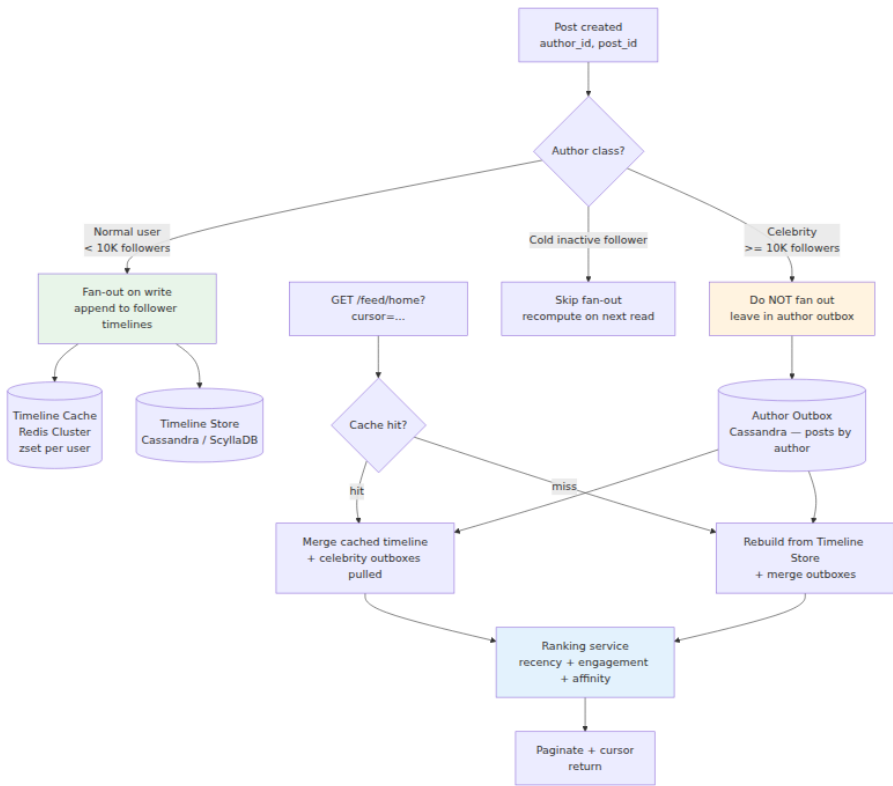


Figure 12-1: Hybrid fan-out. Normal posts are pushed; celebrity posts are pulled at read time. Ranking merges both sources.

Architecture and data flow

```

Client -> Gateway (auth, rate limit) -> Feed Service -> [Timeline
Cache -> Timeline Store]
                                     |
                                     +-> Post Service -> Outbox (Cas
sandra) -> Kafka (post.created)
                                     +-> Social Graph (follows) -> c
ache (Redis) + store (Cassandra/MySQL)
                                     +-> Ranking Service (feature st
ore + model)
                                     +-> Fan-out Workers (consume po
st.created, write timelines)
                                     +-> Media Service (object
store refs)
  
```

The write path is event-driven (Chapter 7): Post Service appends to Outbox + emits `post.created` via transactional outbox (Debezium or dual-write with outbox table); Fan-out Workers consume and write timelines asynchronously so post latency is not coupled to fan-out duration.

Deep dive

Timeline storage — two layers:

```
-- Cassandra – timeline store (materialized per-user timeline, time-ordered)
CREATE TABLE timelines (
  user_id    uuid,
  posted_at  timeuuid,           -- time-ordered clustering key
  post_id    uuid,
  author_id  uuid,
  PRIMARY KEY (user_id, posted_at)
) WITH CLUSTERING ORDER BY (posted_at DESC)
   AND compaction = {'class': 'LeveledCompactionStrategy'};

-- Outbox – posts by author (source of truth for pull path)
CREATE TABLE posts_by_author (
  author_id  uuid,
  posted_at  timeuuid,
  post_id    uuid,
  body       text,
  media_ids  list<uuid>,
  PRIMARY KEY (author_id, posted_at)
) WITH CLUSTERING ORDER BY (posted_at DESC);

-- Social graph – follows (with cache in front)
CREATE TABLE follows (
  follower_id  uuid,
  followee_id  uuid,
  created_at   timestamp,
  PRIMARY KEY (follower_id, followee_id)
);
```



```

# Redis Cluster – hot timeline cache (zset per user, trimmed to 800
entries)
# ZADD timeline:{user_id} {timestamp} {post_id}
# ZREVRANGE timeline:{user_id} {cursor} {cursor+page_size-1}
# Celebrity outboxes are NOT cached here – merged at read time.

# Fan-out worker (Go, pseudo) – push to active followers only
func onPostCreated(ev PostCreated) {
    followers := graph.Followers(ev.AuthorID)           // cached, paginated
    active := filterActive(followers)                   // last_active < 7d
    if len(active) > 10000 { active = active[:10000] } // cap; rest via pull
    pipe := redis.Pipeline()
    for _, uid := range active {
        pipe.ZAdd(ctx, "timeline:"+uid, redis.Z{Score: float64(ev.PostedAt.UnixNano()), Member: ev.PostID})
        pipe.ZRemRangeByRank(ctx, "timeline:"+uid, 0, -801) // trim // async persist to Cassandra in batch
    }
    pipe.Exec(ctx)
}

```

Ranking at read time merges the cached timeline with celebrity outboxes, scores, and paginates. Scoring is a separate service so the feed path does not block on ML inference — precomputed features (affinity, engagement) are in a feature store (Redis/Feast), and the model is a lightweight ranker (logistic regression or two-tower) — heavy models run offline to update features.

```
// Read path – merge + rank + paginate (simplified)
func GetHomeTimeline(ctx context.Context, userID string, cursor
string, n int) ([]Post, string, error) {
    // 1. Followees partitioned into pushed vs pulled
    followees := graph.Followees(userID)
    celebs, normal := partitionByFollowerCount(followees)

    // 2. Pushed timeline (cache → store fallback)
    ids, nextCursor, err := timelineCache.Get(ctx, userID, cursor,
n*2) // over-fetch for merge
    if err != nil { ids, nextCursor, err = timelineStore.Get(ctx, userI
D, cursor, n*2) }

    // 3. Pull celebrity outboxes (parallel, bounded)
    celebPosts := pullOutboxes(ctx, celebs, 50) // cap: at most 50
celeb posts merged

    // 4. Merge, rank, paginate
    merged := mergeByTime(ids, celebPosts)
    scored := ranking.Score(ctx, userID, merged) // feature store
lookup + model
    page := scored[:min(n, len(scored))]
    posts := postStore.Hydrate(ctx, page) // batch get post bodies
    return posts, nextCursor, nil
}
```

```
# Kafka – post.created topic (log, partitioned by author_id for
ordering per author)
# Replication factor 3, acks=all, compaction not needed (retention 7d)
topic: post.created
    partitions: 256
    replication.factor: 3
    min.insync.replicas: 2
    retention.ms: 604800000
producer:
    acks: all
    enable.idempotence: true
    max.in.flight.requests.per.connection: 5
```

Pagination uses `timeuuid` cursor (opaque, stable across ranking changes is hard — most feeds re-rank per page and accept that insertion order may shift; cursor encodes `posted_at` of last item + rank offset).

Consistency: read-your-writes via cache-aside on post: after `POST /posts`, the author's timeline cache is updated synchronously before `201` returns, so the author sees their post on next `GET`. Followers

converge within fan-out lag (typically 200 ms-2 s; celebrity pull path is immediate).

Failure and operations

| Failure | Mitigation |
|--|---|
| Hot celebrity (100M followers) | Never fan out — pull path + per-celebrity outbox cache with short TTL; rate-limit celebrity post frequency |
| Fan-out lag (Kafka/worker backlog) | Monitor <code>consumer_lag</code> ; autoscale workers; shed fan-out to inactive users first (Chapter 11 priority lanes) |
| Cache stampede on miss | Single-flight rebuild + jittered TTL; stale-while-revalidate (serve stale cache while async rebuild runs) |
| Ranking service slow | Bulkhead (Chapter 11) — ranking has its own pool; degrade to chronological merge if ranker times out |
| Cassandra hot partition (celebrity outbox) | Outbox is read-heavy but sharded by author; celebrity outbox gets a dedicated cache layer (Redis replica) |

Case 2 — Chat: real-time messaging

Requirements and scale

Functional: 1:1 and group chat (up to 500 members), message ordering per conversation, exactly-once delivery (no loss, no duplicates visible), typing indicators, presence (online/last seen), read receipts, multi-device sync, message edits/deletes, media attachments, search.

Non-functional: 50M DAU, 1B messages/day (~12K msgs/s avg, 60K peak), p99 send-to-delivery < 150 ms intra-region, < 300 ms cross-region, ordered per conversation, 99.99% delivery (loss only on explicit client failure), works through reconnects.

| Quantity | Assumption | Result |
|-------------|---|--|
| Messages | 1B/day, avg 200 bytes payload | ~200 GB/day payload, ~1 TB/day with metadata/index |
| Connections | 50M DAU, 20% concurrent | 10M concurrent WebSockets |
| Groups | 10% of conversations are groups, avg 20 members | Fan-out 20× on group send |
| Storage | Per-conversation log, 1-year retention hot | ~365 TB/year (before replication) — tier to cold store |

The defining trade-off: ordering and delivery vs. availability

Chat needs per-conversation total order and at-least-once delivery with client-visible exactly-once (dedup). Strong cross-region ordering would require synchronous replication on every send — too slow. The standard choice: **single writer per conversation** (homed to a region or partition leader) gives total order without global sync; cross-region is async with ordering preserved per conversation via sequence numbers. Availability is per-conversation, not global — a partition affects only conversations homed to the partitioned region.

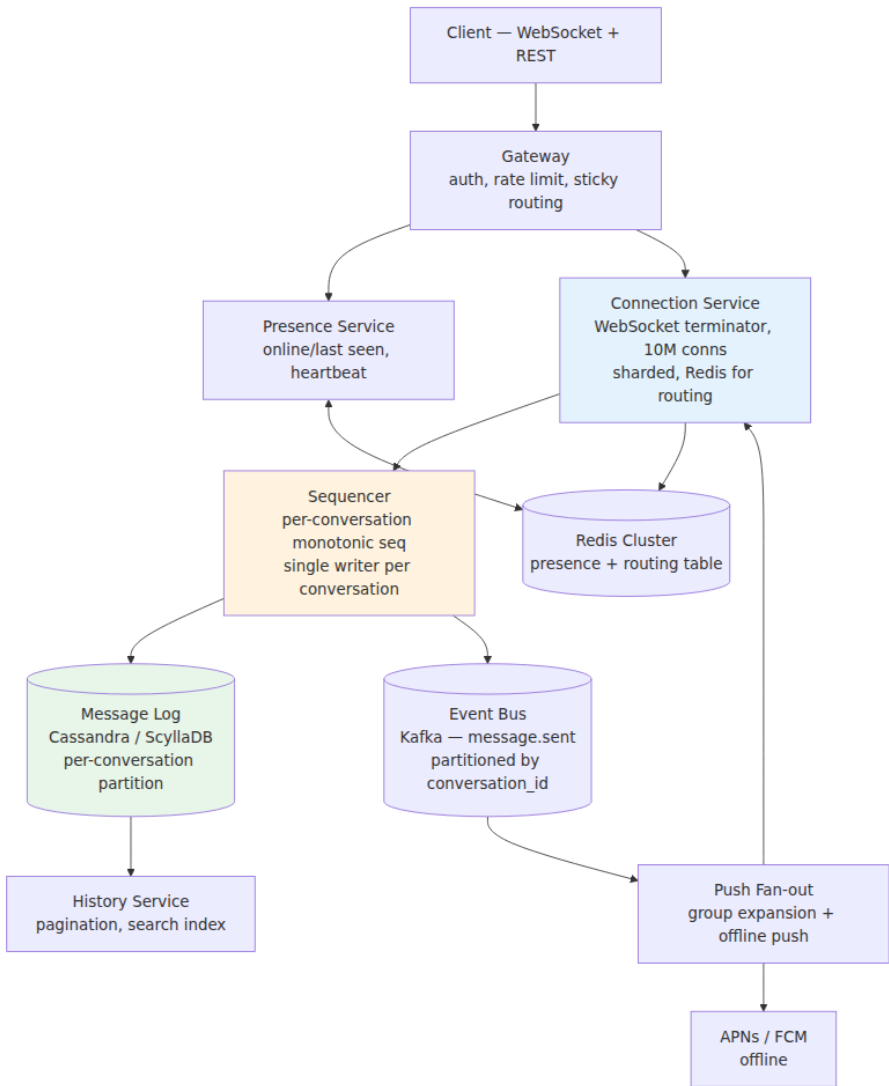


Figure 12-2: Chat architecture. Single-writer sequencer gives per-conversation order; connection service shards WebSockets and routes via a presence/routing table.

Architecture and data flow

Send path (ordered, exactly-once):

```
Client --WebSocket--> Conn Service --gRPC--> Sequencer
(per-conversation leader)
  Sequencer assigns seq = max_seq+1, writes to Cassandra (QUORUM), publishes to Kafka
  -> Fan-out: for each member, route to Conn shard holding their WebSocket (via Redis routing table)
  -> If member offline, enqueue offline push (APNs/FCM) + leave in log for sync on reconnect
```

Why single writer per conversation: without it, two concurrent sends to the same group from different regions would need distributed consensus per message. With it, order is local (one Raft/Kafka partition leader or one DB LWW per conversation) and replication is async.

Deep dive

Message storage — per-conversation ordered log:

```

-- Cassandra – messages per conversation, clustered by sequence number
CREATE TABLE messages (
    conversation_id uuid,
    seq             bigint,           -- monotonic per conversation,
assigned by sequencer
    message_id      uuid,           -- client-generated idempotency key
    sender_id       uuid,
    body            text,
    media_ids       list<uuid>,
    edited_at       timestamp,
    deleted         boolean,
    created_at      timeuuid,
    PRIMARY KEY (conversation_id, seq)
) WITH CLUSTERING ORDER BY (seq ASC);

-- Idempotency – dedup by (conversation_id, message_id) before
assigning seq
CREATE TABLE message_ids (
    conversation_id uuid,
    message_id      uuid,
    seq             bigint,
    PRIMARY KEY ((conversation_id, message_id))
) USING ...; -- lightweight transaction (CAS) or separate dedup store
(Redis SETNX)

-- Inbox per user for multi-device sync (materialized)
CREATE TABLE inbox (
    user_id         uuid,
    conversation_id uuid,
    last_read_seq   bigint,
    last_seq        bigint,
    PRIMARY KEY (user_id, conversation_id)
);

```

```

// Sequencer – single writer per conversation (sharded by
conversation_id hash)
func (s *Sequencer) Send(ctx context.Context, convID, msgID string, body
string) (int64, error) {
    // Idempotency: if msgID already sequenced, return existing seq
    (exactly-once visible)
    if seq, ok := s.dedup.Get(convID, msgID); ok {
        return seq, nil
    }
    // Assign next seq under per-conversation lock (or via Kafka
    partition leader / Raft)
    s.mu.Lock(convID)
    defer s.mu.Unlock(convID)

    seq := s.nextSeq[convID] + 1
    // Write to Cassandra with LWT for seq assignment or idempotent
    insert
    if err := s.store.InsertMessage(ctx, convID, seq, msgID, body);
    err != nil {
        return 0, err
    }
    s.nextSeq[convID] = seq
    s.dedup.Set(convID, msgID, seq)
    s.bus.Publish(ctx, "message.sent", MessageSent{ConvID: convID,
    Seq: seq, MsgID: msgID})
    return seq, nil
}

```

WebSocket and presence:


```

// Connection service – sharded WebSocket terminator
// Each instance holds ~50K connections; sharding via consistent hash
// of user_id
// Routing table in Redis: user_id -> {conn_server_id, last_seen}

func (c *ConnServer) OnMessage(ws *Conn, raw []byte) {
    var m ClientMessage
    json.Unmarshal(raw, &m)
    switch m.Type {
    case "send":
        seq, err := sequencer.Send(ctx, m.ConvID, m.MessageID, m.Body)
        ws.WriteJSON(ServerMessage{Type: "ack", MessageID:
m.MessageID, Seq: seq, Err: err})
        // Fan-out is async via Kafka consumer, not in this hot path
    case "typing":
        c.broadcastTyping(m.ConvID, m.SenderID) // ephemeral, no
persistence
    case "heartbeat":
        presence.Heartbeat(m.UserID) // Redis EXPIRE 60s
    }
}

// Fan-out consumer – group expansion + route to conn shard
func onMessageSent(ev MessageSent) {
    members := groupStore.Members(ev.ConvID)
    for _, uid := range members {
        route, ok := redis.HGet("route:" + uid)
        if ok {
            // Online – push via conn server RPC
            connServers[route.ServerID].Push(uid, ev)
        } else {
            // Offline – enqueue push notification + leave for sync
            pushQueue.Enqueue(uid, ev)
        }
    }
}

```

```
# Gateway – sticky WebSocket routing (hash by user_id so reconnects
land near state)
# Envoy consistent hashing on x-user-id header for WebSocket upgrade
route_config:
  virtual_hosts:
    - name: chat
      routes:
        - match: { prefix: "/ws" }
          route:
            cluster: conn_service
            hash_policy: { header: { header_name: "x-user-id" } }
            timeout: 0s # WebSocket – no timeout
            upgrade_configs: [{ upgrade_type: "websocket" }]
```

Multi-device sync: each device maintains `last_seq` per conversation; on reconnect it calls `GET /conversations/{id}/messages?since_seq={last_seq}` (paginated, ordered) to catch up. The server never deletes from the log within retention — sync is replay.

Ordering guarantee: `seq` is monotonic per conversation, assigned by one writer. Clients sort by `seq`; gaps trigger a sync fetch. No global order across conversations is attempted — it would be expensive and unnecessary.

Presence: heartbeat every 30 s, `SETEX presence:{user_id} 60 online` in Redis; `last_seen` in a durable store updated on disconnect. Presence is eventually consistent and lossy — acceptable.

Failure and operations

| Failure | Mitigation |
|--|--|
| Sequencer leader loss | Raft/Kafka partition leader failover (seconds); clients retry with same <code>message_id</code> — idempotency guarantees no duplicate seq |
| 10M concurrent WebSockets | Shard conn servers (200 × 50K), consistent hash, connection draining on deploy; use <code>S0_REUSEPORT</code> + <code>epoll</code> (Vol 2, Ch 7) |
| Thundering herd on reconnect (region flap) | Exponential backoff + jitter on client reconnect (Chapter 11); gateway rate-limits <code>WS upgrade</code> per IP/user; stagger offline push |

| Failure | Mitigation |
|--|--|
| Group fan-out amplification (500 × 60K msgs/s) | Fan-out is async via Kafka; shed offline push first (Chapter 11 priority lanes); cap group size or shard large groups into chunks |
| Message loss on conn server crash | Log is durable — client re-syncs by <code>since_seq</code> ; at-most-once WebSocket delivery + at-least-once log = exactly-once visible via dedup |
| Cross-region ordering | Conversations homed to a region (Chapter 10 sharding); cross-region delivery is async but per-conversation order is preserved via <code>seq</code> |

Case 3 — Notifications: fan-out to heterogeneous channels

Requirements and scale

Functional: events (like, comment, follow, order update) produce notifications delivered via push (APNs/FCM), email, SMS, and in-app inbox; per-user preferences (which events → which channels, quiet hours, frequency caps, digest vs. immediate); batching (digest every 15 min / hourly / daily); priority (transactional OTP vs. marketing); dedup (do not notify twice for same event).

Non-functional: 100M users, 2B events/day that may produce notifications, 500M notifications/day delivered, p99 event-to-push < 5 s for transactional, < 30 s for marketing, preference update visible within 10 s, survive downstream provider degradation (APNs/FCM/SES throttling).

| Quantity | Assumption | Result |
|-----------|---|-------------------------------|
| Events in | 2B/day | ~23K events/s avg |
| Fan-out | Avg 1.2 channels per event (many filtered by prefs) | ~28K channel deliveries/s avg |
| Push | 60% of deliveries | ~17K pushes/s avg |

| Quantity | Assumption | Result |
|----------------------|------------|----------------------------|
| Email | 30% | ~8K emails/s avg (batched) |
| Deduplication window | 24 h | ~2B dedup keys/day |

The defining trade-off: immediacy vs. batching and preference filtering

Sending every event immediately is simple but noisy (a viral post would generate thousands of pushes per follower) and wasteful (user in quiet hours, digest preference). Batching and filtering reduce load and improve UX but add latency and state (pending batches, preference cache). The architecture must support both on the same pipeline — per-event routing that decides *whether*, *when*, and *via which channel* each notification is delivered.

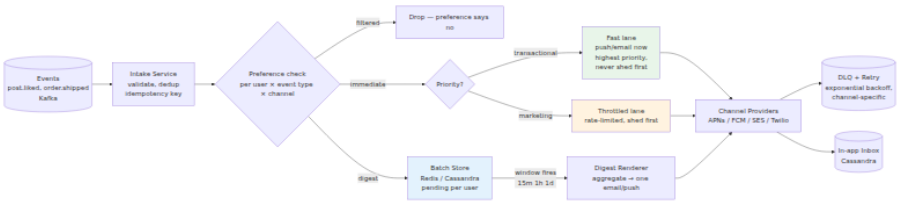


Figure 12-3: Notification pipeline. Preferences filter early; priority decides lane; digests batch; every channel has retry/DLQ.

Architecture and data flow

Event Producers -> Kafka (events) -> Intake (dedup + schema validation)
-> Preference **Service** (Redis cache + PG **store**) -> filter
-> Router: immediate (**priority** lane) vs. batch (pending **store**)
-> Channel workers (push/email/SMS) -> Providers (APNs/FCM/SES/Twilio) with breaker + rate **limit**
-> **In-app** inbox (Cassandra) + DLQ (Kafka) + metrics (delivered/bounce d/throttled)

Deep dive

Preferences — the filter that saves most work:

```

-- Postgres – preference store (strongly consistent, low write rate)
CREATE TABLE notification_prefs (
    user_id    uuid PRIMARY KEY,
    prefs      jsonb NOT NULL, -- { "like": {"push": true, "email":
"digest_daily"}, ... }
    quiet_hours tstzrange,
    digest_at   time,          -- daily digest time in user's tz
    caps        jsonb,         -- { "marketing_push_per_day": 3 }
    updated_at  timestampz DEFAULT now()
);
-- Cache in Redis with 30s TTL + invalidation on update (CDC via
Debezium or direct publish)
-- Preference check is on the hot path – must be <2ms, so local cache
(Caffeine) + Redis.

```

```

// Intake – dedup + preference filter (idempotent, exactly-once
visible)
func HandleEvent(ctx context.Context, ev Event) error {
    key := fmt.Sprintf("notif:%s:%s:%s", ev.Type, ev.ActorID, ev.Target
ID)
    if !dedup.TryOnce(ctx, key, 24*time.Hour) { // Redis SETNX with TTL
        return nil // duplicate
    }
    prefs, _ := prefCache.Get(ctx, ev.TargetUserID)
    channels :=
prefs.ChannelsFor(ev.Type) // e.g. ["push", "inbox"] or
["email:digest"]
    if len(channels) == 0 { return nil } // filtered

    for _, ch := range channels {
        if ch.IsDigest() {
            batchStore.Add(ctx, ev.TargetUserID, ch, ev) // pending
batch
        } else {
            priority := priorityOf(ev.Type) // transactional vs
marketing
            queueFor(ch, priority).Enqueue(ctx, Notification{Event:
ev, Channel: ch, Priority: priority})
        }
    }
    return nil
}

```

Batching / digest:

```

# Digest worker – fires per user per window (Redis sorted set per
user+channel, score=timestamp)
# Cron or Kafka delayed message triggers window close.
def flush_digest(user_id, channel, window):
    events = batch_store.pop_all(user_id, channel, window) # atomic
    pop
    if not events:
        return
    # Aggregate – e.g. "Alice and 12 others liked your post" rather
    than 13 pushes
    rendered = render_digest(events, channel) # template per channel

# Respect caps/quiet hours at flush time too (user may have entered
quiet hours)
    if in_quiet_hours(user_id) or over_cap(user_id, channel):
        # re-batch or drop per policy
        return
    channel_queue(channel, priority="marketing").enqueue(rendered)

```

Channel delivery with resilience (Chapter 11):

```

# Push worker – per-provider breaker + rate limit + retry
# APNs/FCM each get their own breaker and bulkhead so FCM slowness
doesn't block APNs.
providers:
    apns:
        breaker: { failureRateThreshold: 50, windowSize: 100,
waitDurationInOpenState: 30s }
        bulkhead: { maxConcurrentCalls: 100 }
        rate_limit: { rps: 5000 } # provider quota
        retry: { max: 3, backoff: exponential_jitter, retry_on: [503,
429] }
    fcm:
        breaker: { failureRateThreshold: 50, windowSize: 100,
waitDurationInOpenState: 30s }
        bulkhead: { maxConcurrentCalls: 100 }
        rate_limit: { rps: 8000 }
    ses:
        breaker: { failureRateThreshold: 30, windowSize: 100,
waitDurationInOpenState: 60s }
        bulkhead: { maxConcurrentCalls: 50 } # SES throttles aggressively

```

```
// Delivery with fallback: push fails -> still write in-app inbox
// (never lose)
func deliver(n Notification) error {
    err := providerFor(n.Channel).Send(ctx, n)
    if err != nil {
        if isRetryable(err) && retryBudget.Allow() {
            dlq.RetryLater(n, backoff(attempt)) // Kafka delayed retry
            // or SQS delay queue
        } else {
            dlq.DeadLetter(n, err) // manual inspection
        }
    }
    // In-app inbox is always written, even if push/email fails
    inboxStore.Append(ctx, n.UserID, n)
    return nil // intake ack'd; DLQ handles retry
}
```

Inbox storage:

```
-- Cassandra – in-app inbox (per-user, time-ordered, TTL for retention)
CREATE TABLE inbox (
    user_id    uuid,
    notif_id   timeuuid,
    type       text,
    title      text,
    body       text,
    read       boolean,
    PRIMARY KEY (userId, notif_id)
) WITH CLUSTERING ORDER BY (notif_id DESC)
AND default_time_to_live = 2592000; -- 30d
```

Failure and operations

| Failure | Mitigation |
|--|--|
| Notification storm (viral event → millions of like events) | Preference filter drops most; digest collapses remainder; marketing lane shed first (429 to producers); feature flag to disable non-critical types |
| Provider throttling (FCM 429, SES quota) | Per-provider breaker opens, queue backs up, DLQ with backoff; shed marketing, preserve transactional (Chapter 11 priority lanes) |
| Batch store loss | Batches are best-effort — loss means digest is short, not wrong; events can be replayed from Kafka within retention |

| Failure | Mitigation |
|--|---|
| Preference inconsistency (stale cache) | Stale preference may cause one extra/missed notification — acceptable; cache TTL 30 s + invalidation on write bounds staleness |
| Duplicate delivery | Dedup key on (event_type, actor, target, dedup_window) + provider-level idempotency key; at-least-once bus + dedup = exactly-once visible |

Case 4 — Global KV: the storage substrate

Requirements and scale

Functional: GET(key), PUT(key, value), DELETE(key), optional CAS(key, expected_version, value); tunable consistency per operation; TTL; range/bounded iteration is out of scope (use a document/search store — Chapter 5).

Non-functional: 10M RPS (80% reads), p99 read < 10 ms intra-region, < 40 ms cross-region, p99 write < 20 ms intra-region, survive AZ + region loss for critical keys, linear horizontal scale by adding nodes, no single master.

| Quantity | Assumption | Result |
|-------------|--|---|
| Keys | 10B keys, avg value 1 KB | 10 TB logical, ~30 TB with RF=3 |
| Throughput | 10M RPS | ~8M reads, 2M writes |
| Nodes | 100 nodes × 100K RPS each (with cache) | Fits with headroom; scale by adding nodes |
| Replication | RF=3 per DC, 2 DCs | 6 copies globally for critical data |

The defining trade-off: consistency per operation

A global KV cannot be strongly consistent, highly available, and low-latency all at once (CAP/PACELC, Chapter 10). The answer is tunable quorums: the client chooses $W + R$ per operation and the system guarantees $R + W > N$ for strong consistency when needed, eventual

otherwise. Dynamo-style systems push this choice to the caller; Spanner-style systems hide it behind TrueTime and transactions.

We design the Dynamo-style core (the pattern behind Cassandra, DynamoDB, Riak) because it exposes the trade-off most clearly and scales linearly. We then show where Spanner/CockroachDB diverge.

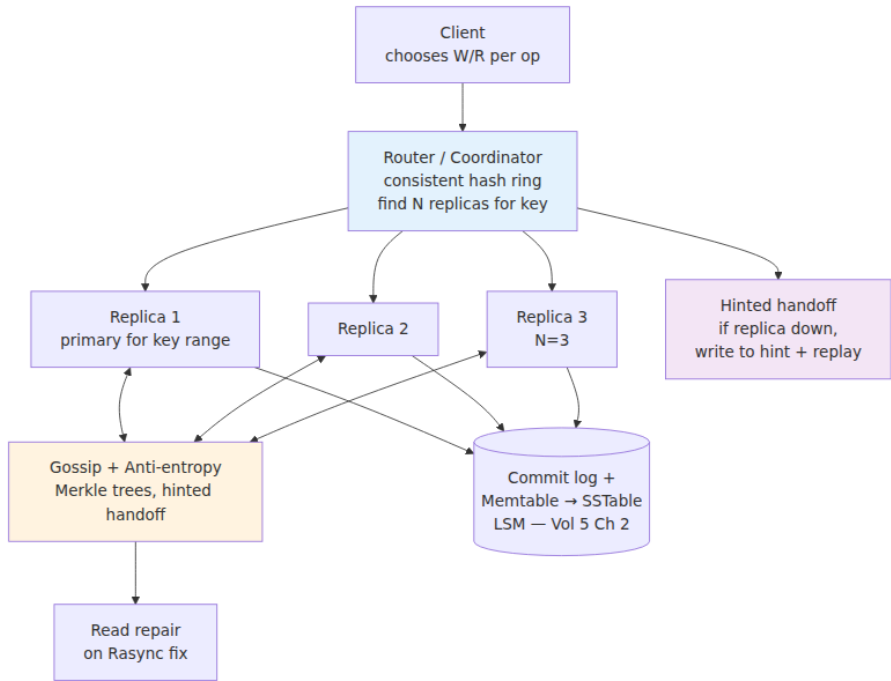


Figure 12-4: Dynamo-style KV. Consistent hashing places keys; coordinator routes to N replicas; gossip + hinted handoff + read repair converge.

Architecture and data flow

Ring and placement:

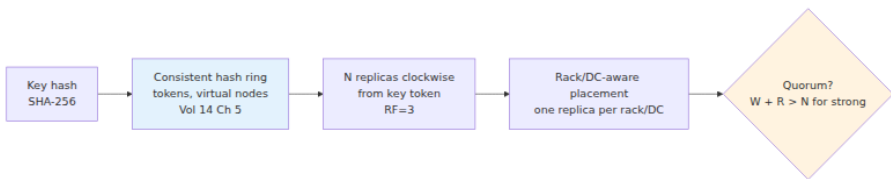


Figure 12-5: From key to replicas. Virtual nodes balance load; rack-aware placement survives rack/DC loss; quorum choice determines consistency.

Deep dive

Consistent hashing (Vol 14, Ch 5):

```
# Ring with virtual nodes – 256 vnodes per physical node
# Tokens are sorted; key's token is first token >= hash(key) clockwise.
import hashlib, bisect

class Ring:
    def __init__(self, nodes, vnodes=256):
        self.ring = [] # sorted list of (token, node)
        for node in nodes:
            for i in range(vnodes):
                token = int(hashlib.sha256(f"{node}#{i}".encode()).hexdigest(), 16)
                self.ring.append((token, node))
            self.ring.sort()

    def replicas(self, key, n=3):
        h = int(hashlib.sha256(key.encode()).hexdigest(), 16)
        idx = bisect.bisect_left(self.ring, (h,))
        seen, out = set(), []
        for j in range(len(self.ring)):
            _, node = self.ring[(idx + j) % len(self.ring)]
            if node not in seen:
                seen.add(node)
                out.append(node)
                if len(out) == n:
                    break
        return out

    def add_node(self, node, vnodes=256):
        # Only O(vnodes) tokens move – only keys whose replica set
        # includes the new vnodes
        for i in range(vnodes):
            token = int(hashlib.sha256(f"{node}#{i}".encode()).hexdigest(), 16)
            bisect.insort(self.ring, (token, node))
```

```

# Cassandra – global KV as CQL (tunable consistency per operation)
# Keyspace with RF 3 per DC, NetworkTopologyStrategy for rack awareness
CREATE KEYSPACE kv WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'us-east': '3',
  'eu-west': '3'
};
CREATE TABLE kv.data (
  k blob PRIMARY KEY,
  v blob,
  version bigint,          -- for CAS / vector clock
  ttl int,
  updated_at timeuuid
) WITH compaction = {'class': 'LeveledCompactionStrategy'};

# Client – choose consistency per operation
# Strong read: SELECT * FROM kv.data WHERE k = ? -- CONSISTENCY
QUORUM (R=2, N=3)
# Fast read:   SELECT * FROM kv.data WHERE k = ? -- CONSISTENCY ONE
(R=1)
# Strong write: INSERT ... USING CONSISTENCY QUORUM -- W=2
# Fast write:  INSERT ... USING CONSISTENCY ONE    -- W=1

```

```

// Coordinator – quorum write + hinted handoff + read repair
func (c *Coordinator) Put(ctx context.Context, key string, value
[]byte, opts PutOpts) error {
    replicas := c.ring.Replicas(key, c.N)
    // opts.W == 1 -> fast, eventually consistent; opts.W == quorum ->
strong
    w := opts.W
    if w == 0 { w = c.defaultW }
    successes := 0
    var wg sync.WaitGroup
    errs := make(chan error, len(replicas))
    for _, r := range replicas {
        wg.Add(1)
        go func(node string) {
            defer wg.Done()
            err := c.rpc.Put(ctx, node, key, value, opts.Version)
            if err != nil {
                // Hinted handoff: store hint locally, replay when node
recovers
                if isDown(err) { c.hints.Store(node, key, value) }
                errs <- err
                return
            }
            errs <- nil
        }(r)
    }
    // Wait for W acks or context deadline
    for i := 0; i < len(replicas); i++ {
        if err := <-errs; err == nil { successes++ }
        if successes >= w { return nil } // quorum satisfied –
remaining writes continue async
    }
    return fmt.Errorf("quorum not reached: %d/%d", successes, w)
}

func (c *Coordinator) Get(ctx context.Context, key string, opts
GetOpts) ([]byte, error) {
    replicas := c.ring.Replicas(key, c.N)
    r := opts.R
    if r == 0 { r = c.defaultR }
    // Read from R replicas, reconcile versions (vector clock /
timestamp), repair if stale
    results := c.scatterGather(ctx, replicas, key, r)
    value, stale := reconcile(results) // LWW or vector clock merge
    if stale {
        go c.readRepair(key, value, results) // async fix out-of-date
replicas
    }
    return value, nil
}

```

Versioning and conflict resolution:

| Strategy | How | When |
|---------------------------------------|---|--|
| Last-write-wins (timestamp) | Highest <code>updated_at</code> wins | Cache, session — loss tolerable |
| Vector clocks | Each replica increments its counter; concurrent versions are siblings the client merges | Shopping cart, collaborative — merge is domain-specific (Vol 6, Ch 11) |
| CRDTs | G-Counter / OR-Set converge without coordination | Counters, sets — limited types but no loss |
| CAS / lightweight transactions | IF <code>version = ?</code> — Paxos round, linearizable | Leader election, unique reservation — low throughput, strong guarantee |

```
-- Cassandra lightweight transaction – linearizable CAS for critical keys
-- Uses Paxos internally; p99 ~30-50ms – use sparingly, not on hot path
UPDATE kv.data SET v = 0x..., version = 4 WHERE k = 0x... IF version = 3;
-- Returns applied=true/false; caller retries on false (optimistic concurrency)
```

Cross-region replication (Chapter 10):

- **Sync (Spanner/CockroachDB path):** every write waits for cross-region quorum — strong but 60–150 ms per write. Use for `CAS` and control-plane keys.
- **Async (Dynamo/Cassandra path):** writes are `LOCAL_QUORUM` in the home region, replicated async via commit-log shipping or Kafka; reads with `LOCAL_QUORUM` are local and eventually consistent. Conflicts resolved by LWW/vector clock.

```

# Cassandra – per-DC quorums keep WAN off the hot path
# Write: LOCAL_QUORUM (2/3 in local DC, async to remote)
# Read:  LOCAL_QUORUM (local, may be stale by replication lag <1s)
# Strong: EACH_QUORUM (2/3 in each DC) – pays WAN, use for critical
keys only
consistency_levels:
  fast: { write: LOCAL_QUORUM, read: LOCAL_ONE }      # lowest
latency
  normal: { write: LOCAL_QUORUM, read: LOCAL_QUORUM }  # eventual,
local
  strong: { write: EACH_QUORUM, read: EACH_QUORUM }
# linearizable, pays WAN

```

Failure and operations

| Failure | Mitigation |
|--|--|
| Ring imbalance (hot keys, uneven vnodes) | Virtual nodes + load-aware token assignment; monitor per-node QPS and storage; move tokens or add nodes (only O(1/N) keys move) |
| Replica down | Hinted handoff (coordinator stores hint, replays on recovery) + anti-entropy (Merkle-tree comparison, repair) |
| Network partition | Tunable quorum: <code>W=1</code> stays available (AP), <code>W=quorum</code> prefers consistency (CP) — choose per operation, not per system |
| Thundering herd on cold key | Cache (Redis) in front with single-flight + jittered TTL; KV p99 stays bounded |
| Clock skew (LWW) | NTP/chrony + <code>max_offset</code> monitoring (Vol 6, Ch 2); prefer vector clocks for mergeable data where skew matters |
| Cross-region lag | <code>LOCAL_QUORUM</code> keeps reads local; monitor <code>replication_lag</code> ; strong reads use <code>EACH_QUORUM</code> only when required |

When to use Spanner/CockroachDB instead: when you need transactions across keys, secondary indexes with strong consistency, or SQL. The Dynamo core is a better fit when you need linear scale, per-operation tunable consistency, and no master — and can live without multi-key transactions (use Saga, Chapter 7, for cross-key workflows).

Cross-cutting lessons

These four cases share structure that generalizes:

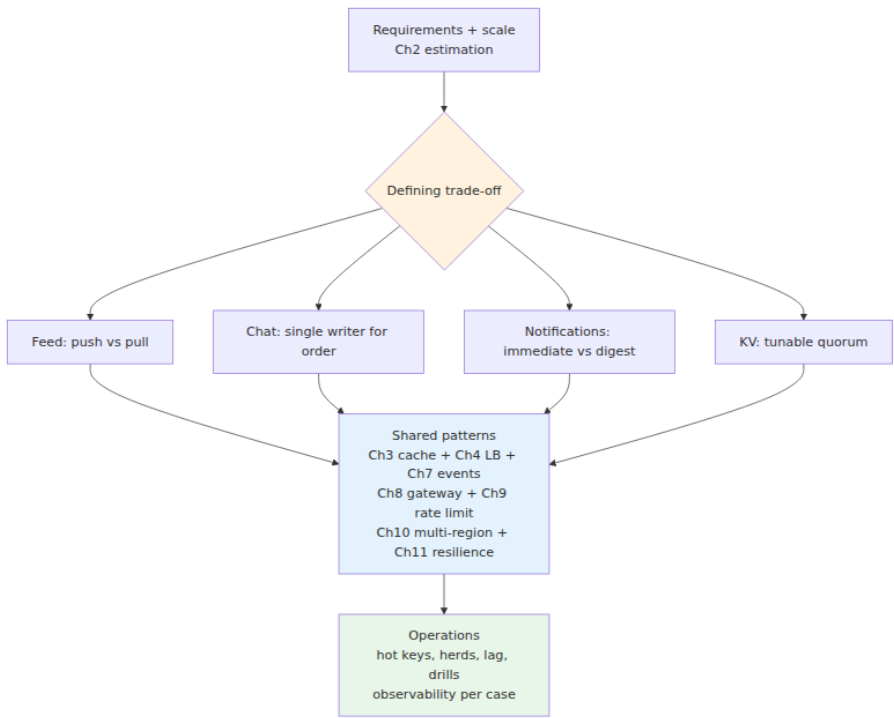


Figure 12-6: Every case is estimation → one defining trade-off → composition of the same patterns → operational hardening. The patterns repeat; the trade-off is what makes each system distinct.

| Pattern from this volume | Where it dominates |
|---------------------------|--|
| Estimation (Ch 2) | Every case — fan-out math, connection count, storage |
| Caching (Ch 3) | Feed timeline cache, chat presence/routing table, KV front-cache |
| Traffic management (Ch 4) | Chat sticky routing, feed fan-out sharding, KV ring routing |

| Pattern from this volume | Where it dominates |
|--------------------------|--|
| Data modeling (Ch 5) | Feed timeline/outbox tables, chat per-conversation log, KV LSM |
| Events (Ch 7) | Feed <code>post.created</code> fan-out, chat <code>message.sent</code> bus, notifications event intake |
| Gateways/Edge (Ch 8) | All cases — auth, rate limiting, WebSocket upgrade, edge cache for feed |
| Rate limiting (Ch 9) | Feed celebrity caps, chat send rate, notifications provider quotas |
| Multi-region (Ch 10) | Chat homed conversations, KV per-DC quorums, feed async replication |
| Resilience (Ch 11) | Feed ranker bulkhead, chat sequencer failover, notifications per-provider breakers, KV hinted handoff |

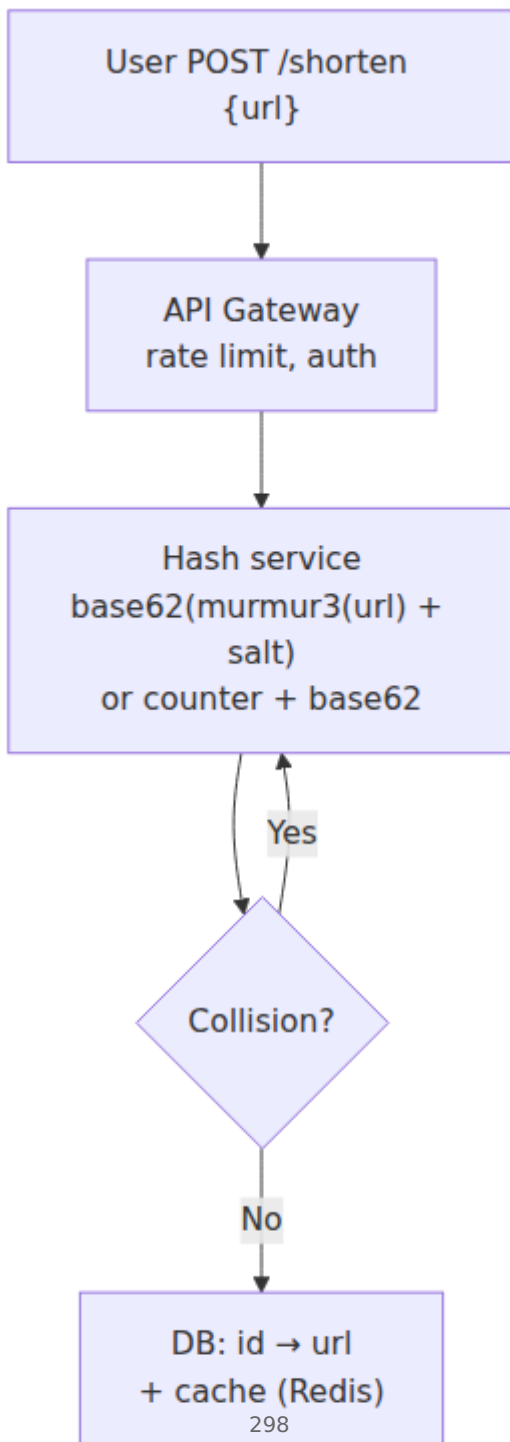
The senior move is not memorizing each architecture — it is recognizing which trade-off dominates a new problem and which patterns compose to bound it. A new case (search, payments, recommender) is another point in the same space.

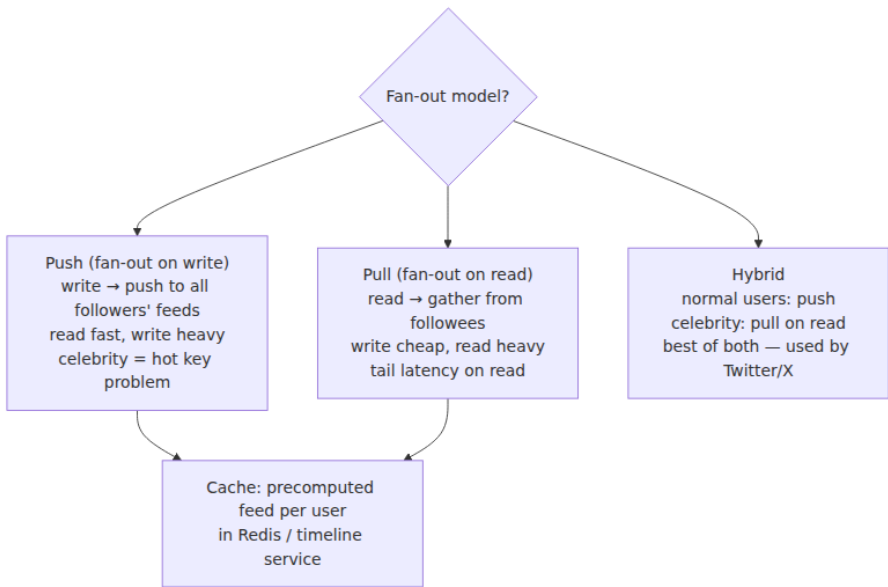
Distributed-systems lens

These cases make concrete why distributed systems is not a separate topic from system design — it is system design at scale. Feed's hybrid fan-out is a consistency/latency trade-off. Chat's single writer is consensus per conversation. Notifications' digest is batching and backpressure. KV's tunable quorum is CAP per operation. Each case chooses AP or CP per dataset and per operation, not per system — the same service is CP for one key and AP for another. That per-operation choice, made explicit and observable, is what lets a system be both fast and correct where it matters.

Key takeaways

- Estimation is the first design step — fan-out amplification, connection count, and storage math decide the architecture before any component is chosen; state assumptions explicitly.
- Feed is hybrid fan-out by necessity — push for normal users (bounded write amplification), pull for celebrities (unbounded fan-out avoided), inactive skipped; ranking merges both sources and degrades to chronological when slow.
- Chat needs per-conversation total order via single writer (sequencer) and exactly-once visible delivery via idempotency keys + `seq`; WebSocket sharding + Redis routing table + async fan-out via log keeps p99 low and reconnects lossless via `since_seq` replay.
- Notifications is a pipeline of filter → route → batch-or-send → provider with per-provider breakers and priority lanes; preferences and digest collapse most work before it reaches providers; transactional never shed before marketing.
- Global KV is a Dynamo-style ring with consistent hashing, virtual nodes, quorum $R+W>N$ for strong when needed, hinted handoff + anti-entropy + read repair for convergence; per-operation consistency (LOCAL vs. EACH quorum) is the API — strong only where required.
- Every case composes the same patterns (cache, LB, events, gateway, rate limit, multi-region, resilience) — the defining trade-off per case is what makes the composition distinct.
- Operability is the design — hot-key, herd, lag, and partition mitigations must be built in and drilled, not added after an outage.





Further reading

- Feed — Twitter (now X) Manhattan and home-timeline architecture; Instagram feed ranking. https://blog.twitter.com/engineering/en_us/topics/infrastructure/2021/processing-billions-of-events-in-real-time-at-twitter and <https://instagram-engineering.com/>
- Chat — WhatsApp architecture (Erlang, Mnesia); Discord's real-time architecture; Slack's channel server. <https://www.washingtonpost.com/news/the-switch/wp/2014/02/19/how-whatsapp-handles-50-billion-messages-a-day/> and <https://discord.com/blog/how-discord-stores-billions-of-messages>
- Notifications — Uber's notification platform; Airbnb's Dynein. <https://www.uber.com/blog/notification-platform/> and <https://medium.com/airbnb-engineering/dynein-building-a-self-service-notification-platform-4a322a3fad7a>
- KV — Dynamo paper (DeCandia et al., SOSP 2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- Cassandra — Architecture and tunable consistency. <https://cassandra.apache.org/doc/latest/cassandra/architecture/overview.html>

- Spanner — TrueTime and externally consistent transactions (Corbett et al., OSDI 2012). <https://research.google/pubs/pub39966/>
- Kleppmann — Designing Data-Intensive Applications, Ch. 5-6, 8-9, 12 (replication, partitioning, consistency, future of data systems).
- Volume 6 — Consistency, consensus, quorums, and CRDTs; Volume 5 — Storage engines (LSM/B-Tree); Volume 10 — Messaging and streaming internals.